General Rings, Ideals, and Morphisms

Release 10.10

The Sage Development Team

Oct 28, 2026

CONTENTS

1 Base Classes for Rings, Algebras and Fields 1

2 Ideals 19

3 Ring Morphisms 51

4 Quotient Rings 89

5 Fraction Fields 119

6 Localization 139

7 Ring Extensions 155

8 Generic Data Structures and Algorithms for Rings 217

9 Utilities 225

10 Derivation 243

11 Indices and Tables 269

Python Module Index 271

Index 273

CHAPTER

ONE

BASE CLASSES FOR RINGS, ALGEBRAS AND FIELDS

1.1 Rings

This module provides the abstract base class *Ring* from which all rings in Sage (used to) derive, as well as a selection of more specific base classes.

Warning

Those classes, except maybe for the lowest ones like *Field*, are being progressively deprecated in favor of the corresponding categories. which are more flexible, in particular with respect to multiple inheritance.

The class inheritance hierarchy is:

- *Ring* (to be deprecated)
  - *Algebra* (deprecated and essentially removed)
  - *CommutativeRing* (deprecated and essentially removed)
    - *NoetherianRing* (deprecated and essentially removed)
    - *CommutativeAlgebra* (deprecated and essentially removed)
    - *IntegralDomain* (deprecated and essentially removed)
      - *DedekindDomain* (deprecated and essentially removed)
      - *PrincipalIdealDomain* (deprecated and essentially removed)

Other subclaasses of *Ring* are

- *Field*
  - FiniteField

Some aspects of this structure may seem strange, but this is an unfortunate consequence of the fact that Cython classes do not support multiple inheritance.

(A distinct but equally awkward issue is that sometimes we may not know *in advance* whether or not a ring belongs in one of these classes; e.g. some orders in number fields are Dedekind domains, but others are not, and we still want to offer a unified interface, so orders are never instances of the deprecated *DedekindDomain* class.)

AUTHORS:

- David Harvey (2006-10-16): changed *CommutativeAlgebra* to derive from *CommutativeRing* instead of from *Algebra*.
- David Loeffler (2009-07-09): documentation fixes, added to reference manual.

- Simon King (2011-03-29): Proper use of the category framework for rings.
- Simon King (2011-05-20): Modify multiplication and _ideal_class_ to support ideals of non-commutative rings.

class sage.rings.ring.**Algebra**

Bases: *Ring*

class sage.rings.ring.**CommutativeAlgebra**

Bases: *Ring*

class sage.rings.ring.**CommutativeRing**

Bases: *Ring*

class sage.rings.ring.**DedekindDomain**

Bases: *Ring*

class sage.rings.ring.**Field**

Bases: *Ring*

Generic field

class sage.rings.ring.**IntegralDomain**

Bases: *Ring*

class sage.rings.ring.**NoetherianRing**

Bases: *Ring*

class sage.rings.ring.**PrincipalIdealDomain**

Bases: *Ring*

class sage.rings.ring.**Ring**

Bases: Parent

Generic ring class.

base_extend(*X*)

EXAMPLES:

```
sage: QQ.base_extend(GF(7))
Traceback (most recent call last):
...
TypeError: no base extension defined
sage: ZZ.base_extend(GF(7))
Finite Field of size 7
```

```
>>> from sage.all import *
>>> QQ.base_extend(GF(Integer(7)))
Traceback (most recent call last):
...
TypeError: no base extension defined
>>> ZZ.base_extend(GF(Integer(7)))
Finite Field of size 7
```

category()

Return the category to which this ring belongs.

> **Note**
>
> This method exists because sometimes a ring is its own base ring. During initialisation of a ring R, it may be checked whether the base ring (hence, the ring itself) is a ring. Hence, it is necessary that `R.category()` tells that `R` is a ring, even *before* its category is properly initialised.

EXAMPLES:

```
sage: FreeAlgebra(QQ, 3, 'x').category()  # todo: use a ring which is not an␣
↪algebra!   # needs sage.combinat sage.modules
Category of algebras with basis over Rational Field
```

```
>>> from sage.all import *
>>> FreeAlgebra(QQ, Integer(3), 'x').category()  # todo: use a ring which is␣
↪not an algebra!   # needs sage.combinat sage.modules
Category of algebras with basis over Rational Field
```

Since a quotient of the integers is its own base ring, and during initialisation of a ring it is tested whether the base ring belongs to the category of rings, the following is an indirect test that the `category()` method of rings returns the category of rings even before the initialisation was successful:

```
sage: I = Integers(15)
sage: I.base_ring() is I
True
sage: I.category()
Join of Category of finite commutative rings
    and Category of subquotients of monoids
    and Category of quotients of semigroups
    and Category of finite enumerated sets
```

```
>>> from sage.all import *
>>> I = Integers(Integer(15))
>>> I.base_ring() is I
True
>>> I.category()
Join of Category of finite commutative rings
    and Category of subquotients of monoids
    and Category of quotients of semigroups
    and Category of finite enumerated sets
```

gens()

Return a tuple whose entries are the generators for this object, in order.

EXAMPLES:

```
sage: ZZ.gens()
(1,)

sage: ZZ["y"].gens()
(y,)

sage: InfinitePolynomialRing(QQ, "a").gens()
(a_*,)
```

```
>>> from sage.all import *
>>> ZZ.gens()
(1,)

>>> ZZ["y"].gens()
(y,)

>>> InfinitePolynomialRing(QQ, "a").gens()
(a_*,)
```

one()

Return the one element of this ring (cached), if it exists.

EXAMPLES:

```
sage: ZZ.one()
1
sage: QQ.one()
1
sage: QQ['x'].one()
1
```

```
>>> from sage.all import *
>>> ZZ.one()
1
>>> QQ.one()
1
>>> QQ['x'].one()
1
```

The result is cached:

```
sage: ZZ.one() is ZZ.one()
True
```

```
>>> from sage.all import *
>>> ZZ.one() is ZZ.one()
True
```

order()

The number of elements of `self`.

EXAMPLES:

```
sage: GF(19).order()
19
sage: QQ.order()
+Infinity
```

```
>>> from sage.all import *
>>> GF(Integer(19)).order()
19
```

(continues on next page)

(continued from previous page)

```
>>> QQ.order()
+Infinity
```

zero()

Return the zero element of this ring (cached).

EXAMPLES:

```
sage: ZZ.zero()
0
sage: QQ.zero()
0
sage: QQ['x'].zero()
0
```

```
>>> from sage.all import *
>>> ZZ.zero()
0
>>> QQ.zero()
0
>>> QQ['x'].zero()
0
```

The result is cached:

```
sage: ZZ.zero() is ZZ.zero()
True
```

```
>>> from sage.all import *
>>> ZZ.zero() is ZZ.zero()
True
```

1.2 Abstract base classes for rings

class sage.rings.abc.**AlgebraicField**

Bases: *AlgebraicField_common*

Abstract base class for AlgebraicField.

This class is defined for the purpose of isinstance tests. It should not be instantiated.

EXAMPLES:

```
sage: import sage.rings.abc
sage: isinstance(QQbar, sage.rings.abc.AlgebraicField)                    #␣
↪needs sage.rings.number_field
True
sage: isinstance(AA, sage.rings.abc.AlgebraicField)                       #␣
↪needs sage.rings.number_field
False
```

```
>>> from sage.all import *
>>> import sage.rings.abc
>>> isinstance(QQbar, sage.rings.abc.AlgebraicField)                    #␣
↪needs sage.rings.number_field
True
>>> isinstance(AA, sage.rings.abc.AlgebraicField)                       #␣
↪needs sage.rings.number_field
False
```

By design, there is a unique direct subclass:

```
sage: sage.rings.abc.AlgebraicField.__subclasses__()                    #␣
↪needs sage.rings.number_field
[<class 'sage.rings.qqbar.AlgebraicField'>]

sage: len(sage.rings.abc.AlgebraicField.__subclasses__()) <= 1
True
```

```
>>> from sage.all import *
>>> sage.rings.abc.AlgebraicField.__subclasses__()                      #␣
↪needs sage.rings.number_field
[<class 'sage.rings.qqbar.AlgebraicField'>]

>>> len(sage.rings.abc.AlgebraicField.__subclasses__()) <= Integer(1)
True
```

class sage.rings.abc.**AlgebraicField_common**

Bases: *Field*

Abstract base class for `AlgebraicField_common`.

This class is defined for the purpose of `isinstance` tests. It should not be instantiated.

EXAMPLES:

```
sage: import sage.rings.abc
sage: isinstance(QQbar, sage.rings.abc.AlgebraicField_common)           #␣
↪needs sage.rings.number_field
True
sage: isinstance(AA, sage.rings.abc.AlgebraicField_common)              #␣
↪needs sage.rings.number_field
True
```

```
>>> from sage.all import *
>>> import sage.rings.abc
>>> isinstance(QQbar, sage.rings.abc.AlgebraicField_common)             #␣
↪needs sage.rings.number_field
True
>>> isinstance(AA, sage.rings.abc.AlgebraicField_common)                #␣
↪needs sage.rings.number_field
True
```

By design, other than the abstract subclasses *AlgebraicField* and *AlgebraicRealField*, there is only one direct implementation subclass:

```
sage: sage.rings.abc.AlgebraicField_common.__subclasses__()                    #␣
↪needs sage.rings.number_field
[<class 'sage.rings.abc.AlgebraicField'>,
 <class 'sage.rings.abc.AlgebraicRealField'>,
 <class 'sage.rings.qqbar.AlgebraicField_common'>]

sage: len(sage.rings.abc.AlgebraicField_common.__subclasses__()) <= 3
True
```

```
>>> from sage.all import *
>>> sage.rings.abc.AlgebraicField_common.__subclasses__()                      #␣
↪needs sage.rings.number_field
[<class 'sage.rings.abc.AlgebraicField'>,
 <class 'sage.rings.abc.AlgebraicRealField'>,
 <class 'sage.rings.qqbar.AlgebraicField_common'>]

>>> len(sage.rings.abc.AlgebraicField_common.__subclasses__()) <= Integer(3)
True
```

class sage.rings.abc.**AlgebraicRealField**

Bases: *AlgebraicField_common*

Abstract base class for `AlgebraicRealField`.

This class is defined for the purpose of `isinstance` tests. It should not be instantiated.

EXAMPLES:

```
sage: import sage.rings.abc
sage: isinstance(QQbar, sage.rings.abc.AlgebraicRealField)                     #␣
↪needs sage.rings.number_field
False
sage: isinstance(AA, sage.rings.abc.AlgebraicRealField)                        #␣
↪needs sage.rings.number_field
True
```

```
>>> from sage.all import *
>>> import sage.rings.abc
>>> isinstance(QQbar, sage.rings.abc.AlgebraicRealField)                       #␣
↪needs sage.rings.number_field
False
>>> isinstance(AA, sage.rings.abc.AlgebraicRealField)                          #␣
↪needs sage.rings.number_field
True
```

By design, there is a unique direct subclass:

```
sage: sage.rings.abc.AlgebraicRealField.__subclasses__()                       #␣
↪needs sage.rings.number_field
[<class 'sage.rings.qqbar.AlgebraicRealField'>]

sage: len(sage.rings.abc.AlgebraicRealField.__subclasses__()) <= 1
True
```

```
>>> from sage.all import *
>>> sage.rings.abc.AlgebraicRealField.__subclasses__()                          #␣
↪needs sage.rings.number_field
[<class 'sage.rings.qqbar.AlgebraicRealField'>]

>>> len(sage.rings.abc.AlgebraicRealField.__subclasses__()) <= Integer(1)
True
```

class sage.rings.abc.**ComplexBallField**

Bases: *Field*

Abstract base class for ComplexBallField.

This class is defined for the purpose of isinstance tests. It should not be instantiated.

EXAMPLES:

```
sage: import sage.rings.abc
sage: isinstance(CBF, sage.rings.abc.ComplexBallField)                          #␣
↪needs sage.libs.flint
True
```

```
>>> from sage.all import *
>>> import sage.rings.abc
>>> isinstance(CBF, sage.rings.abc.ComplexBallField)                            #␣
↪needs sage.libs.flint
True
```

By design, there is a unique direct subclass:

```
sage: sage.rings.abc.ComplexBallField.__subclasses__()                          #␣
↪needs sage.libs.flint
[<class 'sage.rings.complex_arb.ComplexBallField'>]

sage: len(sage.rings.abc.ComplexBallField.__subclasses__()) <= 1
True
```

```
>>> from sage.all import *
>>> sage.rings.abc.ComplexBallField.__subclasses__()                            #␣
↪needs sage.libs.flint
[<class 'sage.rings.complex_arb.ComplexBallField'>]

>>> len(sage.rings.abc.ComplexBallField.__subclasses__()) <= Integer(1)
True
```

class sage.rings.abc.**ComplexDoubleField**

Bases: *Field*

Abstract base class for ComplexDoubleField_class.

This class is defined for the purpose of isinstance tests. It should not be instantiated.

EXAMPLES:

```
sage: import sage.rings.abc
sage: isinstance(CDF, sage.rings.abc.ComplexDoubleField)                    #␣
↪needs sage.rings.complex_double
True
```

```
>>> from sage.all import *
>>> import sage.rings.abc
>>> isinstance(CDF, sage.rings.abc.ComplexDoubleField)                      #␣
↪needs sage.rings.complex_double
True
```

By design, there is a unique direct subclass:

```
sage: sage.rings.abc.ComplexDoubleField.__subclasses__()                    #␣
↪needs sage.rings.complex_double
[<class 'sage.rings.complex_double.ComplexDoubleField_class'>]

sage: len(sage.rings.abc.ComplexDoubleField.__subclasses__()) <= 1
True
```

```
>>> from sage.all import *
>>> sage.rings.abc.ComplexDoubleField.__subclasses__()                      #␣
↪needs sage.rings.complex_double
[<class 'sage.rings.complex_double.ComplexDoubleField_class'>]

>>> len(sage.rings.abc.ComplexDoubleField.__subclasses__()) <= Integer(1)
True
```

class sage.rings.abc.**ComplexField**

Bases: *Field*

Abstract base class for `ComplexField_class`.

This class is defined for the purpose of `isinstance` tests. It should not be instantiated.

EXAMPLES:

```
sage: import sage.rings.abc
sage: isinstance(CC, sage.rings.abc.ComplexField)                           #␣
↪needs sage.rings.real_mpfr
True
```

```
>>> from sage.all import *
>>> import sage.rings.abc
>>> isinstance(CC, sage.rings.abc.ComplexField)                             #␣
↪needs sage.rings.real_mpfr
True
```

By design, there is a unique direct subclass:

```
sage: sage.rings.abc.ComplexField.__subclasses__()                          #␣
↪needs sage.rings.real_mpfr
[<class 'sage.rings.complex_mpfr.ComplexField_class'>]
```

(continues on next page)

(continued from previous page)

```
sage: len(sage.rings.abc.ComplexField.__subclasses__()) <= 1
True
```

```
>>> from sage.all import *
>>> sage.rings.abc.ComplexField.__subclasses__()                              #␣
↪needs sage.rings.real_mpfr
[<class 'sage.rings.complex_mpfr.ComplexField_class'>]

>>> len(sage.rings.abc.ComplexField.__subclasses__()) <= Integer(1)
True
```

class sage.rings.abc.**ComplexIntervalField**

Bases: *Field*

Abstract base class for ComplexIntervalField_class.

This class is defined for the purpose of isinstance tests. It should not be instantiated.

EXAMPLES:

```
sage: import sage.rings.abc
sage: isinstance(CIF, sage.rings.abc.ComplexIntervalField)                    #␣
↪needs sage.rings.complex_interval_field
True
```

```
>>> from sage.all import *
>>> import sage.rings.abc
>>> isinstance(CIF, sage.rings.abc.ComplexIntervalField)                      #␣
↪needs sage.rings.complex_interval_field
True
```

By design, there is a unique direct subclass:

```
sage: sage.rings.abc.ComplexIntervalField.__subclasses__()                    #␣
↪needs sage.rings.complex_interval_field
[<class 'sage.rings.complex_interval_field.ComplexIntervalField_class'>]

sage: len(sage.rings.abc.ComplexIntervalField.__subclasses__()) <= 1
True
```

```
>>> from sage.all import *
>>> sage.rings.abc.ComplexIntervalField.__subclasses__()                      #␣
↪needs sage.rings.complex_interval_field
[<class 'sage.rings.complex_interval_field.ComplexIntervalField_class'>]

>>> len(sage.rings.abc.ComplexIntervalField.__subclasses__()) <= Integer(1)
True
```

class sage.rings.abc.**IntegerModRing**

Bases: object

Abstract base class for IntegerModRing_generic.

This class is defined for the purpose of isinstance tests. It should not be instantiated.

EXAMPLES:

```
sage: import sage.rings.abc
sage: isinstance(Integers(7), sage.rings.abc.IntegerModRing)
True
```

```
>>> from sage.all import *
>>> import sage.rings.abc
>>> isinstance(Integers(Integer(7)), sage.rings.abc.IntegerModRing)
True
```

By design, there is a unique direct subclass:

```
sage: sage.rings.abc.IntegerModRing.__subclasses__()
[<class 'sage.rings.finite_rings.integer_mod_ring.IntegerModRing_generic'>]

sage: len(sage.rings.abc.IntegerModRing.__subclasses__()) <= 1
True
```

```
>>> from sage.all import *
>>> sage.rings.abc.IntegerModRing.__subclasses__()
[<class 'sage.rings.finite_rings.integer_mod_ring.IntegerModRing_generic'>]

>>> len(sage.rings.abc.IntegerModRing.__subclasses__()) <= Integer(1)
True
```

class sage.rings.abc.**NumberField_cyclotomic**

Bases: *Field*

Abstract base class for NumberField_cyclotomic.

This class is defined for the purpose of `isinstance` tests. It should not be instantiated.

EXAMPLES:

```
sage: import sage.rings.abc
sage: K.<zeta> = CyclotomicField(15)                                    #␣
↪needs sage.rings.number_field
sage: isinstance(K, sage.rings.abc.NumberField_cyclotomic)              #␣
↪needs sage.rings.number_field
True
```

```
>>> from sage.all import *
>>> import sage.rings.abc
>>> K = CyclotomicField(Integer(15), names=('zeta',)); (zeta,) = K._first_ngens(1)
↪# needs sage.rings.number_field
>>> isinstance(K, sage.rings.abc.NumberField_cyclotomic)                #␣
↪needs sage.rings.number_field
True
```

By design, there is a unique direct subclass:

```
sage: sage.rings.abc.NumberField_cyclotomic.__subclasses__()            #␣
↪needs sage.rings.number_field
```

(continues on next page)

(continued from previous page)

```
[<class 'sage.rings.number_field.number_field.NumberField_cyclotomic'>]

sage: len(sage.rings.abc.NumberField_cyclotomic.__subclasses__()) <= 1
True
```

```
>>> from sage.all import *
>>> sage.rings.abc.NumberField_cyclotomic.__subclasses__()                    #␣
↪needs sage.rings.number_field
[<class 'sage.rings.number_field.number_field.NumberField_cyclotomic'>]

>>> len(sage.rings.abc.NumberField_cyclotomic.__subclasses__()) <= Integer(1)
True
```

class sage.rings.abc.**NumberField_quadratic**

Bases: *Field*

Abstract base class for `NumberField_quadratic`.

This class is defined for the purpose of `isinstance` tests. It should not be instantiated.

EXAMPLES:

```
sage: import sage.rings.abc
sage: K.<sqrt2> = QuadraticField(2)                                           #␣
↪needs sage.rings.number_field
sage: isinstance(K, sage.rings.abc.NumberField_quadratic)                     #␣
↪needs sage.rings.number_field
True
```

```
>>> from sage.all import *
>>> import sage.rings.abc
>>> K = QuadraticField(Integer(2), names=('sqrt2',)); (sqrt2,) = K._first_ngens(1)
↪# needs sage.rings.number_field
>>> isinstance(K, sage.rings.abc.NumberField_quadratic)                       #␣
↪needs sage.rings.number_field
True
```

By design, there is a unique direct subclass:

```
sage: sage.rings.abc.NumberField_quadratic.__subclasses__()                   #␣
↪needs sage.rings.number_field
[<class 'sage.rings.number_field.number_field.NumberField_quadratic'>]

sage: len(sage.rings.abc.NumberField_quadratic.__subclasses__()) <= 1
True
```

```
>>> from sage.all import *
>>> sage.rings.abc.NumberField_quadratic.__subclasses__()                     #␣
↪needs sage.rings.number_field
[<class 'sage.rings.number_field.number_field.NumberField_quadratic'>]

>>> len(sage.rings.abc.NumberField_quadratic.__subclasses__()) <= Integer(1)
True
```

class sage.rings.abc.**Order**

Bases: object

Abstract base class for Order.

This class is defined for the purpose of isinstance tests. It should not be instantiated.

EXAMPLES:

```
sage: import sage.rings.abc
sage: x = polygen(ZZ, 'x')
sage: K.<a> = NumberField(x^2 + 1); O = K.order(2*a)                    #␣
↪needs sage.rings.number_field
sage: isinstance(O, sage.rings.abc.Order)                               #␣
↪needs sage.rings.number_field
True
```

```
>>> from sage.all import *
>>> import sage.rings.abc
>>> x = polygen(ZZ, 'x')
>>> K = NumberField(x**Integer(2) + Integer(1), names=('a',)); (a,) = K._first_
↪ngens(1); O = K.order(Integer(2)*a)                    # needs sage.
↪rings.number_field
>>> isinstance(O, sage.rings.abc.Order)                                 #␣
↪needs sage.rings.number_field
True
```

By design, there is a unique direct subclass:

```
sage: sage.rings.abc.Order.__subclasses__()                             #␣
↪needs sage.rings.number_field
[<class 'sage.rings.number_field.order.Order'>]

sage: len(sage.rings.abc.Order.__subclasses__()) <= 1
True
```

```
>>> from sage.all import *
>>> sage.rings.abc.Order.__subclasses__()                               #␣
↪needs sage.rings.number_field
[<class 'sage.rings.number_field.order.Order'>]

>>> len(sage.rings.abc.Order.__subclasses__()) <= Integer(1)
True
```

class sage.rings.abc.**RealBallField**

Bases: *Field*

Abstract base class for RealBallField.

This class is defined for the purpose of isinstance tests. It should not be instantiated.

EXAMPLES:

```
sage: import sage.rings.abc
sage: isinstance(RBF, sage.rings.abc.RealBallField)                     #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.libs.flint
True
```

```
>>> from sage.all import *
>>> import sage.rings.abc
>>> isinstance(RBF, sage.rings.abc.RealBallField)                              #␣
↪needs sage.libs.flint
True
```

By design, there is a unique direct subclass:

```
sage: sage.rings.abc.RealBallField.__subclasses__()                            #␣
↪needs sage.libs.flint
[<class 'sage.rings.real_arb.RealBallField'>]

sage: len(sage.rings.abc.RealBallField.__subclasses__()) <= 1
True
```

```
>>> from sage.all import *
>>> sage.rings.abc.RealBallField.__subclasses__()                              #␣
↪needs sage.libs.flint
[<class 'sage.rings.real_arb.RealBallField'>]

>>> len(sage.rings.abc.RealBallField.__subclasses__()) <= Integer(1)
True
```

class sage.rings.abc.**RealDoubleField**

Bases: *Field*

Abstract base class for `RealDoubleField_class`.

This class is defined for the purpose of `isinstance` tests. It should not be instantiated.

EXAMPLES:

```
sage: import sage.rings.abc
sage: isinstance(RDF, sage.rings.abc.RealDoubleField)
True
```

```
>>> from sage.all import *
>>> import sage.rings.abc
>>> isinstance(RDF, sage.rings.abc.RealDoubleField)
True
```

By design, there is a unique direct subclass:

```
sage: sage.rings.abc.RealDoubleField.__subclasses__()
[<class 'sage.rings.real_double.RealDoubleField_class'>]

sage: len(sage.rings.abc.RealDoubleField.__subclasses__()) <= 1
True
```

```
>>> from sage.all import *
>>> sage.rings.abc.RealDoubleField.__subclasses__()
[<class 'sage.rings.real_double.RealDoubleField_class'>]

>>> len(sage.rings.abc.RealDoubleField.__subclasses__()) <= Integer(1)
True
```

class sage.rings.abc.**RealField**

Bases: *Field*

Abstract base class for RealField_class.

This class is defined for the purpose of isinstance tests. It should not be instantiated.

EXAMPLES:

```
sage: import sage.rings.abc
sage: isinstance(RR, sage.rings.abc.RealField)                                  #␣
↪needs sage.rings.real_mpfr
True
```

```
>>> from sage.all import *
>>> import sage.rings.abc
>>> isinstance(RR, sage.rings.abc.RealField)                                    #␣
↪needs sage.rings.real_mpfr
True
```

By design, there is a unique direct subclass:

```
sage: sage.rings.abc.RealField.__subclasses__()                                 #␣
↪needs sage.rings.real_mpfr
[<class 'sage.rings.real_mpfr.RealField_class'>]

sage: len(sage.rings.abc.RealField.__subclasses__()) <= 1
True
```

```
>>> from sage.all import *
>>> sage.rings.abc.RealField.__subclasses__()                                   #␣
↪needs sage.rings.real_mpfr
[<class 'sage.rings.real_mpfr.RealField_class'>]

>>> len(sage.rings.abc.RealField.__subclasses__()) <= Integer(1)
True
```

class sage.rings.abc.**RealIntervalField**

Bases: *Field*

Abstract base class for RealIntervalField_class.

This class is defined for the purpose of isinstance tests. It should not be instantiated.

EXAMPLES:

```
sage: import sage.rings.abc
sage: isinstance(RIF, sage.rings.abc.RealIntervalField)
True
```

```
>>> from sage.all import *
>>> import sage.rings.abc
>>> isinstance(RIF, sage.rings.abc.RealIntervalField)
True
```

By design, there is a unique direct subclass:

```
sage: sage.rings.abc.RealIntervalField.__subclasses__()
[<class 'sage.rings.real_mpfi.RealIntervalField_class'>]

sage: len(sage.rings.abc.RealIntervalField.__subclasses__()) <= 1
True
```

```
>>> from sage.all import *
>>> sage.rings.abc.RealIntervalField.__subclasses__()
[<class 'sage.rings.real_mpfi.RealIntervalField_class'>]

>>> len(sage.rings.abc.RealIntervalField.__subclasses__()) <= Integer(1)
True
```

class sage.rings.abc.**UniversalCyclotomicField**

Bases: *Field*

Abstract base class for `UniversalCyclotomicField`.

This class is defined for the purpose of `isinstance()` tests. It should not be instantiated.

EXAMPLES:

```
sage: import sage.rings.abc
sage: K = UniversalCyclotomicField()                                    #␣
↪needs sage.libs.gap sage.rings.number_field
sage: isinstance(K, sage.rings.abc.UniversalCyclotomicField)            #␣
↪needs sage.libs.gap sage.rings.number_field
True
```

```
>>> from sage.all import *
>>> import sage.rings.abc
>>> K = UniversalCyclotomicField()                                      #␣
↪needs sage.libs.gap sage.rings.number_field
>>> isinstance(K, sage.rings.abc.UniversalCyclotomicField)              #␣
↪needs sage.libs.gap sage.rings.number_field
True
```

By design, there is a unique direct subclass:

```
sage: sage.rings.abc.UniversalCyclotomicField.__subclasses__()          #␣
↪needs sage.libs.gap sage.rings.number_field
[<class 'sage.rings.universal_cyclotomic_field.UniversalCyclotomicField'>]

sage: len(sage.rings.abc.NumberField_cyclotomic.__subclasses__()) <= 1
True
```

```
>>> from sage.all import *
>>> sage.rings.abc.UniversalCyclotomicField.__subclasses__()                    #␣
↪needs sage.libs.gap sage.rings.number_field
[<class 'sage.rings.universal_cyclotomic_field.UniversalCyclotomicField'>]

>>> len(sage.rings.abc.NumberField_cyclotomic.__subclasses__()) <= Integer(1)
True
```

class sage.rings.abc.**pAdicField**

Bases: *Field*

Abstract base class for pAdicFieldGeneric.

This class is defined for the purpose of isinstance tests. It should not be instantiated.

EXAMPLES:

```
sage: import sage.rings.abc
sage: isinstance(Zp(5), sage.rings.abc.pAdicField)                              #␣
↪needs sage.rings.padics
False
sage: isinstance(Qp(5), sage.rings.abc.pAdicField)                              #␣
↪needs sage.rings.padics
True
```

```
>>> from sage.all import *
>>> import sage.rings.abc
>>> isinstance(Zp(Integer(5)), sage.rings.abc.pAdicField)                       ␣
↪     # needs sage.rings.padics
False
>>> isinstance(Qp(Integer(5)), sage.rings.abc.pAdicField)                       ␣
↪     # needs sage.rings.padics
True
```

By design, there is a unique direct subclass:

```
sage: sage.rings.abc.pAdicField.__subclasses__()                                #␣
↪needs sage.rings.padics
[<class 'sage.rings.padics.generic_nodes.pAdicFieldGeneric'>]

sage: len(sage.rings.abc.pAdicField.__subclasses__()) <= 1
True
```

```
>>> from sage.all import *
>>> sage.rings.abc.pAdicField.__subclasses__()                                  #␣
↪needs sage.rings.padics
[<class 'sage.rings.padics.generic_nodes.pAdicFieldGeneric'>]

>>> len(sage.rings.abc.pAdicField.__subclasses__()) <= Integer(1)
True
```

class sage.rings.abc.**pAdicRing**

Bases: *Ring*

Abstract base class for pAdicRingGeneric.

This class is defined for the purpose of `isinstance` tests. It should not be instantiated.

EXAMPLES:

```
sage: import sage.rings.abc
sage: isinstance(Zp(5), sage.rings.abc.pAdicRing)                              #␣
↪needs sage.rings.padics
True
sage: isinstance(Qp(5), sage.rings.abc.pAdicRing)                              #␣
↪needs sage.rings.padics
False
```

```
>>> from sage.all import *
>>> import sage.rings.abc
>>> isinstance(Zp(Integer(5)), sage.rings.abc.pAdicRing)                       ␣
↪     # needs sage.rings.padics
True
>>> isinstance(Qp(Integer(5)), sage.rings.abc.pAdicRing)                       ␣
↪     # needs sage.rings.padics
False
```

By design, there is a unique direct subclass:

```
sage: sage.rings.abc.pAdicRing.__subclasses__()                                #␣
↪needs sage.rings.padics
[<class 'sage.rings.padics.generic_nodes.pAdicRingGeneric'>]

sage: len(sage.rings.abc.pAdicRing.__subclasses__()) <= 1
True
```

```
>>> from sage.all import *
>>> sage.rings.abc.pAdicRing.__subclasses__()                                  #␣
↪needs sage.rings.padics
[<class 'sage.rings.padics.generic_nodes.pAdicRingGeneric'>]

>>> len(sage.rings.abc.pAdicRing.__subclasses__()) <= Integer(1)
True
```

CHAPTER

TWO

IDEALS

2.1 Ideals of commutative rings

Sage provides functionality for computing with ideals. One can create an ideal in any commutative or non-commutative ring R by giving a list of generators, using the notation `R.ideal([a,b,...])`. The case of non-commutative rings is implemented in *noncommutative_ideals*.

A more convenient notation may be `R*[a,b,...]` or `[a,b,...]*R`. If R is non-commutative, the former creates a left and the latter a right ideal, and `R*[a,b,...]*R` creates a two-sided ideal.

`sage.rings.ideal.`**`Cyclic`**(*R*, *n=None*, *homog=False*, *singular=None*)

Ideal of cyclic `n`-roots from 1-st `n` variables of `R` if `R` is coercible to `Singular`.

INPUT:

- `R` – base ring to construct ideal for
- `n` – number of cyclic roots (default: `None`); if `None`, then `n` is set to `R.ngens()`
- `homog` – boolean (default: `False`); if `True` a homogeneous ideal is returned using the last variable in the ideal
- `singular` – Singular instance to use

> **Note**
>
> `R` will be set as the active ring in `Singular`

EXAMPLES:

An example from a multivariate polynomial ring over the rationals:

```
sage: P.<x,y,z> = PolynomialRing(QQ, 3, order='lex')
sage: I = sage.rings.ideal.Cyclic(P); I                                    #␣
↪needs sage.libs.singular
Ideal (x + y + z, x*y + x*z + y*z, x*y*z - 1)
 of Multivariate Polynomial Ring in x, y, z over Rational Field
sage: I.groebner_basis()                                                   #␣
↪needs sage.libs.singular
[x + y + z, y^2 + y*z + z^2, z^3 - 1]
```

```
>>> from sage.all import *
>>> P = PolynomialRing(QQ, Integer(3), order='lex', names=('x', 'y', 'z',)); (x,␣
```

(continues on next page)

(continued from previous page)

```
↪y, z,) = P._first_ngens(3)
>>> I = sage.rings.ideal.Cyclic(P); I                                    #␣
↪needs sage.libs.singular
Ideal (x + y + z, x*y + x*z + y*z, x*y*z - 1)
 of Multivariate Polynomial Ring in x, y, z over Rational Field
>>> I.groebner_basis()                                                   #␣
↪needs sage.libs.singular
[x + y + z, y^2 + y*z + z^2, z^3 - 1]
```

We compute a Groebner basis for cyclic 6, which is a standard benchmark and test ideal:

```
sage: R.<x,y,z,t,u,v> = QQ['x,y,z,t,u,v']
sage: I = sage.rings.ideal.Cyclic(R, 6)                                  #␣
↪needs sage.libs.singular
sage: B = I.groebner_basis()                                             #␣
↪needs sage.libs.singular
sage: len(B)                                                             #␣
↪needs sage.libs.singular
45
```

```
>>> from sage.all import *
>>> R = QQ['x,y,z,t,u,v']; (x, y, z, t, u, v,) = R._first_ngens(6)
>>> I = sage.rings.ideal.Cyclic(R, Integer(6))                                ␣
↪     # needs sage.libs.singular
>>> B = I.groebner_basis()                                               #␣
↪needs sage.libs.singular
>>> len(B)                                                               #␣
↪needs sage.libs.singular
45
```

sage.rings.ideal.**FieldIdeal**(*R*)

Let `q = R.base_ring().order()` and $(x_0, ..., x_n)$ `= R.gens()` then if q is finite this constructor returns

$$\langle x_0^q - x_0, ..., x_n^q - x_n \rangle.$$

We call this ideal the field ideal and the generators the field equations.

EXAMPLES:

The field ideal generated from the polynomial ring over two variables in the finite field of size 2:

```
sage: P.<x,y> = PolynomialRing(GF(2), 2)
sage: I = sage.rings.ideal.FieldIdeal(P); I
Ideal (x^2 + x, y^2 + y) of
 Multivariate Polynomial Ring in x, y over Finite Field of size 2
```

```
>>> from sage.all import *
>>> P = PolynomialRing(GF(Integer(2)), Integer(2), names=('x', 'y',)); (x, y,) =␣
↪P._first_ngens(2)
>>> I = sage.rings.ideal.FieldIdeal(P); I
Ideal (x^2 + x, y^2 + y) of
 Multivariate Polynomial Ring in x, y over Finite Field of size 2
```

Another, similar example:

```
sage: Q.<x1,x2,x3,x4> = PolynomialRing(GF(2^4, name='alpha'), 4)                #␣
↪needs sage.rings.finite_rings
sage: J = sage.rings.ideal.FieldIdeal(Q); J                                     #␣
↪needs sage.rings.finite_rings
Ideal (x1^16 + x1, x2^16 + x2, x3^16 + x3, x4^16 + x4) of
 Multivariate Polynomial Ring in x1, x2, x3, x4
  over Finite Field in alpha of size 2^4
```

```
>>> from sage.all import *
>>> Q = PolynomialRing(GF(Integer(2)**Integer(4), name='alpha'), Integer(4),␣
↪names=('x1', 'x2', 'x3', 'x4',)); (x1, x2, x3, x4,) = Q._first_ngens(4)# needs␣
↪sage.rings.finite_rings
>>> J = sage.rings.ideal.FieldIdeal(Q); J                                       #␣
↪needs sage.rings.finite_rings
Ideal (x1^16 + x1, x2^16 + x2, x3^16 + x3, x4^16 + x4) of
 Multivariate Polynomial Ring in x1, x2, x3, x4
  over Finite Field in alpha of size 2^4
```

sage.rings.ideal.**Ideal**(**args*, ***kwds*)

Create the ideal in ring with given generators.

There are some shorthand notations for creating an ideal, in addition to using the *Ideal()* function:

- `R.ideal(gens, coerce=True)`
- `gens*R`
- `R*gens`

INPUT:

- `R` – a ring (optional; if not given, will try to infer it from `gens`)
- `gens` – list of elements generating the ideal
- `coerce` – boolean (default: `True`); whether `gens` need to be coerced into the ring

OUTPUT: the ideal of ring generated by `gens`

EXAMPLES:

```
sage: R.<x> = ZZ[]
sage: I = R.ideal([4 + 3*x + x^2, 1 + x^2])
sage: I
Ideal (x^2 + 3*x + 4, x^2 + 1) of Univariate Polynomial Ring in x over Integer␣
↪Ring
sage: Ideal(R, [4 + 3*x + x^2, 1 + x^2])
Ideal (x^2 + 3*x + 4, x^2 + 1) of Univariate Polynomial Ring in x over Integer␣
↪Ring
sage: Ideal((4 + 3*x + x^2, 1 + x^2))
Ideal (x^2 + 3*x + 4, x^2 + 1) of Univariate Polynomial Ring in x over Integer␣
↪Ring
```

```
>>> from sage.all import *
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> I = R.ideal([Integer(4) + Integer(3)*x + x**Integer(2), Integer(1) +␣
↪x**Integer(2)])
```

(continues on next page)

(continued from previous page)

```
>>> I
Ideal (x^2 + 3*x + 4, x^2 + 1) of Univariate Polynomial Ring in x over Integer␣
↪Ring
>>> Ideal(R, [Integer(4) + Integer(3)*x + x**Integer(2), Integer(1) +␣
↪x**Integer(2)])
Ideal (x^2 + 3*x + 4, x^2 + 1) of Univariate Polynomial Ring in x over Integer␣
↪Ring
>>> Ideal((Integer(4) + Integer(3)*x + x**Integer(2), Integer(1) + x**Integer(2)))
Ideal (x^2 + 3*x + 4, x^2 + 1) of Univariate Polynomial Ring in x over Integer␣
↪Ring
```

```
sage: ideal(x^2-2*x+1, x^2-1)
Ideal (x^2 - 2*x + 1, x^2 - 1) of Univariate Polynomial Ring in x over Integer␣
↪Ring
sage: ideal([x^2-2*x+1, x^2-1])
Ideal (x^2 - 2*x + 1, x^2 - 1) of Univariate Polynomial Ring in x over Integer␣
↪Ring
sage: l = [x^2-2*x+1, x^2-1]
sage: ideal(f^2 for f in l)
Ideal (x^4 - 4*x^3 + 6*x^2 - 4*x + 1, x^4 - 2*x^2 + 1) of
Univariate Polynomial Ring in x over Integer Ring
```

```
>>> from sage.all import *
>>> ideal(x**Integer(2)-Integer(2)*x+Integer(1), x**Integer(2)-Integer(1))
Ideal (x^2 - 2*x + 1, x^2 - 1) of Univariate Polynomial Ring in x over Integer␣
↪Ring
>>> ideal([x**Integer(2)-Integer(2)*x+Integer(1), x**Integer(2)-Integer(1)])
Ideal (x^2 - 2*x + 1, x^2 - 1) of Univariate Polynomial Ring in x over Integer␣
↪Ring
>>> l = [x**Integer(2)-Integer(2)*x+Integer(1), x**Integer(2)-Integer(1)]
>>> ideal(f**Integer(2) for f in l)
Ideal (x^4 - 4*x^3 + 6*x^2 - 4*x + 1, x^4 - 2*x^2 + 1) of
Univariate Polynomial Ring in x over Integer Ring
```

This example illustrates how Sage finds a common ambient ring for the ideal, even though 1 is in the integers (in this case).

```
sage: R.<t> = ZZ['t']
sage: i = ideal(1,t,t^2)
sage: i
Ideal (1, t, t^2) of Univariate Polynomial Ring in t over Integer Ring
sage: ideal(1/2,t,t^2)
Principal ideal (1) of Univariate Polynomial Ring in t over Rational Field
```

```
>>> from sage.all import *
>>> R = ZZ['t']; (t,) = R._first_ngens(1)
>>> i = ideal(Integer(1),t,t**Integer(2))
>>> i
Ideal (1, t, t^2) of Univariate Polynomial Ring in t over Integer Ring
>>> ideal(Integer(1)/Integer(2),t,t**Integer(2))
Principal ideal (1) of Univariate Polynomial Ring in t over Rational Field
```

This shows that the issues at Issue #1104 are resolved:

```
sage: Ideal(3, 5)
Principal ideal (1) of Integer Ring
sage: Ideal(ZZ, 3, 5)
Principal ideal (1) of Integer Ring
sage: Ideal(2, 4, 6)
Principal ideal (2) of Integer Ring
```

```
>>> from sage.all import *
>>> Ideal(Integer(3), Integer(5))
Principal ideal (1) of Integer Ring
>>> Ideal(ZZ, Integer(3), Integer(5))
Principal ideal (1) of Integer Ring
>>> Ideal(Integer(2), Integer(4), Integer(6))
Principal ideal (2) of Integer Ring
```

You have to provide enough information that Sage can figure out which ring to put the ideal in.

```
sage: I = Ideal([])
Traceback (most recent call last):
...
ValueError: unable to determine which ring to embed the ideal in

sage: I = Ideal()
Traceback (most recent call last):
...
ValueError: need at least one argument
```

```
>>> from sage.all import *
>>> I = Ideal([])
Traceback (most recent call last):
...
ValueError: unable to determine which ring to embed the ideal in

>>> I = Ideal()
Traceback (most recent call last):
...
ValueError: need at least one argument
```

Note that some rings use different ideal implementations than the standard, even if they are PIDs.:

```
sage: R.<x> = GF(5)[]
sage: I = R * (x^2 + 3)
sage: type(I)
<class 'sage.rings.polynomial.ideal.Ideal_1poly_field'>
```

```
>>> from sage.all import *
>>> R = GF(Integer(5))['x']; (x,) = R._first_ngens(1)
>>> I = R * (x**Integer(2) + Integer(3))
>>> type(I)
<class 'sage.rings.polynomial.ideal.Ideal_1poly_field'>
```

You can also pass in a specific ideal type:

```
sage: from sage.rings.ideal import Ideal_pid
sage: I = Ideal(x^2+3,ideal_class=Ideal_pid)
sage: type(I)
<class 'sage.rings.ideal.Ideal_pid'>
```

```
>>> from sage.all import *
>>> from sage.rings.ideal import Ideal_pid
>>> I = Ideal(x**Integer(2)+Integer(3),ideal_class=Ideal_pid)
>>> type(I)
<class 'sage.rings.ideal.Ideal_pid'>
```

class sage.rings.ideal.**Ideal_fractional**(*ring*, *gens*, *coerce=True*, ***kwds*)

Bases: *Ideal_generic*

Fractional ideal of a ring.

See *Ideal()*.

class sage.rings.ideal.**Ideal_generic**(*ring*, *gens*, *coerce=True*, ***kwds*)

Bases: MonoidElement

An ideal.

See *Ideal()*.

absolute_norm()

Return the absolute norm of this ideal.

In the general case, this is just the ideal itself, since the ring it lies in cannot be implicitly assumed to be an extension of anything.

We include this function for compatibility with cases such as ideals in number fields.

Todo

Implement this method.

EXAMPLES:

```
sage: R.<t> = GF(9, names='a')[]                                        #␣
↪needs sage.rings.finite_rings
sage: I = R.ideal(t^4 + t + 1)                                          #␣
↪needs sage.rings.finite_rings
sage: I.absolute_norm()                                                 #␣
↪needs sage.rings.finite_rings
Traceback (most recent call last):
...
NotImplementedError
```

```
>>> from sage.all import *
>>> R = GF(Integer(9), names='a')['t']; (t,) = R._first_ngens(1)# needs sage.
↪rings.finite_rings
>>> I = R.ideal(t**Integer(4) + t + Integer(1))                          ␣
↪              # needs sage.rings.finite_rings
```

(continues on next page)

(continued from previous page)

```
>>> I.absolute_norm()                                                    #␣
↪needs sage.rings.finite_rings
Traceback (most recent call last):
...
NotImplementedError
```

apply_morphism(*phi*)

Apply the morphism `phi` to every element of this ideal. Returns an ideal in the domain of `phi`.

EXAMPLES:

```
sage: psi = CC['x'].hom([-CC['x'].0])
sage: J = ideal([CC['x'].0 + 1]); J
Principal ideal (x + 1.00000000000000) of Univariate Polynomial Ring in x
 over Complex Field with 53 bits of precision
sage: psi(J)
Principal ideal (x - 1.00000000000000) of Univariate Polynomial Ring in x
 over Complex Field with 53 bits of precision
sage: J.apply_morphism(psi)
Principal ideal (x - 1.00000000000000) of Univariate Polynomial Ring in x
 over Complex Field with 53 bits of precision
```

```
>>> from sage.all import *
>>> psi = CC['x'].hom([-CC['x'].gen(0)])
>>> J = ideal([CC['x'].gen(0) + Integer(1)]); J
Principal ideal (x + 1.00000000000000) of Univariate Polynomial Ring in x
 over Complex Field with 53 bits of precision
>>> psi(J)
Principal ideal (x - 1.00000000000000) of Univariate Polynomial Ring in x
 over Complex Field with 53 bits of precision
>>> J.apply_morphism(psi)
Principal ideal (x - 1.00000000000000) of Univariate Polynomial Ring in x
 over Complex Field with 53 bits of precision
```

```
sage: psi = ZZ['x'].hom([-ZZ['x'].0])
sage: J = ideal([ZZ['x'].0, 2]); J
Ideal (x, 2) of Univariate Polynomial Ring in x over Integer Ring
sage: psi(J)
Ideal (-x, 2) of Univariate Polynomial Ring in x over Integer Ring
sage: J.apply_morphism(psi)
Ideal (-x, 2) of Univariate Polynomial Ring in x over Integer Ring
```

```
>>> from sage.all import *
>>> psi = ZZ['x'].hom([-ZZ['x'].gen(0)])
>>> J = ideal([ZZ['x'].gen(0), Integer(2)]); J
Ideal (x, 2) of Univariate Polynomial Ring in x over Integer Ring
>>> psi(J)
Ideal (-x, 2) of Univariate Polynomial Ring in x over Integer Ring
>>> J.apply_morphism(psi)
Ideal (-x, 2) of Univariate Polynomial Ring in x over Integer Ring
```

associated_primes()

Return the list of associated prime ideals of this ideal.

EXAMPLES:

```
sage: R = ZZ['x']
sage: I = R.ideal(7)
sage: I.associated_primes()
Traceback (most recent call last):
...
NotImplementedError
```

```
>>> from sage.all import *
>>> R = ZZ['x']
>>> I = R.ideal(Integer(7))
>>> I.associated_primes()
Traceback (most recent call last):
...
NotImplementedError
```

base_ring()

Return the base ring of this ideal.

EXAMPLES:

```
sage: R = ZZ
sage: I = 3*R; I
Principal ideal (3) of Integer Ring
sage: J = 2*I; J
Principal ideal (6) of Integer Ring
sage: I.base_ring(); J.base_ring()
Integer Ring
Integer Ring
```

```
>>> from sage.all import *
>>> R = ZZ
>>> I = Integer(3)*R; I
Principal ideal (3) of Integer Ring
>>> J = Integer(2)*I; J
Principal ideal (6) of Integer Ring
>>> I.base_ring(); J.base_ring()
Integer Ring
Integer Ring
```

We construct an example of an ideal of a quotient ring:

```
sage: R = PolynomialRing(QQ, 'x'); x = R.gen()
sage: I = R.ideal(x^2 - 2)
sage: I.base_ring()
Rational Field
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, 'x'); x = R.gen()
>>> I = R.ideal(x**Integer(2) - Integer(2))
```

(continues on next page)

(continued from previous page)

```
>>> I.base_ring()
Rational Field
```

And p-adic numbers:

```
sage: R = Zp(7, prec=10); R                                                    #␣
↪needs sage.rings.padics
7-adic Ring with capped relative precision 10
sage: I = 7*R; I                                                               #␣
↪needs sage.rings.padics
Principal ideal (7 + O(7^11)) of 7-adic Ring with capped relative precision 10
sage: I.base_ring()                                                            #␣
↪needs sage.rings.padics
7-adic Ring with capped relative precision 10
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7), prec=Integer(10)); R                                    ␣
↪              # needs sage.rings.padics
7-adic Ring with capped relative precision 10
>>> I = Integer(7)*R; I                                                       ␣
↪     # needs sage.rings.padics
Principal ideal (7 + O(7^11)) of 7-adic Ring with capped relative precision 10
>>> I.base_ring()                                                             #␣
↪needs sage.rings.padics
7-adic Ring with capped relative precision 10
```

category()

Return the category of this ideal.

Note

category is dependent on the ring of the ideal.

EXAMPLES:

```
sage: P.<x> = ZZ[]
sage: I = ZZ.ideal(7)
sage: J = P.ideal(7,x)
sage: K = P.ideal(7)
sage: I.category()
Category of ring ideals in Integer Ring
sage: J.category()
Category of ring ideals in Univariate Polynomial Ring in x
over Integer Ring
sage: K.category()
Category of ring ideals in Univariate Polynomial Ring in x
over Integer Ring
```

```
>>> from sage.all import *
>>> P = ZZ['x']; (x,) = P._first_ngens(1)
>>> I = ZZ.ideal(Integer(7))
```

(continues on next page)

(continued from previous page)

```
>>> J = P.ideal(Integer(7),x)
>>> K = P.ideal(Integer(7))
>>> I.category()
Category of ring ideals in Integer Ring
>>> J.category()
Category of ring ideals in Univariate Polynomial Ring in x
over Integer Ring
>>> K.category()
Category of ring ideals in Univariate Polynomial Ring in x
over Integer Ring
```

embedded_primes()

Return the list of embedded primes of this ideal.

EXAMPLES:

```
sage: R.<x, y> = QQ[]
sage: I = R.ideal(x^2, x*y)
sage: I.embedded_primes()                                                    #␣
↪needs sage.libs.singular
[Ideal (y, x) of Multivariate Polynomial Ring in x, y over Rational Field]
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> I = R.ideal(x**Integer(2), x*y)
>>> I.embedded_primes()                                                      #␣
↪needs sage.libs.singular
[Ideal (y, x) of Multivariate Polynomial Ring in x, y over Rational Field]
```

free_resolution(**args*, ***kwds*)

Return a free resolution of `self`.

For input options, see `FreeResolution`.

EXAMPLES:

```
sage: R.<x> = PolynomialRing(QQ)
sage: I = R.ideal([x^4 + 3*x^2 + 2])
sage: I.free_resolution()                                                    #␣
↪needs sage.modules
S^1 <-- S^1 <-- 0
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, names=('x',)); (x,) = R._first_ngens(1)
>>> I = R.ideal([x**Integer(4) + Integer(3)*x**Integer(2) + Integer(2)])
>>> I.free_resolution()                                                      #␣
↪needs sage.modules
S^1 <-- S^1 <-- 0
```

gen(*i*)

Return the `i`-th generator in the current basis of this ideal.

EXAMPLES:

```
sage: P.<x,y> = PolynomialRing(QQ,2)
sage: I = Ideal([x,y+1]); I
Ideal (x, y + 1) of Multivariate Polynomial Ring in x, y over Rational Field
sage: I.gen(1)
y + 1

sage: ZZ.ideal(5,10).gen()
5
```

```
>>> from sage.all import *
>>> P = PolynomialRing(QQ,Integer(2), names=('x', 'y',)); (x, y,) = P._first_
↪ngens(2)
>>> I = Ideal([x,y+Integer(1)]); I
Ideal (x, y + 1) of Multivariate Polynomial Ring in x, y over Rational Field
>>> I.gen(Integer(1))
y + 1

>>> ZZ.ideal(Integer(5),Integer(10)).gen()
5
```

gens()

Return a set of generators / a basis of `self`.

This is the set of generators provided during creation of this ideal.

EXAMPLES:

```
sage: P.<x,y> = PolynomialRing(QQ,2)
sage: I = Ideal([x,y+1]); I
Ideal (x, y + 1) of Multivariate Polynomial Ring in x, y over Rational Field
sage: I.gens()
[x, y + 1]
```

```
>>> from sage.all import *
>>> P = PolynomialRing(QQ,Integer(2), names=('x', 'y',)); (x, y,) = P._first_
↪ngens(2)
>>> I = Ideal([x,y+Integer(1)]); I
Ideal (x, y + 1) of Multivariate Polynomial Ring in x, y over Rational Field
>>> I.gens()
[x, y + 1]
```

```
sage: ZZ.ideal(5,10).gens()
(5,)
```

```
>>> from sage.all import *
>>> ZZ.ideal(Integer(5),Integer(10)).gens()
(5,)
```

gens_reduced()

Same as *gens()* for this ideal, since there is currently no special `gens_reduced` algorithm implemented for this ring.

This method is provided so that ideals in **Z** have the method `gens_reduced()`, just like ideals of number fields.

EXAMPLES:

```
sage: ZZ.ideal(5).gens_reduced()
(5,)
```

```
>>> from sage.all import *
>>> ZZ.ideal(Integer(5)).gens_reduced()
(5,)
```

graded_free_resolution(**args*, ***kwds*)

Return a graded free resolution of `self`.

For input options, see `GradedFiniteFreeResolution`.

EXAMPLES:

```
sage: R.<x> = PolynomialRing(QQ)
sage: I = R.ideal([x^3])
sage: I.graded_free_resolution()                                                #␣
↪needs sage.modules
S(0) <-- S(-3) <-- 0
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, names=('x',)); (x,) = R._first_ngens(1)
>>> I = R.ideal([x**Integer(3)])
>>> I.graded_free_resolution()                                                  #␣
↪needs sage.modules
S(0) <-- S(-3) <-- 0
```

is_maximal()

Return `True` if the ideal is maximal in the ring containing the ideal.

Todo

This is not implemented for many rings. Implement it!

EXAMPLES:

```
sage: R = ZZ
sage: I = R.ideal(7)
sage: I.is_maximal()
True
sage: R.ideal(16).is_maximal()
False
sage: S = Integers(8)
sage: S.ideal(0).is_maximal()
False
sage: S.ideal(2).is_maximal()
True
sage: S.ideal(4).is_maximal()
False
```

```
>>> from sage.all import *
>>> R = ZZ
>>> I = R.ideal(Integer(7))
>>> I.is_maximal()
True
>>> R.ideal(Integer(16)).is_maximal()
False
>>> S = Integers(Integer(8))
>>> S.ideal(Integer(0)).is_maximal()
False
>>> S.ideal(Integer(2)).is_maximal()
True
>>> S.ideal(Integer(4)).is_maximal()
False
```

is_primary(*P=None*)

Return `True` if this ideal is primary (or P-primary, if a prime ideal P is specified).

Recall that an ideal I is primary if and only if I has a unique associated prime (see page 52 in [AM1969]). If this prime is P, then I is said to be P-primary.

INPUT:

- `P` – (default: `None`) a prime ideal in the same ring

EXAMPLES:

```
sage: R.<x, y> = QQ[]
sage: I = R.ideal([x^2, x*y])
sage: I.is_primary()                                                    #␣
↪needs sage.libs.singular
False
sage: J = I.primary_decomposition()[1]; J                               #␣
↪needs sage.libs.singular
Ideal (y, x^2) of Multivariate Polynomial Ring in x, y over Rational Field
sage: J.is_primary()                                                    #␣
↪needs sage.libs.singular
True
sage: J.is_prime()                                                      #␣
↪needs sage.libs.singular
False
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> I = R.ideal([x**Integer(2), x*y])
>>> I.is_primary()                                                      #␣
↪needs sage.libs.singular
False
>>> J = I.primary_decomposition()[Integer(1)]; J                        ␣
↪     # needs sage.libs.singular
Ideal (y, x^2) of Multivariate Polynomial Ring in x, y over Rational Field
>>> J.is_primary()                                                      #␣
↪needs sage.libs.singular
True
```

(continues on next page)

(continued from previous page)

```
>>> J.is_prime()                                                    #␣
↪needs sage.libs.singular
False
```

Some examples from the Macaulay2 documentation:

```
sage: R.<x, y, z> = GF(101)[]
sage: I = R.ideal([y^6])
sage: I.is_primary()                                                #␣
↪needs sage.libs.singular
True
sage: I.is_primary(R.ideal([y]))                                    #␣
↪needs sage.libs.singular
True
sage: I = R.ideal([x^4, y^7])
sage: I.is_primary()                                                #␣
↪needs sage.libs.singular
True
sage: I = R.ideal([x*y, y^2])
sage: I.is_primary()                                                #␣
↪needs sage.libs.singular
False
```

```
>>> from sage.all import *
>>> R = GF(Integer(101))['x, y, z']; (x, y, z,) = R._first_ngens(3)
>>> I = R.ideal([y**Integer(6)])
>>> I.is_primary()                                                  #␣
↪needs sage.libs.singular
True
>>> I.is_primary(R.ideal([y]))                                      #␣
↪needs sage.libs.singular
True
>>> I = R.ideal([x**Integer(4), y**Integer(7)])
>>> I.is_primary()                                                  #␣
↪needs sage.libs.singular
True
>>> I = R.ideal([x*y, y**Integer(2)])
>>> I.is_primary()                                                  #␣
↪needs sage.libs.singular
False
```

Note

This uses the list of associated primes.

is_prime()

Return `True` if this ideal is prime.

EXAMPLES:

```
sage: R.<x, y> = QQ[]
sage: I = R.ideal([x, y])
sage: I.is_prime()        # a maximal ideal                                    #␣
↪needs sage.libs.singular
True
sage: I = R.ideal([x^2 - y])
sage: I.is_prime()        # a non-maximal prime ideal                          #␣
↪needs sage.libs.singular
True
sage: I = R.ideal([x^2, y])
sage: I.is_prime()        # a non-prime primary ideal                          #␣
↪needs sage.libs.singular
False
sage: I = R.ideal([x^2, x*y])
sage: I.is_prime()        # a non-prime non-primary ideal                      #␣
↪needs sage.libs.singular
False

sage: S = Integers(8)
sage: S.ideal(0).is_prime()
False
sage: S.ideal(2).is_prime()
True
sage: S.ideal(4).is_prime()
False
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> I = R.ideal([x, y])
>>> I.is_prime()        # a maximal ideal                                    #␣
↪needs sage.libs.singular
True
>>> I = R.ideal([x**Integer(2) - y])
>>> I.is_prime()        # a non-maximal prime ideal                          #␣
↪needs sage.libs.singular
True
>>> I = R.ideal([x**Integer(2), y])
>>> I.is_prime()        # a non-prime primary ideal                          #␣
↪needs sage.libs.singular
False
>>> I = R.ideal([x**Integer(2), x*y])
>>> I.is_prime()        # a non-prime non-primary ideal                      #␣
↪needs sage.libs.singular
False

>>> S = Integers(Integer(8))
>>> S.ideal(Integer(0)).is_prime()
False
>>> S.ideal(Integer(2)).is_prime()
True
>>> S.ideal(Integer(4)).is_prime()
False
```

Note that this method is not implemented for all rings where it could be:

```
sage: R.<x> = ZZ[]
sage: I = R.ideal(7)
sage: I.is_prime()        # when implemented, should be True
Traceback (most recent call last):
...
NotImplementedError
```

```
>>> from sage.all import *
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> I = R.ideal(Integer(7))
>>> I.is_prime()        # when implemented, should be True
Traceback (most recent call last):
...
NotImplementedError
```

Note

For general rings, uses the list of associated primes.

is_principal()

Return `True` if the ideal is principal in the ring containing the ideal.

Todo

Code is naive. Only keeps track of ideal generators as set during initialization of the ideal. (Can the base ring change? See example below.)

EXAMPLES:

```
sage: R.<x> = ZZ[]
sage: I = R.ideal(2, x)
sage: I.is_principal()
Traceback (most recent call last):
...
NotImplementedError
sage: J = R.base_extend(QQ).ideal(2, x)
sage: J.is_principal()
True
```

```
>>> from sage.all import *
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> I = R.ideal(Integer(2), x)
>>> I.is_principal()
Traceback (most recent call last):
...
NotImplementedError
>>> J = R.base_extend(QQ).ideal(Integer(2), x)
>>> J.is_principal()
True
```

is_trivial()

Return `True` if this ideal is (0) or (1).

minimal_associated_primes()

Return the list of minimal associated prime ideals of this ideal.

EXAMPLES:

```
sage: R = ZZ['x']
sage: I = R.ideal(7)
sage: I.minimal_associated_primes()
Traceback (most recent call last):
...
NotImplementedError
```

```
>>> from sage.all import *
>>> R = ZZ['x']
>>> I = R.ideal(Integer(7))
>>> I.minimal_associated_primes()
Traceback (most recent call last):
...
NotImplementedError
```

ngens()

Return the number of generators in the basis.

EXAMPLES:

```
sage: P.<x,y> = PolynomialRing(QQ,2)
sage: I = Ideal([x,y+1]); I
Ideal (x, y + 1) of Multivariate Polynomial Ring in x, y over Rational Field
sage: I.ngens()
2

sage: ZZ.ideal(5,10).ngens()
1
```

```
>>> from sage.all import *
>>> P = PolynomialRing(QQ,Integer(2), names=('x', 'y',)); (x, y,) = P._first_
↪ngens(2)
>>> I = Ideal([x,y+Integer(1)]); I
Ideal (x, y + 1) of Multivariate Polynomial Ring in x, y over Rational Field
>>> I.ngens()
2

>>> ZZ.ideal(Integer(5),Integer(10)).ngens()
1
```

norm()

Return the norm of this ideal.

In the general case, this is just the ideal itself, since the ring it lies in cannot be implicitly assumed to be an extension of anything.

We include this function for compatibility with cases such as ideals in number fields.

EXAMPLES:

```
sage: R.<t> = GF(8, names='a')[]                                                #␣
↪needs sage.rings.finite_rings
sage: I = R.ideal(t^4 + t + 1)                                                  #␣
↪needs sage.rings.finite_rings
sage: I.norm()                                                                  #␣
↪needs sage.rings.finite_rings
Principal ideal (t^4 + t + 1) of Univariate Polynomial Ring in t
 over Finite Field in a of size 2^3
```

```
>>> from sage.all import *
>>> R = GF(Integer(8), names='a')['t']; (t,) = R._first_ngens(1)# needs sage.
↪rings.finite_rings
>>> I = R.ideal(t**Integer(4) + t + Integer(1))                                 ␣
↪               # needs sage.rings.finite_rings
>>> I.norm()                                                                    #␣
↪needs sage.rings.finite_rings
Principal ideal (t^4 + t + 1) of Univariate Polynomial Ring in t
 over Finite Field in a of size 2^3
```

primary_decomposition()

Return a decomposition of this ideal into primary ideals.

EXAMPLES:

```
sage: R = ZZ['x']
sage: I = R.ideal(7)
sage: I.primary_decomposition()
Traceback (most recent call last):
...
NotImplementedError
```

```
>>> from sage.all import *
>>> R = ZZ['x']
>>> I = R.ideal(Integer(7))
>>> I.primary_decomposition()
Traceback (most recent call last):
...
NotImplementedError
```

random_element(**args*, ***kwds*)

Return a random element in this ideal.

EXAMPLES:

```
sage: P.<a,b,c> = GF(5)[[]]
sage: I = P.ideal([a^2, a*b + c, c^3])
sage: I.random_element()  # random
2*a^5*c + a^2*b*c^4 + ... + O(a, b, c)^13
```

```
>>> from sage.all import *
>>> P = GF(Integer(5))[['a, b, c']]; (a, b, c,) = P._first_ngens(3)
```

(continues on next page)

(continued from previous page)

```
>>> I = P.ideal([a**Integer(2), a*b + c, c**Integer(3)])
>>> I.random_element()  # random
2*a^5*c + a^2*b*c^4 + ... + O(a, b, c)^13
```

reduce(f)

Return the reduction of the element of f modulo `self`.

This is an element of R that is equivalent modulo I to f where I is `self`.

EXAMPLES:

```
sage: ZZ.ideal(5).reduce(17)
2
sage: parent(ZZ.ideal(5).reduce(17))
Integer Ring
```

```
>>> from sage.all import *
>>> ZZ.ideal(Integer(5)).reduce(Integer(17))
2
>>> parent(ZZ.ideal(Integer(5)).reduce(Integer(17)))
Integer Ring
```

ring()

Return the ring containing this ideal.

EXAMPLES:

```
sage: R = ZZ
sage: I = 3*R; I
Principal ideal (3) of Integer Ring
sage: J = 2*I; J
Principal ideal (6) of Integer Ring
sage: I.ring(); J.ring()
Integer Ring
Integer Ring
```

```
>>> from sage.all import *
>>> R = ZZ
>>> I = Integer(3)*R; I
Principal ideal (3) of Integer Ring
>>> J = Integer(2)*I; J
Principal ideal (6) of Integer Ring
>>> I.ring(); J.ring()
Integer Ring
Integer Ring
```

Note that `self.ring()` is different from `self.base_ring()`

```
sage: R = PolynomialRing(QQ, 'x'); x = R.gen()
sage: I = R.ideal(x^2 - 2)
sage: I.base_ring()
Rational Field
sage: I.ring()
Univariate Polynomial Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, 'x'); x = R.gen()
>>> I = R.ideal(x**Integer(2) - Integer(2))
>>> I.base_ring()
Rational Field
>>> I.ring()
Univariate Polynomial Ring in x over Rational Field
```

Another example using polynomial rings:

```
sage: R = PolynomialRing(QQ, 'x'); x = R.gen()
sage: I = R.ideal(x^2 - 3)
sage: I.ring()
Univariate Polynomial Ring in x over Rational Field
sage: Rbar = R.quotient(I, names='a')                                    #␣
↪needs sage.libs.pari
sage: S = PolynomialRing(Rbar, 'y'); y = Rbar.gen(); S                   #␣
↪needs sage.libs.pari
Univariate Polynomial Ring in y over
 Univariate Quotient Polynomial Ring in a over Rational Field with modulus x^
↪2 - 3
sage: J = S.ideal(y^2 + 1)                                               #␣
↪needs sage.libs.pari
sage: J.ring()                                                           #␣
↪needs sage.libs.pari
Univariate Polynomial Ring in y over
 Univariate Quotient Polynomial Ring in a over Rational Field with modulus x^
↪2 - 3
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, 'x'); x = R.gen()
>>> I = R.ideal(x**Integer(2) - Integer(3))
>>> I.ring()
Univariate Polynomial Ring in x over Rational Field
>>> Rbar = R.quotient(I, names='a')                                      #␣
↪needs sage.libs.pari
>>> S = PolynomialRing(Rbar, 'y'); y = Rbar.gen(); S                     #␣
↪needs sage.libs.pari
Univariate Polynomial Ring in y over
 Univariate Quotient Polynomial Ring in a over Rational Field with modulus x^
↪2 - 3
>>> J = S.ideal(y**Integer(2) + Integer(1))                                 ␣
↪               # needs sage.libs.pari
>>> J.ring()                                                             #␣
↪needs sage.libs.pari
Univariate Polynomial Ring in y over
 Univariate Quotient Polynomial Ring in a over Rational Field with modulus x^
↪2 - 3
```

class sage.rings.ideal.**Ideal_pid**(*ring*, *gens*, *coerce=True*, ***kwds*)

Bases: *Ideal_principal*

An ideal of a principal ideal domain.

See *Ideal()*.

EXAMPLES:

```
sage: I = 8*ZZ
sage: I
Principal ideal (8) of Integer Ring
```

```
>>> from sage.all import *
>>> I = Integer(8)*ZZ
>>> I
Principal ideal (8) of Integer Ring
```

gcd(*other*)

Return the greatest common divisor of the principal ideal with the ideal `other`; that is, the largest principal ideal contained in both the ideal and `other`

Todo

This is not implemented in the case when `other` is neither principal nor when the generator of `self` is contained in `other`. Also, it seems that this class is used only in PIDs–is this redundant?

Note

The second example is broken.

EXAMPLES:

An example in the principal ideal domain **Z**:

```
sage: R = ZZ
sage: I = R.ideal(42)
sage: J = R.ideal(70)
sage: I.gcd(J)
Principal ideal (14) of Integer Ring
sage: J.gcd(I)
Principal ideal (14) of Integer Ring
```

```
>>> from sage.all import *
>>> R = ZZ
>>> I = R.ideal(Integer(42))
>>> J = R.ideal(Integer(70))
>>> I.gcd(J)
Principal ideal (14) of Integer Ring
>>> J.gcd(I)
Principal ideal (14) of Integer Ring
```

is_maximal()

Return whether this ideal is maximal.

Principal ideal domains have Krull dimension 1 (or 0), so an ideal is maximal if and only if it is prime (and nonzero if the ring is not a field).

EXAMPLES:

```
sage: R.<t> = GF(5)[]
sage: p = R.ideal(t^2 + 2)
sage: p.is_maximal()
True
sage: p = R.ideal(t^2 + 1)
sage: p.is_maximal()
False
sage: p = R.ideal(0)
sage: p.is_maximal()
False
sage: p = R.ideal(1)
sage: p.is_maximal()
False
```

```
>>> from sage.all import *
>>> R = GF(Integer(5))['t']; (t,) = R._first_ngens(1)
>>> p = R.ideal(t**Integer(2) + Integer(2))
>>> p.is_maximal()
True
>>> p = R.ideal(t**Integer(2) + Integer(1))
>>> p.is_maximal()
False
>>> p = R.ideal(Integer(0))
>>> p.is_maximal()
False
>>> p = R.ideal(Integer(1))
>>> p.is_maximal()
False
```

is_prime()

Return `True` if the ideal is prime.

This relies on the ring elements having a method `is_irreducible()` implemented, since an ideal (a) is prime iff a is irreducible (or 0).

EXAMPLES:

```
sage: ZZ.ideal(2).is_prime()
True
sage: ZZ.ideal(-2).is_prime()
True
sage: ZZ.ideal(4).is_prime()
False
sage: ZZ.ideal(0).is_prime()
True
sage: R.<x> = QQ[]
sage: P = R.ideal(x^2 + 1); P
Principal ideal (x^2 + 1) of Univariate Polynomial Ring in x over Rational␣
↪Field
sage: P.is_prime()                                                          #␣
↪needs sage.libs.pari
True
```

```
>>> from sage.all import *
>>> ZZ.ideal(Integer(2)).is_prime()
True
>>> ZZ.ideal(-Integer(2)).is_prime()
True
>>> ZZ.ideal(Integer(4)).is_prime()
False
>>> ZZ.ideal(Integer(0)).is_prime()
True
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> P = R.ideal(x**Integer(2) + Integer(1)); P
Principal ideal (x^2 + 1) of Univariate Polynomial Ring in x over Rational␣
↪Field
>>> P.is_prime()                                                    #␣
↪needs sage.libs.pari
True
```

In fields, only the zero ideal is prime:

```
sage: RR.ideal(0).is_prime()
True
sage: RR.ideal(7).is_prime()
False
```

```
>>> from sage.all import *
>>> RR.ideal(Integer(0)).is_prime()
True
>>> RR.ideal(Integer(7)).is_prime()
False
```

radical()

Return the radical of this ideal.

EXAMPLES:

```
sage: ZZ.ideal(12).radical()
Principal ideal (6) of Integer Ring
```

```
>>> from sage.all import *
>>> ZZ.ideal(Integer(12)).radical()
Principal ideal (6) of Integer Ring
```

reduce(f)

Return the reduction of f modulo `self`.

EXAMPLES:

```
sage: I = 8*ZZ
sage: I.reduce(10)
2
sage: n = 10; n.mod(I)
2
```

```
>>> from sage.all import *
>>> I = Integer(8)*ZZ
>>> I.reduce(Integer(10))
2
>>> n = Integer(10); n.mod(I)
2
```

residue_field()

Return the residue class field of this ideal, which must be prime.

Todo

Implement this for more general rings. Currently only defined for **Z** and for number field orders.

EXAMPLES:

```
sage: P = ZZ.ideal(61); P
Principal ideal (61) of Integer Ring
sage: F = P.residue_field(); F
Residue field of Integers modulo 61
sage: pi = F.reduction_map(); pi
Partially defined reduction map:
  From: Rational Field
  To:   Residue field of Integers modulo 61
sage: pi(123/234)
6
sage: pi(1/61)
Traceback (most recent call last):
...
ZeroDivisionError: Cannot reduce rational 1/61 modulo 61: it has negative␣
↪valuation
sage: lift = F.lift_map(); lift
Lifting map:
  From: Residue field of Integers modulo 61
  To:   Integer Ring
sage: lift(F(12345/67890))
33
sage: (12345/67890) % 61
33
```

```
>>> from sage.all import *
>>> P = ZZ.ideal(Integer(61)); P
Principal ideal (61) of Integer Ring
>>> F = P.residue_field(); F
Residue field of Integers modulo 61
>>> pi = F.reduction_map(); pi
Partially defined reduction map:
  From: Rational Field
  To:   Residue field of Integers modulo 61
>>> pi(Integer(123)/Integer(234))
6
```

(continues on next page)

(continued from previous page)

```
>>> pi(Integer(1)/Integer(61))
Traceback (most recent call last):
...
ZeroDivisionError: Cannot reduce rational 1/61 modulo 61: it has negative␣
↪valuation
>>> lift = F.lift_map(); lift
Lifting map:
  From: Residue field of Integers modulo 61
  To:   Integer Ring
>>> lift(F(Integer(12345)/Integer(67890)))
33
>>> (Integer(12345)/Integer(67890)) % Integer(61)
33
```

class sage.rings.ideal.**Ideal_principal**(*ring*, *gens*, *coerce=True*, ***kwds*)

Bases: *Ideal_generic*

A principal ideal.

See *Ideal()*.

divides(*other*)

Return `True` if `self` divides `other`.

EXAMPLES:

```
sage: P.<x> = PolynomialRing(QQ)
sage: I = P.ideal(x)
sage: J = P.ideal(x^2)
sage: I.divides(J)
True
sage: J.divides(I)
False
```

```
>>> from sage.all import *
>>> P = PolynomialRing(QQ, names=('x',)); (x,) = P._first_ngens(1)
>>> I = P.ideal(x)
>>> J = P.ideal(x**Integer(2))
>>> I.divides(J)
True
>>> J.divides(I)
False
```

gen(*i=0*)

Return the generator of the principal ideal.

The generator is an element of the ring containing the ideal.

EXAMPLES:

A simple example in the integers:

```
sage: R = ZZ
sage: I = R.ideal(7)
sage: J = R.ideal(7, 14)
```

(continues on next page)

(continued from previous page)

```
sage: I.gen(); J.gen()
7
7
```

```
>>> from sage.all import *
>>> R = ZZ
>>> I = R.ideal(Integer(7))
>>> J = R.ideal(Integer(7), Integer(14))
>>> I.gen(); J.gen()
7
7
```

Note that the generator belongs to the ring from which the ideal was initialized:

```
sage: R.<x> = ZZ[]
sage: I = R.ideal(x)
sage: J = R.base_extend(QQ).ideal(2,x)
sage: a = I.gen(); a
x
sage: b = J.gen(); b
1
sage: a.base_ring()
Integer Ring
sage: b.base_ring()
Rational Field
```

```
>>> from sage.all import *
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> I = R.ideal(x)
>>> J = R.base_extend(QQ).ideal(Integer(2),x)
>>> a = I.gen(); a
x
>>> b = J.gen(); b
1
>>> a.base_ring()
Integer Ring
>>> b.base_ring()
Rational Field
```

is_principal()

Return `True` if the ideal is principal in the ring containing the ideal. When the ideal construction is explicitly principal (i.e. when we define an ideal with one element) this is always the case.

EXAMPLES:

Note that Sage automatically coerces ideals into principal ideals during initialization:

```
sage: R.<x> = ZZ[]
sage: I = R.ideal(x)
sage: J = R.ideal(2,x)
sage: K = R.base_extend(QQ).ideal(2,x)
sage: I
Principal ideal (x) of Univariate Polynomial Ring in x
```

(continues on next page)

(continued from previous page)

```
over Integer Ring
sage: J
Ideal (2, x) of Univariate Polynomial Ring in x over Integer Ring
sage: K
Principal ideal (1) of Univariate Polynomial Ring in x
over Rational Field
sage: I.is_principal()
True
sage: K.is_principal()
True
```

```
>>> from sage.all import *
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> I = R.ideal(x)
>>> J = R.ideal(Integer(2),x)
>>> K = R.base_extend(QQ).ideal(Integer(2),x)
>>> I
Principal ideal (x) of Univariate Polynomial Ring in x
over Integer Ring
>>> J
Ideal (2, x) of Univariate Polynomial Ring in x over Integer Ring
>>> K
Principal ideal (1) of Univariate Polynomial Ring in x
over Rational Field
>>> I.is_principal()
True
>>> K.is_principal()
True
```

sage.rings.ideal.**Katsura**(*R*, *n=None*, *homog=False*, *singular=None*)

n-th katsura ideal of R if R is coercible to `Singular`.

INPUT:

- `R` – base ring to construct ideal for
- `n` – (default: `None`) which katsura ideal of R. If `None`, then `n` is set to `R.ngens()`
- `homog` – boolean (default: `False`); if `True` a homogeneous ideal is returned using the last variable in the ideal
- `singular` – Singular instance to use

EXAMPLES:

```
sage: P.<x,y,z> = PolynomialRing(QQ, 3)
sage: I = sage.rings.ideal.Katsura(P, 3); I                                    #␣
↪needs sage.libs.singular
Ideal (x + 2*y + 2*z - 1, x^2 + 2*y^2 + 2*z^2 - x, 2*x*y + 2*y*z - y)
of Multivariate Polynomial Ring in x, y, z over Rational Field
```

```
>>> from sage.all import *
>>> P = PolynomialRing(QQ, Integer(3), names=('x', 'y', 'z',)); (x, y, z,) = P._
↪first_ngens(3)
```

(continues on next page)

(continued from previous page)

```
>>> I = sage.rings.ideal.Katsura(P, Integer(3)); I
↪     # needs sage.libs.singular
Ideal (x + 2*y + 2*z - 1, x^2 + 2*y^2 + 2*z^2 - x, 2*x*y + 2*y*z - y)
of Multivariate Polynomial Ring in x, y, z over Rational Field
```

```
sage: Q.<x> = PolynomialRing(QQ, implementation='singular')
↪needs sage.libs.singular
sage: J = sage.rings.ideal.Katsura(Q,1); J
↪needs sage.libs.singular
Ideal (x - 1) of Multivariate Polynomial Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> Q = PolynomialRing(QQ, implementation='singular', names=('x',)); (x,) = Q._
↪first_ngens(1)# needs sage.libs.singular
>>> J = sage.rings.ideal.Katsura(Q,Integer(1)); J
↪     # needs sage.libs.singular
Ideal (x - 1) of Multivariate Polynomial Ring in x over Rational Field
```

2.2 Monoid of ideals in a commutative ring

WARNING: This is used by some rings that are not commutative!

```
sage: MS = MatrixSpace(QQ, 3, 3)
↪needs sage.modules
sage: type(MS.ideal(MS.one()).parent())
↪needs sage.modules
<class 'sage.rings.ideal_monoid.IdealMonoid_c_with_category'>
```

```
>>> from sage.all import *
>>> MS = MatrixSpace(QQ, Integer(3), Integer(3))
↪              # needs sage.modules
>>> type(MS.ideal(MS.one()).parent())
↪needs sage.modules
<class 'sage.rings.ideal_monoid.IdealMonoid_c_with_category'>
```

sage.rings.ideal_monoid.**IdealMonoid**(*R*)

Return the monoid of ideals in the ring R.

EXAMPLES:

```
sage: R = QQ['x']
sage: from sage.rings.ideal_monoid import IdealMonoid
sage: IdealMonoid(R)
Monoid of ideals of Univariate Polynomial Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> R = QQ['x']
>>> from sage.rings.ideal_monoid import IdealMonoid
>>> IdealMonoid(R)
Monoid of ideals of Univariate Polynomial Ring in x over Rational Field
```

class sage.rings.ideal_monoid.**IdealMonoid_c**(*R*)

Bases: Parent

The monoid of ideals in a commutative ring.

Element

alias of *Ideal_generic*

ring()

Return the ring of which this is the ideal monoid.

EXAMPLES:

```
sage: R = QuadraticField(-23, 'a')                                          #␣
↪needs sage.rings.number_field
sage: from sage.rings.ideal_monoid import IdealMonoid
sage: M = IdealMonoid(R); M.ring() is R                                     #␣
↪needs sage.rings.number_field
True
```

```
>>> from sage.all import *
>>> R = QuadraticField(-Integer(23), 'a')                                    ␣
↪     # needs sage.rings.number_field
>>> from sage.rings.ideal_monoid import IdealMonoid
>>> M = IdealMonoid(R); M.ring() is R                                      #␣
↪needs sage.rings.number_field
True
```

2.3 Ideals of non-commutative rings

Generic implementation of one- and two-sided ideals of non-commutative rings.

AUTHOR:

- Simon King (2011-03-21), <simon.king@uni-jena.de>, Issue #7797.

EXAMPLES:

```
sage: MS = MatrixSpace(ZZ,2,2)
sage: MS*MS([0,1,-2,3])
Left Ideal
(
  [ 0  1]
  [-2  3]
)
 of Full MatrixSpace of 2 by 2 dense matrices over Integer Ring
sage: MS([0,1,-2,3])*MS
Right Ideal
(
  [ 0  1]
  [-2  3]
)
 of Full MatrixSpace of 2 by 2 dense matrices over Integer Ring
sage: MS*MS([0,1,-2,3])*MS
Twosided Ideal
```

(continues on next page)

(continued from previous page)

```
(
  [ 0  1]
  [-2  3]
)
 of Full MatrixSpace of 2 by 2 dense matrices over Integer Ring
```

```
>>> from sage.all import *
>>> MS = MatrixSpace(ZZ,Integer(2),Integer(2))
>>> MS*MS([Integer(0),Integer(1),-Integer(2),Integer(3)])
Left Ideal
(
  [ 0  1]
  [-2  3]
)
 of Full MatrixSpace of 2 by 2 dense matrices over Integer Ring
>>> MS([Integer(0),Integer(1),-Integer(2),Integer(3)])*MS
Right Ideal
(
  [ 0  1]
  [-2  3]
)
 of Full MatrixSpace of 2 by 2 dense matrices over Integer Ring
>>> MS*MS([Integer(0),Integer(1),-Integer(2),Integer(3)])*MS
Twosided Ideal
(
  [ 0  1]
  [-2  3]
)
 of Full MatrixSpace of 2 by 2 dense matrices over Integer Ring
```

See `letterplace_ideal` for a more elaborate implementation in the special case of ideals in free algebras.

class sage.rings.noncommutative_ideals.**IdealMonoid_nc**(*R*)

Bases: *IdealMonoid_c*

Base class for the monoid of ideals over a non-commutative ring.

Note

This class is essentially the same as *IdealMonoid_c*, but does not complain about non-commutative rings.

EXAMPLES:

```
sage: MS = MatrixSpace(ZZ,2,2)
sage: MS.ideal_monoid()
Monoid of ideals of Full MatrixSpace of 2 by 2 dense matrices over Integer Ring
```

```
>>> from sage.all import *
>>> MS = MatrixSpace(ZZ,Integer(2),Integer(2))
>>> MS.ideal_monoid()
Monoid of ideals of Full MatrixSpace of 2 by 2 dense matrices over Integer Ring
```

class sage.rings.noncommutative_ideals.**Ideal_nc**(*ring*, *gens*, *coerce=True*, *side='twosided'*)

Bases: *Ideal_generic*

Generic non-commutative ideal.

All fancy stuff such as the computation of Groebner bases must be implemented in sub-classes. See LetterplaceIdeal for an example.

EXAMPLES:

```
sage: MS = MatrixSpace(QQ,2,2)
sage: I = MS*[MS.1,MS.2]; I
Left Ideal
(
  [0 1]
  [0 0],

  [0 0]
  [1 0]
)
 of Full MatrixSpace of 2 by 2 dense matrices over Rational Field
sage: [MS.1,MS.2]*MS
Right Ideal
(
  [0 1]
  [0 0],

  [0 0]
  [1 0]
)
 of Full MatrixSpace of 2 by 2 dense matrices over Rational Field
sage: MS*[MS.1,MS.2]*MS
Twosided Ideal
(
  [0 1]
  [0 0],

  [0 0]
  [1 0]
)
 of Full MatrixSpace of 2 by 2 dense matrices over Rational Field
```

```
>>> from sage.all import *
>>> MS = MatrixSpace(QQ,Integer(2),Integer(2))
>>> I = MS*[MS.gen(1),MS.gen(2)]; I
Left Ideal
(
  [0 1]
  [0 0],
<BLANKLINE>
  [0 0]
  [1 0]
)
 of Full MatrixSpace of 2 by 2 dense matrices over Rational Field
```

(continues on next page)

(continued from previous page)

```
>>> [MS.gen(1),MS.gen(2)]*MS
Right Ideal
(
  [0 1]
  [0 0],
<BLANKLINE>
  [0 0]
  [1 0]
)
 of Full MatrixSpace of 2 by 2 dense matrices over Rational Field
>>> MS*[MS.gen(1),MS.gen(2)]*MS
Twosided Ideal
(
  [0 1]
  [0 0],
<BLANKLINE>
  [0 0]
  [1 0]
)
 of Full MatrixSpace of 2 by 2 dense matrices over Rational Field
```

side()

Return a string that describes the sidedness of this ideal.

EXAMPLES:

```
sage: A = SteenrodAlgebra(2)
sage: IL = A*[A.1+A.2,A.1^2]
sage: IR = [A.1+A.2,A.1^2]*A
sage: IT = A*[A.1+A.2,A.1^2]*A
sage: IL.side()
'left'
sage: IR.side()
'right'
sage: IT.side()
'twosided'
```

```
>>> from sage.all import *
>>> A = SteenrodAlgebra(Integer(2))
>>> IL = A*[A.gen(1)+A.gen(2),A.gen(1)**Integer(2)]
>>> IR = [A.gen(1)+A.gen(2),A.gen(1)**Integer(2)]*A
>>> IT = A*[A.gen(1)+A.gen(2),A.gen(1)**Integer(2)]*A
>>> IL.side()
'left'
>>> IR.side()
'right'
>>> IT.side()
'twosided'
```

CHAPTER

THREE

RING MORPHISMS

3.1 Homomorphisms of rings

We give a large number of examples of ring homomorphisms.

EXAMPLES:

Natural inclusion $\mathbf{Z} \hookrightarrow \mathbf{Q}$:

```
sage: H = Hom(ZZ, QQ)
sage: phi = H([1])
sage: phi(10)
10
sage: phi(3/1)
3
sage: phi(2/3)
Traceback (most recent call last):
...
TypeError: 2/3 fails to convert into the map's domain Integer Ring,
but a `pushforward` method is not properly implemented
```

```
>>> from sage.all import *
>>> H = Hom(ZZ, QQ)
>>> phi = H([Integer(1)])
>>> phi(Integer(10))
10
>>> phi(Integer(3)/Integer(1))
3
>>> phi(Integer(2)/Integer(3))
Traceback (most recent call last):
...
TypeError: 2/3 fails to convert into the map's domain Integer Ring,
but a `pushforward` method is not properly implemented
```

There is no homomorphism in the other direction:

```
sage: H = Hom(QQ, ZZ)
sage: H([1])
Traceback (most recent call last):
...
ValueError: relations do not all (canonically) map to 0
under map determined by images of generators
```

```
>>> from sage.all import *
>>> H = Hom(QQ, ZZ)
>>> H([Integer(1)])
Traceback (most recent call last):
...
ValueError: relations do not all (canonically) map to 0
under map determined by images of generators
```

EXAMPLES:

Reduction to finite field:

```
sage: H = Hom(ZZ, GF(9, 'a'))
sage: phi = H([1])
sage: phi(5)
2
sage: psi = H([4])
sage: psi(5)
2
```

```
>>> from sage.all import *
>>> H = Hom(ZZ, GF(Integer(9), 'a'))
>>> phi = H([Integer(1)])
>>> phi(Integer(5))
2
>>> psi = H([Integer(4)])
>>> psi(Integer(5))
2
```

Map from single variable polynomial ring:

```
sage: R.<x> = ZZ[]
sage: phi = R.hom([2], GF(5)); phi
Ring morphism:
  From: Univariate Polynomial Ring in x over Integer Ring
  To:   Finite Field of size 5
  Defn: x |--> 2
sage: phi(x + 12)
4
```

```
>>> from sage.all import *
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> phi = R.hom([Integer(2)], GF(Integer(5))); phi
Ring morphism:
  From: Univariate Polynomial Ring in x over Integer Ring
  To:   Finite Field of size 5
  Defn: x |--> 2
>>> phi(x + Integer(12))
4
```

Identity map on the real numbers:

```
sage: f = RR.hom([RR(1)]); f
Ring endomorphism of Real Field with 53 bits of precision
  Defn: 1.00000000000000 |--> 1.00000000000000
sage: f(2.5)
2.50000000000000
sage: f = RR.hom([2.0])
Traceback (most recent call last):
...
ValueError: relations do not all (canonically) map to 0
under map determined by images of generators
```

```
>>> from sage.all import *
>>> f = RR.hom([RR(Integer(1))]); f
Ring endomorphism of Real Field with 53 bits of precision
  Defn: 1.00000000000000 |--> 1.00000000000000
>>> f(RealNumber('2.5'))
2.50000000000000
>>> f = RR.hom([RealNumber('2.0')])
Traceback (most recent call last):
...
ValueError: relations do not all (canonically) map to 0
under map determined by images of generators
```

Homomorphism from one precision of field to another.

From smaller to bigger doesn't make sense:

```
sage: R200 = RealField(200)                                                 #␣
↪needs sage.rings.real_mpfr
sage: f = RR.hom( R200 )                                                    #␣
↪needs sage.rings.real_mpfr
Traceback (most recent call last):
...
TypeError: natural coercion morphism from Real Field with 53 bits of precision
to Real Field with 200 bits of precision not defined
```

```
>>> from sage.all import *
>>> R200 = RealField(Integer(200))                                           ␣
↪     # needs sage.rings.real_mpfr
>>> f = RR.hom( R200 )                                                     #␣
↪needs sage.rings.real_mpfr
Traceback (most recent call last):
...
TypeError: natural coercion morphism from Real Field with 53 bits of precision
to Real Field with 200 bits of precision not defined
```

From bigger to small does:

```
sage: f = RR.hom(RealField(15))                                             #␣
↪needs sage.rings.real_mpfr
sage: f(2.5)                                                                #␣
↪needs sage.rings.real_mpfr
2.500
```

(continues on next page)

(continued from previous page)

```
sage: f(RR.pi())                                                                #␣
↪needs sage.rings.real_mpfr
3.142
```

```
>>> from sage.all import *
>>> f = RR.hom(RealField(Integer(15)))                                          ␣
↪      # needs sage.rings.real_mpfr
>>> f(RealNumber('2.5'))                                                        ␣
↪           # needs sage.rings.real_mpfr
2.500
>>> f(RR.pi())                                                                 #␣
↪needs sage.rings.real_mpfr
3.142
```

Inclusion map from the reals to the complexes:

```
sage: i = RR.hom([CC(1)]); i
Ring morphism:
  From: Real Field with 53 bits of precision
  To:   Complex Field with 53 bits of precision
  Defn: 1.00000000000000 |--> 1.00000000000000
sage: i(RR('3.1'))
3.10000000000000
```

```
>>> from sage.all import *
>>> i = RR.hom([CC(Integer(1))]); i
Ring morphism:
  From: Real Field with 53 bits of precision
  To:   Complex Field with 53 bits of precision
  Defn: 1.00000000000000 |--> 1.00000000000000
>>> i(RR('3.1'))
3.10000000000000
```

A map from a multivariate polynomial ring to itself:

```
sage: R.<x,y,z> = PolynomialRing(QQ, 3)
sage: phi = R.hom([y, z, x^2]); phi
Ring endomorphism of Multivariate Polynomial Ring in x, y, z over Rational Field
  Defn: x |--> y
        y |--> z
        z |--> x^2
sage: phi(x + y + z)
x^2 + y + z
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, Integer(3), names=('x', 'y', 'z',)); (x, y, z,) = R._first_
↪ngens(3)
>>> phi = R.hom([y, z, x**Integer(2)]); phi
Ring endomorphism of Multivariate Polynomial Ring in x, y, z over Rational Field
  Defn: x |--> y
        y |--> z
        z |--> x^2
```

(continues on next page)

(continued from previous page)

```
>>> phi(x + y + z)
x^2 + y + z
```

An endomorphism of a quotient of a multi-variate polynomial ring:

```
sage: R.<x,y> = PolynomialRing(QQ)
sage: S.<a,b> = quo(R, ideal(1 + y^2))
sage: phi = S.hom([a^2, -b]); phi
Ring endomorphism of Quotient of Multivariate Polynomial Ring in x, y
 over Rational Field by the ideal (y^2 + 1)
  Defn: a |--> a^2
        b |--> -b
sage: phi(b)
-b
sage: phi(a^2 + b^2)
a^4 - 1
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> S = quo(R, ideal(Integer(1) + y**Integer(2)), names=('a', 'b',)); (a, b,) = S._
↪first_ngens(2)
>>> phi = S.hom([a**Integer(2), -b]); phi
Ring endomorphism of Quotient of Multivariate Polynomial Ring in x, y
 over Rational Field by the ideal (y^2 + 1)
  Defn: a |--> a^2
        b |--> -b
>>> phi(b)
-b
>>> phi(a**Integer(2) + b**Integer(2))
a^4 - 1
```

The reduction map from the integers to the integers modulo 8, viewed as a quotient ring:

```
sage: R = ZZ.quo(8*ZZ)
sage: pi = R.cover(); pi
Ring morphism:
  From: Integer Ring
  To:   Ring of integers modulo 8
  Defn: Natural quotient map
sage: pi.domain()
Integer Ring
sage: pi.codomain()
Ring of integers modulo 8
sage: pi(10)
2
sage: pi.lift()
Set-theoretic ring morphism:
  From: Ring of integers modulo 8
  To:   Integer Ring
  Defn: Choice of lifting map
sage: pi.lift(13)
5
```

```
>>> from sage.all import *
>>> R = ZZ.quo(Integer(8)*ZZ)
>>> pi = R.cover(); pi
Ring morphism:
  From: Integer Ring
  To:   Ring of integers modulo 8
  Defn: Natural quotient map
>>> pi.domain()
Integer Ring
>>> pi.codomain()
Ring of integers modulo 8
>>> pi(Integer(10))
2
>>> pi.lift()
Set-theoretic ring morphism:
  From: Ring of integers modulo 8
  To:   Integer Ring
  Defn: Choice of lifting map
>>> pi.lift(Integer(13))
5
```

Inclusion of `GF(2)` into `GF(4,'a')`:

```
sage: k = GF(2)
sage: i = k.hom(GF(4, 'a'))
sage: i
Ring morphism:
  From: Finite Field of size 2
  To:   Finite Field in a of size 2^2
  Defn: 1 |--> 1
sage: i(0)
0
sage: a = i(1); a.parent()
Finite Field in a of size 2^2
```

```
>>> from sage.all import *
>>> k = GF(Integer(2))
>>> i = k.hom(GF(Integer(4), 'a'))
>>> i
Ring morphism:
  From: Finite Field of size 2
  To:   Finite Field in a of size 2^2
  Defn: 1 |--> 1
>>> i(Integer(0))
0
>>> a = i(Integer(1)); a.parent()
Finite Field in a of size 2^2
```

We next compose the inclusion with reduction from the integers to `GF(2)`:

```
sage: pi = ZZ.hom(k); pi
Natural morphism:
  From: Integer Ring
```

(continues on next page)

(continued from previous page)

```
  To:   Finite Field of size 2
sage: f = i * pi; f
Composite map:
  From: Integer Ring
  To:   Finite Field in a of size 2^2
  Defn:   Natural morphism:
          From: Integer Ring
          To:   Finite Field of size 2
        then
          Ring morphism:
          From: Finite Field of size 2
          To:   Finite Field in a of size 2^2
          Defn: 1 |--> 1
sage: a = f(5); a
1
sage: a.parent()
Finite Field in a of size 2^2
```

```
>>> from sage.all import *
>>> pi = ZZ.hom(k); pi
Natural morphism:
  From: Integer Ring
  To:   Finite Field of size 2
>>> f = i * pi; f
Composite map:
  From: Integer Ring
  To:   Finite Field in a of size 2^2
  Defn:   Natural morphism:
          From: Integer Ring
          To:   Finite Field of size 2
        then
          Ring morphism:
          From: Finite Field of size 2
          To:   Finite Field in a of size 2^2
          Defn: 1 |--> 1
>>> a = f(Integer(5)); a
1
>>> a.parent()
Finite Field in a of size 2^2
```

Inclusion from $\mathbf{Q}$ to the 3-adic field:

```
sage: phi = QQ.hom(Qp(3, print_mode='series'))
sage: phi
Ring morphism:
  From: Rational Field
  To:   3-adic Field with capped relative precision 20
sage: phi.codomain()
3-adic Field with capped relative precision 20
sage: phi(394)
1 + 2*3 + 3^2 + 2*3^3 + 3^4 + 3^5 + O(3^20)
```

```
>>> from sage.all import *
>>> phi = QQ.hom(Qp(Integer(3), print_mode='series'))
>>> phi
Ring morphism:
  From: Rational Field
  To:   3-adic Field with capped relative precision 20
>>> phi.codomain()
3-adic Field with capped relative precision 20
>>> phi(Integer(394))
1 + 2*3 + 3^2 + 2*3^3 + 3^4 + 3^5 + O(3^20)
```

An automorphism of a quotient of a univariate polynomial ring:

```
sage: R.<x> = PolynomialRing(QQ)
sage: S.<sqrt2> = R.quo(x^2 - 2)
sage: sqrt2^2
2
sage: (3+sqrt2)^10
993054*sqrt2 + 1404491
sage: c = S.hom([-sqrt2])
sage: c(1+sqrt2)
-sqrt2 + 1
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, names=('x',)); (x,) = R._first_ngens(1)
>>> S = R.quo(x**Integer(2) - Integer(2), names=('sqrt2',)); (sqrt2,) = S._first_
↪ngens(1)
>>> sqrt2**Integer(2)
2
>>> (Integer(3)+sqrt2)**Integer(10)
993054*sqrt2 + 1404491
>>> c = S.hom([-sqrt2])
>>> c(Integer(1)+sqrt2)
-sqrt2 + 1
```

Note that Sage verifies that the morphism is valid:

```
sage: (1 - sqrt2)^2                                                             #␣
↪needs sage.libs.pari
-2*sqrt2 + 3
sage: c = S.hom([1 - sqrt2])    # this is not valid                             #␣
↪needs sage.libs.pari
Traceback (most recent call last):
...
ValueError: relations do not all (canonically) map to 0
under map determined by images of generators
```

```
>>> from sage.all import *
>>> (Integer(1) - sqrt2)**Integer(2)                                            ␣
↪               # needs sage.libs.pari
-2*sqrt2 + 3
>>> c = S.hom([Integer(1) - sqrt2])    # this is not valid                      ␣
↪     # needs sage.libs.pari
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
ValueError: relations do not all (canonically) map to 0
under map determined by images of generators
```

Endomorphism of power series ring:

```
sage: R.<t> = PowerSeriesRing(QQ, default_prec=10); R
Power Series Ring in t over Rational Field
sage: f = R.hom([t^2]); f
Ring endomorphism of Power Series Ring in t over Rational Field
  Defn: t |--> t^2
sage: s = 1/(1 + t); s
1 - t + t^2 - t^3 + t^4 - t^5 + t^6 - t^7 + t^8 - t^9 + O(t^10)
sage: f(s)
1 - t^2 + t^4 - t^6 + t^8 - t^10 + t^12 - t^14 + t^16 - t^18 + O(t^20)
```

```
>>> from sage.all import *
>>> R = PowerSeriesRing(QQ, default_prec=Integer(10), names=('t',)); (t,) = R._first_
↪ngens(1); R
Power Series Ring in t over Rational Field
>>> f = R.hom([t**Integer(2)]); f
Ring endomorphism of Power Series Ring in t over Rational Field
  Defn: t |--> t^2
>>> s = Integer(1)/(Integer(1) + t); s
1 - t + t^2 - t^3 + t^4 - t^5 + t^6 - t^7 + t^8 - t^9 + O(t^10)
>>> f(s)
1 - t^2 + t^4 - t^6 + t^8 - t^10 + t^12 - t^14 + t^16 - t^18 + O(t^20)
```

Frobenius on a power series ring over a finite field:

```
sage: R.<t> = PowerSeriesRing(GF(5))
sage: f = R.hom([t^5]); f
Ring endomorphism of Power Series Ring in t over Finite Field of size 5
  Defn: t |--> t^5
sage: a = 2 + t + 3*t^2 + 4*t^3 + O(t^4)
sage: b = 1 + t + 2*t^2 + t^3 + O(t^5)
sage: f(a)
2 + t^5 + 3*t^10 + 4*t^15 + O(t^20)
sage: f(b)
1 + t^5 + 2*t^10 + t^15 + O(t^25)
sage: f(a*b)
2 + 3*t^5 + 3*t^10 + t^15 + O(t^20)
sage: f(a)*f(b)
2 + 3*t^5 + 3*t^10 + t^15 + O(t^20)
```

```
>>> from sage.all import *
>>> R = PowerSeriesRing(GF(Integer(5)), names=('t',)); (t,) = R._first_ngens(1)
>>> f = R.hom([t**Integer(5)]); f
Ring endomorphism of Power Series Ring in t over Finite Field of size 5
  Defn: t |--> t^5
>>> a = Integer(2) + t + Integer(3)*t**Integer(2) + Integer(4)*t**Integer(3) +␣
```

(continues on next page)

(continued from previous page)

```
↪O(t**Integer(4))
>>> b = Integer(1) + t + Integer(2)*t**Integer(2) + t**Integer(3) + O(t**Integer(5))
>>> f(a)
2 + t^5 + 3*t^10 + 4*t^15 + O(t^20)
>>> f(b)
1 + t^5 + 2*t^10 + t^15 + O(t^25)
>>> f(a*b)
2 + 3*t^5 + 3*t^10 + t^15 + O(t^20)
>>> f(a)*f(b)
2 + 3*t^5 + 3*t^10 + t^15 + O(t^20)
```

Homomorphism of Laurent series ring:

```
sage: R.<t> = LaurentSeriesRing(QQ, 10)
sage: f = R.hom([t^3 + t]); f
Ring endomorphism of Laurent Series Ring in t over Rational Field
  Defn: t |--> t + t^3
sage: s = 2/t^2 + 1/(1 + t); s
2*t^-2 + 1 - t + t^2 - t^3 + t^4 - t^5 + t^6 - t^7 + t^8 - t^9 + O(t^10)
sage: f(s)
2*t^-2 - 3 - t + 7*t^2 - 2*t^3 - 5*t^4 - 4*t^5 + 16*t^6 - 9*t^7 + O(t^8)
sage: f = R.hom([t^3]); f
Ring endomorphism of Laurent Series Ring in t over Rational Field
  Defn: t |--> t^3
sage: f(s)
2*t^-6 + 1 - t^3 + t^6 - t^9 + t^12 - t^15 + t^18 - t^21 + t^24 - t^27 + O(t^30)
```

```
>>> from sage.all import *
>>> R = LaurentSeriesRing(QQ, Integer(10), names=('t',)); (t,) = R._first_ngens(1)
>>> f = R.hom([t**Integer(3) + t]); f
Ring endomorphism of Laurent Series Ring in t over Rational Field
  Defn: t |--> t + t^3
>>> s = Integer(2)/t**Integer(2) + Integer(1)/(Integer(1) + t); s
2*t^-2 + 1 - t + t^2 - t^3 + t^4 - t^5 + t^6 - t^7 + t^8 - t^9 + O(t^10)
>>> f(s)
2*t^-2 - 3 - t + 7*t^2 - 2*t^3 - 5*t^4 - 4*t^5 + 16*t^6 - 9*t^7 + O(t^8)
>>> f = R.hom([t**Integer(3)]); f
Ring endomorphism of Laurent Series Ring in t over Rational Field
  Defn: t |--> t^3
>>> f(s)
2*t^-6 + 1 - t^3 + t^6 - t^9 + t^12 - t^15 + t^18 - t^21 + t^24 - t^27 + O(t^30)
```

Note that the homomorphism must result in a converging Laurent series, so the valuation of the image of the generator must be positive:

```
sage: R.hom([1/t])
Traceback (most recent call last):
...
ValueError: relations do not all (canonically) map to 0
under map determined by images of generators
sage: R.hom([1])
Traceback (most recent call last):
```

(continues on next page)

(continued from previous page)

```
...
ValueError: relations do not all (canonically) map to 0
under map determined by images of generators
```

```
>>> from sage.all import *
>>> R.hom([Integer(1)/t])
Traceback (most recent call last):
...
ValueError: relations do not all (canonically) map to 0
under map determined by images of generators
>>> R.hom([Integer(1)])
Traceback (most recent call last):
...
ValueError: relations do not all (canonically) map to 0
under map determined by images of generators
```

Complex conjugation on cyclotomic fields:

```
sage: K.<zeta7> = CyclotomicField(7)
sage: c = K.hom([1/zeta7]); c
Ring endomorphism of Cyclotomic Field of order 7 and degree 6
  Defn: zeta7 |--> -zeta7^5 - zeta7^4 - zeta7^3 - zeta7^2 - zeta7 - 1
sage: a = (1+zeta7)^5; a
zeta7^5 + 5*zeta7^4 + 10*zeta7^3 + 10*zeta7^2 + 5*zeta7 + 1
sage: c(a)
5*zeta7^5 + 5*zeta7^4 - 4*zeta7^2 - 5*zeta7 - 4
sage: c(zeta7 + 1/zeta7)       # this element is obviously fixed by inversion
-zeta7^5 - zeta7^4 - zeta7^3 - zeta7^2 - 1
sage: zeta7 + 1/zeta7
-zeta7^5 - zeta7^4 - zeta7^3 - zeta7^2 - 1
```

```
>>> from sage.all import *
>>> K = CyclotomicField(Integer(7), names=('zeta7',)); (zeta7,) = K._first_ngens(1)
>>> c = K.hom([Integer(1)/zeta7]); c
Ring endomorphism of Cyclotomic Field of order 7 and degree 6
  Defn: zeta7 |--> -zeta7^5 - zeta7^4 - zeta7^3 - zeta7^2 - zeta7 - 1
>>> a = (Integer(1)+zeta7)**Integer(5); a
zeta7^5 + 5*zeta7^4 + 10*zeta7^3 + 10*zeta7^2 + 5*zeta7 + 1
>>> c(a)
5*zeta7^5 + 5*zeta7^4 - 4*zeta7^2 - 5*zeta7 - 4
>>> c(zeta7 + Integer(1)/zeta7)       # this element is obviously fixed by inversion
-zeta7^5 - zeta7^4 - zeta7^3 - zeta7^2 - 1
>>> zeta7 + Integer(1)/zeta7
-zeta7^5 - zeta7^4 - zeta7^3 - zeta7^2 - 1
```

Embedding a number field into the reals:

```
sage: R.<x> = PolynomialRing(QQ)
sage: K.<beta> = NumberField(x^3 - 2)
sage: alpha = RR(2)^(1/3); alpha
1.25992104989487
sage: i = K.hom([alpha],check=False); i
```

(continues on next page)

(continued from previous page)

```
Ring morphism:
  From: Number Field in beta with defining polynomial x^3 - 2
  To:   Real Field with 53 bits of precision
  Defn: beta |--> 1.25992104989487
sage: i(beta)
1.25992104989487
sage: i(beta^3)
2.00000000000000
sage: i(beta^2 + 1)
2.58740105196820
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, names=('x',)); (x,) = R._first_ngens(1)
>>> K = NumberField(x**Integer(3) - Integer(2), names=('beta',)); (beta,) = K._first_
↪ngens(1)
>>> alpha = RR(Integer(2))**(Integer(1)/Integer(3)); alpha
1.25992104989487
>>> i = K.hom([alpha],check=False); i
Ring morphism:
  From: Number Field in beta with defining polynomial x^3 - 2
  To:   Real Field with 53 bits of precision
  Defn: beta |--> 1.25992104989487
>>> i(beta)
1.25992104989487
>>> i(beta**Integer(3))
2.00000000000000
>>> i(beta**Integer(2) + Integer(1))
2.58740105196820
```

An example from Jim Carlson:

```
sage: K = QQ  # by the way :-)
sage: R.<a,b,c,d> = K[]; R
Multivariate Polynomial Ring in a, b, c, d over Rational Field
sage: S.<u> = K[]; S
Univariate Polynomial Ring in u over Rational Field
sage: f = R.hom([0,0,0,u], S); f
Ring morphism:
  From: Multivariate Polynomial Ring in a, b, c, d over Rational Field
  To:   Univariate Polynomial Ring in u over Rational Field
  Defn: a |--> 0
        b |--> 0
        c |--> 0
        d |--> u
sage: f(a + b + c + d)
u
sage: f((a+b+c+d)^2)
u^2
```

```
>>> from sage.all import *
>>> K = QQ  # by the way :-)
>>> R = K['a, b, c, d']; (a, b, c, d,) = R._first_ngens(4); R
```

(continues on next page)

(continued from previous page)

```
Multivariate Polynomial Ring in a, b, c, d over Rational Field
>>> S = K['u']; (u,) = S._first_ngens(1); S
Univariate Polynomial Ring in u over Rational Field
>>> f = R.hom([Integer(0),Integer(0),Integer(0),u], S); f
Ring morphism:
  From: Multivariate Polynomial Ring in a, b, c, d over Rational Field
  To:   Univariate Polynomial Ring in u over Rational Field
  Defn: a |--> 0
        b |--> 0
        c |--> 0
        d |--> u
>>> f(a + b + c + d)
u
>>> f((a+b+c+d)**Integer(2))
u^2
```

class sage.rings.morphism.**FrobeniusEndomorphism_generic**

Bases: *RingHomomorphism*

A class implementing Frobenius endomorphisms on rings of prime characteristic.

power()

Return an integer n such that this endomorphism is the n-th power of the absolute (arithmetic) Frobenius.

EXAMPLES:

```
sage: K.<u> = PowerSeriesRing(GF(5))
sage: Frob = K.frobenius_endomorphism()
sage: Frob.power()
1
sage: (Frob^9).power()
9
```

```
>>> from sage.all import *
>>> K = PowerSeriesRing(GF(Integer(5)), names=('u',)); (u,) = K._first_
↪ngens(1)
>>> Frob = K.frobenius_endomorphism()
>>> Frob.power()
1
>>> (Frob**Integer(9)).power()
9
```

class sage.rings.morphism.**RingHomomorphism**

Bases: *RingMap*

Homomorphism of rings.

inverse()

Return the inverse of this ring homomorphism if it exists.

Raises a ZeroDivisionError if the inverse does not exist.

ALGORITHM:

By default, this computes a Gröbner basis of the ideal corresponding to the graph of the ring homomorphism.

EXAMPLES:

```
sage: R.<t> = QQ[]
sage: f = R.hom([2*t - 1], R)
sage: f.inverse()                                                           #␣
↪needs sage.libs.singular
Ring endomorphism of Univariate Polynomial Ring in t over Rational Field
  Defn: t |--> 1/2*t + 1/2
```

```
>>> from sage.all import *
>>> R = QQ['t']; (t,) = R._first_ngens(1)
>>> f = R.hom([Integer(2)*t - Integer(1)], R)
>>> f.inverse()                                                             #␣
↪needs sage.libs.singular
Ring endomorphism of Univariate Polynomial Ring in t over Rational Field
  Defn: t |--> 1/2*t + 1/2
```

The following non-linear homomorphism is not invertible, but it induces an isomorphism on a quotient ring:

```
sage: R.<x,y,z> = QQ[]
sage: f = R.hom([y*z, x*z, x*y], R)
sage: f.inverse()
Traceback (most recent call last):
...
ZeroDivisionError: ring homomorphism not surjective
sage: f.is_injective()
True
sage: Q.<x,y,z> = R.quotient(x*y*z - 1)
sage: g = Q.hom([y*z, x*z, x*y], Q)
sage: g.inverse()
Ring endomorphism of Quotient of Multivariate Polynomial Ring
in x, y, z over Rational Field by the ideal (x*y*z - 1)
  Defn: x |--> y*z
        y |--> x*z
        z |--> x*y
```

```
>>> from sage.all import *
>>> R = QQ['x, y, z']; (x, y, z,) = R._first_ngens(3)
>>> f = R.hom([y*z, x*z, x*y], R)
>>> f.inverse()
Traceback (most recent call last):
...
ZeroDivisionError: ring homomorphism not surjective
>>> f.is_injective()
True
>>> Q = R.quotient(x*y*z - Integer(1), names=('x', 'y', 'z',)); (x, y, z,) =␣
↪Q._first_ngens(3)
>>> g = Q.hom([y*z, x*z, x*y], Q)
>>> g.inverse()
Ring endomorphism of Quotient of Multivariate Polynomial Ring
in x, y, z over Rational Field by the ideal (x*y*z - 1)
  Defn: x |--> y*z
        y |--> x*z
```

(continues on next page)

(continued from previous page)

```
        z |--> x*y
```

Homomorphisms over the integers are supported:

```
sage: S.<x,y> = ZZ[]
sage: f = S.hom([x + 2*y, x + 3*y], S)
sage: f.inverse()                                                           #␣
↪needs sage.libs.singular
Ring endomorphism of Multivariate Polynomial Ring in x, y over Integer Ring
  Defn: x |--> 3*x - 2*y
        y |--> -x + y
sage: (f.inverse() * f).is_identity()                                       #␣
↪needs sage.libs.singular
True
```

```
>>> from sage.all import *
>>> S = ZZ['x, y']; (x, y,) = S._first_ngens(2)
>>> f = S.hom([x + Integer(2)*y, x + Integer(3)*y], S)
>>> f.inverse()                                                             #␣
↪needs sage.libs.singular
Ring endomorphism of Multivariate Polynomial Ring in x, y over Integer Ring
  Defn: x |--> 3*x - 2*y
        y |--> -x + y
>>> (f.inverse() * f).is_identity()                                         #␣
↪needs sage.libs.singular
True
```

The following homomorphism is invertible over the rationals, but not over the integers:

```
sage: g = S.hom([x + y, x - y - 2], S)
sage: g.inverse()                                                           #␣
↪needs sage.libs.singular
Traceback (most recent call last):
...
ZeroDivisionError: ring homomorphism not surjective
sage: R.<x,y> = QQ[x,y]
sage: h = R.hom([x + y, x - y - 2], R)
sage: (h.inverse() * h).is_identity()                                       #␣
↪needs sage.libs.singular
True
```

```
>>> from sage.all import *
>>> g = S.hom([x + y, x - y - Integer(2)], S)
>>> g.inverse()                                                             #␣
↪needs sage.libs.singular
Traceback (most recent call last):
...
ZeroDivisionError: ring homomorphism not surjective
>>> R = QQ[x,y]; (x, y,) = R._first_ngens(2)
>>> h = R.hom([x + y, x - y - Integer(2)], R)
>>> (h.inverse() * h).is_identity()                                         #␣
↪needs sage.libs.singular
```

(continues on next page)

(continued from previous page)

```
True
```

This example by M. Nagata is a wild automorphism:

```
sage: R.<x,y,z> = QQ[]
sage: sigma = R.hom([x - 2*y*(z*x+y^2) - z*(z*x+y^2)^2,
....:                y + z*(z*x+y^2), z], R)
sage: tau = sigma.inverse(); tau                                              #␣
↪needs sage.libs.singular
Ring endomorphism of Multivariate Polynomial Ring in x, y, z over
Rational Field
  Defn: x |--> -y^4*z - 2*x*y^2*z^2 - x^2*z^3 + 2*y^3 + 2*x*y*z + x
        y |--> -y^2*z - x*z^2 + y
        z |--> z
sage: (tau * sigma).is_identity()                                             #␣
↪needs sage.libs.singular
True
```

```
>>> from sage.all import *
>>> R = QQ['x, y, z']; (x, y, z,) = R._first_ngens(3)
>>> sigma = R.hom([x - Integer(2)*y*(z*x+y**Integer(2)) -␣
↪z*(z*x+y**Integer(2))**Integer(2),
...                y + z*(z*x+y**Integer(2)), z], R)
>>> tau = sigma.inverse(); tau                                                #␣
↪needs sage.libs.singular
Ring endomorphism of Multivariate Polynomial Ring in x, y, z over
Rational Field
  Defn: x |--> -y^4*z - 2*x*y^2*z^2 - x^2*z^3 + 2*y^3 + 2*x*y*z + x
        y |--> -y^2*z - x*z^2 + y
        z |--> z
>>> (tau * sigma).is_identity()                                               #␣
↪needs sage.libs.singular
True
```

We compute the triangular automorphism that converts moments to cumulants, as well as its inverse, using the moment generating function. The choice of a term ordering can have a great impact on the computation time of a Gröbner basis, so here we choose a weighted ordering such that the images of the generators are homogeneous polynomials.

```
sage: d = 12
sage: T = TermOrder('wdegrevlex', [1..d])
sage: R = PolynomialRing(QQ, ['x%s' % j for j in (1..d)], order=T)
sage: S.<t> = PowerSeriesRing(R)
sage: egf = S([0] + list(R.gens())).ogf_to_egf().exp(prec=d+1)
sage: phi = R.hom(egf.egf_to_ogf().list()[1:], R)
sage: phi.im_gens()[:5]
[x1,
 x1^2 + x2,
 x1^3 + 3*x1*x2 + x3,
 x1^4 + 6*x1^2*x2 + 3*x2^2 + 4*x1*x3 + x4,
 x1^5 + 10*x1^3*x2 + 15*x1*x2^2 + 10*x1^2*x3 + 10*x2*x3 + 5*x1*x4 + x5]
sage: all(p.is_homogeneous() for p in phi.im_gens())                           #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.libs.singular
True
sage: phi.inverse().im_gens()[:5]                                           #␣
↪needs sage.libs.singular
[x1,
 -x1^2 + x2,
 2*x1^3 - 3*x1*x2 + x3,
 -6*x1^4 + 12*x1^2*x2 - 3*x2^2 - 4*x1*x3 + x4,
 24*x1^5 - 60*x1^3*x2 + 30*x1*x2^2 + 20*x1^2*x3 - 10*x2*x3 - 5*x1*x4 + x5]
sage: (phi.inverse() * phi).is_identity()                                   #␣
↪needs sage.libs.singular
True
```

```
>>> from sage.all import *
>>> d = Integer(12)
>>> T = TermOrder('wdegrevlex', (ellipsis_range(Integer(1),Ellipsis,d)))
>>> R = PolynomialRing(QQ, ['x%s' % j for j in (ellipsis_iter(Integer(1),
↪Ellipsis,d))], order=T)
>>> S = PowerSeriesRing(R, names=('t',)); (t,) = S._first_ngens(1)
>>> egf = S([Integer(0)] + list(R.gens())).ogf_to_egf().exp(prec=d+Integer(1))
>>> phi = R.hom(egf.egf_to_ogf().list()[Integer(1):], R)
>>> phi.im_gens()[:Integer(5)]
[x1,
 x1^2 + x2,
 x1^3 + 3*x1*x2 + x3,
 x1^4 + 6*x1^2*x2 + 3*x2^2 + 4*x1*x3 + x4,
 x1^5 + 10*x1^3*x2 + 15*x1*x2^2 + 10*x1^2*x3 + 10*x2*x3 + 5*x1*x4 + x5]
>>> all(p.is_homogeneous() for p in phi.im_gens())                          #␣
↪needs sage.libs.singular
True
>>> phi.inverse().im_gens()[:Integer(5)]                                     ␣
↪     # needs sage.libs.singular
[x1,
 -x1^2 + x2,
 2*x1^3 - 3*x1*x2 + x3,
 -6*x1^4 + 12*x1^2*x2 - 3*x2^2 - 4*x1*x3 + x4,
 24*x1^5 - 60*x1^3*x2 + 30*x1*x2^2 + 20*x1^2*x3 - 10*x2*x3 - 5*x1*x4 + x5]
>>> (phi.inverse() * phi).is_identity()                                     #␣
↪needs sage.libs.singular
True
```

Automorphisms of number fields as well as Galois fields are supported:

```
sage: K.<zeta7> = CyclotomicField(7)                                        #␣
↪needs sage.rings.number_field
sage: c = K.hom([1/zeta7])                                                  #␣
↪needs sage.rings.number_field
sage: (c.inverse() * c).is_identity()                                       #␣
↪needs sage.libs.singular sage.rings.number_field
True

sage: F.<t> = GF(7^3)                                                       #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.rings.finite_rings
sage: f = F.hom(t^7, F)                                                    #␣
↪needs sage.rings.finite_rings
sage: (f.inverse() * f).is_identity()                                      #␣
↪needs sage.libs.singular sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> K = CyclotomicField(Integer(7), names=('zeta7',)); (zeta7,) = K._first_
↪ngens(1)# needs sage.rings.number_field
>>> c = K.hom([Integer(1)/zeta7])                                          ␣
↪     # needs sage.rings.number_field
>>> (c.inverse() * c).is_identity()                                        #␣
↪needs sage.libs.singular sage.rings.number_field
True

>>> F = GF(Integer(7)**Integer(3), names=('t',)); (t,) = F._first_ngens(1)#␣
↪needs sage.rings.finite_rings
>>> f = F.hom(t**Integer(7), F)                                            ␣
↪     # needs sage.rings.finite_rings
>>> (f.inverse() * f).is_identity()                                        #␣
↪needs sage.libs.singular sage.rings.finite_rings
True
```

An isomorphism between the algebraic torus and the circle over a number field:

```
sage: K.<i> = QuadraticField(-1)
sage: A.<z,w> = K['z,w'].quotient('z*w - 1')
sage: B.<x,y> = K['x,y'].quotient('x^2 + y^2 - 1')
sage: f = A.hom([x + i*y, x - i*y], B)
sage: g = f.inverse()
sage: g.morphism_from_cover().im_gens()
[1/2*z + 1/2*w, (-1/2*i)*z + (1/2*i)*w]
sage: all(g(f(z)) == z for z in A.gens())
True
```

```
>>> from sage.all import *
>>> K = QuadraticField(-Integer(1), names=('i',)); (i,) = K._first_ngens(1)
>>> A = K['z,w'].quotient('z*w - 1', names=('z', 'w',)); (z, w,) = A._first_
↪ngens(2)
>>> B = K['x,y'].quotient('x^2 + y^2 - 1', names=('x', 'y',)); (x, y,) = B._
↪first_ngens(2)
>>> f = A.hom([x + i*y, x - i*y], B)
>>> g = f.inverse()
>>> g.morphism_from_cover().im_gens()
[1/2*z + 1/2*w, (-1/2*i)*z + (1/2*i)*w]
>>> all(g(f(z)) == z for z in A.gens())
True
```

inverse_image(*I*)

Return the inverse image of an ideal or an element in the codomain of this ring homomorphism.

INPUT:

- `I` – an ideal or element in the codomain

OUTPUT:

For an ideal I in the codomain, this returns the largest ideal in the domain whose image is contained in I.

Given an element b in the codomain, this returns an arbitrary element a in the domain such that `self(a) = b` if one such exists. The element a is unique if this ring homomorphism is injective.

EXAMPLES:

```
sage: R.<x,y,z> = QQ[]
sage: S.<u,v> = QQ[]
sage: f = R.hom([u^2, u*v, v^2], S)
sage: I = S.ideal([u^6, u^5*v, u^4*v^2, u^3*v^3])
sage: J = f.inverse_image(I); J                                                 #␣
↪needs sage.libs.singular
Ideal (y^2 - x*z, x*y*z, x^2*z, x^2*y, x^3)
of Multivariate Polynomial Ring in x, y, z over Rational Field
sage: f(J) == I                                                                 #␣
↪needs sage.libs.singular
True
```

```
>>> from sage.all import *
>>> R = QQ['x, y, z']; (x, y, z,) = R._first_ngens(3)
>>> S = QQ['u, v']; (u, v,) = S._first_ngens(2)
>>> f = R.hom([u**Integer(2), u*v, v**Integer(2)], S)
>>> I = S.ideal([u**Integer(6), u**Integer(5)*v, u**Integer(4)*v**Integer(2),␣
↪u**Integer(3)*v**Integer(3)])
>>> J = f.inverse_image(I); J                                                   #␣
↪needs sage.libs.singular
Ideal (y^2 - x*z, x*y*z, x^2*z, x^2*y, x^3)
of Multivariate Polynomial Ring in x, y, z over Rational Field
>>> f(J) == I                                                                   #␣
↪needs sage.libs.singular
True
```

Under the above homomorphism, there exists an inverse image for every element that only involves monomials of even degree:

```
sage: [f.inverse_image(p) for p in [u^2, u^4, u*v + u^3*v^3]]                   #␣
↪needs sage.libs.singular
[x, x^2, x*y*z + y]
sage: f.inverse_image(u*v^2)                                                    #␣
↪needs sage.libs.singular
Traceback (most recent call last):
...
ValueError: element u*v^2 does not have preimage
```

```
>>> from sage.all import *
>>> [f.inverse_image(p) for p in [u**Integer(2), u**Integer(4), u*v +␣
↪u**Integer(3)*v**Integer(3)]]                # needs sage.libs.singular
[x, x^2, x*y*z + y]
>>> f.inverse_image(u*v**Integer(2))                                            ␣
↪      # needs sage.libs.singular
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
ValueError: element u*v^2 does not have preimage
```

The image of the inverse image ideal can be strictly smaller than the original ideal:

```
sage: S.<u,v> = QQ['u,v'].quotient('v^2 - 2')
sage: f = QuadraticField(2).hom([v], S)
sage: I = S.ideal(u + v)
sage: J = f.inverse_image(I)
sage: J.is_zero()
True
sage: f(J) < I
True
```

```
>>> from sage.all import *
>>> S = QQ['u,v'].quotient('v^2 - 2', names=('u', 'v',)); (u, v,) = S._first_
↪ngens(2)
>>> f = QuadraticField(Integer(2)).hom([v], S)
>>> I = S.ideal(u + v)
>>> J = f.inverse_image(I)
>>> J.is_zero()
True
>>> f(J) < I
True
```

Fractional ideals are not yet fully supported:

```
sage: K.<a> = NumberField(QQ['x']('x^2+2'))
sage: f = K.hom([-a], K)
sage: I = K.ideal([a + 1])
sage: f.inverse_image(I)                                                    #␣
↪needs sage.libs.singular
Traceback (most recent call last):
...
NotImplementedError: inverse image not implemented...
sage: f.inverse_image(K.ideal(0)).is_zero()                                 #␣
↪needs sage.libs.singular
True
sage: f.inverse()(I)                                                        #␣
↪needs sage.rings.padics
Fractional ideal (-a + 1)
```

```
>>> from sage.all import *
>>> K = NumberField(QQ['x']('x^2+2'), names=('a',)); (a,) = K._first_ngens(1)
>>> f = K.hom([-a], K)
>>> I = K.ideal([a + Integer(1)])
>>> f.inverse_image(I)                                                      #␣
↪needs sage.libs.singular
Traceback (most recent call last):
...
NotImplementedError: inverse image not implemented...
```

(continues on next page)

(continued from previous page)

```
>>> f.inverse_image(K.ideal(Integer(0))).is_zero()                              ␣
↪     # needs sage.libs.singular
True
>>> f.inverse()(I)                                                             #␣
↪needs sage.rings.padics
Fractional ideal (-a + 1)
```

ALGORITHM:

By default, this computes a Gröbner basis of an ideal related to the graph of the ring homomorphism.

REFERENCES:

- Proposition 2.5.12 [DS2009]

is_invertible()

Return whether this ring homomorphism is bijective.

EXAMPLES:

```
sage: R.<x,y,z> = QQ[]
sage: R.hom([y*z, x*z, x*y], R).is_invertible()                               #␣
↪needs sage.libs.singular
False
sage: Q.<x,y,z> = R.quotient(x*y*z - 1)                                      #␣
↪needs sage.libs.singular
sage: Q.hom([y*z, x*z, x*y], Q).is_invertible()                              #␣
↪needs sage.libs.singular
True
```

```
>>> from sage.all import *
>>> R = QQ['x, y, z']; (x, y, z,) = R._first_ngens(3)
>>> R.hom([y*z, x*z, x*y], R).is_invertible()                                #␣
↪needs sage.libs.singular
False
>>> Q = R.quotient(x*y*z - Integer(1), names=('x', 'y', 'z',)); (x, y, z,) =␣
↪Q._first_ngens(3)# needs sage.libs.singular
>>> Q.hom([y*z, x*z, x*y], Q).is_invertible()                                #␣
↪needs sage.libs.singular
True
```

ALGORITHM:

By default, this requires the computation of a Gröbner basis.

is_surjective()

Return whether this ring homomorphism is surjective.

EXAMPLES:

```
sage: R.<x,y,z> = QQ[]
sage: R.hom([y*z, x*z, x*y], R).is_surjective()                               #␣
↪needs sage.libs.singular
False
sage: Q.<x,y,z> = R.quotient(x*y*z - 1)                                      #␣
↪needs sage.libs.singular
```

(continues on next page)

(continued from previous page)

```
sage: R.hom([y*z, x*z, x*y], Q).is_surjective()                          #␣
↪needs sage.libs.singular
True
```

```
>>> from sage.all import *
>>> R = QQ['x, y, z']; (x, y, z,) = R._first_ngens(3)
>>> R.hom([y*z, x*z, x*y], R).is_surjective()                           #␣
↪needs sage.libs.singular
False
>>> Q = R.quotient(x*y*z - Integer(1), names=('x', 'y', 'z',)); (x, y, z,) =␣
↪Q._first_ngens(3)# needs sage.libs.singular
>>> R.hom([y*z, x*z, x*y], Q).is_surjective()                           #␣
↪needs sage.libs.singular
True
```

ALGORITHM:

By default, this requires the computation of a Gröbner basis.

kernel()

Return the kernel ideal of this ring homomorphism.

EXAMPLES:

```
sage: A.<x,y> = QQ[]
sage: B.<t> = QQ[]
sage: f = A.hom([t^4, t^3 - t^2], B)
sage: f.kernel()                                                        #␣
↪needs sage.libs.singular
Ideal (y^4 - x^3 + 4*x^2*y - 2*x*y^2 + x^2)
of Multivariate Polynomial Ring in x, y over Rational Field
```

```
>>> from sage.all import *
>>> A = QQ['x, y']; (x, y,) = A._first_ngens(2)
>>> B = QQ['t']; (t,) = B._first_ngens(1)
>>> f = A.hom([t**Integer(4), t**Integer(3) - t**Integer(2)], B)
>>> f.kernel()                                                          #␣
↪needs sage.libs.singular
Ideal (y^4 - x^3 + 4*x^2*y - 2*x*y^2 + x^2)
of Multivariate Polynomial Ring in x, y over Rational Field
```

We express a Veronese subring of a polynomial ring as a quotient ring:

```
sage: A.<a,b,c,d> = QQ[]
sage: B.<u,v> = QQ[]
sage: f = A.hom([u^3, u^2*v, u*v^2, v^3], B)
sage: f.kernel() == A.ideal(matrix.hankel([a, b, c], [d]).minors(2))     #␣
↪needs sage.libs.singular
True
sage: Q = A.quotient(f.kernel())                                        #␣
↪needs sage.libs.singular
sage: Q.hom(f.im_gens(), B).is_injective()                              #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.libs.singular
True
```

```
>>> from sage.all import *
>>> A = QQ['a, b, c, d']; (a, b, c, d,) = A._first_ngens(4)
>>> B = QQ['u, v']; (u, v,) = B._first_ngens(2)
>>> f = A.hom([u**Integer(3), u**Integer(2)*v, u*v**Integer(2),␣
↪v**Integer(3)], B)
>>> f.kernel() == A.ideal(matrix.hankel([a, b, c], [d]).minors(Integer(2)))  ␣
↪     # needs sage.libs.singular
True
>>> Q = A.quotient(f.kernel())                                                #␣
↪needs sage.libs.singular
>>> Q.hom(f.im_gens(), B).is_injective()                                      #␣
↪needs sage.libs.singular
True
```

The Steiner-Roman surface:

```
sage: R.<x,y,z> = QQ[]
sage: S = R.quotient(x^2 + y^2 + z^2 - 1)
sage: f = R.hom([x*y, x*z, y*z], S)                                          #␣
↪needs sage.libs.singular
sage: f.kernel()                                                             #␣
↪needs sage.libs.singular
Ideal (x^2*y^2 + x^2*z^2 + y^2*z^2 - x*y*z)
 of Multivariate Polynomial Ring in x, y, z over Rational Field
```

```
>>> from sage.all import *
>>> R = QQ['x, y, z']; (x, y, z,) = R._first_ngens(3)
>>> S = R.quotient(x**Integer(2) + y**Integer(2) + z**Integer(2) - Integer(1))
>>> f = R.hom([x*y, x*z, y*z], S)                                            #␣
↪needs sage.libs.singular
>>> f.kernel()                                                               #␣
↪needs sage.libs.singular
Ideal (x^2*y^2 + x^2*z^2 + y^2*z^2 - x*y*z)
 of Multivariate Polynomial Ring in x, y, z over Rational Field
```

lift (*x=None*)

Return a lifting map associated to this homomorphism, if it has been defined.

If `x` is not `None`, return the value of the lift morphism on `x`.

EXAMPLES:

```
sage: R.<x,y> = QQ[]
sage: f = R.hom([x,x])
sage: f(x+y)
2*x
sage: f.lift()
Traceback (most recent call last):
...
ValueError: no lift map defined
```

(continues on next page)

(continued from previous page)

```
sage: g = R.hom(R)
sage: f._set_lift(g)
sage: f.lift() == g
True
sage: f.lift(x)
x
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> f = R.hom([x,x])
>>> f(x+y)
2*x
>>> f.lift()
Traceback (most recent call last):
...
ValueError: no lift map defined
>>> g = R.hom(R)
>>> f._set_lift(g)
>>> f.lift() == g
True
>>> f.lift(x)
x
```

pushforward(*I*)

Return the pushforward of the ideal I under this ring homomorphism.

EXAMPLES:

```
sage: R.<x,y> = QQ[]; S.<xx,yy> = R.quo([x^2, y^2]); f = S.cover()          #␣
↪needs sage.libs.singular
sage: f.pushforward(R.ideal([x, 3*x + x*y + y^2]))                          #␣
↪needs sage.libs.singular
Ideal (xx, xx*yy + 3*xx) of Quotient of Multivariate Polynomial Ring
 in x, y over Rational Field by the ideal (x^2, y^2)
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2); S = R.quo([x**Integer(2),␣
↪y**Integer(2)], names=('xx', 'yy',)); (xx, yy,) = S._first_ngens(2); f = S.
↪cover()          # needs sage.libs.singular
>>> f.pushforward(R.ideal([x, Integer(3)*x + x*y + y**Integer(2)]))          ␣
↪               # needs sage.libs.singular
Ideal (xx, xx*yy + 3*xx) of Quotient of Multivariate Polynomial Ring
 in x, y over Rational Field by the ideal (x^2, y^2)
```

class sage.rings.morphism.**RingHomomorphism_cover**

Bases: *RingHomomorphism*

A homomorphism induced by quotienting a ring out by an ideal.

EXAMPLES:

```
sage: R.<x,y> = PolynomialRing(QQ, 2)
sage: S.<a,b> = R.quo(x^2 + y^2)                                            #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.libs.singular
sage: phi = S.cover(); phi                                                  #␣
↪needs sage.libs.singular
Ring morphism:
  From: Multivariate Polynomial Ring in x, y over Rational Field
  To:   Quotient of Multivariate Polynomial Ring in x, y over Rational Field
        by the ideal (x^2 + y^2)
  Defn: Natural quotient map
sage: phi(x + y)                                                            #␣
↪needs sage.libs.singular
a + b
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, Integer(2), names=('x', 'y',)); (x, y,) = R._first_
↪ngens(2)
>>> S = R.quo(x**Integer(2) + y**Integer(2), names=('a', 'b',)); (a, b,) = S._
↪first_ngens(2)# needs sage.libs.singular
>>> phi = S.cover(); phi                                                    #␣
↪needs sage.libs.singular
Ring morphism:
  From: Multivariate Polynomial Ring in x, y over Rational Field
  To:   Quotient of Multivariate Polynomial Ring in x, y over Rational Field
        by the ideal (x^2 + y^2)
  Defn: Natural quotient map
>>> phi(x + y)                                                              #␣
↪needs sage.libs.singular
a + b
```

kernel()

Return the kernel of this covering morphism, which is the ideal that was quotiented out by.

EXAMPLES:

```
sage: f = Zmod(6).cover()
sage: f.kernel()
Principal ideal (6) of Integer Ring
```

```
>>> from sage.all import *
>>> f = Zmod(Integer(6)).cover()
>>> f.kernel()
Principal ideal (6) of Integer Ring
```

class sage.rings.morphism.**RingHomomorphism_from_base**

Bases: *RingHomomorphism*

A ring homomorphism determined by a ring homomorphism of the base ring.

AUTHOR:

- Simon King (initial version, 2010-04-30)

EXAMPLES:

We define two polynomial rings and a ring homomorphism:

```
sage: R.<x,y> = QQ[]
sage: S.<z> = QQ[]
sage: f = R.hom([2*z,3*z],S)
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> S = QQ['z']; (z,) = S._first_ngens(1)
>>> f = R.hom([Integer(2)*z,Integer(3)*z],S)
```

Now we construct polynomial rings based on `R` and `S`, and let `f` act on the coefficients:

```
sage: PR.<t> = R[]
sage: PS = S['t']
sage: Pf = PR.hom(f,PS)
sage: Pf
Ring morphism:
  From: Univariate Polynomial Ring in t
        over Multivariate Polynomial Ring in x, y over Rational Field
  To:   Univariate Polynomial Ring in t
        over Univariate Polynomial Ring in z over Rational Field
  Defn: Induced from base ring by
        Ring morphism:
          From: Multivariate Polynomial Ring in x, y over Rational Field
          To:   Univariate Polynomial Ring in z over Rational Field
          Defn: x |--> 2*z
                y |--> 3*z
sage: p = (x - 4*y + 1/13)*t^2 + (1/2*x^2 - 1/3*y^2)*t + 2*y^2 + x
sage: Pf(p)
(-10*z + 1/13)*t^2 - z^2*t + 18*z^2 + 2*z
```

```
>>> from sage.all import *
>>> PR = R['t']; (t,) = PR._first_ngens(1)
>>> PS = S['t']
>>> Pf = PR.hom(f,PS)
>>> Pf
Ring morphism:
  From: Univariate Polynomial Ring in t
        over Multivariate Polynomial Ring in x, y over Rational Field
  To:   Univariate Polynomial Ring in t
        over Univariate Polynomial Ring in z over Rational Field
  Defn: Induced from base ring by
        Ring morphism:
          From: Multivariate Polynomial Ring in x, y over Rational Field
          To:   Univariate Polynomial Ring in z over Rational Field
          Defn: x |--> 2*z
                y |--> 3*z
>>> p = (x - Integer(4)*y + Integer(1)/Integer(13))*t**Integer(2) + (Integer(1)/
↪Integer(2)*x**Integer(2) - Integer(1)/Integer(3)*y**Integer(2))*t +␣
↪Integer(2)*y**Integer(2) + x
>>> Pf(p)
(-10*z + 1/13)*t^2 - z^2*t + 18*z^2 + 2*z
```

Similarly, we can construct the induced homomorphism on a matrix ring over our polynomial rings:

```
sage: MR = MatrixSpace(R, 2, 2)
sage: MS = MatrixSpace(S, 2, 2)
sage: M = MR([x^2 + 1/7*x*y - y^2, -1/2*y^2 + 2*y + 1/6,
....:         4*x^2 - 14*x, 1/2*y^2 + 13/4*x - 2/11*y])
sage: Mf = MR.hom(f, MS)
sage: Mf
Ring morphism:
  From: Full MatrixSpace of 2 by 2 dense matrices
        over Multivariate Polynomial Ring in x, y over Rational Field
  To:   Full MatrixSpace of 2 by 2 dense matrices
        over Univariate Polynomial Ring in z over Rational Field
  Defn: Induced from base ring by
        Ring morphism:
          From: Multivariate Polynomial Ring in x, y over Rational Field
          To:   Univariate Polynomial Ring in z over Rational Field
          Defn: x |--> 2*z
                y |--> 3*z
sage: Mf(M)
[           -29/7*z^2 -9/2*z^2 + 6*z + 1/6]
[       16*z^2 - 28*z   9/2*z^2 + 131/22*z]
```

```
>>> from sage.all import *
>>> MR = MatrixSpace(R, Integer(2), Integer(2))
>>> MS = MatrixSpace(S, Integer(2), Integer(2))
>>> M = MR([x**Integer(2) + Integer(1)/Integer(7)*x*y - y**Integer(2), -
↪Integer(1)/Integer(2)*y**Integer(2) + Integer(2)*y + Integer(1)/Integer(6),
...          Integer(4)*x**Integer(2) - Integer(14)*x, Integer(1)/
↪Integer(2)*y**Integer(2) + Integer(13)/Integer(4)*x - Integer(2)/Integer(11)*y])
>>> Mf = MR.hom(f, MS)
>>> Mf
Ring morphism:
  From: Full MatrixSpace of 2 by 2 dense matrices
        over Multivariate Polynomial Ring in x, y over Rational Field
  To:   Full MatrixSpace of 2 by 2 dense matrices
        over Univariate Polynomial Ring in z over Rational Field
  Defn: Induced from base ring by
        Ring morphism:
          From: Multivariate Polynomial Ring in x, y over Rational Field
          To:   Univariate Polynomial Ring in z over Rational Field
          Defn: x |--> 2*z
                y |--> 3*z
>>> Mf(M)
[           -29/7*z^2 -9/2*z^2 + 6*z + 1/6]
[       16*z^2 - 28*z   9/2*z^2 + 131/22*z]
```

The construction of induced homomorphisms is recursive, and so we have:

```
sage: MPR = MatrixSpace(PR, 2)
sage: MPS = MatrixSpace(PS, 2)
sage: M = MPR([(-x + y)*t^2 + 58*t - 3*x^2 + x*y,
....:          (- 1/7*x*y - 1/40*x)*t^2 + (5*x^2 + y^2)*t + 2*y,
....:          (- 1/3*y + 1)*t^2 + 1/3*x*y + y^2 + 5/2*y + 1/4,
....:          (x + 6*y + 1)*t^2])
```

(continues on next page)

(continued from previous page)

```
sage: MPf = MPR.hom(f, MPS); MPf
Ring morphism:
  From: Full MatrixSpace of 2 by 2 dense matrices over Univariate Polynomial
        Ring in t over Multivariate Polynomial Ring in x, y over Rational Field
  To:   Full MatrixSpace of 2 by 2 dense matrices over Univariate Polynomial
        Ring in t over Univariate Polynomial Ring in z over Rational Field
  Defn: Induced from base ring by
        Ring morphism:
          From: Univariate Polynomial Ring in t
                over Multivariate Polynomial Ring in x, y over Rational Field
          To:   Univariate Polynomial Ring in t
                over Univariate Polynomial Ring in z over Rational Field
          Defn: Induced from base ring by
                Ring morphism:
                  From: Multivariate Polynomial Ring in x, y over Rational Field
                  To:   Univariate Polynomial Ring in z over Rational Field
                  Defn: x |--> 2*z
                        y |--> 3*z
sage: MPf(M)
[                    z*t^2 + 58*t - 6*z^2 (-6/7*z^2 - 1/20*z)*t^2 + 29*z^2*t +␣
↪6*z]
[     (-z + 1)*t^2 + 11*z^2 + 15/2*z + 1/4                          (20*z + 1)*t^
↪2]
```

```
>>> from sage.all import *
>>> MPR = MatrixSpace(PR, Integer(2))
>>> MPS = MatrixSpace(PS, Integer(2))
>>> M = MPR([(-x + y)*t**Integer(2) + Integer(58)*t - Integer(3)*x**Integer(2) +␣
↪x*y,
...           (- Integer(1)/Integer(7)*x*y - Integer(1)/
↪Integer(40)*x)*t**Integer(2) + (Integer(5)*x**Integer(2) + y**Integer(2))*t +␣
↪Integer(2)*y,
...           (- Integer(1)/Integer(3)*y + Integer(1))*t**Integer(2) + Integer(1)/
↪Integer(3)*x*y + y**Integer(2) + Integer(5)/Integer(2)*y + Integer(1)/
↪Integer(4),
...           (x + Integer(6)*y + Integer(1))*t**Integer(2)])
>>> MPf = MPR.hom(f, MPS); MPf
Ring morphism:
  From: Full MatrixSpace of 2 by 2 dense matrices over Univariate Polynomial
        Ring in t over Multivariate Polynomial Ring in x, y over Rational Field
  To:   Full MatrixSpace of 2 by 2 dense matrices over Univariate Polynomial
        Ring in t over Univariate Polynomial Ring in z over Rational Field
  Defn: Induced from base ring by
        Ring morphism:
          From: Univariate Polynomial Ring in t
                over Multivariate Polynomial Ring in x, y over Rational Field
          To:   Univariate Polynomial Ring in t
                over Univariate Polynomial Ring in z over Rational Field
          Defn: Induced from base ring by
                Ring morphism:
                  From: Multivariate Polynomial Ring in x, y over Rational Field
                  To:   Univariate Polynomial Ring in z over Rational Field
```

(continues on next page)

(continued from previous page)

```
                  Defn: x |--> 2*z
                        y |--> 3*z
>>> MPf(M)
[                    z*t^2 + 58*t - 6*z^2 (-6/7*z^2 - 1/20*z)*t^2 + 29*z^2*t +␣
↪6*z]
[    (-z + 1)*t^2 + 11*z^2 + 15/2*z + 1/4                          (20*z + 1)*t^
↪2]
```

inverse()

Return the inverse of this ring homomorphism if the underlying homomorphism of the base ring is invertible.

EXAMPLES:

```
sage: R.<x,y> = QQ[]
sage: S.<a,b> = QQ[]
sage: f = R.hom([a + b, a - b], S)
sage: PR.<t> = R[]
sage: PS = S['t']
sage: Pf = PR.hom(f, PS)
sage: Pf.inverse()                                                          #␣
↪needs sage.libs.singular
Ring morphism:
  From: Univariate Polynomial Ring in t over Multivariate
        Polynomial Ring in a, b over Rational Field
  To:   Univariate Polynomial Ring in t over Multivariate
        Polynomial Ring in x, y over Rational Field
  Defn: Induced from base ring by
        Ring morphism:
          From: Multivariate Polynomial Ring in a, b over Rational Field
          To:   Multivariate Polynomial Ring in x, y over Rational Field
          Defn: a |--> 1/2*x + 1/2*y
                b |--> 1/2*x - 1/2*y
sage: Pf.inverse()(Pf(x*t^2 + y*t))                                         #␣
↪needs sage.libs.singular
x*t^2 + y*t
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> S = QQ['a, b']; (a, b,) = S._first_ngens(2)
>>> f = R.hom([a + b, a - b], S)
>>> PR = R['t']; (t,) = PR._first_ngens(1)
>>> PS = S['t']
>>> Pf = PR.hom(f, PS)
>>> Pf.inverse()                                                            #␣
↪needs sage.libs.singular
Ring morphism:
  From: Univariate Polynomial Ring in t over Multivariate
        Polynomial Ring in a, b over Rational Field
  To:   Univariate Polynomial Ring in t over Multivariate
        Polynomial Ring in x, y over Rational Field
  Defn: Induced from base ring by
        Ring morphism:
```

(continues on next page)

(continued from previous page)

```
          From: Multivariate Polynomial Ring in a, b over Rational Field
          To:   Multivariate Polynomial Ring in x, y over Rational Field
          Defn: a |--> 1/2*x + 1/2*y
                b |--> 1/2*x - 1/2*y
>>> Pf.inverse()(Pf(x*t**Integer(2) + y*t))
 ↪      # needs sage.libs.singular
x*t^2 + y*t
```

underlying_map()

Return the underlying homomorphism of the base ring.

EXAMPLES:

```
sage: R.<x,y> = QQ[]
sage: S.<z> = QQ[]
sage: f = R.hom([2*z, 3*z], S)
sage: MR = MatrixSpace(R, 2)
sage: MS = MatrixSpace(S, 2)
sage: g = MR.hom(f, MS)
sage: g.underlying_map() == f
True
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> S = QQ['z']; (z,) = S._first_ngens(1)
>>> f = R.hom([Integer(2)*z, Integer(3)*z], S)
>>> MR = MatrixSpace(R, Integer(2))
>>> MS = MatrixSpace(S, Integer(2))
>>> g = MR.hom(f, MS)
>>> g.underlying_map() == f
True
```

class sage.rings.morphism.**RingHomomorphism_from_fraction_field**

Bases: *RingHomomorphism*

Morphisms between fraction fields.

inverse()

Return the inverse of this ring homomorphism if it exists.

EXAMPLES:

```
sage: S.<x> = QQ[]
sage: f = S.hom([2*x - 1])
sage: g = f.extend_to_fraction_field()                                  #
 ↪needs sage.libs.singular
sage: g.inverse()                                                       #
 ↪needs sage.libs.singular
Ring endomorphism of Fraction Field of Univariate Polynomial Ring
 in x over Rational Field
  Defn: x |--> 1/2*x + 1/2
```

```
>>> from sage.all import *
>>> S = QQ['x']; (x,) = S._first_ngens(1)
>>> f = S.hom([Integer(2)*x - Integer(1)])
>>> g = f.extend_to_fraction_field()                                    #␣
↪needs sage.libs.singular
>>> g.inverse()                                                         #␣
↪needs sage.libs.singular
Ring endomorphism of Fraction Field of Univariate Polynomial Ring
 in x over Rational Field
  Defn: x |--> 1/2*x + 1/2
```

class sage.rings.morphism.**RingHomomorphism_from_quotient**

Bases: *RingHomomorphism*

A ring homomorphism with domain a generic quotient ring.

INPUT:

- `parent` – a ring homset `Hom(R,S)`
- `phi` – a ring homomorphism `C --> S`, where `C` is the domain of `R.cover()`

OUTPUT: a ring homomorphism

The domain R is a quotient object $C \to R$, and `R.cover()` is the ring homomorphism $\varphi : C \to R$. The condition on the elements `im_gens` of S is that they define a homomorphism $C \to S$ such that each generator of the kernel of φ maps to 0.

EXAMPLES:

```
sage: R.<x, y, z> = PolynomialRing(QQ, 3)
sage: S.<a, b, c> = R.quo(x^3 + y^3 + z^3)
sage: phi = S.hom([b, c, a]); phi
Ring endomorphism of Quotient of Multivariate Polynomial Ring in x, y, z
 over Rational Field by the ideal (x^3 + y^3 + z^3)
  Defn: a |--> b
        b |--> c
        c |--> a
sage: phi(a + b + c)
a + b + c
sage: loads(dumps(phi)) == phi
True
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, Integer(3), names=('x', 'y', 'z',)); (x, y, z,) = R._
↪first_ngens(3)
>>> S = R.quo(x**Integer(3) + y**Integer(3) + z**Integer(3), names=('a', 'b', 'c',
↪)); (a, b, c,) = S._first_ngens(3)
>>> phi = S.hom([b, c, a]); phi
Ring endomorphism of Quotient of Multivariate Polynomial Ring in x, y, z
 over Rational Field by the ideal (x^3 + y^3 + z^3)
  Defn: a |--> b
        b |--> c
        c |--> a
>>> phi(a + b + c)
a + b + c
```

(continues on next page)

(continued from previous page)

```
>>> loads(dumps(phi)) == phi
True
```

Validity of the homomorphism is determined, when possible, and a `TypeError` is raised if there is no homomorphism sending the generators to the given images:

```
sage: S.hom([b^2, c^2, a^2])                                                #␣
↪needs sage.libs.singular
Traceback (most recent call last):
...
ValueError: relations do not all (canonically) map to 0
under map determined by images of generators
```

```
>>> from sage.all import *
>>> S.hom([b**Integer(2), c**Integer(2), a**Integer(2)])                    ␣
↪                          # needs sage.libs.singular
Traceback (most recent call last):
...
ValueError: relations do not all (canonically) map to 0
under map determined by images of generators
```

morphism_from_cover()

Underlying morphism used to define this quotient map, i.e., the morphism from the cover of the domain.

EXAMPLES:

```
sage: R.<x,y> = QQ[]; S.<xx,yy> = R.quo([x^2, y^2])                         #␣
↪needs sage.libs.singular
sage: S.hom([yy,xx]).morphism_from_cover()                                  #␣
↪needs sage.libs.singular
Ring morphism:
  From: Multivariate Polynomial Ring in x, y over Rational Field
  To:   Quotient of Multivariate Polynomial Ring in x, y
        over Rational Field by the ideal (x^2, y^2)
  Defn: x |--> yy
        y |--> xx
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2); S = R.quo([x**Integer(2),␣
↪y**Integer(2)], names=('xx', 'yy',)); (xx, yy,) = S._first_ngens(2)# needs␣
↪sage.libs.singular
>>> S.hom([yy,xx]).morphism_from_cover()                                    #␣
↪needs sage.libs.singular
Ring morphism:
  From: Multivariate Polynomial Ring in x, y over Rational Field
  To:   Quotient of Multivariate Polynomial Ring in x, y
        over Rational Field by the ideal (x^2, y^2)
  Defn: x |--> yy
        y |--> xx
```

class sage.rings.morphism.**RingHomomorphism_im_gens**

Bases: *RingHomomorphism*

A ring homomorphism determined by the images of generators.

base_map()

Return the map on the base ring that is part of the defining data for this morphism. May return `None` if a coercion is used.

EXAMPLES:

```
sage: R.<x> = ZZ[]
sage: K.<i> = NumberField(x^2 + 1)
sage: cc = K.hom([-i])
sage: S.<y> = K[]
sage: phi = S.hom([y^2], base_map=cc)
sage: phi
Ring endomorphism of Univariate Polynomial Ring in y
 over Number Field in i with defining polynomial x^2 + 1
  Defn: y |--> y^2
        with map of base ring
sage: phi(y)
y^2
sage: phi(i*y)
-i*y^2
sage: phi.base_map()
Composite map:
  From: Number Field in i with defining polynomial x^2 + 1
  To:   Univariate Polynomial Ring in y over Number Field in i
        with defining polynomial x^2 + 1
  Defn:   Ring endomorphism of Number Field in i with defining polynomial x^2␣
↪+ 1
          Defn: i |--> -i
        then
          Polynomial base injection morphism:
          From: Number Field in i with defining polynomial x^2 + 1
          To:   Univariate Polynomial Ring in y over Number Field in i
                with defining polynomial x^2 + 1
```

```
>>> from sage.all import *
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> K = NumberField(x**Integer(2) + Integer(1), names=('i',)); (i,) = K._
↪first_ngens(1)
>>> cc = K.hom([-i])
>>> S = K['y']; (y,) = S._first_ngens(1)
>>> phi = S.hom([y**Integer(2)], base_map=cc)
>>> phi
Ring endomorphism of Univariate Polynomial Ring in y
 over Number Field in i with defining polynomial x^2 + 1
  Defn: y |--> y^2
        with map of base ring
>>> phi(y)
y^2
>>> phi(i*y)
-i*y^2
>>> phi.base_map()
Composite map:
```

(continues on next page)

(continued from previous page)

```
  From: Number Field in i with defining polynomial x^2 + 1
  To:   Univariate Polynomial Ring in y over Number Field in i
        with defining polynomial x^2 + 1
  Defn:   Ring endomorphism of Number Field in i with defining polynomial x^2␣
↪+ 1
          Defn: i |--> -i
        then
          Polynomial base injection morphism:
          From: Number Field in i with defining polynomial x^2 + 1
          To:   Univariate Polynomial Ring in y over Number Field in i
                with defining polynomial x^2 + 1
```

im_gens()

Return the images of the generators of the domain.

OUTPUT:

- `list` – a copy of the list of gens (it is safe to change this)

EXAMPLES:

```
sage: R.<x,y> = QQ[]
sage: f = R.hom([x, x + y])
sage: f.im_gens()
[x, x + y]
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> f = R.hom([x, x + y])
>>> f.im_gens()
[x, x + y]
```

We verify that the returned list of images of gens is a copy, so changing it doesn't change `f`:

```
sage: f.im_gens()[0] = 5
sage: f.im_gens()
[x, x + y]
```

```
>>> from sage.all import *
>>> f.im_gens()[Integer(0)] = Integer(5)
>>> f.im_gens()
[x, x + y]
```

class sage.rings.morphism.**RingMap**

Bases: `Morphism`

Set-theoretic map between rings.

class sage.rings.morphism.**RingMap_lift**

Bases: *RingMap*

Given rings R and S such that for any $x \in R$ the function `x.lift()` is an element that naturally coerces to S, this returns the set-theoretic ring map $R \to S$ sending x to `x.lift()`.

EXAMPLES:

```
sage: R.<x,y> = QQ[]
sage: S.<xbar,ybar> = R.quo( (x^2 + y^2, y) )                                  #␣
↪needs sage.libs.singular
sage: S.lift()                                                                 #␣
↪needs sage.libs.singular
Set-theoretic ring morphism:
  From: Quotient of Multivariate Polynomial Ring in x, y
        over Rational Field by the ideal (x^2 + y^2, y)
  To:   Multivariate Polynomial Ring in x, y over Rational Field
  Defn: Choice of lifting map
sage: S.lift() == 0                                                            #␣
↪needs sage.libs.singular
False
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> S = R.quo( (x**Integer(2) + y**Integer(2), y) , names=('xbar', 'ybar',));␣
↪(xbar, ybar,) = S._first_ngens(2)# needs sage.libs.singular
>>> S.lift()                                                                   #␣
↪needs sage.libs.singular
Set-theoretic ring morphism:
  From: Quotient of Multivariate Polynomial Ring in x, y
        over Rational Field by the ideal (x^2 + y^2, y)
  To:   Multivariate Polynomial Ring in x, y over Rational Field
  Defn: Choice of lifting map
>>> S.lift() == Integer(0)                                                      ␣
↪     # needs sage.libs.singular
False
```

Since Issue #11068, it is possible to create quotient rings of non-commutative rings by two-sided ideals. It was needed to modify *RingMap_lift* so that rings can be accepted that are no instances of *sage.rings.ring.Ring*, as in the following example:

```
sage: MS = MatrixSpace(GF(5), 2, 2)
sage: I = MS * [MS.0*MS.1, MS.2+MS.3] * MS
sage: Q = MS.quo(I)
sage: Q.0*Q.1   # indirect doctest
[0 1]
[0 0]
```

```
>>> from sage.all import *
>>> MS = MatrixSpace(GF(Integer(5)), Integer(2), Integer(2))
>>> I = MS * [MS.gen(0)*MS.gen(1), MS.gen(2)+MS.gen(3)] * MS
>>> Q = MS.quo(I)
>>> Q.gen(0)*Q.gen(1)   # indirect doctest
[0 1]
[0 0]
```

3.2 Space of homomorphisms between two rings

sage.rings.homset.**RingHomset**(*R*, *S*, *category=None*)

Construct a space of homomorphisms between the rings `R` and `S`.

For more on homsets, see `Hom()`.

EXAMPLES:

```
sage: Hom(ZZ, QQ) # indirect doctest
Set of Homomorphisms from Integer Ring to Rational Field
```

```
>>> from sage.all import *
>>> Hom(ZZ, QQ) # indirect doctest
Set of Homomorphisms from Integer Ring to Rational Field
```

class sage.rings.homset.**RingHomset_generic**(*R*, *S*, *category=None*)

Bases: `HomsetWithBase`

A generic space of homomorphisms between two rings.

EXAMPLES:

```
sage: Hom(ZZ, QQ)
Set of Homomorphisms from Integer Ring to Rational Field
sage: QQ.Hom(ZZ)
Set of Homomorphisms from Rational Field to Integer Ring
```

```
>>> from sage.all import *
>>> Hom(ZZ, QQ)
Set of Homomorphisms from Integer Ring to Rational Field
>>> QQ.Hom(ZZ)
Set of Homomorphisms from Rational Field to Integer Ring
```

Element

alias of *RingHomomorphism*

has_coerce_map_from(*x*)

The default for coercion maps between ring homomorphism spaces is very restrictive (until more implementation work is done).

Currently this checks if the domains and the codomains are equal.

EXAMPLES:

```
sage: H = Hom(ZZ, QQ)
sage: H2 = Hom(QQ, ZZ)
sage: H.has_coerce_map_from(H2)
False
```

```
>>> from sage.all import *
>>> H = Hom(ZZ, QQ)
>>> H2 = Hom(QQ, ZZ)
>>> H.has_coerce_map_from(H2)
False
```

natural_map()

Return the natural map from the domain to the codomain.

The natural map is the coercion map from the domain ring to the codomain ring.

EXAMPLES:

```
sage: H = Hom(ZZ, QQ)
sage: H.natural_map()
Natural morphism:
  From: Integer Ring
  To:   Rational Field
```

```
>>> from sage.all import *
>>> H = Hom(ZZ, QQ)
>>> H.natural_map()
Natural morphism:
  From: Integer Ring
  To:   Rational Field
```

zero()

Return the zero element of this homset.

EXAMPLES:

Since a ring homomorphism maps 1 to 1, there can only be a zero morphism when mapping to the trivial ring:

```
sage: Hom(ZZ, Zmod(1)).zero()
Ring morphism:
  From: Integer Ring
  To:   Ring of integers modulo 1
  Defn: 1 |--> 0
sage: Hom(ZZ, Zmod(2)).zero()
Traceback (most recent call last):
...
ValueError: homset has no zero element
```

```
>>> from sage.all import *
>>> Hom(ZZ, Zmod(Integer(1))).zero()
Ring morphism:
  From: Integer Ring
  To:   Ring of integers modulo 1
  Defn: 1 |--> 0
>>> Hom(ZZ, Zmod(Integer(2))).zero()
Traceback (most recent call last):
...
ValueError: homset has no zero element
```

class sage.rings.homset.**RingHomset_quo_ring**(*R*, *S*, *category=None*)

Bases: *RingHomset_generic*

Space of ring homomorphisms where the domain is a (formal) quotient ring.

EXAMPLES:

```
sage: R.<x,y> = PolynomialRing(QQ, 2)
sage: S.<a,b> = R.quotient(x^2 + y^2)                                          #␣
↪needs sage.libs.singular
sage: phi = S.hom([b,a]); phi                                                  #␣
↪needs sage.libs.singular
Ring endomorphism of Quotient of Multivariate Polynomial Ring in x, y
 over Rational Field by the ideal (x^2 + y^2)
  Defn: a |--> b
        b |--> a
sage: phi(a)                                                                   #␣
↪needs sage.libs.singular
b
sage: phi(b)                                                                   #␣
↪needs sage.libs.singular
a
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, Integer(2), names=('x', 'y',)); (x, y,) = R._first_
↪ngens(2)
>>> S = R.quotient(x**Integer(2) + y**Integer(2), names=('a', 'b',)); (a, b,) = S.
↪_first_ngens(2)# needs sage.libs.singular
>>> phi = S.hom([b,a]); phi                                                    #␣
↪needs sage.libs.singular
Ring endomorphism of Quotient of Multivariate Polynomial Ring in x, y
 over Rational Field by the ideal (x^2 + y^2)
  Defn: a |--> b
        b |--> a
>>> phi(a)                                                                     #␣
↪needs sage.libs.singular
b
>>> phi(b)                                                                     #␣
↪needs sage.libs.singular
a
```

Element

alias of *RingHomomorphism_from_quotient*

CHAPTER

FOUR

QUOTIENT RINGS

4.1 Quotient Rings

AUTHORS:

- William Stein
- Simon King (2011-04): Put it into the category framework, use the new coercion model.
- Simon King (2011-04): Quotients of non-commutative rings by twosided ideals.

Todo

The following skipped tests should be removed once Issue #13999 is fixed:

```
sage: TestSuite(S).run(skip=['_test_nonzero_equal', '_test_elements', '_test_zero'])
```

```
>>> from sage.all import *
>>> TestSuite(S).run(skip=['_test_nonzero_equal', '_test_elements', '_test_zero'])
```

In Issue #11068, non-commutative quotient rings R/I were implemented. The only requirement is that the two-sided ideal I provides a `reduce` method so that `I.reduce(x)` is the normal form of an element x with respect to I (i.e., we have `I.reduce(x) == I.reduce(y)` if $x-y \in I$, and `x - I.reduce(x) in I`). Here is a toy example:

```
sage: from sage.rings.noncommutative_ideals import Ideal_nc
sage: from itertools import product
sage: class PowerIdeal(Ideal_nc):
....:     def __init__(self, R, n):
....:         self._power = n
....:         self._power = n
....:         Ideal_nc.__init__(self, R, [R.prod(m) for m in product(R.gens(),␣
↪repeat=n)])
....:     def reduce(self, x):
....:         R = self.ring()
....:         return add([c*R(m) for m,c in x if len(m)<self._power],R(0))
sage: F.<x,y,z> = FreeAlgebra(QQ, 3)
sage: I3 = PowerIdeal(F,3); I3
Twosided Ideal (x^3, x^2*y, x^2*z, x*y*x, x*y^2, x*y*z, x*z*x, x*z*y,
x*z^2, y*x^2, y*x*y, y*x*z, y^2*x, y^3, y^2*z, y*z*x, y*z*y, y*z^2,
z*x^2, z*x*y, z*x*z, z*y*x, z*y^2, z*y*z, z^2*x, z^2*y, z^3) of
Free Algebra on 3 generators (x, y, z) over Rational Field
```

```
>>> from sage.all import *
>>> from sage.rings.noncommutative_ideals import Ideal_nc
>>> from itertools import product
>>> class PowerIdeal(Ideal_nc):
...     def __init__(self, R, n):
...         self._power = n
...         self._power = n
...         Ideal_nc.__init__(self, R, [R.prod(m) for m in product(R.gens(),␣
↪repeat=n)])
...     def reduce(self, x):
...         R = self.ring()
...         return add([c*R(m) for m,c in x if len(m)<self._power],R(Integer(0)))
>>> F = FreeAlgebra(QQ, Integer(3), names=('x', 'y', 'z',)); (x, y, z,) = F._first_
↪ngens(3)
>>> I3 = PowerIdeal(F,Integer(3)); I3
Twosided Ideal (x^3, x^2*y, x^2*z, x*y*x, x*y^2, x*y*z, x*z*x, x*z*y,
x*z^2, y*x^2, y*x*y, y*x*z, y^2*x, y^3, y^2*z, y*z*x, y*z*y, y*z^2,
z*x^2, z*x*y, z*x*z, z*y*x, z*y^2, z*y*z, z^2*x, z^2*y, z^3) of
Free Algebra on 3 generators (x, y, z) over Rational Field
```

Free algebras have a custom quotient method that serves at creating finite dimensional quotients defined by multiplication matrices. We are bypassing it, so that we obtain the default quotient:

```
sage: Q3.<a,b,c> = F.quotient(I3)
sage: Q3
Quotient of Free Algebra on 3 generators (x, y, z) over Rational Field by
the ideal (x^3, x^2*y, x^2*z, x*y*x, x*y^2, x*y*z, x*z*x, x*z*y, x*z^2,
y*x^2, y*x*y, y*x*z, y^2*x, y^3, y^2*z, y*z*x, y*z*y, y*z^2, z*x^2, z*x*y,
z*x*z, z*y*x, z*y^2, z*y*z, z^2*x, z^2*y, z^3)
sage: (a+b+2)^4
16 + 32*a + 32*b + 24*a^2 + 24*a*b + 24*b*a + 24*b^2
sage: Q3.is_commutative()
False
```

```
>>> from sage.all import *
>>> Q3 = F.quotient(I3, names=('a', 'b', 'c',)); (a, b, c,) = Q3._first_ngens(3)
>>> Q3
Quotient of Free Algebra on 3 generators (x, y, z) over Rational Field by
the ideal (x^3, x^2*y, x^2*z, x*y*x, x*y^2, x*y*z, x*z*x, x*z*y, x*z^2,
y*x^2, y*x*y, y*x*z, y^2*x, y^3, y^2*z, y*z*x, y*z*y, y*z^2, z*x^2, z*x*y,
z*x*z, z*y*x, z*y^2, z*y*z, z^2*x, z^2*y, z^3)
>>> (a+b+Integer(2))**Integer(4)
16 + 32*a + 32*b + 24*a^2 + 24*a*b + 24*b*a + 24*b^2
>>> Q3.is_commutative()
False
```

Even though Q_3 is not commutative, there is commutativity for products of degree three:

```
sage: a*(b*c)-(b*c)*a==F.zero()
True
```

```
>>> from sage.all import *
>>> a*(b*c)-(b*c)*a==F.zero()
```

(continues on next page)

(continued from previous page)

```
True
```

If we quotient out all terms of degree two then of course the resulting quotient ring is commutative:

```
sage: I2 = PowerIdeal(F,2); I2
Twosided Ideal (x^2, x*y, x*z, y*x, y^2, y*z, z*x, z*y, z^2) of Free Algebra
on 3 generators (x, y, z) over Rational Field
sage: Q2.<a,b,c> = F.quotient(I2)
sage: Q2.is_commutative()
True
sage: (a+b+2)^4
16 + 32*a + 32*b
```

```
>>> from sage.all import *
>>> I2 = PowerIdeal(F,Integer(2)); I2
Twosided Ideal (x^2, x*y, x*z, y*x, y^2, y*z, z*x, z*y, z^2) of Free Algebra
on 3 generators (x, y, z) over Rational Field
>>> Q2 = F.quotient(I2, names=('a', 'b', 'c',)); (a, b, c,) = Q2._first_ngens(3)
>>> Q2.is_commutative()
True
>>> (a+b+Integer(2))**Integer(4)
16 + 32*a + 32*b
```

Since Issue #7797, there is an implementation of free algebras based on Singular's implementation of the Letterplace Algebra. Our letterplace wrapper allows to provide the above toy example more easily:

```
sage: from itertools import product
sage: F.<x,y,z> = FreeAlgebra(QQ, implementation='letterplace')
sage: Q3 = F.quo(F*[F.prod(m) for m in product(F.gens(), repeat=3)]*F)
sage: Q3
Quotient of Free Associative Unital Algebra on 3 generators (x, y, z)
 over Rational Field by the ideal (x*x*x, x*x*y, x*x*z, x*y*x, x*y*y, x*y*z,
  x*z*x, x*z*y, x*z*z, y*x*x, y*x*y, y*x*z, y*y*x, y*y*y, y*y*z, y*z*x, y*z*y,
  y*z*z, z*x*x, z*x*y, z*x*z, z*y*x, z*y*y, z*y*z, z*z*x, z*z*y, z*z*z)
sage: Q3.0*Q3.1 - Q3.1*Q3.0
xbar*ybar - ybar*xbar
sage: Q3.0*(Q3.1*Q3.2) - (Q3.1*Q3.2)*Q3.0
0
sage: Q2 = F.quo(F*[F.prod(m) for m in product(F.gens(), repeat=2)]*F)
sage: Q2.is_commutative()
True
```

```
>>> from sage.all import *
>>> from itertools import product
>>> F = FreeAlgebra(QQ, implementation='letterplace', names=('x', 'y', 'z',)); (x, y,␣
↪z,) = F._first_ngens(3)
>>> Q3 = F.quo(F*[F.prod(m) for m in product(F.gens(), repeat=Integer(3))]*F)
>>> Q3
Quotient of Free Associative Unital Algebra on 3 generators (x, y, z)
 over Rational Field by the ideal (x*x*x, x*x*y, x*x*z, x*y*x, x*y*y, x*y*z,
  x*z*x, x*z*y, x*z*z, y*x*x, y*x*y, y*x*z, y*y*x, y*y*y, y*y*z, y*z*x, y*z*y,
  y*z*z, z*x*x, z*x*y, z*x*z, z*y*x, z*y*y, z*y*z, z*z*x, z*z*y, z*z*z)
```

(continues on next page)

(continued from previous page)

```
>>> Q3.gen(0)*Q3.gen(1) - Q3.gen(1)*Q3.gen(0)
xbar*ybar - ybar*xbar
>>> Q3.gen(0)*(Q3.gen(1)*Q3.gen(2)) - (Q3.gen(1)*Q3.gen(2))*Q3.gen(0)
0
>>> Q2 = F.quo(F*[F.prod(m) for m in product(F.gens(), repeat=Integer(2))]*F)
>>> Q2.is_commutative()
True
```

sage.rings.quotient_ring.**QuotientRing**(*R*, *I*, *names=None*, ***kwds*)

Create a quotient ring of the ring R by the twosided ideal I.

Variables are labeled by `names` (if the quotient ring is a quotient of a polynomial ring). If `names` isn't given, 'bar' will be appended to the variable names in R.

INPUT:

- `R` – a ring
- `I` – a twosided ideal of R
- `names` – (optional) a list of strings to be used as names for the variables in the quotient ring R/I
- further named arguments that will be passed to the constructor of the quotient ring instance

OUTPUT: R/I - the quotient ring R mod the ideal I

ASSUMPTION:

`I` has a method `I.reduce(x)` returning the normal form of elements $x \in R$. In other words, it is required that `I.reduce(x)==I.reduce(y)` $\iff x - y \in I$, and `x-I.reduce(x) in I`, for all $x, y \in R$.

EXAMPLES:

Some simple quotient rings with the integers:

```
sage: R = QuotientRing(ZZ, 7*ZZ); R
Quotient of Integer Ring by the ideal (7)
sage: R.gens()
(1,)
sage: 1*R(3); 6*R(3); 7*R(3)
3
4
0
```

```
>>> from sage.all import *
>>> R = QuotientRing(ZZ, Integer(7)*ZZ); R
Quotient of Integer Ring by the ideal (7)
>>> R.gens()
(1,)
>>> Integer(1)*R(Integer(3)); Integer(6)*R(Integer(3)); Integer(7)*R(Integer(3))
3
4
0
```

```
sage: S = QuotientRing(ZZ,ZZ.ideal(8)); S
Quotient of Integer Ring by the ideal (8)
```

(continues on next page)

(continued from previous page)

```
sage: 2*S(4)
0
```

```
>>> from sage.all import *
>>> S = QuotientRing(ZZ,ZZ.ideal(Integer(8))); S
Quotient of Integer Ring by the ideal (8)
>>> Integer(2)*S(Integer(4))
0
```

With polynomial rings (note that the variable name of the quotient ring can be specified as shown below):

```
sage: P.<x> = QQ[]
sage: R.<xx> = QuotientRing(P, P.ideal(x^2 + 1))
sage: R
Univariate Quotient Polynomial Ring in xx over Rational Field
 with modulus x^2 + 1
sage: R.gens(); R.gen()
(xx,)
xx
sage: for n in range(4): xx^n
1
xx
-1
-xx
```

```
>>> from sage.all import *
>>> P = QQ['x']; (x,) = P._first_ngens(1)
>>> R = QuotientRing(P, P.ideal(x**Integer(2) + Integer(1)), names=('xx',)); (xx,
↪) = R._first_ngens(1)
>>> R
Univariate Quotient Polynomial Ring in xx over Rational Field
 with modulus x^2 + 1
>>> R.gens(); R.gen()
(xx,)
xx
>>> for n in range(Integer(4)): xx**n
1
xx
-1
-xx
```

```
sage: P.<x> = QQ[]
sage: S = QuotientRing(P, P.ideal(x^2 - 2))
sage: S
Univariate Quotient Polynomial Ring in xbar over Rational Field
 with modulus x^2 - 2
sage: xbar = S.gen(); S.gen()
xbar
sage: for n in range(3): xbar^n
1
xbar
2
```

```
>>> from sage.all import *
>>> P = QQ['x']; (x,) = P._first_ngens(1)
>>> S = QuotientRing(P, P.ideal(x**Integer(2) - Integer(2)))
>>> S
Univariate Quotient Polynomial Ring in xbar over Rational Field
 with modulus x^2 - 2
>>> xbar = S.gen(); S.gen()
xbar
>>> for n in range(Integer(3)): xbar**n
1
xbar
2
```

Sage coerces objects into ideals when possible:

```
sage: P.<x> = QQ[]
sage: R = QuotientRing(P, x^2 + 1); R
Univariate Quotient Polynomial Ring in xbar over Rational Field
 with modulus x^2 + 1
```

```
>>> from sage.all import *
>>> P = QQ['x']; (x,) = P._first_ngens(1)
>>> R = QuotientRing(P, x**Integer(2) + Integer(1)); R
Univariate Quotient Polynomial Ring in xbar over Rational Field
 with modulus x^2 + 1
```

By Noether's homomorphism theorems, the quotient of a quotient ring of R is just the quotient of R by the sum of the ideals. In this example, we end up modding out the ideal (x) from the ring $\mathbf{Q}[x, y]$:

```
sage: R.<x,y> = PolynomialRing(QQ, 2)
sage: S.<a,b> = QuotientRing(R, R.ideal(1 + y^2))
sage: T.<c,d> = QuotientRing(S, S.ideal(a))
sage: T
Quotient of Multivariate Polynomial Ring in x, y over Rational Field
 by the ideal (x, y^2 + 1)
sage: R.gens(); S.gens(); T.gens()
(x, y)
(a, b)
(0, d)
sage: for n in range(4): d^n
1
d
-1
-d
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, Integer(2), names=('x', 'y',)); (x, y,) = R._first_
↪ngens(2)
>>> S = QuotientRing(R, R.ideal(Integer(1) + y**Integer(2)), names=('a', 'b',));␣
↪(a, b,) = S._first_ngens(2)
>>> T = QuotientRing(S, S.ideal(a), names=('c', 'd',)); (c, d,) = T._first_
↪ngens(2)
>>> T
```

(continues on next page)

(continued from previous page)

```
Quotient of Multivariate Polynomial Ring in x, y over Rational Field
 by the ideal (x, y^2 + 1)
>>> R.gens(); S.gens(); T.gens()
(x, y)
(a, b)
(0, d)
>>> for n in range(Integer(4)): d**n
1
d
-1
-d
```

class sage.rings.quotient_ring.**QuotientRingIdeal_generic**(*ring*, *gens*, *coerce=True*, ***kwds*)

Bases: *Ideal_generic*

Specialized class for quotient-ring ideals.

EXAMPLES:

```
sage: Zmod(9).ideal([-6,9])
Ideal (3, 0) of Ring of integers modulo 9
```

```
>>> from sage.all import *
>>> Zmod(Integer(9)).ideal([-Integer(6),Integer(9)])
Ideal (3, 0) of Ring of integers modulo 9
```

radical()

Return the radical of this ideal.

EXAMPLES:

```
sage: Zmod(16).ideal(4).radical()
Principal ideal (2) of Ring of integers modulo 16
```

```
>>> from sage.all import *
>>> Zmod(Integer(16)).ideal(Integer(4)).radical()
Principal ideal (2) of Ring of integers modulo 16
```

class sage.rings.quotient_ring.**QuotientRingIdeal_principal**(*ring*, *gens*, *coerce=True*, ***kwds*)

Bases: *Ideal_principal*, *QuotientRingIdeal_generic*

Specialized class for principal quotient-ring ideals.

EXAMPLES:

```
sage: Zmod(9).ideal(-33)
Principal ideal (3) of Ring of integers modulo 9
```

```
>>> from sage.all import *
>>> Zmod(Integer(9)).ideal(-Integer(33))
Principal ideal (3) of Ring of integers modulo 9
```

class sage.rings.quotient_ring.**QuotientRing_generic**(*R*, *I*, *names*, *category=None*)

Bases: *QuotientRing_nc*, *Ring*

Create a quotient ring of a *commutative* ring R by the ideal I.

EXAMPLES:

```
sage: R.<x> = PolynomialRing(ZZ)
sage: I = R.ideal([4 + 3*x + x^2, 1 + x^2])
sage: S = R.quotient_ring(I); S
Quotient of Univariate Polynomial Ring in x over Integer Ring
 by the ideal (x^2 + 3*x + 4, x^2 + 1)
```

```
>>> from sage.all import *
>>> R = PolynomialRing(ZZ, names=('x',)); (x,) = R._first_ngens(1)
>>> I = R.ideal([Integer(4) + Integer(3)*x + x**Integer(2), Integer(1) +␣
↪x**Integer(2)])
>>> S = R.quotient_ring(I); S
Quotient of Univariate Polynomial Ring in x over Integer Ring
 by the ideal (x^2 + 3*x + 4, x^2 + 1)
```

class sage.rings.quotient_ring.**QuotientRing_nc**(*R*, *I*, *names*, *category=None*)

Bases: `Parent`

The quotient ring of R by a twosided ideal I.

This class is for rings that are not in the category `Rings().Commutative()`.

EXAMPLES:

Here is a quotient of a free algebra by a twosided homogeneous ideal:

```
sage: F.<x,y,z> = FreeAlgebra(QQ, implementation='letterplace')
sage: I = F * [x*y + y*z, x^2 + x*y - y*x - y^2]*F
sage: Q.<a,b,c> = F.quo(I); Q
Quotient of Free Associative Unital Algebra on 3 generators (x, y, z) over␣
↪Rational Field
 by the ideal (x*y + y*z, x*x + x*y - y*x - y*y)
sage: a*b
-b*c
sage: a^3
-b*c*a - b*c*b - b*c*c
```

```
>>> from sage.all import *
>>> F = FreeAlgebra(QQ, implementation='letterplace', names=('x', 'y', 'z',)); (x,
↪ y, z,) = F._first_ngens(3)
>>> I = F * [x*y + y*z, x**Integer(2) + x*y - y*x - y**Integer(2)]*F
>>> Q = F.quo(I, names=('a', 'b', 'c',)); (a, b, c,) = Q._first_ngens(3); Q
Quotient of Free Associative Unital Algebra on 3 generators (x, y, z) over␣
↪Rational Field
 by the ideal (x*y + y*z, x*x + x*y - y*x - y*y)
>>> a*b
-b*c
>>> a**Integer(3)
-b*c*a - b*c*b - b*c*c
```

A quotient of a quotient is just the quotient of the original top ring by the sum of two ideals:

```
sage: J = Q * [a^3 - b^3] * Q
sage: R.<i,j,k> = Q.quo(J); R
Quotient of
 Free Associative Unital Algebra on 3 generators (x, y, z) over Rational Field
 by the ideal (-y*y*z - y*z*x - 2*y*z*z, x*y + y*z, x*x + x*y - y*x - y*y)
sage: i^3
-j*k*i - j*k*j - j*k*k
sage: j^3
-j*k*i - j*k*j - j*k*k
```

```
>>> from sage.all import *
>>> J = Q * [a**Integer(3) - b**Integer(3)] * Q
>>> R = Q.quo(J, names=('i', 'j', 'k',)); (i, j, k,) = R._first_ngens(3); R
Quotient of
 Free Associative Unital Algebra on 3 generators (x, y, z) over Rational Field
 by the ideal (-y*y*z - y*z*x - 2*y*z*z, x*y + y*z, x*x + x*y - y*x - y*y)
>>> i**Integer(3)
-j*k*i - j*k*j - j*k*k
>>> j**Integer(3)
-j*k*i - j*k*j - j*k*k
```

For rings that *do* inherit from *Ring*, we provide a subclass *QuotientRing_generic*, for backwards compatibility.

EXAMPLES:

```
sage: R.<x> = PolynomialRing(ZZ,'x')
sage: I = R.ideal([4 + 3*x + x^2, 1 + x^2])
sage: S = R.quotient_ring(I); S
Quotient of Univariate Polynomial Ring in x over Integer Ring
 by the ideal (x^2 + 3*x + 4, x^2 + 1)
```

```
>>> from sage.all import *
>>> R = PolynomialRing(ZZ,'x', names=('x',)); (x,) = R._first_ngens(1)
>>> I = R.ideal([Integer(4) + Integer(3)*x + x**Integer(2), Integer(1) +␣
↪x**Integer(2)])
>>> S = R.quotient_ring(I); S
Quotient of Univariate Polynomial Ring in x over Integer Ring
 by the ideal (x^2 + 3*x + 4, x^2 + 1)
```

```
sage: R.<x,y> = PolynomialRing(QQ)
sage: S.<a,b> = R.quo(x^2 + y^2)
sage: a^2 + b^2 == 0
True
sage: S(0) == a^2 + b^2
True
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> S = R.quo(x**Integer(2) + y**Integer(2), names=('a', 'b',)); (a, b,) = S._
↪first_ngens(2)
>>> a**Integer(2) + b**Integer(2) == Integer(0)
```

(continues on next page)

(continued from previous page)

```
True
>>> S(Integer(0)) == a**Integer(2) + b**Integer(2)
True
```

Again, a quotient of a quotient is just the quotient of the original top ring by the sum of two ideals.

```
sage: R.<x,y> = PolynomialRing(QQ, 2)
sage: S.<a,b> = R.quo(1 + y^2)
sage: T.<c,d> = S.quo(a)
sage: T
Quotient of Multivariate Polynomial Ring in x, y over Rational Field
 by the ideal (x, y^2 + 1)
sage: T.gens()
(0, d)
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, Integer(2), names=('x', 'y',)); (x, y,) = R._first_
↪ngens(2)
>>> S = R.quo(Integer(1) + y**Integer(2), names=('a', 'b',)); (a, b,) = S._first_
↪ngens(2)
>>> T = S.quo(a, names=('c', 'd',)); (c, d,) = T._first_ngens(2)
>>> T
Quotient of Multivariate Polynomial Ring in x, y over Rational Field
 by the ideal (x, y^2 + 1)
>>> T.gens()
(0, d)
```

Element

alias of *QuotientRingElement*

ambient()

alias of *cover_ring()*.

characteristic()

Return the characteristic of the quotient ring.

Todo

Not yet implemented!

EXAMPLES:

```
sage: Q = QuotientRing(ZZ,7*ZZ)
sage: Q.characteristic()
Traceback (most recent call last):
...
NotImplementedError
```

```
>>> from sage.all import *
>>> Q = QuotientRing(ZZ,Integer(7)*ZZ)
>>> Q.characteristic()
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
NotImplementedError
```

construction()

Return the functorial construction of `self`.

EXAMPLES:

```
sage: R.<x> = PolynomialRing(ZZ,'x')
sage: I = R.ideal([4 + 3*x + x^2, 1 + x^2])
sage: R.quotient_ring(I).construction()
(QuotientFunctor, Univariate Polynomial Ring in x over Integer Ring)

sage: F.<x,y,z> = FreeAlgebra(QQ, implementation='letterplace')
sage: I = F * [x*y + y*z, x^2 + x*y - y*x - y^2] * F
sage: Q = F.quo(I)
sage: Q.construction()
(QuotientFunctor,
 Free Associative Unital Algebra on 3 generators (x, y, z) over Rational␣
↪Field)
```

```
>>> from sage.all import *
>>> R = PolynomialRing(ZZ,'x', names=('x',)); (x,) = R._first_ngens(1)
>>> I = R.ideal([Integer(4) + Integer(3)*x + x**Integer(2), Integer(1) +␣
↪x**Integer(2)])
>>> R.quotient_ring(I).construction()
(QuotientFunctor, Univariate Polynomial Ring in x over Integer Ring)

>>> F = FreeAlgebra(QQ, implementation='letterplace', names=('x', 'y', 'z',));
↪ (x, y, z,) = F._first_ngens(3)
>>> I = F * [x*y + y*z, x**Integer(2) + x*y - y*x - y**Integer(2)] * F
>>> Q = F.quo(I)
>>> Q.construction()
(QuotientFunctor,
 Free Associative Unital Algebra on 3 generators (x, y, z) over Rational␣
↪Field)
```

cover()

The covering ring homomorphism $R \to R/I$, equipped with a section.

EXAMPLES:

```
sage: R = ZZ.quo(3 * ZZ)
sage: pi = R.cover()
sage: pi
Ring morphism:
  From: Integer Ring
  To:   Ring of integers modulo 3
  Defn: Natural quotient map
sage: pi(5)
2
sage: l = pi.lift()
```

```
>>> from sage.all import *
>>> R = ZZ.quo(Integer(3) * ZZ)
>>> pi = R.cover()
>>> pi
Ring morphism:
  From: Integer Ring
  To:   Ring of integers modulo 3
  Defn: Natural quotient map
>>> pi(Integer(5))
2
>>> l = pi.lift()
```

```
sage: R.<x,y>  = PolynomialRing(QQ)
sage: Q = R.quo((x^2, y^2))
sage: pi = Q.cover()
sage: pi(x^3 + y)
ybar
sage: l = pi.lift(x + y^3)
sage: l
x
sage: l = pi.lift(); l
Set-theoretic ring morphism:
  From: Quotient of Multivariate Polynomial Ring in x, y over Rational Field
        by the ideal (x^2, y^2)
  To:   Multivariate Polynomial Ring in x, y over Rational Field
  Defn: Choice of lifting map
sage: l(x + y^3)
x
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> Q = R.quo((x**Integer(2), y**Integer(2)))
>>> pi = Q.cover()
>>> pi(x**Integer(3) + y)
ybar
>>> l = pi.lift(x + y**Integer(3))
>>> l
x
>>> l = pi.lift(); l
Set-theoretic ring morphism:
  From: Quotient of Multivariate Polynomial Ring in x, y over Rational Field
        by the ideal (x^2, y^2)
  To:   Multivariate Polynomial Ring in x, y over Rational Field
  Defn: Choice of lifting map
>>> l(x + y**Integer(3))
x
```

cover_ring()

Return the cover ring of the quotient ring: that is, the original ring R from which we modded out an ideal, I.

EXAMPLES:

```
sage: Q = QuotientRing(ZZ, 7 * ZZ)
sage: Q.cover_ring()
Integer Ring
```

```
>>> from sage.all import *
>>> Q = QuotientRing(ZZ, Integer(7) * ZZ)
>>> Q.cover_ring()
Integer Ring
```

```
sage: P.<x> = QQ[]
sage: Q = QuotientRing(P, x^2 + 1)
sage: Q.cover_ring()
Univariate Polynomial Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> P = QQ['x']; (x,) = P._first_ngens(1)
>>> Q = QuotientRing(P, x**Integer(2) + Integer(1))
>>> Q.cover_ring()
Univariate Polynomial Ring in x over Rational Field
```

defining_ideal()

Return the ideal generating this quotient ring.

EXAMPLES:

In the integers:

```
sage: Q = QuotientRing(ZZ,7*ZZ)
sage: Q.defining_ideal()
Principal ideal (7) of Integer Ring
```

```
>>> from sage.all import *
>>> Q = QuotientRing(ZZ,Integer(7)*ZZ)
>>> Q.defining_ideal()
Principal ideal (7) of Integer Ring
```

An example involving a quotient of a quotient. By Noether's homomorphism theorems, this is actually a quotient by a sum of two ideals:

```
sage: R.<x,y> = PolynomialRing(QQ, 2)
sage: S.<a,b> = QuotientRing(R, R.ideal(1 + y^2))
sage: T.<c,d> = QuotientRing(S, S.ideal(a))
sage: S.defining_ideal()
Ideal (y^2 + 1) of Multivariate Polynomial Ring in x, y over Rational Field
sage: T.defining_ideal()
Ideal (x, y^2 + 1) of Multivariate Polynomial Ring in x, y over Rational Field
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, Integer(2), names=('x', 'y',)); (x, y,) = R._first_
↪ngens(2)
>>> S = QuotientRing(R, R.ideal(Integer(1) + y**Integer(2)), names=('a', 'b',
↪)); (a, b,) = S._first_ngens(2)
```

(continues on next page)

(continued from previous page)

```
>>> T = QuotientRing(S, S.ideal(a), names=('c', 'd',)); (c, d,) = T._first_
↪ngens(2)
>>> S.defining_ideal()
Ideal (y^2 + 1) of Multivariate Polynomial Ring in x, y over Rational Field
>>> T.defining_ideal()
Ideal (x, y^2 + 1) of Multivariate Polynomial Ring in x, y over Rational Field
```

gen (*i=0*)

Return the i-th generator for this quotient ring.

EXAMPLES:

```
sage: R = QuotientRing(ZZ, 7*ZZ)
sage: R.gen(0)
1
```

```
>>> from sage.all import *
>>> R = QuotientRing(ZZ, Integer(7)*ZZ)
>>> R.gen(Integer(0))
1
```

```
sage: R.<x,y> = PolynomialRing(QQ,2)
sage: S.<a,b> = QuotientRing(R, R.ideal(1 + y^2))
sage: T.<c,d> = QuotientRing(S, S.ideal(a))
sage: T
Quotient of Multivariate Polynomial Ring in x, y over Rational Field
 by the ideal (x, y^2 + 1)
sage: R.gen(0); R.gen(1)
x
y
sage: S.gen(0); S.gen(1)
a
b
sage: T.gen(0); T.gen(1)
0
d
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ,Integer(2), names=('x', 'y',)); (x, y,) = R._first_
↪ngens(2)
>>> S = QuotientRing(R, R.ideal(Integer(1) + y**Integer(2)), names=('a', 'b',
↪)); (a, b,) = S._first_ngens(2)
>>> T = QuotientRing(S, S.ideal(a), names=('c', 'd',)); (c, d,) = T._first_
↪ngens(2)
>>> T
Quotient of Multivariate Polynomial Ring in x, y over Rational Field
 by the ideal (x, y^2 + 1)
>>> R.gen(Integer(0)); R.gen(Integer(1))
x
y
>>> S.gen(Integer(0)); S.gen(Integer(1))
a
```

(continues on next page)

(continued from previous page)

```
b
>>> T.gen(Integer(0)); T.gen(Integer(1))
0
d
```

gens()

Return a tuple containing generators of `self`.

EXAMPLES:

```
sage: R.<x,y> = PolynomialRing(QQ)
sage: S = R.quotient_ring(x^2 + y^2)
sage: S.gens()
(xbar, ybar)
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> S = R.quotient_ring(x**Integer(2) + y**Integer(2))
>>> S.gens()
(xbar, ybar)
```

ideal(**gens*, ***kwds*)

Return the ideal of `self` with the given generators.

EXAMPLES:

```
sage: R.<x,y> = PolynomialRing(QQ)
sage: S = R.quotient_ring(x^2 + y^2)
sage: S.ideal()
Ideal (0) of Quotient of Multivariate Polynomial Ring in x, y
 over Rational Field by the ideal (x^2 + y^2)
sage: S.ideal(x + y + 1)
Ideal (xbar + ybar + 1) of Quotient of Multivariate Polynomial Ring in x, y
 over Rational Field by the ideal (x^2 + y^2)
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> S = R.quotient_ring(x**Integer(2) + y**Integer(2))
>>> S.ideal()
Ideal (0) of Quotient of Multivariate Polynomial Ring in x, y
 over Rational Field by the ideal (x^2 + y^2)
>>> S.ideal(x + y + Integer(1))
Ideal (xbar + ybar + 1) of Quotient of Multivariate Polynomial Ring in x, y
 over Rational Field by the ideal (x^2 + y^2)
```

is_commutative()

Tell whether this quotient ring is commutative.

Note

This is certainly the case if the cover ring is commutative. Otherwise, if this ring has a finite number of

generators, it is tested whether they commute. If the number of generators is infinite, a `NotImplementedError` is raised.

AUTHOR:

- Simon King (2011-03-23): See Issue #7797.

EXAMPLES:

Any quotient of a commutative ring is commutative:

```
sage: P.<a,b,c> = QQ[]
sage: P.quo(P.random_element()).is_commutative()
True
```

```
>>> from sage.all import *
>>> P = QQ['a, b, c']; (a, b, c,) = P._first_ngens(3)
>>> P.quo(P.random_element()).is_commutative()
True
```

The non-commutative case is more interesting:

```
sage: F.<x,y,z> = FreeAlgebra(QQ, implementation='letterplace')
sage: I = F * [x*y + y*z, x^2 + x*y - y*x - y^2] * F
sage: Q = F.quo(I)
sage: Q.is_commutative()
False
sage: Q.1*Q.2 == Q.2*Q.1
False
```

```
>>> from sage.all import *
>>> F = FreeAlgebra(QQ, implementation='letterplace', names=('x', 'y', 'z',));
↪ (x, y, z,) = F._first_ngens(3)
>>> I = F * [x*y + y*z, x**Integer(2) + x*y - y*x - y**Integer(2)] * F
>>> Q = F.quo(I)
>>> Q.is_commutative()
False
>>> Q.gen(1)*Q.gen(2) == Q.gen(2)*Q.gen(1)
False
```

In the next example, the generators apparently commute:

```
sage: J = F * [x*y - y*x, x*z - z*x, y*z - z*y, x^3 - y^3] * F
sage: R = F.quo(J)
sage: R.is_commutative()
True
```

```
>>> from sage.all import *
>>> J = F * [x*y - y*x, x*z - z*x, y*z - z*y, x**Integer(3) - y**Integer(3)]␣
↪* F
>>> R = F.quo(J)
>>> R.is_commutative()
True
```

is_field(*proof=True*)

Return `True` if the quotient ring is a field. Checks to see if the defining ideal is maximal.

is_integral_domain(*proof=True*)

With `proof` equal to `True` (the default), this function may raise a `NotImplementedError`.

When `proof` is `False`, if `True` is returned, then `self` is definitely an integral domain. If the function returns `False`, then either `self` is not an integral domain or it was unable to determine whether or not `self` is an integral domain.

EXAMPLES:

```
sage: R.<x,y> = QQ[]
sage: R.quo(x^2 - y).is_integral_domain()
True
sage: R.quo(x^2 - y^2).is_integral_domain()
False
sage: R.quo(x^2 - y^2).is_integral_domain(proof=False)
False
sage: R.<a,b,c> = ZZ[]
sage: Q = R.quotient_ring([a, b])
sage: Q.is_integral_domain()
Traceback (most recent call last):
...
NotImplementedError
sage: Q.is_integral_domain(proof=False)
False
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> R.quo(x**Integer(2) - y).is_integral_domain()
True
>>> R.quo(x**Integer(2) - y**Integer(2)).is_integral_domain()
False
>>> R.quo(x**Integer(2) - y**Integer(2)).is_integral_domain(proof=False)
False
>>> R = ZZ['a, b, c']; (a, b, c,) = R._first_ngens(3)
>>> Q = R.quotient_ring([a, b])
>>> Q.is_integral_domain()
Traceback (most recent call last):
...
NotImplementedError
>>> Q.is_integral_domain(proof=False)
False
```

is_noetherian()

Return `True` if this ring is Noetherian.

EXAMPLES:

```
sage: R = QuotientRing(ZZ, 102 * ZZ)
sage: R.is_noetherian()
True

sage: P.<x> = QQ[]
```

(continues on next page)

(continued from previous page)

```
sage: R = QuotientRing(P, x^2 + 1)
sage: R.is_noetherian()
True
```

```
>>> from sage.all import *
>>> R = QuotientRing(ZZ, Integer(102) * ZZ)
>>> R.is_noetherian()
True

>>> P = QQ['x']; (x,) = P._first_ngens(1)
>>> R = QuotientRing(P, x**Integer(2) + Integer(1))
>>> R.is_noetherian()
True
```

If the cover ring of `self` is not Noetherian, we currently have no way of testing whether `self` is Noetherian, so we raise an error:

```
sage: R.<x> = InfinitePolynomialRing(QQ)
sage: R.is_noetherian()
False
sage: I = R.ideal([x[1]^2, x[2]])
sage: S = R.quotient(I)
sage: S.is_noetherian()
Traceback (most recent call last):
...
NotImplementedError
```

```
>>> from sage.all import *
>>> R = InfinitePolynomialRing(QQ, names=('x',)); (x,) = R._first_ngens(1)
>>> R.is_noetherian()
False
>>> I = R.ideal([x[Integer(1)]**Integer(2), x[Integer(2)]])
>>> S = R.quotient(I)
>>> S.is_noetherian()
Traceback (most recent call last):
...
NotImplementedError
```

lift (*x=None*)

Return the lifting map to the cover, or the image of an element under the lifting map.

Note

The category framework imposes that `Q.lift(x)` returns the image of an element x under the lifting map. For backwards compatibility, we let `Q.lift()` return the lifting map.

EXAMPLES:

```
sage: R.<x,y> = PolynomialRing(QQ, 2)
sage: S = R.quotient(x^2 + y^2)
sage: S.lift()
```

(continues on next page)

(continued from previous page)

```
Set-theoretic ring morphism:
  From: Quotient of Multivariate Polynomial Ring in x, y over Rational Field
        by the ideal (x^2 + y^2)
  To:   Multivariate Polynomial Ring in x, y over Rational Field
  Defn: Choice of lifting map
sage: S.lift(S.0) == x
True
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, Integer(2), names=('x', 'y',)); (x, y,) = R._first_
↪ngens(2)
>>> S = R.quotient(x**Integer(2) + y**Integer(2))
>>> S.lift()
Set-theoretic ring morphism:
  From: Quotient of Multivariate Polynomial Ring in x, y over Rational Field
        by the ideal (x^2 + y^2)
  To:   Multivariate Polynomial Ring in x, y over Rational Field
  Defn: Choice of lifting map
>>> S.lift(S.gen(0)) == x
True
```

lifting_map()

Return the lifting map to the cover.

EXAMPLES:

```
sage: R.<x,y> = PolynomialRing(QQ, 2)
sage: S = R.quotient(x^2 + y^2)
sage: pi = S.cover(); pi
Ring morphism:
  From: Multivariate Polynomial Ring in x, y over Rational Field
  To:   Quotient of Multivariate Polynomial Ring in x, y over Rational Field
        by the ideal (x^2 + y^2)
  Defn: Natural quotient map
sage: L = S.lifting_map(); L
Set-theoretic ring morphism:
  From: Quotient of Multivariate Polynomial Ring in x, y over Rational Field
        by the ideal (x^2 + y^2)
  To:   Multivariate Polynomial Ring in x, y over Rational Field
  Defn: Choice of lifting map
sage: L(S.0)
x
sage: L(S.1)
y
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, Integer(2), names=('x', 'y',)); (x, y,) = R._first_
↪ngens(2)
>>> S = R.quotient(x**Integer(2) + y**Integer(2))
>>> pi = S.cover(); pi
Ring morphism:
  From: Multivariate Polynomial Ring in x, y over Rational Field
```

(continues on next page)

(continued from previous page)

```
  To:   Quotient of Multivariate Polynomial Ring in x, y over Rational Field
        by the ideal (x^2 + y^2)
  Defn: Natural quotient map
>>> L = S.lifting_map(); L
Set-theoretic ring morphism:
  From: Quotient of Multivariate Polynomial Ring in x, y over Rational Field
        by the ideal (x^2 + y^2)
  To:   Multivariate Polynomial Ring in x, y over Rational Field
  Defn: Choice of lifting map
>>> L(S.gen(0))
x
>>> L(S.gen(1))
y
```

Note that some reduction may be applied so that the lift of a reduction need not equal the original element:

```
sage: z = pi(x^3 + 2*y^2); z
-xbar*ybar^2 + 2*ybar^2
sage: L(z)
-x*y^2 + 2*y^2
sage: L(z) == x^3 + 2*y^2
False
```

```
>>> from sage.all import *
>>> z = pi(x**Integer(3) + Integer(2)*y**Integer(2)); z
-xbar*ybar^2 + 2*ybar^2
>>> L(z)
-x*y^2 + 2*y^2
>>> L(z) == x**Integer(3) + Integer(2)*y**Integer(2)
False
```

Test that there also is a lift for rings that are no instances of *Ring* (see Issue #11068):

```
sage: MS = MatrixSpace(GF(5), 2, 2)
sage: I = MS * [MS.0*MS.1, MS.2 + MS.3] * MS
sage: Q = MS.quo(I)
sage: Q.lift()
Set-theoretic ring morphism:
  From: Quotient of Full MatrixSpace of 2 by 2 dense matrices
        over Finite Field of size 5 by the ideal
(
  [0 1]
  [0 0],

  [0 0]
  [1 1]
)

  To:   Full MatrixSpace of 2 by 2 dense matrices over Finite Field of size 5
  Defn: Choice of lifting map
```

```
>>> from sage.all import *
>>> MS = MatrixSpace(GF(Integer(5)), Integer(2), Integer(2))
>>> I = MS * [MS.gen(0)*MS.gen(1), MS.gen(2) + MS.gen(3)] * MS
>>> Q = MS.quo(I)
>>> Q.lift()
Set-theoretic ring morphism:
  From: Quotient of Full MatrixSpace of 2 by 2 dense matrices
        over Finite Field of size 5 by the ideal
(
  [0 1]
  [0 0],
<BLANKLINE>
  [0 0]
  [1 1]
)
<BLANKLINE>
  To:   Full MatrixSpace of 2 by 2 dense matrices over Finite Field of size 5
  Defn: Choice of lifting map
```

ngens()

Return the number of generators for this quotient ring.

Todo

Note that `ngens` counts 0 as a generator. Does this make sense? That is, since 0 only generates itself and the fact that this is true for all rings, is there a way to "knock it off" of the generators list if a generator of some original ring is modded out?

EXAMPLES:

```
sage: R = QuotientRing(ZZ, 7*ZZ)
sage: R.gens(); R.ngens()
(1,)
1
```

```
>>> from sage.all import *
>>> R = QuotientRing(ZZ, Integer(7)*ZZ)
>>> R.gens(); R.ngens()
(1,)
1
```

```
sage: R.<x,y> = PolynomialRing(QQ,2)
sage: S.<a,b> = QuotientRing(R, R.ideal(1 + y^2))
sage: T.<c,d> = QuotientRing(S, S.ideal(a))
sage: T
Quotient of Multivariate Polynomial Ring in x, y over Rational Field
 by the ideal (x, y^2 + 1)
sage: R.gens(); S.gens(); T.gens()
(x, y)
(a, b)
(0, d)
```

(continues on next page)

(continued from previous page)

```
sage: R.ngens(); S.ngens(); T.ngens()
2
2
2
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ,Integer(2), names=('x', 'y',)); (x, y,) = R._first_
↪ngens(2)
>>> S = QuotientRing(R, R.ideal(Integer(1) + y**Integer(2)), names=('a', 'b',
↪)); (a, b,) = S._first_ngens(2)
>>> T = QuotientRing(S, S.ideal(a), names=('c', 'd',)); (c, d,) = T._first_
↪ngens(2)
>>> T
Quotient of Multivariate Polynomial Ring in x, y over Rational Field
 by the ideal (x, y^2 + 1)
>>> R.gens(); S.gens(); T.gens()
(x, y)
(a, b)
(0, d)
>>> R.ngens(); S.ngens(); T.ngens()
2
2
2
```

random_element()

Return a random element of this quotient ring obtained by sampling a random element of the cover ring and reducing it modulo the defining ideal.

EXAMPLES:

```
sage: R.<x,y> = QQ[]
sage: S = R.quotient([x^3, y^2])
sage: S.random_element()  # random
-8/5*xbar^2 + 3/2*xbar*ybar + 2*xbar - 4/23
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> S = R.quotient([x**Integer(3), y**Integer(2)])
>>> S.random_element()  # random
-8/5*xbar^2 + 3/2*xbar*ybar + 2*xbar - 4/23
```

retract(x)

The image of an element of the cover ring under the quotient map.

INPUT:

- `x` – an element of the cover ring

OUTPUT: the image of the given element in `self`

EXAMPLES:

```
sage: R.<x,y> = PolynomialRing(QQ, 2)
sage: S = R.quotient(x^2 + y^2)
```

(continues on next page)

(continued from previous page)

```
sage: S.retract((x+y)^2)
2*xbar*ybar
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, Integer(2), names=('x', 'y',)); (x, y,) = R._first_
↪ngens(2)
>>> S = R.quotient(x**Integer(2) + y**Integer(2))
>>> S.retract((x+y)**Integer(2))
2*xbar*ybar
```

term_order()

Return the term order of this ring.

EXAMPLES:

```
sage: P.<a,b,c> = PolynomialRing(QQ)
sage: I = Ideal([a^2 - a, b^2 - b, c^2 - c])
sage: Q = P.quotient(I)
sage: Q.term_order()
Degree reverse lexicographic term order
```

```
>>> from sage.all import *
>>> P = PolynomialRing(QQ, names=('a', 'b', 'c',)); (a, b, c,) = P._first_
↪ngens(3)
>>> I = Ideal([a**Integer(2) - a, b**Integer(2) - b, c**Integer(2) - c])
>>> Q = P.quotient(I)
>>> Q.term_order()
Degree reverse lexicographic term order
```

4.2 Quotient Ring Elements

AUTHORS:

- William Stein

class sage.rings.quotient_ring_element.**QuotientRingElement**(*parent*, *rep*, *reduce=True*)

Bases: RingElement

An element of a quotient ring R/I.

INPUT:

- parent – the ring R/I
- rep – a representative of the element in R; this is used as the internal representation of the element
- reduce – boolean (default: True); if True, then the internal representation of the element is rep reduced modulo the ideal I

EXAMPLES:

```
sage: R.<x> = PolynomialRing(ZZ)
sage: S.<xbar> = R.quo((4 + 3*x + x^2, 1 + x^2)); S
Quotient of Univariate Polynomial Ring in x over Integer Ring
```

(continues on next page)

(continued from previous page)

```
 by the ideal (x^2 + 3*x + 4, x^2 + 1)
sage: v = S.gens(); v
(xbar,)
```

```
>>> from sage.all import *
>>> R = PolynomialRing(ZZ, names=('x',)); (x,) = R._first_ngens(1)
>>> S = R.quo((Integer(4) + Integer(3)*x + x**Integer(2), Integer(1) +␣
↪x**Integer(2)), names=('xbar',)); (xbar,) = S._first_ngens(1); S
Quotient of Univariate Polynomial Ring in x over Integer Ring
 by the ideal (x^2 + 3*x + 4, x^2 + 1)
>>> v = S.gens(); v
(xbar,)
```

```
sage: loads(v[0].dumps()) == v[0]
True
```

```
>>> from sage.all import *
>>> loads(v[Integer(0)].dumps()) == v[Integer(0)]
True
```

```
sage: R.<x,y> = PolynomialRing(QQ, 2)
sage: S = R.quo(x^2 + y^2); S
Quotient of Multivariate Polynomial Ring in x, y over Rational Field
 by the ideal (x^2 + y^2)
sage: S.gens()                                                              #␣
↪needs sage.libs.singular
(xbar, ybar)
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, Integer(2), names=('x', 'y',)); (x, y,) = R._first_
↪ngens(2)
>>> S = R.quo(x**Integer(2) + y**Integer(2)); S
Quotient of Multivariate Polynomial Ring in x, y over Rational Field
 by the ideal (x^2 + y^2)
>>> S.gens()                                                                #␣
↪needs sage.libs.singular
(xbar, ybar)
```

We name each of the generators.

```
sage: S.<a,b> = R.quotient(x^2 + y^2)
sage: a
a
sage: b
b
sage: a^2 + b^2 == 0
True
sage: b.lift()
y
sage: (a^3 + b^2).lift()
-x*y^2 + y^2
```

```
>>> from sage.all import *
>>> S = R.quotient(x**Integer(2) + y**Integer(2), names=('a', 'b',)); (a, b,) = S.
↪_first_ngens(2)
>>> a
a
>>> b
b
>>> a**Integer(2) + b**Integer(2) == Integer(0)
True
>>> b.lift()
y
>>> (a**Integer(3) + b**Integer(2)).lift()
-x*y^2 + y^2
```

is_unit()

Return `True` if `self` is a unit in the quotient ring.

EXAMPLES:

```
sage: R.<x,y> = QQ[]; S.<a,b> = R.quo(1 - x*y); type(a)                    #␣
↪needs sage.libs.singular
<class 'sage.rings.quotient_ring.QuotientRing_generic_with_category.element_
↪class'>
sage: a*b                                                                  #␣
↪needs sage.libs.singular
1
sage: S(2).is_unit()                                                       #␣
↪needs sage.libs.singular
True
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2); S = R.quo(Integer(1) - x*y,␣
↪names=('a', 'b',)); (a, b,) = S._first_ngens(2); type(a)                  ␣
↪ # needs sage.libs.singular
<class 'sage.rings.quotient_ring.QuotientRing_generic_with_category.element_
↪class'>
>>> a*b                                                                    #␣
↪needs sage.libs.singular
1
>>> S(Integer(2)).is_unit()                                                ␣
↪     # needs sage.libs.singular
True
```

Check that Issue #29469 is fixed:

```
sage: a.is_unit()                                                          #␣
↪needs sage.libs.singular
True
sage: (a+b).is_unit()                                                      #␣
↪needs sage.libs.singular
False
```

```
>>> from sage.all import *
>>> a.is_unit()                                                              #␣
↪needs sage.libs.singular
True
>>> (a+b).is_unit()                                                          #␣
↪needs sage.libs.singular
False
```

lc()

Return the leading coefficient of this quotient ring element.

EXAMPLES:

```
sage: R.<x,y,z> = PolynomialRing(GF(7), 3, order='lex')
sage: I = sage.rings.ideal.FieldIdeal(R)
sage: Q = R.quo(I)
sage: f = Q(z*y + 2*x)
sage: f.lc()
2
```

```
>>> from sage.all import *
>>> R = PolynomialRing(GF(Integer(7)), Integer(3), order='lex', names=('x', 'y
↪', 'z',)); (x, y, z,) = R._first_ngens(3)
>>> I = sage.rings.ideal.FieldIdeal(R)
>>> Q = R.quo(I)
>>> f = Q(z*y + Integer(2)*x)
>>> f.lc()
2
```

lift()

If `self` is an element of R/I, then return `self` as an element of R.

EXAMPLES:

```
sage: R.<x,y> = QQ[]; S.<a,b> = R.quo(x^2 + y^2); type(a)                  #␣
↪needs sage.libs.singular
<class 'sage.rings.quotient_ring.QuotientRing_generic_with_category.element_
↪class'>
sage: a.lift()                                                             #␣
↪needs sage.libs.singular
x
sage: (3/5*(a + a^2 + b^2)).lift()                                         #␣
↪needs sage.libs.singular
3/5*x
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2); S = R.quo(x**Integer(2) +␣
↪y**Integer(2), names=('a', 'b',)); (a, b,) = S._first_ngens(2); type(a)    ␣
↪              # needs sage.libs.singular
<class 'sage.rings.quotient_ring.QuotientRing_generic_with_category.element_
↪class'>
>>> a.lift()                                                               #␣
↪needs sage.libs.singular
```

(continues on next page)

(continued from previous page)

```
x
>>> (Integer(3)/Integer(5)*(a + a**Integer(2) + b**Integer(2))).lift()
↪                                    # needs sage.libs.singular
3/5*x
```

lm()

Return the leading monomial of this quotient ring element.

EXAMPLES:

```
sage: R.<x,y,z> = PolynomialRing(GF(7), 3, order='lex')
sage: I = sage.rings.ideal.FieldIdeal(R)
sage: Q = R.quo(I)
sage: f = Q(z*y + 2*x)
sage: f.lm()
xbar
```

```
>>> from sage.all import *
>>> R = PolynomialRing(GF(Integer(7)), Integer(3), order='lex', names=('x', 'y
↪', 'z',)); (x, y, z,) = R._first_ngens(3)
>>> I = sage.rings.ideal.FieldIdeal(R)
>>> Q = R.quo(I)
>>> f = Q(z*y + Integer(2)*x)
>>> f.lm()
xbar
```

lt()

Return the leading term of this quotient ring element.

EXAMPLES:

```
sage: R.<x,y,z> = PolynomialRing(GF(7), 3, order='lex')
sage: I = sage.rings.ideal.FieldIdeal(R)
sage: Q = R.quo(I)
sage: f = Q(z*y + 2*x)
sage: f.lt()
2*xbar
```

```
>>> from sage.all import *
>>> R = PolynomialRing(GF(Integer(7)), Integer(3), order='lex', names=('x', 'y
↪', 'z',)); (x, y, z,) = R._first_ngens(3)
>>> I = sage.rings.ideal.FieldIdeal(R)
>>> Q = R.quo(I)
>>> f = Q(z*y + Integer(2)*x)
>>> f.lt()
2*xbar
```

monomials()

Return the monomials in `self`.

OUTPUT: list of monomials

EXAMPLES:

```
sage: R.<x,y> = QQ[]; S.<a,b> = R.quo(x^2 + y^2); type(a)
<class 'sage.rings.quotient_ring.QuotientRing_generic_with_category.element_
↪class'>
sage: a.monomials()
[a]
sage: (a + a*b).monomials()
[a*b, a]
sage: R.zero().monomials()
[]
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2); S = R.quo(x**Integer(2) +␣
↪y**Integer(2), names=('a', 'b',)); (a, b,) = S._first_ngens(2); type(a)
<class 'sage.rings.quotient_ring.QuotientRing_generic_with_category.element_
↪class'>
>>> a.monomials()
[a]
>>> (a + a*b).monomials()
[a*b, a]
>>> R.zero().monomials()
[]
```

reduce(G)

Reduce this quotient ring element by a set of quotient ring elements `G`.

INPUT:

- `G` – list of quotient ring elements

Warning

This method is not guaranteed to return unique minimal results. For quotients of polynomial rings, use `reduce()` on the ideal generated by `G`, instead.

EXAMPLES:

```
sage: P.<a,b,c,d,e> = PolynomialRing(GF(2), 5, order='lex')
sage: I1 = ideal([a*b + c*d + 1, a*c*e + d*e,
....:             a*b*e + c*e, b*c + c*d*e + 1])
sage: Q = P.quotient(sage.rings.ideal.FieldIdeal(P))
sage: I2 = ideal([Q(f) for f in I1.gens()])
sage: f = Q((a*b + c*d + 1)^2  + e)
sage: f.reduce(I2.gens())
ebar
```

```
>>> from sage.all import *
>>> P = PolynomialRing(GF(Integer(2)), Integer(5), order='lex', names=('a', 'b
↪', 'c', 'd', 'e',)); (a, b, c, d, e,) = P._first_ngens(5)
>>> I1 = ideal([a*b + c*d + Integer(1), a*c*e + d*e,
...             a*b*e + c*e, b*c + c*d*e + Integer(1)])
>>> Q = P.quotient(sage.rings.ideal.FieldIdeal(P))
>>> I2 = ideal([Q(f) for f in I1.gens()])
```

(continues on next page)

(continued from previous page)

```
>>> f = Q((a*b + c*d + Integer(1))**Integer(2)  + e)
>>> f.reduce(I2.gens())
ebar
```

Notice that the result above is not minimal:

```
sage: I2.reduce(f)                                                          #␣
↪needs sage.libs.singular
0
```

```
>>> from sage.all import *
>>> I2.reduce(f)                                                            #␣
↪needs sage.libs.singular
0
```

variables()

Return all variables occurring in `self`.

OUTPUT:

A tuple of linear monomials, one for each variable occurring in `self`.

EXAMPLES:

```
sage: R.<x,y> = QQ[]; S.<a,b> = R.quo(x^2 + y^2); type(a)
<class 'sage.rings.quotient_ring.QuotientRing_generic_with_category.element_
↪class'>
sage: a.variables()
(a,)
sage: b.variables()
(b,)
sage: s = a^2 + b^2 + 1; s
1
sage: s.variables()
()
sage: (a + b).variables()
(a, b)
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2); S = R.quo(x**Integer(2) +␣
↪y**Integer(2), names=('a', 'b',)); (a, b,) = S._first_ngens(2); type(a)
<class 'sage.rings.quotient_ring.QuotientRing_generic_with_category.element_
↪class'>
>>> a.variables()
(a,)
>>> b.variables()
(b,)
>>> s = a**Integer(2) + b**Integer(2) + Integer(1); s
1
>>> s.variables()
()
>>> (a + b).variables()
(a, b)
```

CHAPTER

FIVE

FRACTION FIELDS

5.1 Fraction Field of Integral Domains

AUTHORS:

- William Stein (with input from David Joyner, David Kohel, and Joe Wetherell)
- Burcin Erocal
- Julian Rüth (2017-06-27): embedding into the field of fractions and its section

EXAMPLES:

Quotienting is a constructor for an element of the fraction field:

```
sage: R.<x> = QQ[]
sage: (x^2-1)/(x+1)
x - 1
sage: parent((x^2-1)/(x+1))
Fraction Field of Univariate Polynomial Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> (x**Integer(2)-Integer(1))/(x+Integer(1))
x - 1
>>> parent((x**Integer(2)-Integer(1))/(x+Integer(1)))
Fraction Field of Univariate Polynomial Ring in x over Rational Field
```

The GCD is not taken (since it doesn't converge sometimes) in the inexact case:

```
sage: Z.<z> = CC[]
sage: I = CC.gen()
sage: (1+I+z)/(z+0.1*I)
(z + 1.00000000000000 + I)/(z + 0.100000000000000*I)
sage: (1+I*z)/(z+1.1)
(I*z + 1.00000000000000)/(z + 1.10000000000000)
```

```
>>> from sage.all import *
>>> Z = CC['z']; (z,) = Z._first_ngens(1)
>>> I = CC.gen()
>>> (Integer(1)+I+z)/(z+RealNumber('0.1')*I)
(z + 1.00000000000000 + I)/(z + 0.100000000000000*I)
```

(continues on next page)

(continued from previous page)

```
>>> (Integer(1)+I*z)/(z+RealNumber('1.1'))
(I*z + 1.00000000000000)/(z + 1.10000000000000)
```

sage.rings.fraction_field.**FractionField**(*R*, *names=None*)

Create the fraction field of the integral domain `R`.

INPUT:

- `R` – an integral domain
- `names` – ignored

EXAMPLES:

We create some example fraction fields:

```
sage: FractionField(IntegerRing())
Rational Field
sage: FractionField(PolynomialRing(RationalField(),'x'))
Fraction Field of Univariate Polynomial Ring in x over Rational Field
sage: FractionField(PolynomialRing(IntegerRing(),'x'))
Fraction Field of Univariate Polynomial Ring in x over Integer Ring
sage: FractionField(PolynomialRing(RationalField(),2,'x'))
Fraction Field of Multivariate Polynomial Ring in x0, x1 over Rational Field
```

```
>>> from sage.all import *
>>> FractionField(IntegerRing())
Rational Field
>>> FractionField(PolynomialRing(RationalField(),'x'))
Fraction Field of Univariate Polynomial Ring in x over Rational Field
>>> FractionField(PolynomialRing(IntegerRing(),'x'))
Fraction Field of Univariate Polynomial Ring in x over Integer Ring
>>> FractionField(PolynomialRing(RationalField(),Integer(2),'x'))
Fraction Field of Multivariate Polynomial Ring in x0, x1 over Rational Field
```

Dividing elements often implicitly creates elements of the fraction field:

```
sage: x = PolynomialRing(RationalField(), 'x').gen()
sage: f = x/(x+1)
sage: g = x**3/(x+1)
sage: f/g
1/x^2
sage: g/f
x^2
```

```
>>> from sage.all import *
>>> x = PolynomialRing(RationalField(), 'x').gen()
>>> f = x/(x+Integer(1))
>>> g = x**Integer(3)/(x+Integer(1))
>>> f/g
1/x^2
>>> g/f
x^2
```

The input must be an integral domain:

```
sage: Frac(Integers(4))
Traceback (most recent call last):
...
TypeError: R must be an integral domain
```

```
>>> from sage.all import *
>>> Frac(Integers(Integer(4)))
Traceback (most recent call last):
...
TypeError: R must be an integral domain
```

class sage.rings.fraction_field.**FractionFieldEmbedding**

Bases: DefaultConvertMap_unique

The embedding of an integral domain into its field of fractions.

EXAMPLES:

```
sage: R.<x> = QQ[]
sage: f = R.fraction_field().coerce_map_from(R); f
Coercion map:
  From: Univariate Polynomial Ring in x over Rational Field
  To:   Fraction Field of Univariate Polynomial Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> f = R.fraction_field().coerce_map_from(R); f
Coercion map:
  From: Univariate Polynomial Ring in x over Rational Field
  To:   Fraction Field of Univariate Polynomial Ring in x over Rational Field
```

is_injective()

Return whether this map is injective.

EXAMPLES:

The map from an integral domain to its fraction field is always injective:

```
sage: R.<x> = QQ[]
sage: R.fraction_field().coerce_map_from(R).is_injective()
True
```

```
>>> from sage.all import *
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> R.fraction_field().coerce_map_from(R).is_injective()
True
```

is_surjective()

Return whether this map is surjective.

EXAMPLES:

```
sage: R.<x> = QQ[]
sage: R.fraction_field().coerce_map_from(R).is_surjective()
False
```

```
>>> from sage.all import *
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> R.fraction_field().coerce_map_from(R).is_surjective()
False
```

section()

Return a section of this map.

EXAMPLES:

```
sage: R.<x> = QQ[]
sage: R.fraction_field().coerce_map_from(R).section()
Section map:
  From: Fraction Field of Univariate Polynomial Ring in x over Rational Field
  To:   Univariate Polynomial Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> R.fraction_field().coerce_map_from(R).section()
Section map:
  From: Fraction Field of Univariate Polynomial Ring in x over Rational Field
  To:   Univariate Polynomial Ring in x over Rational Field
```

class sage.rings.fraction_field.**FractionFieldEmbeddingSection**

Bases: Section

The section of the embedding of an integral domain into its field of fractions.

EXAMPLES:

```
sage: R.<x> = QQ[]
sage: f = R.fraction_field().coerce_map_from(R).section(); f
Section map:
  From: Fraction Field of Univariate Polynomial Ring in x over Rational Field
  To:   Univariate Polynomial Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> f = R.fraction_field().coerce_map_from(R).section(); f
Section map:
  From: Fraction Field of Univariate Polynomial Ring in x over Rational Field
  To:   Univariate Polynomial Ring in x over Rational Field
```

class sage.rings.fraction_field.**FractionField_1poly_field**(*R*, *element_class=<class 'sage.rings.fraction_field_element.FractionFieldElement_1poly_field'>*)

Bases: *FractionField_generic*

The fraction field of a univariate polynomial ring over a field.

Many of the functions here are included for coherence with number fields.

class_number()

Here for compatibility with number fields and function fields.

EXAMPLES:

```
sage: R.<t> = GF(5)[]; K = R.fraction_field()
sage: K.class_number()
1
```

```
>>> from sage.all import *
>>> R = GF(Integer(5))['t']; (t,) = R._first_ngens(1); K = R.fraction_field()
>>> K.class_number()
1
```

function_field()

Return the isomorphic function field.

EXAMPLES:

```
sage: R.<t> = GF(5)[]
sage: K = R.fraction_field()
sage: K.function_field()
Rational function field in t over Finite Field of size 5
```

```
>>> from sage.all import *
>>> R = GF(Integer(5))['t']; (t,) = R._first_ngens(1)
>>> K = R.fraction_field()
>>> K.function_field()
Rational function field in t over Finite Field of size 5
```

See also

sage.rings.function_field.function_field_rational.RationalFunctionField.field()

maximal_order()

Return the maximal order in this fraction field.

EXAMPLES:

```
sage: K = FractionField(GF(5)['t'])
sage: K.maximal_order()
Univariate Polynomial Ring in t over Finite Field of size 5
```

```
>>> from sage.all import *
>>> K = FractionField(GF(Integer(5))['t'])
>>> K.maximal_order()
Univariate Polynomial Ring in t over Finite Field of size 5
```

ring_of_integers()

Return the ring of integers in this fraction field.

EXAMPLES:

```
sage: K = FractionField(GF(5)['t'])
sage: K.ring_of_integers()
Univariate Polynomial Ring in t over Finite Field of size 5
```

```
>>> from sage.all import *
>>> K = FractionField(GF(Integer(5))['t'])
>>> K.ring_of_integers()
Univariate Polynomial Ring in t over Finite Field of size 5
```

class sage.rings.fraction_field.**FractionField_generic**(*R*, *element_class=<class 'sage.rings.fraction_field_element.FractionFieldElement'>*, *category=Category of quotient fields*)

Bases: *Field*

The fraction field of an integral domain.

base_ring()

Return the base ring of `self`.

This is the base ring of the ring which this fraction field is the fraction field of.

EXAMPLES:

```
sage: R = Frac(ZZ['t'])
sage: R.base_ring()
Integer Ring
```

```
>>> from sage.all import *
>>> R = Frac(ZZ['t'])
>>> R.base_ring()
Integer Ring
```

characteristic()

Return the characteristic of this fraction field.

EXAMPLES:

```
sage: R = Frac(ZZ['t'])
sage: R.base_ring()
Integer Ring
sage: R = Frac(ZZ['t']); R.characteristic()
0
sage: R = Frac(GF(5)['w']); R.characteristic()
5
```

```
>>> from sage.all import *
>>> R = Frac(ZZ['t'])
>>> R.base_ring()
Integer Ring
>>> R = Frac(ZZ['t']); R.characteristic()
0
>>> R = Frac(GF(Integer(5))['w']); R.characteristic()
5
```

construction()

EXAMPLES:

```
sage: Frac(ZZ['x']).construction()
(FractionField, Univariate Polynomial Ring in x over Integer Ring)
sage: K = Frac(GF(3)['t'])
sage: f, R = K.construction()
sage: f(R)
Fraction Field of Univariate Polynomial Ring in t
 over Finite Field of size 3
sage: f(R) == K
True
```

```
>>> from sage.all import *
>>> Frac(ZZ['x']).construction()
(FractionField, Univariate Polynomial Ring in x over Integer Ring)
>>> K = Frac(GF(Integer(3))['t'])
>>> f, R = K.construction()
>>> f(R)
Fraction Field of Univariate Polynomial Ring in t
 over Finite Field of size 3
>>> f(R) == K
True
```

gen(*i=0*)

Return the `i`-th generator of `self`.

EXAMPLES:

```
sage: R = Frac(PolynomialRing(QQ,'z',10)); R
Fraction Field of Multivariate Polynomial Ring
 in z0, z1, z2, z3, z4, z5, z6, z7, z8, z9 over Rational Field
sage: R.0
z0
sage: R.gen(3)
z3
sage: R.3
z3
```

```
>>> from sage.all import *
>>> R = Frac(PolynomialRing(QQ,'z',Integer(10))); R
Fraction Field of Multivariate Polynomial Ring
 in z0, z1, z2, z3, z4, z5, z6, z7, z8, z9 over Rational Field
>>> R.gen(0)
z0
>>> R.gen(Integer(3))
z3
>>> R.gen(3)
z3
```

is_exact()

Return if `self` is exact which is if the underlying ring is exact.

EXAMPLES:

```
sage: Frac(ZZ['x']).is_exact()
True
sage: Frac(CDF['x']).is_exact()                                                 #␣
↪needs sage.rings.complex_double
False
```

```
>>> from sage.all import *
>>> Frac(ZZ['x']).is_exact()
True
>>> Frac(CDF['x']).is_exact()                                                   #␣
↪needs sage.rings.complex_double
False
```

is_field(*proof=True*)

Return `True`, since the fraction field is a field.

EXAMPLES:

```
sage: Frac(ZZ).is_field()
True
```

```
>>> from sage.all import *
>>> Frac(ZZ).is_field()
True
```

is_finite()

Tells whether this fraction field is finite.

> **Note**
>
> A fraction field is finite if and only if the associated integral domain is finite.

EXAMPLES:

```
sage: Frac(QQ['a','b','c']).is_finite()
False
```

```
>>> from sage.all import *
>>> Frac(QQ['a','b','c']).is_finite()
False
```

ngens()

This is the same as for the parent object.

EXAMPLES:

```
sage: R = Frac(PolynomialRing(QQ,'z',10)); R
Fraction Field of Multivariate Polynomial Ring
 in z0, z1, z2, z3, z4, z5, z6, z7, z8, z9 over Rational Field
sage: R.ngens()
10
```

```
>>> from sage.all import *
>>> R = Frac(PolynomialRing(QQ,'z',Integer(10))); R
Fraction Field of Multivariate Polynomial Ring
 in z0, z1, z2, z3, z4, z5, z6, z7, z8, z9 over Rational Field
>>> R.ngens()
10
```

random_element(**args*, ***kwds*)

Return a random element in this fraction field.

The arguments are passed to the random generator of the underlying ring.

EXAMPLES:

```
sage: F = ZZ['x'].fraction_field()
sage: F.random_element()  # random
(2*x - 8)/(-x^2 + x)
```

```
>>> from sage.all import *
>>> F = ZZ['x'].fraction_field()
>>> F.random_element()  # random
(2*x - 8)/(-x^2 + x)
```

```
sage: f = F.random_element(degree=5)
sage: f.numerator().degree() == f.denominator().degree()
True
sage: f.denominator().degree() <= 5
True
sage: while f.numerator().degree() != 5:
....:     f = F.random_element(degree=5)
```

```
>>> from sage.all import *
>>> f = F.random_element(degree=Integer(5))
>>> f.numerator().degree() == f.denominator().degree()
True
>>> f.denominator().degree() <= Integer(5)
True
>>> while f.numerator().degree() != Integer(5):
...     f = F.random_element(degree=Integer(5))
```

ring()

Return the ring that this is the fraction field of.

EXAMPLES:

```
sage: R = Frac(QQ['x,y'])
sage: R
Fraction Field of Multivariate Polynomial Ring in x, y over Rational Field
sage: R.ring()
Multivariate Polynomial Ring in x, y over Rational Field
```

```
>>> from sage.all import *
>>> R = Frac(QQ['x,y'])
```

(continues on next page)

(continued from previous page)

```
>>> R
Fraction Field of Multivariate Polynomial Ring in x, y over Rational Field
>>> R.ring()
Multivariate Polynomial Ring in x, y over Rational Field
```

some_elements()

Return some elements in this field.

EXAMPLES:

```
sage: R.<x> = QQ[]
sage: R.fraction_field().some_elements()
[0,
 1,
 x,
 2*x,
 x/(x^2 + 2*x + 1),
 1/x^2,
 ...
 (2*x^2 + 2)/(x^2 + 2*x + 1),
 (2*x^2 + 2)/x^3,
 (2*x^2 + 2)/(x^2 - 1),
 2]
```

```
>>> from sage.all import *
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> R.fraction_field().some_elements()
[0,
 1,
 x,
 2*x,
 x/(x^2 + 2*x + 1),
 1/x^2,
 ...
 (2*x^2 + 2)/(x^2 + 2*x + 1),
 (2*x^2 + 2)/x^3,
 (2*x^2 + 2)/(x^2 - 1),
 2]
```

variable_names_recursive(*depth=+Infinity*)

Return the variable names of the underlying ring recursively.

INPUT:

- `depth` – integer or *Infinity*

EXAMPLES:

```
sage: R = QQ['x']['y']
sage: K = R.fraction_field()
sage: K.variable_names_recursive()
('x', 'y')
sage: K.variable_names_recursive(1)
('y',)
```

```
>>> from sage.all import *
>>> R = QQ['x']['y']
>>> K = R.fraction_field()
>>> K.variable_names_recursive()
('x', 'y')
>>> K.variable_names_recursive(Integer(1))
('y',)
```

5.2 Fraction Field Elements

AUTHORS:

- William Stein (input from David Joyner, David Kohel, and Joe Wetherell)
- Sebastian Pancratz (2010-01-06): Rewrite of addition, multiplication and derivative to use Henrici's algorithms [Hor1972]

class sage.rings.fraction_field_element.**FractionFieldElement**

Bases: FieldElement

EXAMPLES:

```
sage: K = FractionField(PolynomialRing(QQ, 'x'))
sage: K
Fraction Field of Univariate Polynomial Ring in x over Rational Field
sage: loads(K.dumps()) == K
True
sage: x = K.gen()
sage: f = (x^3 + x)/(17 - x^19); f
(-x^3 - x)/(x^19 - 17)
sage: loads(f.dumps()) == f
True
```

```
>>> from sage.all import *
>>> K = FractionField(PolynomialRing(QQ, 'x'))
>>> K
Fraction Field of Univariate Polynomial Ring in x over Rational Field
>>> loads(K.dumps()) == K
True
>>> x = K.gen()
>>> f = (x**Integer(3) + x)/(Integer(17) - x**Integer(19)); f
(-x^3 - x)/(x^19 - 17)
>>> loads(f.dumps()) == f
True
```

denominator()

Return the denominator of `self`.

EXAMPLES:

```
sage: R.<x,y> = ZZ[]
sage: f = x/y + 1; f
(x + y)/y
```

(continues on next page)

(continued from previous page)

```
sage: f.denominator()
y
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> f = x/y + Integer(1); f
(x + y)/y
>>> f.denominator()
y
```

is_one()

Return `True` if this element is equal to one.

EXAMPLES:

```
sage: F = ZZ['x,y'].fraction_field()
sage: x,y = F.gens()
sage: (x/x).is_one()
True
sage: (x/y).is_one()
False
```

```
>>> from sage.all import *
>>> F = ZZ['x,y'].fraction_field()
>>> x,y = F.gens()
>>> (x/x).is_one()
True
>>> (x/y).is_one()
False
```

is_square(*root=False*)

Return whether or not `self` is a perfect square.

If the optional argument `root` is `True`, then also returns a square root (or `None`, if the fraction field element is not square).

INPUT:

- `root` – whether or not to also return a square root (default: `False`)

OUTPUT:

- boolean; whether or not a square
- object (optional); an actual square root if found, and `None` otherwise

EXAMPLES:

```
sage: R.<t> = QQ[]
sage: (1/t).is_square()
False
sage: (1/t^6).is_square()
True
sage: ((1+t)^4/t^6).is_square()
True
```

(continues on next page)

(continued from previous page)

```
sage: (4*(1+t)^4/t^6).is_square()
True
sage: (2*(1+t)^4/t^6).is_square()
False
sage: ((1+t)/t^6).is_square()
False

sage: (4*(1+t)^4/t^6).is_square(root=True)
(True, (2*t^2 + 4*t + 2)/t^3)
sage: (2*(1+t)^4/t^6).is_square(root=True)
(False, None)

sage: R.<x> = QQ[]
sage: a = 2*(x+1)^2 / (2*(x-1)^2); a
(x^2 + 2*x + 1)/(x^2 - 2*x + 1)
sage: a.is_square()
True
sage: (0/x).is_square()
True
```

```
>>> from sage.all import *
>>> R = QQ['t']; (t,) = R._first_ngens(1)
>>> (Integer(1)/t).is_square()
False
>>> (Integer(1)/t**Integer(6)).is_square()
True
>>> ((Integer(1)+t)**Integer(4)/t**Integer(6)).is_square()
True
>>> (Integer(4)*(Integer(1)+t)**Integer(4)/t**Integer(6)).is_square()
True
>>> (Integer(2)*(Integer(1)+t)**Integer(4)/t**Integer(6)).is_square()
False
>>> ((Integer(1)+t)/t**Integer(6)).is_square()
False

>>> (Integer(4)*(Integer(1)+t)**Integer(4)/t**Integer(6)).is_square(root=True)
(True, (2*t^2 + 4*t + 2)/t^3)
>>> (Integer(2)*(Integer(1)+t)**Integer(4)/t**Integer(6)).is_square(root=True)
(False, None)

>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> a = Integer(2)*(x+Integer(1))**Integer(2) / (Integer(2)*(x-
↪Integer(1))**Integer(2)); a
(x^2 + 2*x + 1)/(x^2 - 2*x + 1)
>>> a.is_square()
True
>>> (Integer(0)/x).is_square()
True
```

is_zero()

Return `True` if this element is equal to zero.

EXAMPLES:

```
sage: F = ZZ['x,y'].fraction_field()
sage: x,y = F.gens()
sage: t = F(0)/x
sage: t.is_zero()
True
sage: u = 1/x - 1/x
sage: u.is_zero()
True
sage: u.parent() is F
True
```

```
>>> from sage.all import *
>>> F = ZZ['x,y'].fraction_field()
>>> x,y = F.gens()
>>> t = F(Integer(0))/x
>>> t.is_zero()
True
>>> u = Integer(1)/x - Integer(1)/x
>>> u.is_zero()
True
>>> u.parent() is F
True
```

nth_root(*n*)

Return a n-th root of this element.

EXAMPLES:

```
sage: R = QQ['t'].fraction_field()
sage: t = R.gen()
sage: p = (t+1)^3 / (t^2+t-1)^3
sage: p.nth_root(3)
(t + 1)/(t^2 + t - 1)

sage: p = (t+1) / (t-1)
sage: p.nth_root(2)
Traceback (most recent call last):
...
ValueError: not a 2nd power
```

```
>>> from sage.all import *
>>> R = QQ['t'].fraction_field()
>>> t = R.gen()
>>> p = (t+Integer(1))**Integer(3) / (t**Integer(2)+t-Integer(1))**Integer(3)
>>> p.nth_root(Integer(3))
(t + 1)/(t^2 + t - 1)

>>> p = (t+Integer(1)) / (t-Integer(1))
>>> p.nth_root(Integer(2))
Traceback (most recent call last):
...
ValueError: not a 2nd power
```

numerator()

Return the numerator of `self`.

EXAMPLES:

```
sage: R.<x,y> = ZZ[]
sage: f = x/y + 1; f
(x + y)/y
sage: f.numerator()
x + y
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> f = x/y + Integer(1); f
(x + y)/y
>>> f.numerator()
x + y
```

reduce()

Reduce this fraction.

Divides out the gcd of the numerator and denominator. If the denominator becomes a unit, it becomes 1. Additionally, depending on the base ring, the leading coefficients of the numerator and the denominator may be normalized to 1.

Automatically called for exact rings, but because it may be numerically unstable for inexact rings it must be called manually in that case.

EXAMPLES:

```
sage: R.<x> = RealField(10)[]                                                #␣
↪needs sage.rings.real_mpfr
sage: f = (x^2+2*x+1)/(x+1); f                                               #␣
↪needs sage.rings.real_mpfr
(x^2 + 2.0*x + 1.0)/(x + 1.0)
sage: f.reduce(); f                                                          #␣
↪needs sage.rings.real_mpfr
x + 1.0
```

```
>>> from sage.all import *
>>> R = RealField(Integer(10))['x']; (x,) = R._first_ngens(1)# needs sage.
↪rings.real_mpfr
>>> f = (x**Integer(2)+Integer(2)*x+Integer(1))/(x+Integer(1)); f           ␣
↪                                  # needs sage.rings.real_mpfr
(x^2 + 2.0*x + 1.0)/(x + 1.0)
>>> f.reduce(); f                                                          #␣
↪needs sage.rings.real_mpfr
x + 1.0
```

specialization(*D=None*, *phi=None*)

Return the specialization of a fraction element of a polynomial ring.

subs(*in_dict=None*, **args*, ***kwds*)

Substitute variables in the numerator and denominator of `self`.

If a dictionary is passed, the keys are mapped to generators of the parent ring. Otherwise, the arguments are transmitted unchanged to the method `subs` of the numerator and the denominator.

EXAMPLES:

```
sage: x, y = PolynomialRing(ZZ, 2, 'xy').gens()
sage: f = x^2 + y + x^2*y^2 + 5
sage: (1/f).subs(x=5)
1/(25*y^2 + y + 30)
```

```
>>> from sage.all import *
>>> x, y = PolynomialRing(ZZ, Integer(2), 'xy').gens()
>>> f = x**Integer(2) + y + x**Integer(2)*y**Integer(2) + Integer(5)
>>> (Integer(1)/f).subs(x=Integer(5))
1/(25*y^2 + y + 30)
```

valuation(*v=None*)

Return the valuation of `self`, assuming that the numerator and denominator have valuation functions defined on them.

EXAMPLES:

```
sage: x = PolynomialRing(RationalField(),'x').gen()
sage: f = (x^3 + x)/(x^2 - 2*x^3)
sage: f
(-1/2*x^2 - 1/2)/(x^2 - 1/2*x)
sage: f.valuation()
-1
sage: f.valuation(x^2 + 1)
1
```

```
>>> from sage.all import *
>>> x = PolynomialRing(RationalField(),'x').gen()
>>> f = (x**Integer(3) + x)/(x**Integer(2) - Integer(2)*x**Integer(3))
>>> f
(-1/2*x^2 - 1/2)/(x^2 - 1/2*x)
>>> f.valuation()
-1
>>> f.valuation(x**Integer(2) + Integer(1))
1
```

class sage.rings.fraction_field_element.**FractionFieldElement_1poly_field**

Bases: *FractionFieldElement*

A fraction field element where the parent is the fraction field of a univariate polynomial ring over a field.

Many of the functions here are included for coherence with number fields.

is_integral()

Return whether this element is actually a polynomial.

EXAMPLES:

```
sage: R.<t> = QQ[]
sage: elt = (t^2 + t - 2) / (t + 2); elt # == (t + 2)*(t - 1)/(t + 2)
```

(continues on next page)

(continued from previous page)

```
t - 1
sage: elt.is_integral()
True
sage: elt = (t^2 - t) / (t+2); elt # == t*(t - 1)/(t + 2)
(t^2 - t)/(t + 2)
sage: elt.is_integral()
False
```

```
>>> from sage.all import *
>>> R = QQ['t']; (t,) = R._first_ngens(1)
>>> elt = (t**Integer(2) + t - Integer(2)) / (t + Integer(2)); elt # == (t +␣
↪2)*(t - 1)/(t + 2)
t - 1
>>> elt.is_integral()
True
>>> elt = (t**Integer(2) - t) / (t+Integer(2)); elt # == t*(t - 1)/(t + 2)
(t^2 - t)/(t + 2)
>>> elt.is_integral()
False
```

reduce()

Pick a normalized representation of `self`.

In particular, for any a == b, after normalization they will have the same numerator and denominator.

EXAMPLES:

For univariate rational functions over a field, we have:

```
sage: R.<x> = QQ[]
sage: (2 + 2*x) / (4*x) # indirect doctest
(1/2*x + 1/2)/x
```

```
>>> from sage.all import *
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> (Integer(2) + Integer(2)*x) / (Integer(4)*x) # indirect doctest
(1/2*x + 1/2)/x
```

Compare with:

```
sage: R.<x> = ZZ[]
sage: (2 + 2*x) / (4*x)
(x + 1)/(2*x)
```

```
>>> from sage.all import *
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> (Integer(2) + Integer(2)*x) / (Integer(4)*x)
(x + 1)/(2*x)
```

support()

Return a sorted list of primes dividing either the numerator or denominator of this element.

EXAMPLES:

```
sage: R.<t> = QQ[]
sage: h = (t^14 + 2*t^12 - 4*t^11 - 8*t^9 + 6*t^8 + 12*t^6 - 4*t^5
....:       - 8*t^3 + t^2 + 2)/(t^6 + 6*t^5 + 9*t^4 - 2*t^2 - 12*t - 18)
sage: h.support()                                                           #␣
↪needs sage.libs.pari
[t - 1, t + 3, t^2 + 2, t^2 + t + 1, t^4 - 2]
```

```
>>> from sage.all import *
>>> R = QQ['t']; (t,) = R._first_ngens(1)
>>> h = (t**Integer(14) + Integer(2)*t**Integer(12) -␣
↪Integer(4)*t**Integer(11) - Integer(8)*t**Integer(9) +␣
↪Integer(6)*t**Integer(8) + Integer(12)*t**Integer(6) -␣
↪Integer(4)*t**Integer(5)
...       - Integer(8)*t**Integer(3) + t**Integer(2) + Integer(2))/
↪(t**Integer(6) + Integer(6)*t**Integer(5) + Integer(9)*t**Integer(4) -␣
↪Integer(2)*t**Integer(2) - Integer(12)*t - Integer(18))
>>> h.support()                                                           #␣
↪needs sage.libs.pari
[t - 1, t + 3, t^2 + 2, t^2 + t + 1, t^4 - 2]
```

sage.rings.fraction_field_element.**make_element**(*parent*, *numerator*, *denominator*)

Used for unpickling *FractionFieldElement* objects (and subclasses).

EXAMPLES:

```
sage: from sage.rings.fraction_field_element import make_element
sage: R = ZZ['x,y']
sage: x,y = R.gens()
sage: F = R.fraction_field()
sage: make_element(F, 1 + x, 1 + y)
(x + 1)/(y + 1)
```

```
>>> from sage.all import *
>>> from sage.rings.fraction_field_element import make_element
>>> R = ZZ['x,y']
>>> x,y = R.gens()
>>> F = R.fraction_field()
>>> make_element(F, Integer(1) + x, Integer(1) + y)
(x + 1)/(y + 1)
```

sage.rings.fraction_field_element.**make_element_old**(*parent*, *cdict*)

Used for unpickling old *FractionFieldElement* pickles.

EXAMPLES:

```
sage: from sage.rings.fraction_field_element import make_element_old
sage: R.<x,y> = ZZ[]
sage: F = R.fraction_field()
sage: make_element_old(F, {'_FractionFieldElement__numerator': x + y,
....:                      '_FractionFieldElement__denominator': x - y})
(x + y)/(x - y)
```

```
>>> from sage.all import *
>>> from sage.rings.fraction_field_element import make_element_old
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> F = R.fraction_field()
>>> make_element_old(F, {'_FractionFieldElement__numerator': x + y,
...                      '_FractionFieldElement__denominator': x - y})
(x + y)/(x - y)
```

CHAPTER

SIX

LOCALIZATION

6.1 Localization

Localization is an important ring construction tool. Whenever you have to extend a given integral domain such that it contains the inverses of a finite set of elements but should allow non injective homomorphic images this construction will be needed. See the example on Ariki-Koike algebras below for such an application.

EXAMPLES:

```
sage: LZ = Localization(ZZ, (5,11))
sage: m = matrix(LZ, [[5, 7], [0,11]])
sage: m.parent()
Full MatrixSpace of 2 by 2 dense matrices over Integer Ring localized at (5, 11)
sage: ~m      # parent of inverse is different: see documentation of m.__invert__
[  1/5 -7/55]
[    0  1/11]
sage: _.parent()
Full MatrixSpace of 2 by 2 dense matrices over Rational Field
sage: mi = matrix(LZ, ~m)
sage: mi.parent()
Full MatrixSpace of 2 by 2 dense matrices over Integer Ring localized at (5, 11)
sage: mi == ~m
True
```

```
>>> from sage.all import *
>>> LZ = Localization(ZZ, (Integer(5),Integer(11)))
>>> m = matrix(LZ, [[Integer(5), Integer(7)], [Integer(0),Integer(11)]])
>>> m.parent()
Full MatrixSpace of 2 by 2 dense matrices over Integer Ring localized at (5, 11)
>>> ~m      # parent of inverse is different: see documentation of m.__invert__
[  1/5 -7/55]
[    0  1/11]
>>> _.parent()
Full MatrixSpace of 2 by 2 dense matrices over Rational Field
>>> mi = matrix(LZ, ~m)
>>> mi.parent()
Full MatrixSpace of 2 by 2 dense matrices over Integer Ring localized at (5, 11)
>>> mi == ~m
True
```

The next example defines the most general ring containing the coefficients of the irreducible representations of the Ariki-Koike algebra corresponding to the three colored permutations on three elements:

```
sage: R.<u0, u1, u2, q> = ZZ[]
sage: u = [u0, u1, u2]
sage: S = Set(u)
sage: I = S.cartesian_product(S)
sage: add_units = u + [q, q + 1] + [ui - uj for ui, uj in I if ui != uj]
sage: add_units += [q*ui - uj for ui, uj in I if ui != uj]
sage: L = R.localization(tuple(add_units)); L
Multivariate Polynomial Ring in u0, u1, u2, q over Integer Ring localized at
(q, q + 1, u2, u1 - u2, u1, u0 - u1, u0 - u2, u0, u2*q - u0, u2*q - u1, u1*q - u0,
 u1*q - u2, u0*q - u1, u0*q - u2)
```

```
>>> from sage.all import *
>>> R = ZZ['u0, u1, u2, q']; (u0, u1, u2, q,) = R._first_ngens(4)
>>> u = [u0, u1, u2]
>>> S = Set(u)
>>> I = S.cartesian_product(S)
>>> add_units = u + [q, q + Integer(1)] + [ui - uj for ui, uj in I if ui != uj]
>>> add_units += [q*ui - uj for ui, uj in I if ui != uj]
>>> L = R.localization(tuple(add_units)); L
Multivariate Polynomial Ring in u0, u1, u2, q over Integer Ring localized at
(q, q + 1, u2, u1 - u2, u1, u0 - u1, u0 - u2, u0, u2*q - u0, u2*q - u1, u1*q - u0,
 u1*q - u2, u0*q - u1, u0*q - u2)
```

Define the representation matrices (of one of the three dimensional irreducible representations):

```
sage: m1 = matrix(L, [[u1, 0, 0], [0, u0, 0], [0, 0, u0]])
sage: m2 = matrix(L, [[(u0*q - u0)/(u0 - u1), (u0*q - u1)/(u0 - u1), 0],
....:                 [(-u1*q + u0)/(u0 - u1), (-u1*q + u1)/(u0 - u1), 0],
....:                 [0, 0, -1]])
sage: m3 = matrix(L, [[-1, 0, 0],
....:                 [0, u0*(1 - q)/(u1*q - u0), q*(u1 - u0)/(u1*q - u0)],
....:                 [0, (u1*q^2 - u0)/(u1*q - u0), (u1*q^ 2 - u1*q)/(u1*q - u0)]])
sage: m1.base_ring() == L
True
```

```
>>> from sage.all import *
>>> m1 = matrix(L, [[u1, Integer(0), Integer(0)], [Integer(0), u0, Integer(0)],␣
↪[Integer(0), Integer(0), u0]])
>>> m2 = matrix(L, [[(u0*q - u0)/(u0 - u1), (u0*q - u1)/(u0 - u1), Integer(0)],
...                 [(-u1*q + u0)/(u0 - u1), (-u1*q + u1)/(u0 - u1), Integer(0)],
...                 [Integer(0), Integer(0), -Integer(1)]])
>>> m3 = matrix(L, [[-Integer(1), Integer(0), Integer(0)],
...                 [Integer(0), u0*(Integer(1) - q)/(u1*q - u0), q*(u1 - u0)/(u1*q -␣
↪u0)],
...                 [Integer(0), (u1*q**Integer(2) - u0)/(u1*q - u0), (u1*q**␣
↪Integer(2) - u1*q)/(u1*q - u0)]])
>>> m1.base_ring() == L
True
```

Check relations of the Ariki-Koike algebra:

```
sage: m1*m2*m1*m2 == m2*m1*m2*m1
True
```

(continues on next page)

(continued from previous page)

```
sage: m2*m3*m2 == m3*m2*m3
True
sage: m1*m3 == m3*m1
True
sage: m1**3 - (u0+u1+u2)*m1**2 + (u0*u1+u0*u2+u1*u2)*m1 - u0*u1*u2 == 0
True
sage: m2**2 - (q-1)*m2 - q == 0
True
sage: m3**2 - (q-1)*m3 - q == 0
True
sage: ~m1 in m1.parent()
True
sage: ~m2 in m2.parent()
True
sage: ~m3 in m3.parent()
True
```

```
>>> from sage.all import *
>>> m1*m2*m1*m2 == m2*m1*m2*m1
True
>>> m2*m3*m2 == m3*m2*m3
True
>>> m1*m3 == m3*m1
True
>>> m1**Integer(3) - (u0+u1+u2)*m1**Integer(2) + (u0*u1+u0*u2+u1*u2)*m1 - u0*u1*u2 ==␣
↪Integer(0)
True
>>> m2**Integer(2) - (q-Integer(1))*m2 - q == Integer(0)
True
>>> m3**Integer(2) - (q-Integer(1))*m3 - q == Integer(0)
True
>>> ~m1 in m1.parent()
True
>>> ~m2 in m2.parent()
True
>>> ~m3 in m3.parent()
True
```

Obtain specializations in positive characteristic:

```
sage: Fp = GF(17)
sage: f = L.hom((3,5,7,11), codomain=Fp); f
Ring morphism:
  From: Multivariate Polynomial Ring in u0, u1, u2, q over Integer Ring localized at
        (q, q + 1, u2, u1 - u2, u1, u0 - u1, u0 - u2, u0, u2*q - u0, u2*q - u1,
         u1*q - u0, u1*q - u2, u0*q - u1, u0*q - u2)
  To:   Finite Field of size 17
  Defn: u0 |--> 3
        u1 |--> 5
        u2 |--> 7
        q |--> 11
sage: mFp1 = matrix({k: f(v) for k, v in m1.dict().items()}); mFp1
```

(continues on next page)

(continued from previous page)

```
[5 0 0]
[0 3 0]
[0 0 3]
sage: mFp1.base_ring()
Finite Field of size 17
sage: mFp2 = matrix({k: f(v) for k, v in m2.dict().items()}); mFp2
[ 2  3  0]
[ 9  8  0]
[ 0  0 16]
sage: mFp3 = matrix({k: f(v) for k, v in m3.dict().items()}); mFp3
[16  0  0]
[ 0  4  5]
[ 0  7  6]
```

```
>>> from sage.all import *
>>> Fp = GF(Integer(17))
>>> f = L.hom((Integer(3),Integer(5),Integer(7),Integer(11)), codomain=Fp); f
Ring morphism:
  From: Multivariate Polynomial Ring in u0, u1, u2, q over Integer Ring localized at
        (q, q + 1, u2, u1 - u2, u1, u0 - u1, u0 - u2, u0, u2*q - u0, u2*q - u1,
         u1*q - u0, u1*q - u2, u0*q - u1, u0*q - u2)
  To:   Finite Field of size 17
  Defn: u0 |--> 3
        u1 |--> 5
        u2 |--> 7
        q |--> 11
>>> mFp1 = matrix({k: f(v) for k, v in m1.dict().items()}); mFp1
[5 0 0]
[0 3 0]
[0 0 3]
>>> mFp1.base_ring()
Finite Field of size 17
>>> mFp2 = matrix({k: f(v) for k, v in m2.dict().items()}); mFp2
[ 2  3  0]
[ 9  8  0]
[ 0  0 16]
>>> mFp3 = matrix({k: f(v) for k, v in m3.dict().items()}); mFp3
[16  0  0]
[ 0  4  5]
[ 0  7  6]
```

Obtain specializations in characteristic 0:

```
sage: fQ = L.hom((3,5,7,11), codomain=QQ); fQ
Ring morphism:
  From: Multivariate Polynomial Ring in u0, u1, u2, q over Integer Ring
        localized at (q, q + 1, u2, u1 - u2, u1, u0 - u1, u0 - u2, u0, u2*q - u0,
        u2*q - u1, u1*q - u0, u1*q - u2, u0*q - u1, u0*q - u2)
  To:   Rational Field
  Defn: u0 |--> 3
        u1 |--> 5
        u2 |--> 7
```

(continues on next page)

(continued from previous page)

```
        q |--> 11

sage: mQ1 = matrix({k: fQ(v) for k, v in m1.dict().items()}); mQ1
[5 0 0]
[0 3 0]
[0 0 3]
sage: mQ1.base_ring()
Rational Field
sage: mQ2 = matrix({k: fQ(v) for k, v in m2.dict().items()}); mQ2
[-15 -14   0]
[ 26  25   0]
[  0   0  -1]
sage: mQ3 = matrix({k: fQ(v) for k, v in m3.dict().items()}); mQ3
[    -1      0      0]
[     0 -15/26  11/26]
[     0 301/26 275/26]

sage: S.<x, y, z, t> = QQ[]
sage: T = S.quo(x + y + z)
sage: F = T.fraction_field()
sage: fF = L.hom((x, y, z, t), codomain=F); fF
Ring morphism:
  From: Multivariate Polynomial Ring in u0, u1, u2, q over Integer Ring
        localized at (q, q + 1, u2, u1 - u2, u1, u0 - u1, u0 - u2, u0, u2*q - u0,
        u2*q - u1, u1*q - u0, u1*q - u2, u0*q - u1, u0*q - u2)
  To:   Fraction Field of Quotient of Multivariate Polynomial Ring in x, y, z, t
        over Rational Field by the ideal (x + y + z)
  Defn: u0 |--> -ybar - zbar
        u1 |--> ybar
        u2 |--> zbar
        q |--> tbar
sage: mF1 = matrix({k: fF(v) for k, v in m1.dict().items()}); mF1
[        ybar           0           0]
[           0 -ybar - zbar           0]
[           0           0 -ybar - zbar]
sage: mF1.base_ring() == F
True
```

```
>>> from sage.all import *
>>> fQ = L.hom((Integer(3),Integer(5),Integer(7),Integer(11)), codomain=QQ); fQ
Ring morphism:
  From: Multivariate Polynomial Ring in u0, u1, u2, q over Integer Ring
        localized at (q, q + 1, u2, u1 - u2, u1, u0 - u1, u0 - u2, u0, u2*q - u0,
        u2*q - u1, u1*q - u0, u1*q - u2, u0*q - u1, u0*q - u2)
  To:   Rational Field
  Defn: u0 |--> 3
        u1 |--> 5
        u2 |--> 7
        q |--> 11

>>> mQ1 = matrix({k: fQ(v) for k, v in m1.dict().items()}); mQ1
[5 0 0]
```

(continues on next page)

(continued from previous page)

```
[0 3 0]
[0 0 3]
>>> mQ1.base_ring()
Rational Field
>>> mQ2 = matrix({k: fQ(v) for k, v in m2.dict().items()}); mQ2
[-15 -14   0]
[ 26  25   0]
[  0   0  -1]
>>> mQ3 = matrix({k: fQ(v) for k, v in m3.dict().items()}); mQ3
[    -1      0      0]
[     0 -15/26  11/26]
[     0 301/26 275/26]

>>> S = QQ['x, y, z, t']; (x, y, z, t,) = S._first_ngens(4)
>>> T = S.quo(x + y + z)
>>> F = T.fraction_field()
>>> fF = L.hom((x, y, z, t), codomain=F); fF
Ring morphism:
  From: Multivariate Polynomial Ring in u0, u1, u2, q over Integer Ring
        localized at (q, q + 1, u2, u1 - u2, u1, u0 - u1, u0 - u2, u0, u2*q - u0,
        u2*q - u1, u1*q - u0, u1*q - u2, u0*q - u1, u0*q - u2)
  To:   Fraction Field of Quotient of Multivariate Polynomial Ring in x, y, z, t
        over Rational Field by the ideal (x + y + z)
  Defn: u0 |--> -ybar - zbar
        u1 |--> ybar
        u2 |--> zbar
        q |--> tbar
>>> mF1 = matrix({k: fF(v) for k, v in m1.dict().items()}); mF1
[        ybar            0            0]
[           0 -ybar - zbar            0]
[           0            0 -ybar - zbar]
>>> mF1.base_ring() == F
True
```

AUTHORS:

- Sebastian Oehms 2019-12-09: initial version.
- Sebastian Oehms 2022-03-05: fix some corner cases and add `factor()` (Issue #33463)

class sage.rings.localization.**Localization**(*base_ring*, *extra_units*, *names=None*, *normalize=True*, *category=None*, *warning=True*)

Bases: `Parent`, `UniqueRepresentation`

The localization generalizes the construction of the field of fractions of an integral domain to an arbitrary ring. Given a (not necessarily commutative) ring R and a subset S of R, there exists a ring $R[S^{-1}]$ together with the ring homomorphism $R \longrightarrow R[S^{-1}]$ that "inverts" S; that is, the homomorphism maps elements in S to unit elements in $R[S^{-1}]$ and, moreover, any ring homomorphism from R that "inverts" S uniquely factors through $R[S^{-1}]$.

The ring $R[S^{-1}]$ is called the *localization* of R with respect to S. For example, if R is a commutative ring and f an element in R, then the localization consists of elements of the form $r/f, r \in R, n \geq 0$ (to be precise, $R[f^{-1}] = R[t]/(ft-1)$).

The above text is taken from $Wikipedia$. The construction here used for this class relies on the construction of

the field of fraction and is therefore restricted to integral domains.

Accordingly, the base ring must be in the category of `IntegralDomains`. Furthermore, the base ring should support `sage.structure.element.CommutativeRingElement.divides()` and the exact division operator `//` (`sage.structure.element.Element.__floordiv__()`) in order to guarantee a successful application.

INPUT:

- `base_ring` – a ring in the category of `IntegralDomains`
- `extra_units` – tuple of elements of `base_ring` which should be turned into units
- `category` – (default: `None`) passed to `Parent`
- `warning` – boolean (default: `True`); to suppress a warning which is thrown if `self` cannot be represented uniquely

REFERENCES:

- Wikipedia article Ring_(mathematics)#Localization

EXAMPLES:

```
sage: L = Localization(ZZ, (3,5))
sage: 1/45 in L
True
sage: 1/43 in L
False

sage: Localization(L, (7,11))
Integer Ring localized at (3, 5, 7, 11)
sage: _.is_subring(QQ)
True

sage: L(~7)
Traceback (most recent call last):
...
ValueError: factor 7 of denominator is not a unit

sage: Localization(Zp(7), (3, 5))                                              #␣
↪needs sage.rings.padics
Traceback (most recent call last):
...
ValueError: all given elements are invertible in
7-adic Ring with capped relative precision 20

sage: R.<x> = ZZ[]
sage: L = R.localization(x**2 + 1)
sage: s = (x+5)/(x**2+1)
sage: s in L
True
sage: t = (x+5)/(x**2+2)
sage: t in L
False
sage: L(t)
Traceback (most recent call last):
...
TypeError: fraction must have unit denominator
```

(continues on next page)

(continued from previous page)

```
sage: L(s) in R
False
sage: y = L(x)
sage: g = L(s)
sage: g.parent()
Univariate Polynomial Ring in x over Integer Ring localized at (x^2 + 1,)
sage: f = (y+5)/(y**2+1); f
(x + 5)/(x^2 + 1)
sage: f == g
True
sage: (y+5)/(y**2+2)
Traceback (most recent call last):
...
ValueError: factor x^2 + 2 of denominator is not a unit

sage: Lau.<u, v> = LaurentPolynomialRing(ZZ)                                    #␣
↪needs sage.modules
sage: LauL = Lau.localization(u + 1)                                            #␣
↪needs sage.modules
sage: LauL(~u).parent()                                                         #␣
↪needs sage.modules
Multivariate Polynomial Ring in u, v over Integer Ring localized at (v, u, u + 1)
```

```
>>> from sage.all import *
>>> L = Localization(ZZ, (Integer(3),Integer(5)))
>>> Integer(1)/Integer(45) in L
True
>>> Integer(1)/Integer(43) in L
False

>>> Localization(L, (Integer(7),Integer(11)))
Integer Ring localized at (3, 5, 7, 11)
>>> _.is_subring(QQ)
True

>>> L(~Integer(7))
Traceback (most recent call last):
...
ValueError: factor 7 of denominator is not a unit

>>> Localization(Zp(Integer(7)), (Integer(3), Integer(5)))                      ␣
↪                       # needs sage.rings.padics
Traceback (most recent call last):
...
ValueError: all given elements are invertible in
7-adic Ring with capped relative precision 20

>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> L = R.localization(x**Integer(2) + Integer(1))
>>> s = (x+Integer(5))/(x**Integer(2)+Integer(1))
>>> s in L
True
```

(continues on next page)

(continued from previous page)

```
>>> t = (x+Integer(5))/(x**Integer(2)+Integer(2))
>>> t in L
False
>>> L(t)
Traceback (most recent call last):
...
TypeError: fraction must have unit denominator
>>> L(s) in R
False
>>> y = L(x)
>>> g = L(s)
>>> g.parent()
Univariate Polynomial Ring in x over Integer Ring localized at (x^2 + 1,)
>>> f = (y+Integer(5))/(y**Integer(2)+Integer(1)); f
(x + 5)/(x^2 + 1)
>>> f == g
True
>>> (y+Integer(5))/(y**Integer(2)+Integer(2))
Traceback (most recent call last):
...
ValueError: factor x^2 + 2 of denominator is not a unit

>>> Lau = LaurentPolynomialRing(ZZ, names=('u', 'v',)); (u, v,) = Lau._first_
↪ngens(2)# needs sage.modules
>>> LauL = Lau.localization(u + Integer(1))                                    ␣
↪     # needs sage.modules
>>> LauL(~u).parent()                                                         #␣
↪needs sage.modules
Multivariate Polynomial Ring in u, v over Integer Ring localized at (v, u, u + 1)
```

More examples will be shown typing `sage.rings.localization?`

Element

alias of *LocalizationElement*

characteristic()

Return the characteristic of `self`.

EXAMPLES:

```
sage: R.<a> = GF(5)[]
sage: L = R.localization((a**2 - 3, a))
sage: L.characteristic()
5
```

```
>>> from sage.all import *
>>> R = GF(Integer(5))['a']; (a,) = R._first_ngens(1)
>>> L = R.localization((a**Integer(2) - Integer(3), a))
>>> L.characteristic()
5
```

fraction_field()

Return the fraction field of `self`.

EXAMPLES:

```
sage: R.<a> = GF(5)[]
sage: L = Localization(R, (a**2 - 3, a))
sage: L.fraction_field()
Fraction Field of Univariate Polynomial Ring in a over Finite Field of size 5
sage: L.is_subring(_)
True
```

```
>>> from sage.all import *
>>> R = GF(Integer(5))['a']; (a,) = R._first_ngens(1)
>>> L = Localization(R, (a**Integer(2) - Integer(3), a))
>>> L.fraction_field()
Fraction Field of Univariate Polynomial Ring in a over Finite Field of size 5
>>> L.is_subring(_)
True
```

gen(*i*)

Return the `i`-th generator of `self` which is the `i`-th generator of the base ring.

EXAMPLES:

```
sage: R.<x, y> = ZZ[]
sage: R.localization((x**2 + 1, y - 1)).gen(0)
x

sage: ZZ.localization(2).gen(0)
1
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> R.localization((x**Integer(2) + Integer(1), y - Integer(1))).
↪gen(Integer(0))
x

>>> ZZ.localization(Integer(2)).gen(Integer(0))
1
```

gens()

Return a tuple whose entries are the generators for this object, in order.

EXAMPLES:

```
sage: R.<x, y> = ZZ[]
sage: Localization(R, (x**2 + 1, y - 1)).gens()
(x, y)

sage: Localization(ZZ, 2).gens()
(1,)
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> Localization(R, (x**Integer(2) + Integer(1), y - Integer(1))).gens()
```

(continues on next page)

(continued from previous page)

```
(x, y)

>>> Localization(ZZ, Integer(2)).gens()
(1,)
```

is_field(*proof=True*)

Return `True` if this ring is a field.

INPUT:

- `proof` – boolean (default: `True`); determines what to do in unknown cases

ALGORITHM:

If the parameter `proof` is set to `True`, the returned value is correct but the method might throw an error. Otherwise, if it is set to `False`, the method returns `True` if it can establish that `self` is a field and `False` otherwise.

EXAMPLES:

```
sage: R = ZZ.localization((2, 3))
sage: R.is_field()
False
```

```
>>> from sage.all import *
>>> R = ZZ.localization((Integer(2), Integer(3)))
>>> R.is_field()
False
```

krull_dimension()

Return the Krull dimension of this localization.

Since the current implementation just allows integral domains as base ring and localization at a finite set of elements the spectrum of `self` is open in the irreducible spectrum of its base ring. Therefore, by density we may take the dimension from there.

EXAMPLES:

```
sage: R = ZZ.localization((2, 3))
sage: R.krull_dimension()
1
```

```
>>> from sage.all import *
>>> R = ZZ.localization((Integer(2), Integer(3)))
>>> R.krull_dimension()
1
```

ngens()

Return the number of generators of `self` according to the same method for the base ring.

EXAMPLES:

```
sage: R.<x, y> = ZZ[]
sage: Localization(R, (x**2 + 1, y - 1)).ngens()
2
```

(continues on next page)

(continued from previous page)

```
sage: Localization(ZZ, 2).ngens()
1
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> Localization(R, (x**Integer(2) + Integer(1), y - Integer(1))).ngens()
2

>>> Localization(ZZ, Integer(2)).ngens()
1
```

class sage.rings.localization.**LocalizationElement**(*parent*, *x*)

Bases: IntegralDomainElement

Element class for localizations of integral domains.

INPUT:

- parent – instance of *Localization*
- x – instance of *FractionFieldElement* whose parent is the fraction field of the parent's base ring

EXAMPLES:

```
sage: from sage.rings.localization import LocalizationElement
sage: P.<x,y,z> = GF(5)[]
sage: L = P.localization((x, y*z - x))
sage: LocalizationElement(L, 4/(y*z-x)**2)
(-1)/(y^2*z^2 - 2*x*y*z + x^2)
sage: _.parent()
Multivariate Polynomial Ring in x, y, z over Finite Field of size 5
 localized at (x, y*z - x)
```

```
>>> from sage.all import *
>>> from sage.rings.localization import LocalizationElement
>>> P = GF(Integer(5))['x, y, z']; (x, y, z,) = P._first_ngens(3)
>>> L = P.localization((x, y*z - x))
>>> LocalizationElement(L, Integer(4)/(y*z-x)**Integer(2))
(-1)/(y^2*z^2 - 2*x*y*z + x^2)
>>> _.parent()
Multivariate Polynomial Ring in x, y, z over Finite Field of size 5
 localized at (x, y*z - x)
```

denominator()

Return the denominator of self.

EXAMPLES:

```
sage: L = Localization(ZZ, (3,5))
sage: L(7/15).denominator()
15
```

```
>>> from sage.all import *
>>> L = Localization(ZZ, (Integer(3),Integer(5)))
>>> L(Integer(7)/Integer(15)).denominator()
15
```

factor(*proof=None*)

Return the factorization of this polynomial.

INPUT:

- `proof` – (optional) if given it is passed to the corresponding method of the numerator of `self`

EXAMPLES:

```
sage: P.<X, Y> = QQ['x, y']
sage: L = P.localization(X - Y)
sage: x, y = L.gens()
sage: p = (x^2 - y^2)/(x-y)^2                                                  #␣
↪needs sage.libs.singular
sage: p.factor()                                                               #␣
↪needs sage.libs.singular
(1/(x - y)) * (x + y)
```

```
>>> from sage.all import *
>>> P = QQ['x, y']; (X, Y,) = P._first_ngens(2)
>>> L = P.localization(X - Y)
>>> x, y = L.gens()
>>> p = (x**Integer(2) - y**Integer(2))/(x-y)**Integer(2)                      ␣
↪                            # needs sage.libs.singular
>>> p.factor()                                                                 #␣
↪needs sage.libs.singular
(1/(x - y)) * (x + y)
```

inverse_of_unit()

Return the inverse of `self`.

EXAMPLES:

```
sage: P.<x,y,z> = ZZ[]
sage: L = Localization(P, x*y*z)
sage: L(x*y*z).inverse_of_unit()                                               #␣
↪needs sage.libs.singular
1/(x*y*z)
sage: L(z).inverse_of_unit()                                                   #␣
↪needs sage.libs.singular
1/z
```

```
>>> from sage.all import *
>>> P = ZZ['x, y, z']; (x, y, z,) = P._first_ngens(3)
>>> L = Localization(P, x*y*z)
>>> L(x*y*z).inverse_of_unit()                                                 #␣
↪needs sage.libs.singular
1/(x*y*z)
>>> L(z).inverse_of_unit()                                                     #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.libs.singular
1/z
```

is_unit()

Return `True` if `self` is a unit.

EXAMPLES:

```
sage: P.<x,y,z> = QQ[]
sage: L = P.localization((x, y*z))
sage: L(y*z).is_unit()
True
sage: L(z).is_unit()
True
sage: L(x*y*z).is_unit()
True
```

```
>>> from sage.all import *
>>> P = QQ['x, y, z']; (x, y, z,) = P._first_ngens(3)
>>> L = P.localization((x, y*z))
>>> L(y*z).is_unit()
True
>>> L(z).is_unit()
True
>>> L(x*y*z).is_unit()
True
```

numerator()

Return the numerator of `self`.

EXAMPLES:

```
sage: L = ZZ.localization((3,5))
sage: L(7/15).numerator()
7
```

```
>>> from sage.all import *
>>> L = ZZ.localization((Integer(3),Integer(5)))
>>> L(Integer(7)/Integer(15)).numerator()
7
```

sage.rings.localization.**normalize_extra_units**(*base_ring*, *add_units*, *warning=True*)

Function to normalize input data.

The given list will be replaced by a list of the involved prime factors (if possible).

INPUT:

- `base_ring` – a ring in the category of `IntegralDomains`
- `add_units` – list of elements from base ring
- `warning` – boolean (default: `True`); to suppress a warning which is thrown if no normalization was possible

OUTPUT: list of all prime factors of the elements of the given list

EXAMPLES:

```
sage: from sage.rings.localization import normalize_extra_units
sage: normalize_extra_units(ZZ, [3, -15, 45, 9, 2, 50])
[2, 3, 5]
sage: P.<x,y,z> = ZZ[]
sage: normalize_extra_units(P,                                                  #␣
↪needs sage.libs.pari
....:                       [3*x, z*y**2, 2*z, 18*(x*y*z)**2, x*z, 6*x*z, 5])
[2, 3, 5, z, y, x]
sage: P.<x,y,z> = QQ[]
sage: normalize_extra_units(P,                                                  #␣
↪needs sage.libs.pari
....:                       [3*x, z*y**2, 2*z, 18*(x*y*z)**2, x*z, 6*x*z, 5])
[z, y, x]

sage: R.<x, y> = ZZ[]
sage: Q.<a, b> = R.quo(x**2 - 5)
sage: p = b**2 - 5
sage: p == (b-a)*(b+a)
True
sage: normalize_extra_units(Q, [p])                                             #␣
↪needs sage.libs.pari
doctest:...: UserWarning: Localization may not be represented uniquely
[b^2 - 5]
sage: normalize_extra_units(Q, [p], warning=False)                              #␣
↪needs sage.libs.pari
[b^2 - 5]
```

```
>>> from sage.all import *
>>> from sage.rings.localization import normalize_extra_units
>>> normalize_extra_units(ZZ, [Integer(3), -Integer(15), Integer(45), Integer(9),␣
↪Integer(2), Integer(50)])
[2, 3, 5]
>>> P = ZZ['x, y, z']; (x, y, z,) = P._first_ngens(3)
>>> normalize_extra_units(P,                                                    #␣
↪needs sage.libs.pari
...                       [Integer(3)*x, z*y**Integer(2), Integer(2)*z,␣
↪Integer(18)*(x*y*z)**Integer(2), x*z, Integer(6)*x*z, Integer(5)])
[2, 3, 5, z, y, x]
>>> P = QQ['x, y, z']; (x, y, z,) = P._first_ngens(3)
>>> normalize_extra_units(P,                                                    #␣
↪needs sage.libs.pari
...                       [Integer(3)*x, z*y**Integer(2), Integer(2)*z,␣
↪Integer(18)*(x*y*z)**Integer(2), x*z, Integer(6)*x*z, Integer(5)])
[z, y, x]

>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> Q = R.quo(x**Integer(2) - Integer(5), names=('a', 'b',)); (a, b,) = Q._first_
↪ngens(2)
>>> p = b**Integer(2) - Integer(5)
>>> p == (b-a)*(b+a)
True
>>> normalize_extra_units(Q, [p])                                               #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.libs.pari
doctest:...: UserWarning: Localization may not be represented uniquely
[b^2 - 5]
>>> normalize_extra_units(Q, [p], warning=False)                              #␣
↪needs sage.libs.pari
[b^2 - 5]
```

CHAPTER

SEVEN

RING EXTENSIONS

7.1 Extension of rings

Sage offers the possibility to work with ring extensions L/K as actual parents and perform meaningful operations on them and their elements.

The simplest way to build an extension is to use the method `sage.categories.commutative_rings.CommutativeRings.ParentMethods.over()` on the top ring, that is L. For example, the following line constructs the extension of finite fields $\mathbf{F}_{5^4}/\mathbf{F}_{5^2}$:

```
sage: GF(5^4).over(GF(5^2))
Field in z4 with defining polynomial x^2 + (4*z2 + 3)*x + z2 over its base
```

```
>>> from sage.all import *
>>> GF(Integer(5)**Integer(4)).over(GF(Integer(5)**Integer(2)))
Field in z4 with defining polynomial x^2 + (4*z2 + 3)*x + z2 over its base
```

By default, Sage reuses the canonical generator of the top ring (here $z_4 \in \mathbf{F}_{5^4}$), together with its name. However, the user can customize them by passing in appropriate arguments:

```
sage: F = GF(5^2)
sage: k = GF(5^4)
sage: z4 = k.gen()
sage: K.<a> = k.over(F, gen=1-z4); K
Field in a with defining polynomial x^2 + z2*x + 4 over its base
```

```
>>> from sage.all import *
>>> F = GF(Integer(5)**Integer(2))
>>> k = GF(Integer(5)**Integer(4))
>>> z4 = k.gen()
>>> K = k.over(F, gen=Integer(1)-z4, names=('a',)); (a,) = K._first_ngens(1); K
Field in a with defining polynomial x^2 + z2*x + 4 over its base
```

The base of the extension is available via the method `base()` (or equivalently `base_ring`):

```
sage: K.base()
Finite Field in z2 of size 5^2
```

```
>>> from sage.all import *
>>> K.base()
Finite Field in z2 of size 5^2
```

It is also possible to build an extension on top of another extension, obtaining this way a tower of extensions:

```
sage: L.<b> = GF(5^8).over(K); L
Field in b with defining polynomial x^2 + (4*z2 + 3*a)*x + 1 - a over its base
sage: L.base()
Field in a with defining polynomial x^2 + z2*x + 4 over its base
sage: L.base().base()
Finite Field in z2 of size 5^2
```

```
>>> from sage.all import *
>>> L = GF(Integer(5)**Integer(8)).over(K, names=('b',)); (b,) = L._first_ngens(1); L
Field in b with defining polynomial x^2 + (4*z2 + 3*a)*x + 1 - a over its base
>>> L.base()
Field in a with defining polynomial x^2 + z2*x + 4 over its base
>>> L.base().base()
Finite Field in z2 of size 5^2
```

The method *bases()* gives access to the complete list of rings in a tower:

```
sage: L.bases()
[Field in b with defining polynomial x^2 + (4*z2 + 3*a)*x + 1 - a over its base,
 Field in a with defining polynomial x^2 + z2*x + 4 over its base,
 Finite Field in z2 of size 5^2]
```

```
>>> from sage.all import *
>>> L.bases()
[Field in b with defining polynomial x^2 + (4*z2 + 3*a)*x + 1 - a over its base,
 Field in a with defining polynomial x^2 + z2*x + 4 over its base,
 Finite Field in z2 of size 5^2]
```

Once we have constructed an extension (or a tower of extensions), we have interesting methods attached to it. As a basic example, one can compute a basis of the top ring over any base in the tower:

```
sage: L.basis_over(K)
[1, b]
sage: L.basis_over(F)
[1, a, b, a*b]
```

```
>>> from sage.all import *
>>> L.basis_over(K)
[1, b]
>>> L.basis_over(F)
[1, a, b, a*b]
```

When the base is omitted, the default is the natural base of the extension:

```
sage: L.basis_over()
[1, b]
```

```
>>> from sage.all import *
>>> L.basis_over()
[1, b]
```

The method *sage.rings.ring_extension_element.RingExtensionWithBasisElement.vector()* computes the coordinates of an element according to the above basis:

```
sage: u = a + 2*b + 3*a*b
sage: u.vector()   # over K
(a, 2 + 3*a)
sage: u.vector(F)
(0, 1, 2, 3)
```

```
>>> from sage.all import *
>>> u = a + Integer(2)*b + Integer(3)*a*b
>>> u.vector()   # over K
(a, 2 + 3*a)
>>> u.vector(F)
(0, 1, 2, 3)
```

One can also compute traces and norms with respect to any base of the tower:

```
sage: u.trace()           # over K
(2*z2 + 1) + (2*z2 + 1)*a
sage: u.trace(F)
z2 + 1
sage: u.trace().trace()   # over K, then over F
z2 + 1
sage: u.norm()            # over K
(z2 + 1) + (4*z2 + 2)*a
sage: u.norm(F)
2*z2 + 2
```

```
>>> from sage.all import *
>>> u.trace()           # over K
(2*z2 + 1) + (2*z2 + 1)*a
>>> u.trace(F)
z2 + 1
>>> u.trace().trace()   # over K, then over F
z2 + 1
>>> u.norm()            # over K
(z2 + 1) + (4*z2 + 2)*a
>>> u.norm(F)
2*z2 + 2
```

And minimal polynomials:

```
sage: u.minpoly()
x^2 + ((3*z2 + 4) + (3*z2 + 4)*a)*x + (z2 + 1) + (4*z2 + 2)*a
sage: u.minpoly(F)
x^4 + (4*z2 + 4)*x^3 + x^2 + (z2 + 1)*x + 2*z2 + 2
```

```
>>> from sage.all import *
>>> u.minpoly()
x^2 + ((3*z2 + 4) + (3*z2 + 4)*a)*x + (z2 + 1) + (4*z2 + 2)*a
>>> u.minpoly(F)
x^4 + (4*z2 + 4)*x^3 + x^2 + (z2 + 1)*x + 2*z2 + 2
```

AUTHOR:

- Xavier Caruso (2019)

class sage.rings.ring_extension.**RingExtensionFactory**

Bases: UniqueFactory

Factory for ring extensions.

create_key_and_extra_args(*ring*, *defining_morphism=None*, *gens=None*, *names=None*, *constructors=None*)

Create a key and return it together with a list of constructors of the object.

INPUT:

- `ring` – a commutative ring
- `defining_morphism` – a ring homomorphism or a commutative ring or `None` (default: `None`); the defining morphism of this extension or its base (if it coerces to `ring`)
- `gens` – list of generators of this extension (over its base) or `None` (default: `None`)
- `names` – list or a tuple of variable names or `None` (default: `None`)
- `constructors` – list of constructors; each constructor is a pair $(class, arguments)$ where $class$ is the class implementing the extension and $arguments$ is the dictionary of arguments to pass in to init function

create_object(*version*, *key*, ***extra_args*)

Return the object associated to a given key.

class sage.rings.ring_extension.**RingExtensionFractionField**

Bases: *RingExtension_generic*

A class for ring extensions of the form ` extrm{Frac}(A)/A`.

Element

alias of *RingExtensionFractionFieldElement*

ring()

Return the ring whose fraction field is this extension.

EXAMPLES:

```
sage: x = polygen(ZZ, 'x')
sage: A.<a> = ZZ.extension(x^2 - 2)
sage: OK = A.over()
sage: K = OK.fraction_field(); K
Fraction Field of
 Maximal Order generated by a in Number Field in a with defining polynomial x^
↪2 - 2 over its base
sage: K.ring()
Maximal Order generated by a in Number Field in a with defining polynomial x^
↪2 - 2 over its base
sage: K.ring() is OK
True
```

```
>>> from sage.all import *
>>> x = polygen(ZZ, 'x')
>>> A = ZZ.extension(x**Integer(2) - Integer(2), names=('a',)); (a,) = A._
```

(continues on next page)

(continued from previous page)

```
↪first_ngens(1)
>>> OK = A.over()
>>> K = OK.fraction_field(); K
Fraction Field of
 Maximal Order generated by a in Number Field in a with defining polynomial x^
↪2 - 2 over its base
>>> K.ring()
Maximal Order generated by a in Number Field in a with defining polynomial x^
↪2 - 2 over its base
>>> K.ring() is OK
True
```

class sage.rings.ring_extension.**RingExtensionWithBasis**

Bases: *RingExtension_generic*

A class for finite free ring extensions equipped with a basis.

Element

alias of *RingExtensionWithBasisElement*

basis_over(*base=None*)

Return a basis of this extension over `base`.

INPUT:

- `base` – a commutative ring (which might be itself an extension)

EXAMPLES:

```
sage: F.<a> = GF(5^2).over()  # over GF(5)
sage: K.<b> = GF(5^4).over(F)
sage: L.<c> = GF(5^12).over(K)
sage: L.basis_over(K)
[1, c, c^2]
sage: L.basis_over(F)
[1, b, c, b*c, c^2, b*c^2]
sage: L.basis_over(GF(5))
[1, a, b, a*b, c, a*c, b*c, a*b*c, c^2, a*c^2, b*c^2, a*b*c^2]
```

```
>>> from sage.all import *
>>> F = GF(Integer(5)**Integer(2)).over(names=('a',)); (a,) = F._first_
↪ngens(1)# over GF(5)
>>> K = GF(Integer(5)**Integer(4)).over(F, names=('b',)); (b,) = K._first_
↪ngens(1)
>>> L = GF(Integer(5)**Integer(12)).over(K, names=('c',)); (c,) = L._first_
↪ngens(1)
>>> L.basis_over(K)
[1, c, c^2]
>>> L.basis_over(F)
[1, b, c, b*c, c^2, b*c^2]
>>> L.basis_over(GF(Integer(5)))
[1, a, b, a*b, c, a*c, b*c, a*b*c, c^2, a*c^2, b*c^2, a*b*c^2]
```

If `base` is omitted, it is set to its default which is the base of the extension:

```
sage: L.basis_over()
[1, c, c^2]

sage: K.basis_over()
[1, b]
```

```
>>> from sage.all import *
>>> L.basis_over()
[1, c, c^2]

>>> K.basis_over()
[1, b]
```

Note that `base` must be an explicit base over which the extension has been defined (as listed by the method *bases()*):

```
sage: L.degree_over(GF(5^6))
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field in z6 of size 5^6
```

```
>>> from sage.all import *
>>> L.degree_over(GF(Integer(5)**Integer(6)))
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field in z6 of size 5^6
```

fraction_field(*extend_base=False*)

Return the fraction field of this extension.

INPUT:

- `extend_base` – boolean (default: `False`)

If `extend_base` is `False`, the fraction field of the extension L/K is defined as $\mathrm{Frac}(L)/L/K$, except is L is already a field in which base the fraction field of L/K is L/K itself.

If `extend_base` is `True`, the fraction field of the extension L/K is defined as $\mathrm{Frac}(L)/\mathrm{Frac}(K)$ (provided that the defining morphism extends to the fraction fields, i.e. is injective).

EXAMPLES:

```
sage: x = polygen(ZZ, 'x')
sage: A.<a> = ZZ.extension(x^2 - 5)
sage: OK = A.over()   # over ZZ
sage: OK
Order of conductor 2 generated by a in Number Field in a with defining␣
↪polynomial x^2 - 5 over its base
sage: K1 = OK.fraction_field(); K1
Fraction Field of Order of conductor 2 generated by a in Number Field in a
 with defining polynomial x^2 - 5 over its base
sage: K1.bases()
[Fraction Field of Order of conductor 2 generated by a in Number Field in a
  with defining polynomial x^2 - 5 over its base,
 Order of conductor 2 generated by a in Number Field in a
```

(continues on next page)

(continued from previous page)

```
  with defining polynomial x^2 - 5 over its base,
 Integer Ring]
sage: K2 = OK.fraction_field(extend_base=True); K2
Fraction Field of Order of conductor 2 generated by a
  in Number Field in a with defining polynomial x^2 - 5 over its base
sage: K2.bases()
[Fraction Field of Order of conductor 2 generated by a
  in Number Field in a with defining polynomial x^2 - 5 over its base,
 Rational Field]
```

```
>>> from sage.all import *
>>> x = polygen(ZZ, 'x')
>>> A = ZZ.extension(x**Integer(2) - Integer(5), names=('a',)); (a,) = A._
↪first_ngens(1)
>>> OK = A.over()   # over ZZ
>>> OK
Order of conductor 2 generated by a in Number Field in a with defining␣
↪polynomial x^2 - 5 over its base
>>> K1 = OK.fraction_field(); K1
Fraction Field of Order of conductor 2 generated by a in Number Field in a
 with defining polynomial x^2 - 5 over its base
>>> K1.bases()
[Fraction Field of Order of conductor 2 generated by a in Number Field in a
  with defining polynomial x^2 - 5 over its base,
 Order of conductor 2 generated by a in Number Field in a
  with defining polynomial x^2 - 5 over its base,
 Integer Ring]
>>> K2 = OK.fraction_field(extend_base=True); K2
Fraction Field of Order of conductor 2 generated by a
  in Number Field in a with defining polynomial x^2 - 5 over its base
>>> K2.bases()
[Fraction Field of Order of conductor 2 generated by a
  in Number Field in a with defining polynomial x^2 - 5 over its base,
 Rational Field]
```

Note that there is no coercion map between K_1 and K_2:

```
sage: K1.has_coerce_map_from(K2)
False
sage: K2.has_coerce_map_from(K1)
False
```

```
>>> from sage.all import *
>>> K1.has_coerce_map_from(K2)
False
>>> K2.has_coerce_map_from(K1)
False
```

We check that when the extension is a field, its fraction field does not change:

```
sage: K1.fraction_field() is K1
True
```

(continues on next page)

(continued from previous page)

```
sage: K2.fraction_field() is K2
True
```

```
>>> from sage.all import *
>>> K1.fraction_field() is K1
True
>>> K2.fraction_field() is K2
True
```

free_module(*base=None*, *map=True*)

Return a free module V over `base` which is isomorphic to this ring

INPUT:

- `base` – a commutative ring (which might be itself an extension) or `None` (default: `None`)
- `map` – boolean (default: `True`); whether to return isomorphisms between this ring and V

OUTPUT:

- A finite-rank free module V over `base`
- The isomorphism from V to this ring corresponding to the basis output by the method *basis_over()* (only included if `map` is `True`)
- The reverse isomorphism of the isomorphism above (only included if `map` is `True`)

EXAMPLES:

```
sage: F = GF(11)
sage: K.<a> = GF(11^2).over()
sage: L.<b> = GF(11^6).over(K)
```

```
>>> from sage.all import *
>>> F = GF(Integer(11))
>>> K = GF(Integer(11)**Integer(2)).over(names=('a',)); (a,) = K._first_
↪ngens(1)
>>> L = GF(Integer(11)**Integer(6)).over(K, names=('b',)); (b,) = L._first_
↪ngens(1)
```

Forgetting a part of the multiplicative structure, the field L can be viewed as a vector space of dimension 3 over K, equipped with a distinguished basis, namely $(1, b, b^2)$:

```
sage: V, i, j = L.free_module(K)
sage: V
Vector space of dimension 3 over
 Field in a with defining polynomial x^2 + 7*x + 2 over its base
sage: i
Generic map:
  From: Vector space of dimension 3 over
        Field in a with defining polynomial x^2 + 7*x + 2 over its base
  To:   Field in b with defining polynomial
        x^3 + (7 + 2*a)*x^2 + (2 - a)*x - a over its base
sage: j
Generic map:
  From: Field in b with defining polynomial
```

(continues on next page)

(continued from previous page)

```
        x^3 + (7 + 2*a)*x^2 + (2 - a)*x - a over its base
  To:   Vector space of dimension 3 over
        Field in a with defining polynomial x^2 + 7*x + 2 over its base
sage: j(b)
(0, 1, 0)
sage: i((1, a, a+1))
1 + a*b + (1 + a)*b^2
```

```
>>> from sage.all import *
>>> V, i, j = L.free_module(K)
>>> V
Vector space of dimension 3 over
 Field in a with defining polynomial x^2 + 7*x + 2 over its base
>>> i
Generic map:
  From: Vector space of dimension 3 over
        Field in a with defining polynomial x^2 + 7*x + 2 over its base
  To:   Field in b with defining polynomial
        x^3 + (7 + 2*a)*x^2 + (2 - a)*x - a over its base
>>> j
Generic map:
  From: Field in b with defining polynomial
        x^3 + (7 + 2*a)*x^2 + (2 - a)*x - a over its base
  To:   Vector space of dimension 3 over
        Field in a with defining polynomial x^2 + 7*x + 2 over its base
>>> j(b)
(0, 1, 0)
>>> i((Integer(1), a, a+Integer(1)))
1 + a*b + (1 + a)*b^2
```

Similarly, one can view L as a F-vector space of dimension 6:

```
sage: V, i, j, = L.free_module(F)
sage: V
Vector space of dimension 6 over Finite Field of size 11
```

```
>>> from sage.all import *
>>> V, i, j, = L.free_module(F)
>>> V
Vector space of dimension 6 over Finite Field of size 11
```

In this case, the isomorphisms between V and L are given by the basis $(1, a, b, ab, b^2, ab^2)$:

sage: j(a*b) (0, 0, 0, 1, 0, 0) sage: i((1,2,3,4,5,6)) (1 + 2*a) + (3 + 4*a)*b + (5 + 6*a)*b^2

When `base` is omitted, the default is the base of this extension:

```
sage: L.free_module(map=False)
Vector space of dimension 3 over
 Field in a with defining polynomial x^2 + 7*x + 2 over its base
```

```
>>> from sage.all import *
>>> L.free_module(map=False)
```

(continues on next page)

(continued from previous page)

```
Vector space of dimension 3 over
 Field in a with defining polynomial x^2 + 7*x + 2 over its base
```

Note that `base` must be an explicit base over which the extension has been defined (as listed by the method *bases()*):

```
sage: L.degree(GF(11^3))
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field in z3 of size 11^3
```

```
>>> from sage.all import *
>>> L.degree(GF(Integer(11)**Integer(3)))
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field in z3 of size 11^3
```

class sage.rings.ring_extension.**RingExtensionWithGen**

Bases: *RingExtensionWithBasis*

A class for finite free ring extensions generated by a single element

fraction_field(*extend_base=False*)

Return the fraction field of this extension.

INPUT:

- `extend_base` – boolean (default: `False`)

If `extend_base` is `False`, the fraction field of the extension L/K is defined as $\mathrm{Frac}(L)/L/K$, except is L is already a field in which base the fraction field of L/K is L/K itself.

If `extend_base` is `True`, the fraction field of the extension L/K is defined as $\mathrm{Frac}(L)/\mathrm{Frac}(K)$ (provided that the defining morphism extends to the fraction fields, i.e. is injective).

EXAMPLES:

```
sage: x = polygen(ZZ, 'x')
sage: A.<a> = ZZ.extension(x^2 - 5)
sage: OK = A.over()   # over ZZ
sage: OK
Order of conductor 2 generated by a in Number Field in a
 with defining polynomial x^2 - 5 over its base
sage: K1 = OK.fraction_field(); K1
Fraction Field of Order of conductor 2 generated by a
 in Number Field in a with defining polynomial x^2 - 5 over its base
sage: K1.bases()
[Fraction Field of Order of conductor 2 generated by a
  in Number Field in a with defining polynomial x^2 - 5 over its base,
 Order of conductor 2 generated by a in Number Field in a
  with defining polynomial x^2 - 5 over its base,
 Integer Ring]
sage: K2 = OK.fraction_field(extend_base=True); K2
Fraction Field of Order of conductor 2 generated by a
 in Number Field in a with defining polynomial x^2 - 5 over its base
```

(continues on next page)

(continued from previous page)

```
sage: K2.bases()
[Fraction Field of Order of conductor 2 generated by a
  in Number Field in a with defining polynomial x^2 - 5 over its base,
 Rational Field]
```

```
>>> from sage.all import *
>>> x = polygen(ZZ, 'x')
>>> A = ZZ.extension(x**Integer(2) - Integer(5), names=('a',)); (a,) = A._
↪first_ngens(1)
>>> OK = A.over()   # over ZZ
>>> OK
Order of conductor 2 generated by a in Number Field in a
 with defining polynomial x^2 - 5 over its base
>>> K1 = OK.fraction_field(); K1
Fraction Field of Order of conductor 2 generated by a
 in Number Field in a with defining polynomial x^2 - 5 over its base
>>> K1.bases()
[Fraction Field of Order of conductor 2 generated by a
  in Number Field in a with defining polynomial x^2 - 5 over its base,
 Order of conductor 2 generated by a in Number Field in a
  with defining polynomial x^2 - 5 over its base,
 Integer Ring]
>>> K2 = OK.fraction_field(extend_base=True); K2
Fraction Field of Order of conductor 2 generated by a
 in Number Field in a with defining polynomial x^2 - 5 over its base
>>> K2.bases()
[Fraction Field of Order of conductor 2 generated by a
  in Number Field in a with defining polynomial x^2 - 5 over its base,
 Rational Field]
```

Note that there is no coercion map between K_1 and K_2:

```
sage: K1.has_coerce_map_from(K2)
False
sage: K2.has_coerce_map_from(K1)
False
```

```
>>> from sage.all import *
>>> K1.has_coerce_map_from(K2)
False
>>> K2.has_coerce_map_from(K1)
False
```

We check that when the extension is a field, its fraction field does not change:

```
sage: K1.fraction_field() is K1
True
sage: K2.fraction_field() is K2
True
```

```
>>> from sage.all import *
>>> K1.fraction_field() is K1
```

(continues on next page)

(continued from previous page)

```
True
>>> K2.fraction_field() is K2
True
```

gens(*base=None*)

Return the generators of this extension over `base`.

INPUT:

- `base` – a commutative ring (which might be itself an extension) or `None` (default: `None`)

EXAMPLES:

```
sage: K.<a> = GF(5^2).over()   # over GF(5)
sage: K.gens()
(a,)
sage: L.<b> = GF(5^4).over(K)
sage: L.gens()
(b,)
sage: L.gens(GF(5))
(b, a)
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(2)).over(names=('a',)); (a,) = K._first_
↪ngens(1)# over GF(5)
>>> K.gens()
(a,)
>>> L = GF(Integer(5)**Integer(4)).over(K, names=('b',)); (b,) = L._first_
↪ngens(1)
>>> L.gens()
(b,)
>>> L.gens(GF(Integer(5)))
(b, a)
```

modulus(*var='x'*)

Return the defining polynomial of this extension, that is the minimal polynomial of the given generator of this extension.

INPUT:

- `var` – a variable name (default: `x`)

EXAMPLES:

```
sage: K.<u> = GF(7^10).over(GF(7^2)); K
Field in u with defining polynomial x^5 + (6*z2 + 4)*x^4
 + (3*z2 + 5)*x^3 + (2*z2 + 2)*x^2 + 4*x + 6*z2 over its base
sage: P = K.modulus(); P
x^5 + (6*z2 + 4)*x^4 + (3*z2 + 5)*x^3 + (2*z2 + 2)*x^2 + 4*x + 6*z2
sage: P(u)
0
```

```
>>> from sage.all import *
>>> K = GF(Integer(7)**Integer(10)).over(GF(Integer(7)**Integer(2)), names=('u
```

(continues on next page)

(continued from previous page)

```
↪',)); (u,) = K._first_ngens(1); K
Field in u with defining polynomial x^5 + (6*z2 + 4)*x^4
 + (3*z2 + 5)*x^3 + (2*z2 + 2)*x^2 + 4*x + 6*z2 over its base
>>> P = K.modulus(); P
x^5 + (6*z2 + 4)*x^4 + (3*z2 + 5)*x^3 + (2*z2 + 2)*x^2 + 4*x + 6*z2
>>> P(u)
0
```

We can use a different variable name:

```
sage: K.modulus('y')
y^5 + (6*z2 + 4)*y^4 + (3*z2 + 5)*y^3 + (2*z2 + 2)*y^2 + 4*y + 6*z2
```

```
>>> from sage.all import *
>>> K.modulus('y')
y^5 + (6*z2 + 4)*y^4 + (3*z2 + 5)*y^3 + (2*z2 + 2)*y^2 + 4*y + 6*z2
```

class sage.rings.ring_extension.**RingExtension_generic**

Bases: Parent

A generic class for all ring extensions.

Element

alias of *RingExtensionElement*

absolute_base()

Return the absolute base of this extension.

By definition, the absolute base of an iterated extension $K_n/\cdots K_2/K_1$ is the ring K_1.

EXAMPLES:

```
sage: F = GF(5^2).over()    # over GF(5)
sage: K = GF(5^4).over(F)
sage: L = GF(5^12).over(K)
sage: F.absolute_base()
Finite Field of size 5
sage: K.absolute_base()
Finite Field of size 5
sage: L.absolute_base()
Finite Field of size 5
```

```
>>> from sage.all import *
>>> F = GF(Integer(5)**Integer(2)).over()    # over GF(5)
>>> K = GF(Integer(5)**Integer(4)).over(F)
>>> L = GF(Integer(5)**Integer(12)).over(K)
>>> F.absolute_base()
Finite Field of size 5
>>> K.absolute_base()
Finite Field of size 5
>>> L.absolute_base()
Finite Field of size 5
```

See also

base(), *bases()*, *is_defined_over()*

absolute_degree()

Return the degree of this extension over its absolute base.

EXAMPLES:

```
sage: A = GF(5^4).over(GF(5^2))
sage: B = GF(5^12).over(A)
sage: A.absolute_degree()
2
sage: B.absolute_degree()
6
```

```
>>> from sage.all import *
>>> A = GF(Integer(5)**Integer(4)).over(GF(Integer(5)**Integer(2)))
>>> B = GF(Integer(5)**Integer(12)).over(A)
>>> A.absolute_degree()
2
>>> B.absolute_degree()
6
```

See also

degree(), *relative_degree()*

backend(*force=False*)

Return the backend of this extension.

INPUT:

- `force` – boolean (default: `False`); if `False`, raise an error if the backend is not exposed

EXAMPLES:

```
sage: K = GF(5^3)
sage: E = K.over()
sage: E
Field in z3 with defining polynomial x^3 + 3*x + 3 over its base
sage: E.backend()
Finite Field in z3 of size 5^3
sage: E.backend() is K
True
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3))
>>> E = K.over()
>>> E
Field in z3 with defining polynomial x^3 + 3*x + 3 over its base
>>> E.backend()
```

(continues on next page)

(continued from previous page)

```
Finite Field in z3 of size 5^3
>>> E.backend() is K
True
```

base()

Return the base of this extension.

EXAMPLES:

```
sage: F = GF(5^2)
sage: K = GF(5^4).over(F)
sage: K.base()
Finite Field in z2 of size 5^2
```

```
>>> from sage.all import *
>>> F = GF(Integer(5)**Integer(2))
>>> K = GF(Integer(5)**Integer(4)).over(F)
>>> K.base()
Finite Field in z2 of size 5^2
```

In case of iterated extensions, the base is itself an extension:

```
sage: L = GF(5^8).over(K)
sage: L.base()
Field in z4 with defining polynomial x^2 + (4*z2 + 3)*x + z2 over its base
sage: L.base() is K
True
```

```
>>> from sage.all import *
>>> L = GF(Integer(5)**Integer(8)).over(K)
>>> L.base()
Field in z4 with defining polynomial x^2 + (4*z2 + 3)*x + z2 over its base
>>> L.base() is K
True
```

See also

bases(), *absolute_base()*, *is_defined_over()*

bases()

Return the list of successive bases of this extension (including itself).

EXAMPLES:

```
sage: F = GF(5^2).over()  # over GF(5)
sage: K = GF(5^4).over(F)
sage: L = GF(5^12).over(K)
sage: F.bases()
[Field in z2 with defining polynomial x^2 + 4*x + 2 over its base,
 Finite Field of size 5]
sage: K.bases()
```

(continues on next page)

(continued from previous page)

```
[Field in z4 with defining polynomial x^2 + (3 - z2)*x + z2 over its base,
 Field in z2 with defining polynomial x^2 + 4*x + 2 over its base,
 Finite Field of size 5]
sage: L.bases()
[Field in z12 with defining polynomial
  x^3 + (1 + (2 - z2)*z4)*x^2 + (2 + 2*z4)*x - z4 over its base,
 Field in z4 with defining polynomial x^2 + (3 - z2)*x + z2 over its base,
 Field in z2 with defining polynomial x^2 + 4*x + 2 over its base,
 Finite Field of size 5]
```

```
>>> from sage.all import *
>>> F = GF(Integer(5)**Integer(2)).over()  # over GF(5)
>>> K = GF(Integer(5)**Integer(4)).over(F)
>>> L = GF(Integer(5)**Integer(12)).over(K)
>>> F.bases()
[Field in z2 with defining polynomial x^2 + 4*x + 2 over its base,
 Finite Field of size 5]
>>> K.bases()
[Field in z4 with defining polynomial x^2 + (3 - z2)*x + z2 over its base,
 Field in z2 with defining polynomial x^2 + 4*x + 2 over its base,
 Finite Field of size 5]
>>> L.bases()
[Field in z12 with defining polynomial
  x^3 + (1 + (2 - z2)*z4)*x^2 + (2 + 2*z4)*x - z4 over its base,
 Field in z4 with defining polynomial x^2 + (3 - z2)*x + z2 over its base,
 Field in z2 with defining polynomial x^2 + 4*x + 2 over its base,
 Finite Field of size 5]
```

See also

base(), *absolute_base()*, *is_defined_over()*

characteristic()

Return the characteristic of the extension as a ring.

OUTPUT: a prime number or zero

EXAMPLES:

```
sage: F = GF(5^2).over()    # over GF(5)
sage: K = GF(5^4).over(F)
sage: L = GF(5^12).over(K)
sage: F.characteristic()
5
sage: K.characteristic()
5
sage: L.characteristic()
5
```

```
>>> from sage.all import *
>>> F = GF(Integer(5)**Integer(2)).over()    # over GF(5)
```

(continues on next page)

(continued from previous page)

```
>>> K = GF(Integer(5)**Integer(4)).over(F)
>>> L = GF(Integer(5)**Integer(12)).over(K)
>>> F.characteristic()
5
>>> K.characteristic()
5
>>> L.characteristic()
5
```

```
sage: F = RR.over(ZZ)
sage: F.characteristic()
0
```

```
>>> from sage.all import *
>>> F = RR.over(ZZ)
>>> F.characteristic()
0
```

```
sage: F = GF(11)
sage: A.<x> = F[]
sage: K = Frac(F).over(F)
sage: K.characteristic()
11
```

```
>>> from sage.all import *
>>> F = GF(Integer(11))
>>> A = F['x']; (x,) = A._first_ngens(1)
>>> K = Frac(F).over(F)
>>> K.characteristic()
11
```

```
sage: E = GF(7).over(ZZ)
sage: E.characteristic()
7
```

```
>>> from sage.all import *
>>> E = GF(Integer(7)).over(ZZ)
>>> E.characteristic()
7
```

construction()

Return the functorial construction of this extension, if defined.

EXAMPLES:

```
sage: E = GF(5^3).over()
sage: E.construction()
```

```
>>> from sage.all import *
>>> E = GF(Integer(5)**Integer(3)).over()
>>> E.construction()
```

defining_morphism(*base=None*)

Return the defining morphism of this extension over `base`.

INPUT:

- `base` – a commutative ring (which might be itself an extension) or `None` (default: `None`)

EXAMPLES:

```
sage: F = GF(5^2)
sage: K = GF(5^4).over(F)
sage: L = GF(5^12).over(K)
sage: K.defining_morphism()
Ring morphism:
  From: Finite Field in z2 of size 5^2
  To:   Field in z4 with defining polynomial x^2 + (4*z2 + 3)*x + z2 over its␣
↪base
  Defn: z2 |--> z2
sage: L.defining_morphism()
Ring morphism:
  From: Field in z4 with defining polynomial x^2 + (4*z2 + 3)*x + z2 over its␣
↪base
  To:   Field in z12 with defining polynomial
        x^3 + (1 + (4*z2 + 2)*z4)*x^2 + (2 + 2*z4)*x - z4 over its base
  Defn: z4 |--> z4
```

```
>>> from sage.all import *
>>> F = GF(Integer(5)**Integer(2))
>>> K = GF(Integer(5)**Integer(4)).over(F)
>>> L = GF(Integer(5)**Integer(12)).over(K)
>>> K.defining_morphism()
Ring morphism:
  From: Finite Field in z2 of size 5^2
  To:   Field in z4 with defining polynomial x^2 + (4*z2 + 3)*x + z2 over its␣
↪base
  Defn: z2 |--> z2
>>> L.defining_morphism()
Ring morphism:
  From: Field in z4 with defining polynomial x^2 + (4*z2 + 3)*x + z2 over its␣
↪base
  To:   Field in z12 with defining polynomial
        x^3 + (1 + (4*z2 + 2)*z4)*x^2 + (2 + 2*z4)*x - z4 over its base
  Defn: z4 |--> z4
```

One can also pass in a base over which the extension is explicitly defined (see also *is_defined_over()*):

```
sage: L.defining_morphism(F)
Ring morphism:
  From: Finite Field in z2 of size 5^2
  To:   Field in z12 with defining polynomial
        x^3 + (1 + (4*z2 + 2)*z4)*x^2 + (2 + 2*z4)*x - z4 over its base
  Defn: z2 |--> z2
sage: L.defining_morphism(GF(5))
Traceback (most recent call last):
```

(continues on next page)

(continued from previous page)

```
...
ValueError: not (explicitly) defined over Finite Field of size 5
```

```
>>> from sage.all import *
>>> L.defining_morphism(F)
Ring morphism:
  From: Finite Field in z2 of size 5^2
  To:   Field in z12 with defining polynomial
        x^3 + (1 + (4*z2 + 2)*z4)*x^2 + (2 + 2*z4)*x - z4 over its base
  Defn: z2 |--> z2
>>> L.defining_morphism(GF(Integer(5)))
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field of size 5
```

degree(*base*)

Return the degree of this extension over `base`.

INPUT:

- `base` – a commutative ring (which might be itself an extension)

EXAMPLES:

```
sage: A = GF(5^4).over(GF(5^2))
sage: B = GF(5^12).over(A)
sage: A.degree(GF(5^2))
2
sage: B.degree(A)
3
sage: B.degree(GF(5^2))
6
```

```
>>> from sage.all import *
>>> A = GF(Integer(5)**Integer(4)).over(GF(Integer(5)**Integer(2)))
>>> B = GF(Integer(5)**Integer(12)).over(A)
>>> A.degree(GF(Integer(5)**Integer(2)))
2
>>> B.degree(A)
3
>>> B.degree(GF(Integer(5)**Integer(2)))
6
```

Note that `base` must be an explicit base over which the extension has been defined (as listed by the method *bases()*):

```
sage: A.degree(GF(5))
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field of size 5
```

```
>>> from sage.all import *
>>> A.degree(GF(Integer(5)))
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field of size 5
```

See also

relative_degree(), *absolute_degree()*

degree_over(*base=None*)

Return the degree of this extension over `base`.

INPUT:

- `base` – a commutative ring (which might be itself an extension) or `None` (default: `None`)

EXAMPLES:

```
sage: F = GF(5^2)
sage: K = GF(5^4).over(F)
sage: L = GF(5^12).over(K)
sage: K.degree_over(F)
2
sage: L.degree_over(K)
3
sage: L.degree_over(F)
6
```

```
>>> from sage.all import *
>>> F = GF(Integer(5)**Integer(2))
>>> K = GF(Integer(5)**Integer(4)).over(F)
>>> L = GF(Integer(5)**Integer(12)).over(K)
>>> K.degree_over(F)
2
>>> L.degree_over(K)
3
>>> L.degree_over(F)
6
```

If `base` is omitted, the degree is computed over the base of the extension:

```
sage: K.degree_over()
2
sage: L.degree_over()
3
```

```
>>> from sage.all import *
>>> K.degree_over()
2
>>> L.degree_over()
3
```

Note that `base` must be an explicit base over which the extension has been defined (as listed by the method *bases()*):

```
sage: K.degree_over(GF(5))
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field of size 5
```

```
>>> from sage.all import *
>>> K.degree_over(GF(Integer(5)))
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field of size 5
```

fraction_field(*extend_base=False*)

Return the fraction field of this extension.

INPUT:

- `extend_base` – boolean (default: `False`)

If `extend_base` is `False`, the fraction field of the extension L/K is defined as $\mathrm{Frac}(L)/L/K$, except if L is already a field in which base the fraction field of L/K is L/K itself.

If `extend_base` is `True`, the fraction field of the extension L/K is defined as $\mathrm{Frac}(L)/\mathrm{Frac}(K)$ (provided that the defining morphism extends to the fraction fields, i.e. is injective).

EXAMPLES:

```
sage: x = polygen(ZZ, 'x')
sage: A.<a> = ZZ.extension(x^2 - 5)
sage: OK = A.over()   # over ZZ
sage: OK
Order of conductor 2 generated by a in Number Field in a
 with defining polynomial x^2 - 5 over its base
sage: K1 = OK.fraction_field(); K1
Fraction Field of Order of conductor 2 generated by a
 in Number Field in a with defining polynomial x^2 - 5 over its base
sage: K1.bases()
[Fraction Field of Order of conductor 2 generated by a
 in Number Field in a with defining polynomial x^2 - 5 over its base,
 Order of conductor 2 generated by a in Number Field in a
 with defining polynomial x^2 - 5 over its base,
 Integer Ring]
sage: K2 = OK.fraction_field(extend_base=True); K2
Fraction Field of Order of conductor 2 generated by a
 in Number Field in a with defining polynomial x^2 - 5 over its base
sage: K2.bases()
[Fraction Field of Order of conductor 2 generated by a
 in Number Field in a with defining polynomial x^2 - 5 over its base,
 Rational Field]
```

```
>>> from sage.all import *
>>> x = polygen(ZZ, 'x')
>>> A = ZZ.extension(x**Integer(2) - Integer(5), names=('a',)); (a,) = A._
↪first_ngens(1)
>>> OK = A.over()   # over ZZ
>>> OK
```

(continues on next page)

(continued from previous page)

```
Order of conductor 2 generated by a in Number Field in a
 with defining polynomial x^2 - 5 over its base
>>> K1 = OK.fraction_field(); K1
Fraction Field of Order of conductor 2 generated by a
 in Number Field in a with defining polynomial x^2 - 5 over its base
>>> K1.bases()
[Fraction Field of Order of conductor 2 generated by a
 in Number Field in a with defining polynomial x^2 - 5 over its base,
 Order of conductor 2 generated by a in Number Field in a
 with defining polynomial x^2 - 5 over its base,
 Integer Ring]
>>> K2 = OK.fraction_field(extend_base=True); K2
Fraction Field of Order of conductor 2 generated by a
 in Number Field in a with defining polynomial x^2 - 5 over its base
>>> K2.bases()
[Fraction Field of Order of conductor 2 generated by a
 in Number Field in a with defining polynomial x^2 - 5 over its base,
 Rational Field]
```

Note that there is no coercion between K_1 and K_2:

```
sage: K1.has_coerce_map_from(K2)
False
sage: K2.has_coerce_map_from(K1)
False
```

```
>>> from sage.all import *
>>> K1.has_coerce_map_from(K2)
False
>>> K2.has_coerce_map_from(K1)
False
```

We check that when the extension is a field, its fraction field does not change:

```
sage: K1.fraction_field() is K1
True
sage: K2.fraction_field() is K2
True
```

```
>>> from sage.all import *
>>> K1.fraction_field() is K1
True
>>> K2.fraction_field() is K2
True
```

from_base_ring(*r*)

Return the canonical embedding of `r` into this extension.

INPUT:

- `r` – an element of the base of the ring of this extension

EXAMPLES:

```
sage: k = GF(5)
sage: K.<u> = GF(5^2).over(k)
sage: L.<v> = GF(5^4).over(K)
sage: x = L.from_base_ring(k(2)); x
2
sage: x.parent()
Field in v with defining polynomial x^2 + (3 - u)*x + u over its base
sage: x = L.from_base_ring(u); x
u
sage: x.parent()
Field in v with defining polynomial x^2 + (3 - u)*x + u over its base
```

```
>>> from sage.all import *
>>> k = GF(Integer(5))
>>> K = GF(Integer(5)**Integer(2)).over(k, names=('u',)); (u,) = K._first_
↪ngens(1)
>>> L = GF(Integer(5)**Integer(4)).over(K, names=('v',)); (v,) = L._first_
↪ngens(1)
>>> x = L.from_base_ring(k(Integer(2))); x
2
>>> x.parent()
Field in v with defining polynomial x^2 + (3 - u)*x + u over its base
>>> x = L.from_base_ring(u); x
u
>>> x.parent()
Field in v with defining polynomial x^2 + (3 - u)*x + u over its base
```

gen()

Return the first generator of this extension.

EXAMPLES:

```
sage: K = GF(5^2).over()    # over GF(5)
sage: x = K.gen(); x
z2
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(2)).over()    # over GF(5)
>>> x = K.gen(); x
z2
```

Observe that the generator lives in the extension:

```
sage: x.parent()
Field in z2 with defining polynomial x^2 + 4*x + 2 over its base
sage: x.parent() is K
True
```

```
>>> from sage.all import *
>>> x.parent()
Field in z2 with defining polynomial x^2 + 4*x + 2 over its base
>>> x.parent() is K
True
```

gens(*base=None*)

Return the generators of this extension over `base`.

INPUT:

- `base` – a commutative ring (which might be itself an extension) or `None` (default: `None`); if omitted, use the base of this extension

EXAMPLES:

```
sage: K.<a> = GF(5^2).over()  # over GF(5)
sage: K.gens()
(a,)
sage: L.<b> = GF(5^4).over(K)
sage: L.gens()
(b,)
sage: L.gens(GF(5))
(b, a)

sage: S.<x> = QQ[]
sage: T.<y> = S[]
sage: T.over(S).gens()
(y,)
sage: T.over(QQ).gens()
(y, x)
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(2)).over(names=('a',)); (a,) = K._first_
↪ngens(1)# over GF(5)
>>> K.gens()
(a,)
>>> L = GF(Integer(5)**Integer(4)).over(K, names=('b',)); (b,) = L._first_
↪ngens(1)
>>> L.gens()
(b,)
>>> L.gens(GF(Integer(5)))
(b, a)

>>> S = QQ['x']; (x,) = S._first_ngens(1)
>>> T = S['y']; (y,) = T._first_ngens(1)
>>> T.over(S).gens()
(y,)
>>> T.over(QQ).gens()
(y, x)
```

hom(*im_gens*, *codomain=None*, *base_map=None*, *category=None*, *check=True*)

Return the unique homomorphism from this extension to `codomain` that sends `self.gens()` to the entries of `im_gens` and induces the map `base_map` on the base ring.

INPUT:

- `im_gens` – the images of the generators of this extension
- `codomain` – the codomain of the homomorphism; if omitted, it is set to the smallest parent containing all the entries of `im_gens`

- `base_map` – a map from one of the bases of this extension into something that coerces into the codomain; if omitted, coercion maps are used
- `category` – the category of the resulting morphism
- `check` – boolean (default: `True`); whether to verify that the images of generators extend to define a map (using only canonical coercions)

EXAMPLES:

```
sage: K.<a> = GF(5^2).over()     # over GF(5)
sage: L.<b> = GF(5^6).over(K)
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(2)).over(names=('a',)); (a,) = K._first_
↪ngens(1)# over GF(5)
>>> L = GF(Integer(5)**Integer(6)).over(K, names=('b',)); (b,) = L._first_
↪ngens(1)
```

We define (by hand) the relative Frobenius endomorphism of the extension L/K:

```
sage: L.hom([b^25])
Ring endomorphism of
 Field in b with defining polynomial x^3 + (2 + 2*a)*x - a over its base
  Defn: b |--> 2 + 2*a*b + (2 - a)*b^2
```

```
>>> from sage.all import *
>>> L.hom([b**Integer(25)])
Ring endomorphism of
 Field in b with defining polynomial x^3 + (2 + 2*a)*x - a over its base
  Defn: b |--> 2 + 2*a*b + (2 - a)*b^2
```

Defining the absolute Frobenius of L is a bit more complicated because it is not a homomorphism of K-algebras. For this reason, the construction `L.hom([b^5])` fails:

```
sage: L.hom([b^5])
Traceback (most recent call last):
...
ValueError: images do not define a valid homomorphism
```

```
>>> from sage.all import *
>>> L.hom([b**Integer(5)])
Traceback (most recent call last):
...
ValueError: images do not define a valid homomorphism
```

What we need is to specify a base map:

```
sage: FrobK = K.hom([a^5])
sage: FrobL = L.hom([b^5], base_map=FrobK); FrobL
Ring endomorphism of
 Field in b with defining polynomial x^3 + (2 + 2*a)*x - a over its base
  Defn: b |--> (-1 + a) + (1 + 2*a)*b + a*b^2
        with map on base ring:
        a |--> 1 - a
```

```
>>> from sage.all import *
>>> FrobK = K.hom([a**Integer(5)])
>>> FrobL = L.hom([b**Integer(5)], base_map=FrobK); FrobL
Ring endomorphism of
 Field in b with defining polynomial x^3 + (2 + 2*a)*x - a over its base
  Defn: b |--> (-1 + a) + (1 + 2*a)*b + a*b^2
        with map on base ring:
        a |--> 1 - a
```

As a shortcut, we may use the following construction:

```
sage: phi = L.hom([b^5, a^5]); phi
Ring endomorphism of
 Field in b with defining polynomial x^3 + (2 + 2*a)*x - a over its base
  Defn: b |--> (-1 + a) + (1 + 2*a)*b + a*b^2
        with map on base ring:
        a |--> 1 - a
sage: phi == FrobL
True
```

```
>>> from sage.all import *
>>> phi = L.hom([b**Integer(5), a**Integer(5)]); phi
Ring endomorphism of
 Field in b with defining polynomial x^3 + (2 + 2*a)*x - a over its base
  Defn: b |--> (-1 + a) + (1 + 2*a)*b + a*b^2
        with map on base ring:
        a |--> 1 - a
>>> phi == FrobL
True
```

is_defined_over(*base*)

Return whether or not `base` is one of the bases of this extension.

INPUT:

- `base` – a commutative ring, which might be itself an extension

EXAMPLES:

```
sage: A = GF(5^4).over(GF(5^2))
sage: B = GF(5^12).over(A)
sage: A.is_defined_over(GF(5^2))
True
sage: A.is_defined_over(GF(5))
False

sage: B.is_defined_over(A)
True
sage: B.is_defined_over(GF(5^4))
True
sage: B.is_defined_over(GF(5^2))
True
sage: B.is_defined_over(GF(5))
False
```

```
>>> from sage.all import *
>>> A = GF(Integer(5)**Integer(4)).over(GF(Integer(5)**Integer(2)))
>>> B = GF(Integer(5)**Integer(12)).over(A)
>>> A.is_defined_over(GF(Integer(5)**Integer(2)))
True
>>> A.is_defined_over(GF(Integer(5)))
False

>>> B.is_defined_over(A)
True
>>> B.is_defined_over(GF(Integer(5)**Integer(4)))
True
>>> B.is_defined_over(GF(Integer(5)**Integer(2)))
True
>>> B.is_defined_over(GF(Integer(5)))
False
```

Note that an extension is defined over itself:

```
sage: A.is_defined_over(A)
True
sage: A.is_defined_over(GF(5^4))
True
```

```
>>> from sage.all import *
>>> A.is_defined_over(A)
True
>>> A.is_defined_over(GF(Integer(5)**Integer(4)))
True
```

See also

base(), *bases()*, *absolute_base()*

is_field(*proof=True*)

Return whether or not this extension is a field.

INPUT:

- `proof` – boolean (default: `False`)

EXAMPLES:

```
sage: K = GF(5^5).over()  # over GF(5)
sage: K.is_field()
True

sage: S.<x> = QQ[]
sage: A = S.over(QQ)
sage: A.is_field()
False

sage: B = A.fraction_field()
```

(continues on next page)

(continued from previous page)

```
sage: B.is_field()
True
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(5)).over()   # over GF(5)
>>> K.is_field()
True

>>> S = QQ['x']; (x,) = S._first_ngens(1)
>>> A = S.over(QQ)
>>> A.is_field()
False

>>> B = A.fraction_field()
>>> B.is_field()
True
```

is_finite_over(*base=None*)

Return whether or not this extension is finite over `base` (as a module).

INPUT:

- `base` – a commutative ring (which might be itself an extension) or `None` (default: `None`)

EXAMPLES:

```
sage: K = GF(5^2).over()   # over GF(5)
sage: L = GF(5^4).over(K)
sage: L.is_finite_over(K)
True
sage: L.is_finite_over(GF(5))
True
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(2)).over()   # over GF(5)
>>> L = GF(Integer(5)**Integer(4)).over(K)
>>> L.is_finite_over(K)
True
>>> L.is_finite_over(GF(Integer(5)))
True
```

If `base` is omitted, it is set to its default which is the base of the extension:

```
sage: L.is_finite_over()
True
```

```
>>> from sage.all import *
>>> L.is_finite_over()
True
```

is_free_over(*base=None*)

Return `True` if this extension is free (as a module) over `base`

INPUT:

- `base` – a commutative ring (which might be itself an extension) or `None` (default: `None`)

EXAMPLES:

```
sage: K = GF(5^2).over()  # over GF(5)
sage: L = GF(5^4).over(K)
sage: L.is_free_over(K)
True
sage: L.is_free_over(GF(5))
True
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(2)).over()  # over GF(5)
>>> L = GF(Integer(5)**Integer(4)).over(K)
>>> L.is_free_over(K)
True
>>> L.is_free_over(GF(Integer(5)))
True
```

If `base` is omitted, it is set to its default which is the base of the extension:

```
sage: L.is_free_over()
True
```

```
>>> from sage.all import *
>>> L.is_free_over()
True
```

ngens(*base=None*)

Return the number of generators of this extension over `base`.

INPUT:

- `base` – a commutative ring (which might be itself an extension) or `None` (default: `None`)

EXAMPLES:

```
sage: K = GF(5^2).over()   # over GF(5)
sage: K.gens()
(z2,)
sage: K.ngens()
1
sage: L = GF(5^4).over(K)
sage: L.gens(GF(5))
(z4, z2)
sage: L.ngens(GF(5))
2
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(2)).over()   # over GF(5)
>>> K.gens()
(z2,)
>>> K.ngens()
1
>>> L = GF(Integer(5)**Integer(4)).over(K)
```

(continues on next page)

(continued from previous page)

```
>>> L.gens(GF(Integer(5)))
(z4, z2)
>>> L.ngens(GF(Integer(5)))
2
```

print_options(***options*)

Update the printing options of this extension.

INPUT:

- `over` – integer or `Infinity` (default: `0`); the maximum number of bases included in the printing of this extension
- `base` – a base over which this extension is finite free; elements in this extension will be printed as a linear combinaison of a basis of this extension over the given base

EXAMPLES:

```
sage: A.<a> = GF(5^2).over()    # over GF(5)
sage: B.<b> = GF(5^4).over(A)
sage: C.<c> = GF(5^12).over(B)
sage: D.<d> = GF(5^24).over(C)
```

```
>>> from sage.all import *
>>> A = GF(Integer(5)**Integer(2)).over(names=('a',)); (a,) = A._first_
↪ngens(1)# over GF(5)
>>> B = GF(Integer(5)**Integer(4)).over(A, names=('b',)); (b,) = B._first_
↪ngens(1)
>>> C = GF(Integer(5)**Integer(12)).over(B, names=('c',)); (c,) = C._first_
↪ngens(1)
>>> D = GF(Integer(5)**Integer(24)).over(C, names=('d',)); (d,) = D._first_
↪ngens(1)
```

Observe what happens when we modify the option `over`:

```
sage: D
Field in d with defining polynomial
 x^2 + ((1 - a) + ((1 + 2*a) - b)*c + ((2 + a) + (1 - a)*b)*c^2)*x + c over␣
↪its base
sage: D.print_options(over=2)
sage: D
Field in d with defining polynomial x^2 + ((1 - a) + ((1 + 2*a) - b)*c + ((2␣
↪+ a) + (1 - a)*b)*c^2)*x + c over
Field in c with defining polynomial x^3 + (1 + (2 - a)*b)*x^2 + (2 + 2*b)*x -␣
↪b over
Field in b with defining polynomial x^2 + (3 - a)*x + a over its base
sage: D.print_options(over=Infinity)
sage: D
Field in d with defining polynomial x^2 + ((1 - a) + ((1 + 2*a) - b)*c + ((2␣
↪+ a) + (1 - a)*b)*c^2)*x + c over
Field in c with defining polynomial x^3 + (1 + (2 - a)*b)*x^2 + (2 + 2*b)*x -␣
↪b over
Field in b with defining polynomial x^2 + (3 - a)*x + a over
```

(continues on next page)

(continued from previous page)

```
Field in a with defining polynomial x^2 + 4*x + 2 over
Finite Field of size 5
```

```
>>> from sage.all import *
>>> D
Field in d with defining polynomial
 x^2 + ((1 - a) + ((1 + 2*a) - b)*c + ((2 + a) + (1 - a)*b)*c^2)*x + c over␣
↪its base
>>> D.print_options(over=Integer(2))
>>> D
Field in d with defining polynomial x^2 + ((1 - a) + ((1 + 2*a) - b)*c + ((2␣
↪+ a) + (1 - a)*b)*c^2)*x + c over
Field in c with defining polynomial x^3 + (1 + (2 - a)*b)*x^2 + (2 + 2*b)*x -␣
↪b over
Field in b with defining polynomial x^2 + (3 - a)*x + a over its base
>>> D.print_options(over=Infinity)
>>> D
Field in d with defining polynomial x^2 + ((1 - a) + ((1 + 2*a) - b)*c + ((2␣
↪+ a) + (1 - a)*b)*c^2)*x + c over
Field in c with defining polynomial x^3 + (1 + (2 - a)*b)*x^2 + (2 + 2*b)*x -␣
↪b over
Field in b with defining polynomial x^2 + (3 - a)*x + a over
Field in a with defining polynomial x^2 + 4*x + 2 over
Finite Field of size 5
```

Now the option `base`:

```
sage: d^2
-c + ((-1 + a) + ((-1 + 3*a) + b)*c + ((3 - a) + (-1 + a)*b)*c^2)*d
sage: D.basis_over(B)
[1, c, c^2, d, c*d, c^2*d]
sage: D.print_options(base=B)
sage: d^2
-c + (-1 + a)*d + ((-1 + 3*a) + b)*c*d + ((3 - a) + (-1 + a)*b)*c^2*d
sage: D.basis_over(A)
[1, b, c, b*c, c^2, b*c^2, d, b*d, c*d, b*c*d, c^2*d, b*c^2*d]
sage: D.print_options(base=A)
sage: d^2
-c + (-1 + a)*d + (-1 + 3*a)*c*d + b*c*d + (3 - a)*c^2*d + (-1 + a)*b*c^2*d
```

```
>>> from sage.all import *
>>> d**Integer(2)
-c + ((-1 + a) + ((-1 + 3*a) + b)*c + ((3 - a) + (-1 + a)*b)*c^2)*d
>>> D.basis_over(B)
[1, c, c^2, d, c*d, c^2*d]
>>> D.print_options(base=B)
>>> d**Integer(2)
-c + (-1 + a)*d + ((-1 + 3*a) + b)*c*d + ((3 - a) + (-1 + a)*b)*c^2*d
>>> D.basis_over(A)
[1, b, c, b*c, c^2, b*c^2, d, b*d, c*d, b*c*d, c^2*d, b*c^2*d]
>>> D.print_options(base=A)
>>> d**Integer(2)
```

(continues on next page)

(continued from previous page)

```
-c + (-1 + a)*d + (-1 + 3*a)*c*d + b*c*d + (3 - a)*c^2*d + (-1 + a)*b*c^2*d
```

random_element()

Return a random element in this extension.

EXAMPLES:

```
sage: K = GF(5^2).over()    # over GF(5)
sage: x = K.random_element(); x    # random
3 + z2
sage: x.parent()
Field in z2 with defining polynomial x^2 + 4*x + 2 over its base
sage: x.parent() is K
True
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(2)).over()    # over GF(5)
>>> x = K.random_element(); x    # random
3 + z2
>>> x.parent()
Field in z2 with defining polynomial x^2 + 4*x + 2 over its base
>>> x.parent() is K
True
```

relative_degree()

Return the degree of this extension over its base.

EXAMPLES:

```
sage: A = GF(5^4).over(GF(5^2))
sage: A.relative_degree()
2
```

```
>>> from sage.all import *
>>> A = GF(Integer(5)**Integer(4)).over(GF(Integer(5)**Integer(2)))
>>> A.relative_degree()
2
```

See also

degree(), *absolute_degree()*

sage.rings.ring_extension.**common_base**(*K*, *L*, *degree*)

Return a common base on which `K` and `L` are defined.

INPUT:

- `K` – a commutative ring
- `L` – a commutative ring
- `degree` – boolean; if `True`, return the degree of `K` and `L` over their common base

EXAMPLES:

```
sage: from sage.rings.ring_extension import common_base

sage: common_base(GF(5^3), GF(5^7), False)
Finite Field of size 5
sage: common_base(GF(5^3), GF(5^7), True)
(Finite Field of size 5, 3, 7)

sage: common_base(GF(5^3), GF(7^5), False)
Traceback (most recent call last):
...
NotImplementedError: unable to find a common base
```

```
>>> from sage.all import *
>>> from sage.rings.ring_extension import common_base

>>> common_base(GF(Integer(5)**Integer(3)), GF(Integer(5)**Integer(7)), False)
Finite Field of size 5
>>> common_base(GF(Integer(5)**Integer(3)), GF(Integer(5)**Integer(7)), True)
(Finite Field of size 5, 3, 7)

>>> common_base(GF(Integer(5)**Integer(3)), GF(Integer(7)**Integer(5)), False)
Traceback (most recent call last):
...
NotImplementedError: unable to find a common base
```

When `degree` is set to `True`, we only look up for bases on which both `K` and `L` are finite:

```
sage: S.<x> = QQ[]
sage: common_base(S, QQ, False)
Rational Field
sage: common_base(S, QQ, True)
Traceback (most recent call last):
...
NotImplementedError: unable to find a common base
```

```
>>> from sage.all import *
>>> S = QQ['x']; (x,) = S._first_ngens(1)
>>> common_base(S, QQ, False)
Rational Field
>>> common_base(S, QQ, True)
Traceback (most recent call last):
...
NotImplementedError: unable to find a common base
```

`sage.rings.ring_extension.`**`generators`**(*ring*, *base*)

Return the generators of `ring` over `base`.

INPUT:

- `ring` – a commutative ring
- `base` – a commutative ring

EXAMPLES:

```
sage: from sage.rings.ring_extension import generators
sage: S.<x> = QQ[]
sage: T.<y> = S[]

sage: generators(T, S)
(y,)
sage: generators(T, QQ)
(y, x)
```

```
>>> from sage.all import *
>>> from sage.rings.ring_extension import generators
>>> S = QQ['x']; (x,) = S._first_ngens(1)
>>> T = S['y']; (y,) = T._first_ngens(1)

>>> generators(T, S)
(y,)
>>> generators(T, QQ)
(y, x)
```

sage.rings.ring_extension.**tower_bases**(*ring*, *degree*)

Return the list of bases of `ring` (including itself); if degree is `True`, restrict to finite extensions and return in addition the degree of `ring` over each base.

INPUT:

- `ring` – a commutative ring
- `degree` – boolean

EXAMPLES:

```
sage: from sage.rings.ring_extension import tower_bases
sage: S.<x> = QQ[]
sage: T.<y> = S[]
sage: tower_bases(T, False)
([Univariate Polynomial Ring in y over
   Univariate Polynomial Ring in x over Rational Field,
  Univariate Polynomial Ring in x over Rational Field,
  Rational Field],
 [])
sage: tower_bases(T, True)
([Univariate Polynomial Ring in y over
   Univariate Polynomial Ring in x over Rational Field],
 [1])

sage: K.<a> = Qq(5^2)
sage: L.<w> = K.extension(x^3 - 5)
sage: tower_bases(L, True)
([5-adic Eisenstein Extension Field in w defined by x^3 - 5 over its base field,
  5-adic Unramified Extension Field in a defined by x^2 + 4*x + 2,
  5-adic Field with capped relative precision 20],
 [1, 3, 6])
```

```
>>> from sage.all import *
>>> from sage.rings.ring_extension import tower_bases
>>> S = QQ['x']; (x,) = S._first_ngens(1)
>>> T = S['y']; (y,) = T._first_ngens(1)
>>> tower_bases(T, False)
([Univariate Polynomial Ring in y over
   Univariate Polynomial Ring in x over Rational Field,
  Univariate Polynomial Ring in x over Rational Field,
  Rational Field],
 [])
>>> tower_bases(T, True)
([Univariate Polynomial Ring in y over
   Univariate Polynomial Ring in x over Rational Field],
 [1])

>>> K = Qq(Integer(5)**Integer(2), names=('a',)); (a,) = K._first_ngens(1)
>>> L = K.extension(x**Integer(3) - Integer(5), names=('w',)); (w,) = L._first_
↪ngens(1)
>>> tower_bases(L, True)
([5-adic Eisenstein Extension Field in w defined by x^3 - 5 over its base field,
  5-adic Unramified Extension Field in a defined by x^2 + 4*x + 2,
  5-adic Field with capped relative precision 20],
 [1, 3, 6])
```

sage.rings.ring_extension.**variable_names**(*ring*, *base*)

Return the variable names of the generators of `ring` over `base`.

INPUT:

- `ring` – a commutative ring
- `base` – a commutative ring

EXAMPLES:

```
sage: from sage.rings.ring_extension import variable_names
sage: S.<x> = QQ[]
sage: T.<y> = S[]

sage: variable_names(T, S)
('y',)
sage: variable_names(T, QQ)
('y', 'x')
```

```
>>> from sage.all import *
>>> from sage.rings.ring_extension import variable_names
>>> S = QQ['x']; (x,) = S._first_ngens(1)
>>> T = S['y']; (y,) = T._first_ngens(1)

>>> variable_names(T, S)
('y',)
>>> variable_names(T, QQ)
('y', 'x')
```

7.2 Elements lying in extension of rings

AUTHOR:

- Xavier Caruso (2019)

class sage.rings.ring_extension_element.**RingExtensionElement**

Bases: CommutativeAlgebraElement

Generic class for elements lying in ring extensions.

additive_order()

Return the additive order of this element.

EXAMPLES:

```
sage: K.<a> = GF(5^4).over(GF(5^2))                                    #␣
↪needs sage.rings.finite_rings
sage: a.additive_order()                                               #␣
↪needs sage.rings.finite_rings
5
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(4)).over(GF(Integer(5)**Integer(2)), names=('a
↪',)); (a,) = K._first_ngens(1)# needs sage.rings.finite_rings
>>> a.additive_order()                                                 #␣
↪needs sage.rings.finite_rings
5
```

backend(*force=False*)

Return the backend of this element.

INPUT:

- `force` – boolean (default: `False`); if `False`, raise an error if the backend is not exposed

EXAMPLES:

```
sage: F = GF(5^2)
sage: K.<z> = GF(5^4).over(F)
sage: x = z^10
sage: x
(z2 + 2) + (3*z2 + 1)*z
sage: y = x.backend()
sage: y
4*z4^3 + 2*z4^2 + 4*z4 + 4
sage: y.parent()
Finite Field in z4 of size 5^4
```

```
>>> from sage.all import *
>>> F = GF(Integer(5)**Integer(2))
>>> K = GF(Integer(5)**Integer(4)).over(F, names=('z',)); (z,) = K._first_
↪ngens(1)
>>> x = z**Integer(10)
>>> x
(z2 + 2) + (3*z2 + 1)*z
```

(continues on next page)

(continued from previous page)

```
>>> y = x.backend()
>>> y
4*z4^3 + 2*z4^2 + 4*z4 + 4
>>> y.parent()
Finite Field in z4 of size 5^4
```

in_base()

Return this element as an element of the base.

EXAMPLES:

```
sage: F = GF(5^2)
sage: K.<z> = GF(5^4).over(F)
sage: x = z^3 + z^2 + z + 4
sage: y = x.in_base()
sage: y
z2 + 1
sage: y.parent()
Finite Field in z2 of size 5^2
```

```
>>> from sage.all import *
>>> F = GF(Integer(5)**Integer(2))
>>> K = GF(Integer(5)**Integer(4)).over(F, names=('z',)); (z,) = K._first_
↪ngens(1)
>>> x = z**Integer(3) + z**Integer(2) + z + Integer(4)
>>> y = x.in_base()
>>> y
z2 + 1
>>> y.parent()
Finite Field in z2 of size 5^2
```

When the element is not in the base, an error is raised:

```
sage: z.in_base()                                                          #␣
↪needs sage.rings.finite_rings
Traceback (most recent call last):
...
ValueError: z is not in the base
```

```
>>> from sage.all import *
>>> z.in_base()                                                            #␣
↪needs sage.rings.finite_rings
Traceback (most recent call last):
...
ValueError: z is not in the base
```

```
sage: S.<X> = F[]
sage: E = S.over(F)
sage: f = E(1)
sage: g = f.in_base(); g
1
```

(continues on next page)

(continued from previous page)

```
sage: g.parent()
Finite Field in z2 of size 5^2
```

```
>>> from sage.all import *
>>> S = F['X']; (X,) = S._first_ngens(1)
>>> E = S.over(F)
>>> f = E(Integer(1))
>>> g = f.in_base(); g
1
>>> g.parent()
Finite Field in z2 of size 5^2
```

is_nilpotent()

Return whether if this element is nilpotent in this ring.

EXAMPLES:

```
sage: A.<x> = PolynomialRing(QQ)
sage: E = A.over(QQ)
sage: E(0).is_nilpotent()
True
sage: E(x).is_nilpotent()
False
```

```
>>> from sage.all import *
>>> A = PolynomialRing(QQ, names=('x',)); (x,) = A._first_ngens(1)
>>> E = A.over(QQ)
>>> E(Integer(0)).is_nilpotent()
True
>>> E(x).is_nilpotent()
False
```

is_prime()

Return whether this element is a prime element in this ring.

EXAMPLES:

```
sage: A.<x> = PolynomialRing(QQ)
sage: E = A.over(QQ)
sage: E(x^2 + 1).is_prime()                                                 #␣
↪needs sage.libs.pari
True
sage: E(x^2 - 1).is_prime()                                                 #␣
↪needs sage.libs.pari
False
```

```
>>> from sage.all import *
>>> A = PolynomialRing(QQ, names=('x',)); (x,) = A._first_ngens(1)
>>> E = A.over(QQ)
>>> E(x**Integer(2) + Integer(1)).is_prime()                                ␣
↪               # needs sage.libs.pari
True
```

(continues on next page)

(continued from previous page)

```
>>> E(x**Integer(2) - Integer(1)).is_prime()                                    ␣
↪               # needs sage.libs.pari
False
```

is_square(*root=False*)

Return whether this element is a square in this ring.

INPUT:

- `root` – boolean (default: `False`); if `True`, return also a square root

EXAMPLES:

```
sage: K.<a> = GF(5^3).over()
sage: a.is_square()
False
sage: a.is_square(root=True)
(False, None)
sage: b = a + 1
sage: b.is_square()
True
sage: b.is_square(root=True)
(True, 2 + 3*a + a^2)
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3)).over(names=('a',)); (a,) = K._first_
↪ngens(1)
>>> a.is_square()
False
>>> a.is_square(root=True)
(False, None)
>>> b = a + Integer(1)
>>> b.is_square()
True
>>> b.is_square(root=True)
(True, 2 + 3*a + a^2)
```

is_unit()

Return whether if this element is a unit in this ring.

EXAMPLES:

```
sage: A.<x> = PolynomialRing(QQ)
sage: E = A.over(QQ)
sage: E(4).is_unit()
True
sage: E(x).is_unit()
False
```

```
>>> from sage.all import *
>>> A = PolynomialRing(QQ, names=('x',)); (x,) = A._first_ngens(1)
>>> E = A.over(QQ)
>>> E(Integer(4)).is_unit()
```

(continues on next page)

(continued from previous page)

```
True
>>> E(x).is_unit()
False
```

multiplicative_order()

Return the multiplicite order of this element.

EXAMPLES:

```
sage: K.<a> = GF(5^4).over(GF(5^2))                                    #␣
↪needs sage.rings.finite_rings
sage: a.multiplicative_order()                                         #␣
↪needs sage.rings.finite_rings
624
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(4)).over(GF(Integer(5)**Integer(2)), names=('a
↪',)); (a,) = K._first_ngens(1)# needs sage.rings.finite_rings
>>> a.multiplicative_order()                                           #␣
↪needs sage.rings.finite_rings
624
```

sqrt(*extend=True*, *all=False*, *name=None*)

Return a square root or all square roots of this element.

INPUT:

- `extend` – boolean (default: `True`); if `True`, return a square root in an extension ring, if necessary. Otherwise, raise a `ValueError` if the root is not in the ring.
- `all` – boolean (default: `False`); if `True`, return all square roots of this element, instead of just one
- `name` – required when `extend=True` and `self` is not a square; this will be the name of the generator extension

Note

The option `extend=True` is often not implemented.

EXAMPLES:

```
sage: K.<a> = GF(5^3).over()
sage: b = a + 1
sage: b.sqrt()
2 + 3*a + a^2
sage: b.sqrt(all=True)
[2 + 3*a + a^2, 3 + 2*a - a^2]
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3)).over(names=('a',)); (a,) = K._first_
↪ngens(1)
>>> b = a + Integer(1)
>>> b.sqrt()
```

(continues on next page)

(continued from previous page)

```
2 + 3*a + a^2
>>> b.sqrt(all=True)
[2 + 3*a + a^2, 3 + 2*a - a^2]
```

class sage.rings.ring_extension_element.**RingExtensionFractionFieldElement**

Bases: *RingExtensionElement*

A class for elements lying in fraction fields of ring extensions.

denominator()

Return the denominator of this element.

EXAMPLES:

```
sage: R.<x> = ZZ[]
sage: A.<a> = ZZ.extension(x^2 - 2)
sage: OK = A.over()  # over ZZ
sage: K = OK.fraction_field(); K
Fraction Field of
 Maximal Order generated by a in Number Field in a
 with defining polynomial x^2 - 2 over its base
sage: x = K(1/a); x
a/2
sage: denom = x.denominator(); denom
2
```

```
>>> from sage.all import *
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> A = ZZ.extension(x**Integer(2) - Integer(2), names=('a',)); (a,) = A._
↪first_ngens(1)
>>> OK = A.over()  # over ZZ
>>> K = OK.fraction_field(); K
Fraction Field of
 Maximal Order generated by a in Number Field in a
 with defining polynomial x^2 - 2 over its base
>>> x = K(Integer(1)/a); x
a/2
>>> denom = x.denominator(); denom
2
```

The denominator is an element of the ring which was used to construct the fraction field:

```
sage: denom.parent()                                                    #␣
↪needs sage.rings.number_field
Maximal Order generated by a in Number Field in a with defining polynomial x^
↪2 - 2 over its base
sage: denom.parent() is OK                                              #␣
↪needs sage.rings.number_field
True
```

```
>>> from sage.all import *
>>> denom.parent()                                                      #␣
↪needs sage.rings.number_field
```

(continues on next page)

(continued from previous page)

```
Maximal Order generated by a in Number Field in a with defining polynomial x^
↪2 - 2 over its base
>>> denom.parent() is OK                                                    #␣
↪needs sage.rings.number_field
True
```

numerator()

Return the numerator of this element.

EXAMPLES:

```
sage: x = polygen(ZZ, 'x')
sage: A.<a> = ZZ.extension(x^2 - 2)
sage: OK = A.over()  # over ZZ
sage: K = OK.fraction_field(); K
Fraction Field of Maximal Order generated by a in Number Field in a
 with defining polynomial x^2 - 2 over its base
sage: x = K(1/a); x
a/2
sage: num = x.numerator(); num
a
```

```
>>> from sage.all import *
>>> x = polygen(ZZ, 'x')
>>> A = ZZ.extension(x**Integer(2) - Integer(2), names=('a',)); (a,) = A._
↪first_ngens(1)
>>> OK = A.over()  # over ZZ
>>> K = OK.fraction_field(); K
Fraction Field of Maximal Order generated by a in Number Field in a
 with defining polynomial x^2 - 2 over its base
>>> x = K(Integer(1)/a); x
a/2
>>> num = x.numerator(); num
a
```

The numerator is an element of the ring which was used to construct the fraction field:

```
sage: num.parent()                                                          #␣
↪needs sage.rings.number_field
Maximal Order generated by a in Number Field in a
 with defining polynomial x^2 - 2 over its base
sage: num.parent() is OK                                                    #␣
↪needs sage.rings.number_field
True
```

```
>>> from sage.all import *
>>> num.parent()                                                            #␣
↪needs sage.rings.number_field
Maximal Order generated by a in Number Field in a
 with defining polynomial x^2 - 2 over its base
>>> num.parent() is OK                                                      #␣
↪needs sage.rings.number_field
```

(continues on next page)

(continued from previous page)

```
True
```

class sage.rings.ring_extension_element.**RingExtensionWithBasisElement**

Bases: *RingExtensionElement*

A class for elements lying in finite free extensions.

charpoly(*base=None*, *var='x'*)

Return the characteristic polynomial of this element over `base`.

INPUT:

- `base` – a commutative ring (which might be itself an extension) or `None`

EXAMPLES:

```
sage: F = GF(5)
sage: K.<a> = GF(5^3).over(F)
sage: L.<b> = GF(5^6).over(K)
sage: u = a/(1+b)
sage: chi = u.charpoly(K); chi
x^2 + (1 + 2*a + 3*a^2)*x + 3 + 2*a^2
```

```
>>> from sage.all import *
>>> F = GF(Integer(5))
>>> K = GF(Integer(5)**Integer(3)).over(F, names=('a',)); (a,) = K._first_
↪ngens(1)
>>> L = GF(Integer(5)**Integer(6)).over(K, names=('b',)); (b,) = L._first_
↪ngens(1)
>>> u = a/(Integer(1)+b)
>>> chi = u.charpoly(K); chi
x^2 + (1 + 2*a + 3*a^2)*x + 3 + 2*a^2
```

We check that the charpoly has coefficients in the base ring:

```
sage: chi.base_ring()                                                    #␣
↪needs sage.rings.finite_rings
Field in a with defining polynomial x^3 + 3*x + 3 over its base
sage: chi.base_ring() is K                                               #␣
↪needs sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> chi.base_ring()                                                      #␣
↪needs sage.rings.finite_rings
Field in a with defining polynomial x^3 + 3*x + 3 over its base
>>> chi.base_ring() is K                                                 #␣
↪needs sage.rings.finite_rings
True
```

and that it annihilates u:

```
sage: chi(u)                                                             #␣
↪needs sage.rings.finite_rings
0
```

```
>>> from sage.all import *
>>> chi(u)                                                                    #␣
↪needs sage.rings.finite_rings
0
```

Similarly, one can compute the characteristic polynomial over F:

```
sage: u.charpoly(F)                                                           #␣
↪needs sage.rings.finite_rings
x^6 + x^4 + 2*x^3 + 3*x + 4
```

```
>>> from sage.all import *
>>> u.charpoly(F)                                                             #␣
↪needs sage.rings.finite_rings
x^6 + x^4 + 2*x^3 + 3*x + 4
```

A different variable name can be specified:

```
sage: u.charpoly(F, var='t')                                                  #␣
↪needs sage.rings.finite_rings
t^6 + t^4 + 2*t^3 + 3*t + 4
```

```
>>> from sage.all import *
>>> u.charpoly(F, var='t')                                                    #␣
↪needs sage.rings.finite_rings
t^6 + t^4 + 2*t^3 + 3*t + 4
```

If `base` is omitted, it is set to its default which is the base of the extension:

```
sage: u.charpoly()                                                            #␣
↪needs sage.rings.finite_rings
x^2 + (1 + 2*a + 3*a^2)*x + 3 + 2*a^2
```

```
>>> from sage.all import *
>>> u.charpoly()                                                              #␣
↪needs sage.rings.finite_rings
x^2 + (1 + 2*a + 3*a^2)*x + 3 + 2*a^2
```

Note that `base` must be an explicit base over which the extension has been defined (as listed by the method *bases()*):

```
sage: u.charpoly(GF(5^2))                                                     #␣
↪needs sage.rings.finite_rings
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field in z2 of size 5^2
```

```
>>> from sage.all import *
>>> u.charpoly(GF(Integer(5)**Integer(2)))                                    ␣
↪               # needs sage.rings.finite_rings
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field in z2 of size 5^2
```

matrix(*base=None*)

Return the matrix of the multiplication by this element (in the basis output by *basis_over()*).

INPUT:

- `base` – a commutative ring (which might be itself an extension) or `None`

EXAMPLES:

```
sage: K.<a> = GF(5^3).over()   # over GF(5)
sage: L.<b> = GF(5^6).over(K)
sage: u = a/(1+b)
sage: u
(2 + a + 3*a^2) + (3 + 3*a + a^2)*b
sage: b*u
(3 + 2*a^2) + (2 + 2*a - a^2)*b
sage: u.matrix(K)
[2 + a + 3*a^2 3 + 3*a + a^2]
[    3 + 2*a^2 2 + 2*a - a^2]
sage: u.matrix(GF(5))
[2 1 3 3 3 1]
[1 3 1 2 0 3]
[2 3 3 1 3 0]
[3 0 2 2 2 4]
[4 2 0 3 0 2]
[0 4 2 4 2 0]
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3)).over(names=('a',)); (a,) = K._first_
↪ngens(1)# over GF(5)
>>> L = GF(Integer(5)**Integer(6)).over(K, names=('b',)); (b,) = L._first_
↪ngens(1)
>>> u = a/(Integer(1)+b)
>>> u
(2 + a + 3*a^2) + (3 + 3*a + a^2)*b
>>> b*u
(3 + 2*a^2) + (2 + 2*a - a^2)*b
>>> u.matrix(K)
[2 + a + 3*a^2 3 + 3*a + a^2]
[    3 + 2*a^2 2 + 2*a - a^2]
>>> u.matrix(GF(Integer(5)))
[2 1 3 3 3 1]
[1 3 1 2 0 3]
[2 3 3 1 3 0]
[3 0 2 2 2 4]
[4 2 0 3 0 2]
[0 4 2 4 2 0]
```

If `base` is omitted, it is set to its default which is the base of the extension:

```
sage: u.matrix()                                                              #␣
↪needs sage.rings.finite_rings
[2 + a + 3*a^2 3 + 3*a + a^2]
[    3 + 2*a^2 2 + 2*a - a^2]
```

```
>>> from sage.all import *
>>> u.matrix()                                                                #␣
↪needs sage.rings.finite_rings
[2 + a + 3*a^2 3 + 3*a + a^2]
[    3 + 2*a^2 2 + 2*a - a^2]
```

Note that `base` must be an explicit base over which the extension has been defined (as listed by the method *bases()*):

```
sage: u.matrix(GF(5^2))                                                       #␣
↪needs sage.rings.finite_rings
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field in z2 of size 5^2
```

```
>>> from sage.all import *
>>> u.matrix(GF(Integer(5)**Integer(2)))                                      ␣
↪               # needs sage.rings.finite_rings
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field in z2 of size 5^2
```

minpoly(*base=None*, *var='x'*)

Return the minimal polynomial of this element over `base`.

INPUT:

- `base` – a commutative ring (which might be itself an extension) or `None`

EXAMPLES:

```
sage: F = GF(5)
sage: K.<a> = GF(5^3).over(F)
sage: L.<b> = GF(5^6).over(K)
sage: u = 1 / (a+b)
sage: chi = u.minpoly(K); chi
x^2 + (2*a + a^2)*x - 1 + a
```

```
>>> from sage.all import *
>>> F = GF(Integer(5))
>>> K = GF(Integer(5)**Integer(3)).over(F, names=('a',)); (a,) = K._first_
↪ngens(1)
>>> L = GF(Integer(5)**Integer(6)).over(K, names=('b',)); (b,) = L._first_
↪ngens(1)
>>> u = Integer(1) / (a+b)
>>> chi = u.minpoly(K); chi
x^2 + (2*a + a^2)*x - 1 + a
```

We check that the minimal polynomial has coefficients in the base ring:

```
sage: chi.base_ring()                                                         #␣
↪needs sage.rings.finite_rings
Field in a with defining polynomial x^3 + 3*x + 3 over its base
sage: chi.base_ring() is K                                                    #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> chi.base_ring()                                                    #␣
↪needs sage.rings.finite_rings
Field in a with defining polynomial x^3 + 3*x + 3 over its base
>>> chi.base_ring() is K                                               #␣
↪needs sage.rings.finite_rings
True
```

and that it annihilates u:

```
sage: chi(u)                                                           #␣
↪needs sage.rings.finite_rings
0
```

```
>>> from sage.all import *
>>> chi(u)                                                             #␣
↪needs sage.rings.finite_rings
0
```

Similarly, one can compute the minimal polynomial over F:

```
sage: u.minpoly(F)                                                     #␣
↪needs sage.rings.finite_rings
x^6 + 4*x^5 + x^4 + 2*x^2 + 3
```

```
>>> from sage.all import *
>>> u.minpoly(F)                                                       #␣
↪needs sage.rings.finite_rings
x^6 + 4*x^5 + x^4 + 2*x^2 + 3
```

A different variable name can be specified:

```
sage: u.minpoly(F, var='t')                                            #␣
↪needs sage.rings.finite_rings
t^6 + 4*t^5 + t^4 + 2*t^2 + 3
```

```
>>> from sage.all import *
>>> u.minpoly(F, var='t')                                              #␣
↪needs sage.rings.finite_rings
t^6 + 4*t^5 + t^4 + 2*t^2 + 3
```

If `base` is omitted, it is set to its default which is the base of the extension:

```
sage: u.minpoly()                                                      #␣
↪needs sage.rings.finite_rings
x^2 + (2*a + a^2)*x - 1 + a
```

```
>>> from sage.all import *
>>> u.minpoly()                                                              #␣
↪needs sage.rings.finite_rings
x^2 + (2*a + a^2)*x - 1 + a
```

Note that `base` must be an explicit base over which the extension has been defined (as listed by the method *bases()*):

```
sage: u.minpoly(GF(5^2))                                                     #␣
↪needs sage.rings.finite_rings
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field in z2 of size 5^2
```

```
>>> from sage.all import *
>>> u.minpoly(GF(Integer(5)**Integer(2)))                                    ␣
↪               # needs sage.rings.finite_rings
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field in z2 of size 5^2
```

norm(*base=None*)

Return the norm of this element over `base`.

INPUT:

- `base` – a commutative ring (which might be itself an extension) or `None`

EXAMPLES:

```
sage: F = GF(5)
sage: K.<a> = GF(5^3).over(F)
sage: L.<b> = GF(5^6).over(K)
sage: u = a/(1+b)
sage: nr = u.norm(K); nr
3 + 2*a^2
```

```
>>> from sage.all import *
>>> F = GF(Integer(5))
>>> K = GF(Integer(5)**Integer(3)).over(F, names=('a',)); (a,) = K._first_
↪ngens(1)
>>> L = GF(Integer(5)**Integer(6)).over(K, names=('b',)); (b,) = L._first_
↪ngens(1)
>>> u = a/(Integer(1)+b)
>>> nr = u.norm(K); nr
3 + 2*a^2
```

We check that the norm lives in the base ring:

```
sage: nr.parent()                                                            #␣
↪needs sage.rings.finite_rings
Field in a with defining polynomial x^3 + 3*x + 3 over its base
sage: nr.parent() is K                                                       #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> nr.parent()                                                          #␣
↪needs sage.rings.finite_rings
Field in a with defining polynomial x^3 + 3*x + 3 over its base
>>> nr.parent() is K                                                     #␣
↪needs sage.rings.finite_rings
True
```

Similarly, one can compute the norm over F:

```
sage: u.norm(F)                                                          #␣
↪needs sage.rings.finite_rings
4
```

```
>>> from sage.all import *
>>> u.norm(F)                                                            #␣
↪needs sage.rings.finite_rings
4
```

We check the transitivity of the norm:

```
sage: u.norm(F) == nr.norm(F)                                            #␣
↪needs sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> u.norm(F) == nr.norm(F)                                              #␣
↪needs sage.rings.finite_rings
True
```

If `base` is omitted, it is set to its default which is the base of the extension:

```
sage: u.norm()                                                           #␣
↪needs sage.rings.finite_rings
3 + 2*a^2
```

```
>>> from sage.all import *
>>> u.norm()                                                             #␣
↪needs sage.rings.finite_rings
3 + 2*a^2
```

Note that `base` must be an explicit base over which the extension has been defined (as listed by the method *bases()*):

```
sage: u.norm(GF(5^2))                                                    #␣
↪needs sage.rings.finite_rings
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field in z2 of size 5^2
```

```
>>> from sage.all import *
>>> u.norm(GF(Integer(5)**Integer(2)))
↪               # needs sage.rings.finite_rings
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field in z2 of size 5^2
```

polynomial (*base=None*, *var='x'*)

Return a polynomial (in one or more variables) over `base` whose evaluation at the generators of the parent equals this element.

INPUT:

- `base` – a commutative ring (which might be itself an extension) or `None`

EXAMPLES:

```
sage: F.<a> = GF(5^2).over()  # over GF(5)
sage: K.<b> = GF(5^4).over(F)
sage: L.<c> = GF(5^12).over(K)
sage: u = 1/(a + b + c); u
(2 + (-1 - a)*b) + ((2 + 3*a) + (1 - a)*b)*c + ((-1 - a) - a*b)*c^2
sage: P = u.polynomial(K); P
((-1 - a) - a*b)*x^2 + ((2 + 3*a) + (1 - a)*b)*x + 2 + (-1 - a)*b
sage: P.base_ring() is K
True
sage: P(c) == u
True
```

```
>>> from sage.all import *
>>> F = GF(Integer(5)**Integer(2)).over(names=('a',)); (a,) = F._first_
↪ngens(1)# over GF(5)
>>> K = GF(Integer(5)**Integer(4)).over(F, names=('b',)); (b,) = K._first_
↪ngens(1)
>>> L = GF(Integer(5)**Integer(12)).over(K, names=('c',)); (c,) = L._first_
↪ngens(1)
>>> u = Integer(1)/(a + b + c); u
(2 + (-1 - a)*b) + ((2 + 3*a) + (1 - a)*b)*c + ((-1 - a) - a*b)*c^2
>>> P = u.polynomial(K); P
((-1 - a) - a*b)*x^2 + ((2 + 3*a) + (1 - a)*b)*x + 2 + (-1 - a)*b
>>> P.base_ring() is K
True
>>> P(c) == u
True
```

When the base is F, we obtain a bivariate polynomial:

```
sage: P = u.polynomial(F); P                                                #␣
↪needs sage.rings.finite_rings
(-a)*x0^2*x1 + (-1 - a)*x0^2 + (1 - a)*x0*x1 + (2 + 3*a)*x0 + (-1 - a)*x1 + 2
```

```
>>> from sage.all import *
>>> P = u.polynomial(F); P                                                  #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.rings.finite_rings
(-a)*x0^2*x1 + (-1 - a)*x0^2 + (1 - a)*x0*x1 + (2 + 3*a)*x0 + (-1 - a)*x1 + 2
```

We check that its value at the generators is the element we started with:

```
sage: L.gens(F)                                                                #␣
↪needs sage.rings.finite_rings
(c, b)
sage: P(c, b) == u                                                             #␣
↪needs sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> L.gens(F)                                                                  #␣
↪needs sage.rings.finite_rings
(c, b)
>>> P(c, b) == u                                                               #␣
↪needs sage.rings.finite_rings
True
```

Similarly, when the base is `GF(5)`, we get a trivariate polynomial:

sage: P = u.polynomial(GF(5)); P # needs sage.rings.finite_rings -x0^2*x1*x2 - x0^2*x2 - x0*x1*x2 - x0^2 + x0*x1 - 2*x0*x2 - x1*x2 + 2*x0 - x1 + 2 sage: P(c, b, a) == u # needs sage.rings.finite_rings True

Different variable names can be specified:

```
sage: u.polynomial(GF(5), var='y')                                             #␣
↪needs sage.rings.finite_rings
-y0^2*y1*y2 - y0^2*y2 - y0*y1*y2 - y0^2 + y0*y1 - 2*y0*y2 - y1*y2 + 2*y0 - y1␣
↪+ 2
sage: u.polynomial(GF(5), var=['x','y','z'])                                   #␣
↪needs sage.rings.finite_rings
-x^2*y*z - x^2*z - x*y*z - x^2 + x*y - 2*x*z - y*z + 2*x - y + 2
```

```
>>> from sage.all import *
>>> u.polynomial(GF(Integer(5)), var='y')                                      ␣
↪     # needs sage.rings.finite_rings
-y0^2*y1*y2 - y0^2*y2 - y0*y1*y2 - y0^2 + y0*y1 - 2*y0*y2 - y1*y2 + 2*y0 - y1␣
↪+ 2
>>> u.polynomial(GF(Integer(5)), var=['x','y','z'])                           ␣
↪     # needs sage.rings.finite_rings
-x^2*y*z - x^2*z - x*y*z - x^2 + x*y - 2*x*z - y*z + 2*x - y + 2
```

If `base` is omitted, it is set to its default which is the base of the extension:

```
sage: u.polynomial()                                                           #␣
↪needs sage.rings.finite_rings
((-1 - a) - a*b)*x^2 + ((2 + 3*a) + (1 - a)*b)*x + 2 + (-1 - a)*b
```

```
>>> from sage.all import *
>>> u.polynomial()                                                             #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.rings.finite_rings
((-1 - a) - a*b)*x^2 + ((2 + 3*a) + (1 - a)*b)*x + 2 + (-1 - a)*b
```

Note that `base` must be an explicit base over which the extension has been defined (as listed by the method *bases()*):

```
sage: u.polynomial(GF(5^3))                                                 #␣
↪needs sage.rings.finite_rings
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field in z3 of size 5^3
```

```
>>> from sage.all import *
>>> u.polynomial(GF(Integer(5)**Integer(3)))                                ␣
↪               # needs sage.rings.finite_rings
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field in z3 of size 5^3
```

trace(*base=None*)

Return the trace of this element over `base`.

INPUT:

- `base` – a commutative ring (which might be itself an extension) or `None`

EXAMPLES:

```
sage: F = GF(5)
sage: K.<a> = GF(5^3).over(F)
sage: L.<b> = GF(5^6).over(K)
sage: u = a/(1+b)
sage: tr = u.trace(K); tr
-1 + 3*a + 2*a^2
```

```
>>> from sage.all import *
>>> F = GF(Integer(5))
>>> K = GF(Integer(5)**Integer(3)).over(F, names=('a',)); (a,) = K._first_
↪ngens(1)
>>> L = GF(Integer(5)**Integer(6)).over(K, names=('b',)); (b,) = L._first_
↪ngens(1)
>>> u = a/(Integer(1)+b)
>>> tr = u.trace(K); tr
-1 + 3*a + 2*a^2
```

We check that the trace lives in the base ring:

```
sage: tr.parent()                                                           #␣
↪needs sage.rings.finite_rings
Field in a with defining polynomial x^3 + 3*x + 3 over its base
sage: tr.parent() is K                                                      #␣
↪needs sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> tr.parent()                                                              #␣
↪needs sage.rings.finite_rings
Field in a with defining polynomial x^3 + 3*x + 3 over its base
>>> tr.parent() is K                                                         #␣
↪needs sage.rings.finite_rings
True
```

Similarly, one can compute the trace over F:

```
sage: u.trace(F)                                                             #␣
↪needs sage.rings.finite_rings
0
```

```
>>> from sage.all import *
>>> u.trace(F)                                                               #␣
↪needs sage.rings.finite_rings
0
```

We check the transitivity of the trace:

```
sage: u.trace(F) == tr.trace(F)                                              #␣
↪needs sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> u.trace(F) == tr.trace(F)                                                #␣
↪needs sage.rings.finite_rings
True
```

If `base` is omitted, it is set to its default which is the base of the extension:

```
sage: u.trace()                                                              #␣
↪needs sage.rings.finite_rings
-1 + 3*a + 2*a^2
```

```
>>> from sage.all import *
>>> u.trace()                                                                #␣
↪needs sage.rings.finite_rings
-1 + 3*a + 2*a^2
```

Note that `base` must be an explicit base over which the extension has been defined (as listed by the method *bases()*):

```
sage: u.trace(GF(5^2))                                                       #␣
↪needs sage.rings.finite_rings
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field in z2 of size 5^2
```

```
>>> from sage.all import *
>>> u.trace(GF(Integer(5)**Integer(2)))                                      ␣
```

(continues on next page)

(continued from previous page)

```
↪                # needs sage.rings.finite_rings
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field in z2 of size 5^2
```

vector(*base=None*)

Return the vector of coordinates of this element over `base` (in the basis output by the method *basis_over()*).

INPUT:

- `base` – a commutative ring (which might be itself an extension) or `None`

EXAMPLES:

```
sage: F = GF(5)
sage: K.<a> = GF(5^2).over()  # over F
sage: L.<b> = GF(5^6).over(K)
sage: x = (a+b)^4; x
(-1 + a) + (3 + a)*b + (1 - a)*b^2
sage: x.vector(K)                    # basis is (1, b, b^2)
(-1 + a, 3 + a, 1 - a)
sage: x.vector(F)                    # basis is (1, a, b, a*b, b^2, a*b^2)
(4, 1, 3, 1, 1, 4)
```

```
>>> from sage.all import *
>>> F = GF(Integer(5))
>>> K = GF(Integer(5)**Integer(2)).over(names=('a',)); (a,) = K._first_
↪ngens(1)# over F
>>> L = GF(Integer(5)**Integer(6)).over(K, names=('b',)); (b,) = L._first_
↪ngens(1)
>>> x = (a+b)**Integer(4); x
(-1 + a) + (3 + a)*b + (1 - a)*b^2
>>> x.vector(K)                    # basis is (1, b, b^2)
(-1 + a, 3 + a, 1 - a)
>>> x.vector(F)                    # basis is (1, a, b, a*b, b^2, a*b^2)
(4, 1, 3, 1, 1, 4)
```

If `base` is omitted, it is set to its default which is the base of the extension:

```
sage: x.vector()                                                              #␣
↪needs sage.rings.finite_rings
(-1 + a, 3 + a, 1 - a)
```

```
>>> from sage.all import *
>>> x.vector()                                                              #␣
↪needs sage.rings.finite_rings
(-1 + a, 3 + a, 1 - a)
```

Note that `base` must be an explicit base over which the extension has been defined (as listed by the method *bases()*):

```
sage: x.vector(GF(5^3))                                                    #␣
↪needs sage.rings.finite_rings
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field in z3 of size 5^3
```

```
>>> from sage.all import *
>>> x.vector(GF(Integer(5)**Integer(3)))                                    ␣
↪               # needs sage.rings.finite_rings
Traceback (most recent call last):
...
ValueError: not (explicitly) defined over Finite Field in z3 of size 5^3
```

7.3 Morphisms between extension of rings

AUTHOR:

- Xavier Caruso (2019)

class sage.rings.ring_extension_morphism.**MapFreeModuleToRelativeRing**

Bases: Map

Base class of the module isomorphism between a ring extension and a free module over one of its bases.

is_injective()

Return whether this morphism is injective.

EXAMPLES:

```
sage: K = GF(11^6).over(GF(11^3))                                          #␣
↪needs sage.rings.finite_rings
sage: V, i, j = K.free_module()                                            #␣
↪needs sage.rings.finite_rings
sage: i.is_injective()                                                     #␣
↪needs sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> K = GF(Integer(11)**Integer(6)).over(GF(Integer(11)**Integer(3)))      ␣
↪                                  # needs sage.rings.finite_rings
>>> V, i, j = K.free_module()                                              #␣
↪needs sage.rings.finite_rings
>>> i.is_injective()                                                       #␣
↪needs sage.rings.finite_rings
True
```

is_surjective()

Return whether this morphism is surjective.

EXAMPLES:

```
sage: K = GF(11^6).over(GF(11^3))                                          #␣
↪needs sage.rings.finite_rings
```

(continues on next page)

(continued from previous page)

```
sage: V, i, j = K.free_module()                                                 #␣
↪needs sage.rings.finite_rings
sage: i.is_surjective()                                                         #␣
↪needs sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> K = GF(Integer(11)**Integer(6)).over(GF(Integer(11)**Integer(3)))          ␣
↪                                  # needs sage.rings.finite_rings
>>> V, i, j = K.free_module()                                                   #␣
↪needs sage.rings.finite_rings
>>> i.is_surjective()                                                          #␣
↪needs sage.rings.finite_rings
True
```

class sage.rings.ring_extension_morphism.**MapRelativeRingToFreeModule**

Bases: Map

Base class of the module isomorphism between a ring extension and a free module over one of its bases.

is_injective()

Return whether this morphism is injective.

EXAMPLES:

```
sage: K = GF(11^6).over(GF(11^3))                                               #␣
↪needs sage.rings.finite_rings
sage: V, i, j = K.free_module()                                                 #␣
↪needs sage.rings.finite_rings
sage: j.is_injective()                                                          #␣
↪needs sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> K = GF(Integer(11)**Integer(6)).over(GF(Integer(11)**Integer(3)))          ␣
↪                                  # needs sage.rings.finite_rings
>>> V, i, j = K.free_module()                                                   #␣
↪needs sage.rings.finite_rings
>>> j.is_injective()                                                           #␣
↪needs sage.rings.finite_rings
True
```

is_surjective()

Return whether this morphism is injective.

EXAMPLES:

```
sage: K = GF(11^6).over(GF(11^3))                                               #␣
↪needs sage.rings.finite_rings
sage: V, i, j = K.free_module()                                                 #␣
↪needs sage.rings.finite_rings
sage: j.is_surjective()                                                         #␣
↪needs sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> K = GF(Integer(11)**Integer(6)).over(GF(Integer(11)**Integer(3)))
↪                                    # needs sage.rings.finite_rings
>>> V, i, j = K.free_module()                                          #␣
↪needs sage.rings.finite_rings
>>> j.is_surjective()                                                  #␣
↪needs sage.rings.finite_rings
True
```

class sage.rings.ring_extension_morphism.**RingExtensionBackendIsomorphism**

Bases: *RingExtensionHomomorphism*

A class for implementating isomorphisms taking an element of the backend to its ring extension.

class sage.rings.ring_extension_morphism.**RingExtensionBackendReverseIsomorphism**

Bases: *RingExtensionHomomorphism*

A class for implementating isomorphisms from a ring extension to its backend.

class sage.rings.ring_extension_morphism.**RingExtensionHomomorphism**

Bases: *RingMap*

A class for ring homomorphisms between extensions.

base_map()

Return the base map of this morphism or just `None` if the base map is a coercion map.

EXAMPLES:

```
sage: F = GF(5)
sage: K.<a> = GF(5^2).over(F)                                          #␣
↪needs sage.rings.finite_rings
sage: L.<b> = GF(5^6).over(K)                                          #␣
↪needs sage.rings.finite_rings
```

```
>>> from sage.all import *
>>> F = GF(Integer(5))
>>> K = GF(Integer(5)**Integer(2)).over(F, names=('a',)); (a,) = K._first_
↪ngens(1)# needs sage.rings.finite_rings
>>> L = GF(Integer(5)**Integer(6)).over(K, names=('b',)); (b,) = L._first_
↪ngens(1)# needs sage.rings.finite_rings
```

We define the absolute Frobenius of L:

```
sage: FrobL = L.hom([b^5, a^5]); FrobL                                 #␣
↪needs sage.rings.finite_rings
Ring endomorphism of
 Field in b with defining polynomial x^3 + (2 + 2*a)*x - a over its base
  Defn: b |--> (-1 + a) + (1 + 2*a)*b + a*b^2
        with map on base ring:
        a |--> 1 - a
sage: FrobL.base_map()                                                 #␣
↪needs sage.rings.finite_rings
Ring morphism:
  From: Field in a with defining polynomial x^2 + 4*x + 2 over its base
```

(continues on next page)

(continued from previous page)

```
  To:   Field in b with defining polynomial x^3 + (2 + 2*a)*x - a over its␣
↪base
  Defn: a |--> 1 - a
```

```
>>> from sage.all import *
>>> FrobL = L.hom([b**Integer(5), a**Integer(5)]); FrobL                    ␣
↪               # needs sage.rings.finite_rings
Ring endomorphism of
 Field in b with defining polynomial x^3 + (2 + 2*a)*x - a over its base
  Defn: b |--> (-1 + a) + (1 + 2*a)*b + a*b^2
        with map on base ring:
        a |--> 1 - a
>>> FrobL.base_map()                                                        #␣
↪needs sage.rings.finite_rings
Ring morphism:
  From: Field in a with defining polynomial x^2 + 4*x + 2 over its base
  To:   Field in b with defining polynomial x^3 + (2 + 2*a)*x - a over its␣
↪base
  Defn: a |--> 1 - a
```

The square of `FrobL` acts trivially on K; in other words, it has a trivial base map:

```
sage: phi = FrobL^2; phi                                                   #␣
↪needs sage.rings.finite_rings
Ring endomorphism of
 Field in b with defining polynomial x^3 + (2 + 2*a)*x - a over its base
  Defn: b |--> 2 + 2*a*b + (2 - a)*b^2
sage: phi.base_map()                                                       #␣
↪needs sage.rings.finite_rings
```

```
>>> from sage.all import *
>>> phi = FrobL**Integer(2); phi                                           ␣
↪      # needs sage.rings.finite_rings
Ring endomorphism of
 Field in b with defining polynomial x^3 + (2 + 2*a)*x - a over its base
  Defn: b |--> 2 + 2*a*b + (2 - a)*b^2
>>> phi.base_map()                                                         #␣
↪needs sage.rings.finite_rings
```

is_identity()

Return whether this morphism is the identity.

EXAMPLES:

```
sage: K.<a> = GF(5^2).over()    # over GF(5)
sage: FrobK = K.hom([a^5])
sage: FrobK.is_identity()
False
sage: (FrobK^2).is_identity()
True
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(2)).over(names=('a',)); (a,) = K._first_
↪ngens(1)# over GF(5)
>>> FrobK = K.hom([a**Integer(5)])
>>> FrobK.is_identity()
False
>>> (FrobK**Integer(2)).is_identity()
True
```

Coercion maps are not considered as identity morphisms:

```
sage: L.<b> = GF(5^6).over(K)
sage: iota = L.defining_morphism(); iota
Ring morphism:
  From: Field in a with defining polynomial x^2 + 4*x + 2 over its base
  To:   Field in b with defining polynomial x^3 + (2 + 2*a)*x - a over its␣
↪base
  Defn: a |--> a
sage: iota.is_identity()
False
```

```
>>> from sage.all import *
>>> L = GF(Integer(5)**Integer(6)).over(K, names=('b',)); (b,) = L._first_
↪ngens(1)
>>> iota = L.defining_morphism(); iota
Ring morphism:
  From: Field in a with defining polynomial x^2 + 4*x + 2 over its base
  To:   Field in b with defining polynomial x^3 + (2 + 2*a)*x - a over its␣
↪base
  Defn: a |--> a
>>> iota.is_identity()
False
```

is_injective()

Return whether this morphism is injective.

EXAMPLES:

```
sage: K = GF(5^10).over(GF(5^5))
sage: iota = K.defining_morphism(); iota
Ring morphism:
  From: Finite Field in z5 of size 5^5
  To:   Field in z10 with defining polynomial
        x^2 + (2*z5^3 + 2*z5^2 + 4*z5 + 4)*x + z5 over its base
  Defn: z5 |--> z5
sage: iota.is_injective()
True

sage: K = GF(7).over(ZZ)
sage: iota = K.defining_morphism(); iota
Ring morphism:
  From: Integer Ring
  To:   Finite Field of size 7 over its base
```

(continues on next page)

(continued from previous page)

```
  Defn: 1 |--> 1
sage: iota.is_injective()
False
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(10)).over(GF(Integer(5)**Integer(5)))
>>> iota = K.defining_morphism(); iota
Ring morphism:
  From: Finite Field in z5 of size 5^5
  To:   Field in z10 with defining polynomial
        x^2 + (2*z5^3 + 2*z5^2 + 4*z5 + 4)*x + z5 over its base
  Defn: z5 |--> z5
>>> iota.is_injective()
True

>>> K = GF(Integer(7)).over(ZZ)
>>> iota = K.defining_morphism(); iota
Ring morphism:
  From: Integer Ring
  To:   Finite Field of size 7 over its base
  Defn: 1 |--> 1
>>> iota.is_injective()
False
```

is_surjective()

Return whether this morphism is surjective.

EXAMPLES:

```
sage: K = GF(5^10).over(GF(5^5))
sage: iota = K.defining_morphism(); iota
Ring morphism:
  From: Finite Field in z5 of size 5^5
  To:   Field in z10 with defining polynomial
        x^2 + (2*z5^3 + 2*z5^2 + 4*z5 + 4)*x + z5 over its base
  Defn: z5 |--> z5
sage: iota.is_surjective()
False

sage: K = GF(7).over(ZZ)
sage: iota = K.defining_morphism(); iota
Ring morphism:
  From: Integer Ring
  To:   Finite Field of size 7 over its base
  Defn: 1 |--> 1
sage: iota.is_surjective()
True
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(10)).over(GF(Integer(5)**Integer(5)))
>>> iota = K.defining_morphism(); iota
Ring morphism:
```

(continues on next page)

(continued from previous page)

```
  From: Finite Field in z5 of size 5^5
  To:   Field in z10 with defining polynomial
        x^2 + (2*z5^3 + 2*z5^2 + 4*z5 + 4)*x + z5 over its base
  Defn: z5 |--> z5
>>> iota.is_surjective()
False

>>> K = GF(Integer(7)).over(ZZ)
>>> iota = K.defining_morphism(); iota
Ring morphism:
  From: Integer Ring
  To:   Finite Field of size 7 over its base
  Defn: 1 |--> 1
>>> iota.is_surjective()
True
```

CHAPTER

EIGHT

GENERIC DATA STRUCTURES AND ALGORITHMS FOR RINGS

8.1 Generic data structures and algorithms for rings

AUTHORS:

- Lorenz Panny (2022): *ProductTree*, *prod_with_derivative()*

class sage.rings.generic.**ProductTree**(*leaves*)

Bases: object

A simple binary product tree, i.e., a tree of ring elements in which every node equals the product of its children. (In particular, the *root* equals the product of all *leaves*.)

Product trees are a very useful building block for fast computer algebra. For example, a quasilinear-time Discrete Fourier Transform (the famous *Fast* Fourier Transform) can be implemented as follows using the *remainders()* method of this class:

```
sage: from sage.rings.generic import ProductTree
sage: F = GF(65537)
sage: a = F(1111)
sage: assert a.multiplicative_order() == 1024
sage: R.<x> = F[]
sage: ms = [x - a^i for i in range(1024)]               # roots of unity
sage: ys = [F.random_element() for _ in range(1024)]    # input vector
sage: tree = ProductTree(ms)
sage: zs = tree.remainders(R(ys))                       # compute FFT!
sage: zs == [R(ys) % m for m in ms]
True
```

```
>>> from sage.all import *
>>> from sage.rings.generic import ProductTree
>>> F = GF(Integer(65537))
>>> a = F(Integer(1111))
>>> assert a.multiplicative_order() == Integer(1024)
>>> R = F['x']; (x,) = R._first_ngens(1)
>>> ms = [x - a**i for i in range(Integer(1024))]               # roots of unity
>>> ys = [F.random_element() for _ in range(Integer(1024))]    # input vector
>>> tree = ProductTree(ms)
>>> zs = tree.remainders(R(ys))                       # compute FFT!
>>> zs == [R(ys) % m for m in ms]
True
```

Similarly, the *interpolation()* method can be used to implement the inverse Fast Fourier Transform:

```
sage: tree.interpolation(zs).padded_list(len(ys)) == ys                    #␣
↪needs sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> tree.interpolation(zs).padded_list(len(ys)) == ys                      #␣
↪needs sage.rings.finite_rings
True
```

This class encodes the tree as *layers*: Layer 0 is just a tuple of the leaves. Layer $i+1$ is obtained from layer i by replacing each pair of two adjacent elements by their product, starting from the left. (If the length is odd, the unpaired element at the end is simply copied as is.) This iteration stops as soon as it yields a layer containing only a single element (the root).

Note

Use this class if you need the `remainders()` method. To compute just the product, `prod()` is likely faster.

INPUT:

- `leaves` – an iterable of elements in a common ring

EXAMPLES:

```
sage: from sage.rings.generic import ProductTree
sage: R.<x> = GF(101)[]
sage: vs = [x - i for i in range(1,10)]
sage: tree = ProductTree(vs)
sage: tree.root()
x^9 + 56*x^8 + 62*x^7 + 44*x^6 + 47*x^5 + 42*x^4 + 15*x^3 + 11*x^2 + 12*x + 13
sage: tree.remainders(x^7 + x + 1)
[3, 30, 70, 27, 58, 72, 98, 98, 23]
sage: tree.remainders(x^100)
[1, 1, 1, 1, 1, 1, 1, 1, 1]
```

```
>>> from sage.all import *
>>> from sage.rings.generic import ProductTree
>>> R = GF(Integer(101))['x']; (x,) = R._first_ngens(1)
>>> vs = [x - i for i in range(Integer(1),Integer(10))]
>>> tree = ProductTree(vs)
>>> tree.root()
x^9 + 56*x^8 + 62*x^7 + 44*x^6 + 47*x^5 + 42*x^4 + 15*x^3 + 11*x^2 + 12*x + 13
>>> tree.remainders(x**Integer(7) + x + Integer(1))
[3, 30, 70, 27, 58, 72, 98, 98, 23]
>>> tree.remainders(x**Integer(100))
[1, 1, 1, 1, 1, 1, 1, 1, 1]
```

```
sage: vs = prime_range(100)
sage: tree = ProductTree(vs)
sage: tree.root().factor()
2 * 3 * 5 * 7 * 11 * 13 * 17 * 19 * 23 * 29 * 31 * 37 * 41 * 43 * 47 * 53 * 59 *␣
↪61 * 67 * 71 * 73 * 79 * 83 * 89 * 97
```

(continues on next page)

(continued from previous page)

```
sage: tree.remainders(3599)
[1, 2, 4, 1, 2, 11, 12, 8, 11, 3, 3, 10, 32, 30, 27, 48, 0, 0, 48, 49, 22, 44, 30,
↪ 39, 10]
```

```
>>> from sage.all import *
>>> vs = prime_range(Integer(100))
>>> tree = ProductTree(vs)
>>> tree.root().factor()
2 * 3 * 5 * 7 * 11 * 13 * 17 * 19 * 23 * 29 * 31 * 37 * 41 * 43 * 47 * 53 * 59 *␣
↪61 * 67 * 71 * 73 * 79 * 83 * 89 * 97
>>> tree.remainders(Integer(3599))
[1, 2, 4, 1, 2, 11, 12, 8, 11, 3, 3, 10, 32, 30, 27, 48, 0, 0, 48, 49, 22, 44, 30,
↪ 39, 10]
```

We can access the individual layers of the tree:

```
sage: tree.layers                                                                #␣
↪needs sage.libs.pari
[(2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,␣
↪79, 83, 89, 97),
 (6, 35, 143, 323, 667, 1147, 1763, 2491, 3599, 4757, 5767, 7387, 97),
 (210, 46189, 765049, 4391633, 17120443, 42600829, 97),
 (9699690, 3359814435017, 729345064647247, 97),
 (32589158477190044730, 70746471270782959),
 (2305567963945518424753102147331756070,)]
```

```
>>> from sage.all import *
>>> tree.layers                                                                  #␣
↪needs sage.libs.pari
[(2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,␣
↪79, 83, 89, 97),
 (6, 35, 143, 323, 667, 1147, 1763, 2491, 3599, 4757, 5767, 7387, 97),
 (210, 46189, 765049, 4391633, 17120443, 42600829, 97),
 (9699690, 3359814435017, 729345064647247, 97),
 (32589158477190044730, 70746471270782959),
 (2305567963945518424753102147331756070,)]
```

CRT_bases()

Return a tuple of tuples containing all CRT bases needed to compute the *interpolation()* from the roots of this product tree to the root by following the tree structure.

In more detail, the return value is a three-fold nested tuple `msss` such that:

- The tuple `msss` runs parallel to pairs of adjacent layers of the tree, i.e., each of its entries corresponds to a layer of "parents" and a layer of "children".
- Each tuple `mss` inside `msss` runs parallel to the parents inside its associated pair of layers.
- Each tuple `ms` inside `mss` runs parallel to the children of its associated parent.

…and the following property holds: The entries of `ms` are valid CRT coefficients for computing a CRT interpolation modulo its associated children.

EXAMPLES:

```
sage: from sage.rings.generic import ProductTree
sage: tree = ProductTree([2,3,5,7])
sage: ms0, ms1 = tree.CRT_bases()
sage: tree.remainders(ms0[0][0])
[1, 0, 3, 3]
sage: tree.remainders(ms0[0][1])
[0, 1, 4, 4]
sage: tree.remainders(ms0[1][0])
[1, 0, 1, 0]
sage: tree.remainders(ms0[1][1])
[1, 0, 0, 1]
sage: tree.remainders(ms1[0][0] * ms0[0][0])
[1, 0, 0, 0]
sage: tree.remainders(ms1[0][0] * ms0[0][1])
[0, 1, 0, 0]
sage: tree.remainders(ms1[0][1] * ms0[1][0])
[0, 0, 1, 0]
sage: tree.remainders(ms1[0][1] * ms0[1][1])
[0, 0, 0, 1]
```

```
>>> from sage.all import *
>>> from sage.rings.generic import ProductTree
>>> tree = ProductTree([Integer(2),Integer(3),Integer(5),Integer(7)])
>>> ms0, ms1 = tree.CRT_bases()
>>> tree.remainders(ms0[Integer(0)][Integer(0)])
[1, 0, 3, 3]
>>> tree.remainders(ms0[Integer(0)][Integer(1)])
[0, 1, 4, 4]
>>> tree.remainders(ms0[Integer(1)][Integer(0)])
[1, 0, 1, 0]
>>> tree.remainders(ms0[Integer(1)][Integer(1)])
[1, 0, 0, 1]
>>> tree.remainders(ms1[Integer(0)][Integer(0)] * ms0[Integer(0)][Integer(0)])
[1, 0, 0, 0]
>>> tree.remainders(ms1[Integer(0)][Integer(0)] * ms0[Integer(0)][Integer(1)])
[0, 1, 0, 0]
>>> tree.remainders(ms1[Integer(0)][Integer(1)] * ms0[Integer(1)][Integer(0)])
[0, 0, 1, 0]
>>> tree.remainders(ms1[Integer(0)][Integer(1)] * ms0[Integer(1)][Integer(1)])
[0, 0, 0, 1]
```

interpolation(*xs*)

Given a sequence `xs` of values, one per leaf, return a single element x which is congruent to the ith value in `xs` modulo the ith leaf, for all i.

This is an explicit version of the Chinese remainder theorem; see also `CRT()`. Using this product tree is faster for repeated calls since the required CRT bases are cached after the first run.

The base ring must support the `xgcd()` function for this method to work.

EXAMPLES:

```
sage: from sage.rings.generic import ProductTree
sage: vs = prime_range(100)
```

(continues on next page)

(continued from previous page)

```
sage: tree = ProductTree(vs)
sage: tree.interpolation([1, 1, 2, 1, 9, 1, 7, 15, 8, 20, 15, 6, 27, 11, 2, 6,
↪ 0, 25, 49, 5, 51, 4, 19, 74, 13])
1085749272377676749812331719267
```

```
>>> from sage.all import *
>>> from sage.rings.generic import ProductTree
>>> vs = prime_range(Integer(100))
>>> tree = ProductTree(vs)
>>> tree.interpolation([Integer(1), Integer(1), Integer(2), Integer(1),␣
↪Integer(9), Integer(1), Integer(7), Integer(15), Integer(8), Integer(20),␣
↪Integer(15), Integer(6), Integer(27), Integer(11), Integer(2), Integer(6),␣
↪Integer(0), Integer(25), Integer(49), Integer(5), Integer(51), Integer(4),␣
↪Integer(19), Integer(74), Integer(13)])
1085749272377676749812331719267
```

This method is faster than `CRT()` for repeated calls with the same moduli:

```
sage: vs = prime_range(1000,2000)
sage: rs = lambda: [randrange(1,100) for _ in vs]
sage: tree = ProductTree(vs)
sage: %timeit CRT(rs(), vs)              # not tested
372 µs ± 3.34 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
sage: %timeit tree.interpolation(rs())  # not tested
146 µs ± 479 ns per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
```

```
>>> from sage.all import *
>>> vs = prime_range(Integer(1000),Integer(2000))
>>> rs = lambda: [randrange(Integer(1),Integer(100)) for _ in vs]
>>> tree = ProductTree(vs)
>>> %timeit CRT(rs(), vs)              # not tested
372 µs ± 3.34 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
>>> %timeit tree.interpolation(rs())  # not tested
146 µs ± 479 ns per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
```

leaves()

Return a tuple containing the leaves of this product tree.

EXAMPLES:

```
sage: from sage.rings.generic import ProductTree
sage: R.<x> = GF(101)[]
sage: vs = [x - i for i in range(1,10)]
sage: tree = ProductTree(vs)
sage: tree.leaves()
(x + 100, x + 99, x + 98, ..., x + 93, x + 92)
sage: tree.leaves() == tuple(vs)
True
```

```
>>> from sage.all import *
>>> from sage.rings.generic import ProductTree
>>> R = GF(Integer(101))['x']; (x,) = R._first_ngens(1)
```

(continues on next page)

(continued from previous page)

```
>>> vs = [x - i for i in range(Integer(1),Integer(10))]
>>> tree = ProductTree(vs)
>>> tree.leaves()
(x + 100, x + 99, x + 98, ..., x + 93, x + 92)
>>> tree.leaves() == tuple(vs)
True
```

remainders(x)

Given a value x, return a list of all remainders of x modulo the leaves of this product tree.

The base ring must support the `%` operator for this method to work.

INPUT:

- `x` – an element of the base ring of this product tree

EXAMPLES:

```
sage: from sage.rings.generic import ProductTree
sage: vs = prime_range(100)
sage: tree = ProductTree(vs)
sage: n = 10857492723776767498123331719267
sage: tree.remainders(n)
[1, 1, 2, 1, 9, 1, 7, 15, 8, 20, 15, 6, 27, 11, 2, 6, 0, 25, 49, 5, 51, 4, 19,
↪ 74, 13]
sage: [n % v for v in vs]
[1, 1, 2, 1, 9, 1, 7, 15, 8, 20, 15, 6, 27, 11, 2, 6, 0, 25, 49, 5, 51, 4, 19,
↪ 74, 13]
```

```
>>> from sage.all import *
>>> from sage.rings.generic import ProductTree
>>> vs = prime_range(Integer(100))
>>> tree = ProductTree(vs)
>>> n = Integer(10857492723776767498123331719267)
>>> tree.remainders(n)
[1, 1, 2, 1, 9, 1, 7, 15, 8, 20, 15, 6, 27, 11, 2, 6, 0, 25, 49, 5, 51, 4, 19,
↪ 74, 13]
>>> [n % v for v in vs]
[1, 1, 2, 1, 9, 1, 7, 15, 8, 20, 15, 6, 27, 11, 2, 6, 0, 25, 49, 5, 51, 4, 19,
↪ 74, 13]
```

root()

Return the value at the root of this product tree (i.e., the product of all leaves).

EXAMPLES:

```
sage: from sage.rings.generic import ProductTree
sage: R.<x> = GF(101)[]
sage: vs = [x - i for i in range(1,10)]
sage: tree = ProductTree(vs)
sage: tree.root()
x^9 + 56*x^8 + 62*x^7 + 44*x^6 + 47*x^5 + 42*x^4 + 15*x^3 + 11*x^2 + 12*x + 13
sage: tree.root() == prod(vs)
True
```

```
>>> from sage.all import *
>>> from sage.rings.generic import ProductTree
>>> R = GF(Integer(101))['x']; (x,) = R._first_ngens(1)
>>> vs = [x - i for i in range(Integer(1),Integer(10))]
>>> tree = ProductTree(vs)
>>> tree.root()
x^9 + 56*x^8 + 62*x^7 + 44*x^6 + 47*x^5 + 42*x^4 + 15*x^3 + 11*x^2 + 12*x + 13
>>> tree.root() == prod(vs)
True
```

sage.rings.generic.**prod_with_derivative**(*pairs*)

Given an iterable of pairs $(f, \partial f)$ of ring elements, return the pair $(\prod f, \partial \prod f)$, assuming ∂ is an operator obeying the standard product rule.

This function is entirely algebraic, hence still works when the elements f and ∂f are all passed through some ring homomorphism first. One particularly useful instance of this is evaluating the derivative of a product of polynomials at a point without fully expanding the product; see the second example below.

INPUT:

- `pairs` – an iterable of tuples $(f, \partial f)$ of elements of a common ring

ALGORITHM: Repeated application of the product rule.

EXAMPLES:

```
sage: from sage.rings.generic import prod_with_derivative
sage: R.<x> = ZZ[]
sage: fs = [x^2 + 2*x + 3, 4*x + 5, 6*x^7 + 8*x + 9]
sage: prod(fs)
24*x^10 + 78*x^9 + 132*x^8 + 90*x^7 + 32*x^4 + 140*x^3 + 293*x^2 + 318*x + 135
sage: prod(fs).derivative()
240*x^9 + 702*x^8 + 1056*x^7 + 630*x^6 + 128*x^3 + 420*x^2 + 586*x + 318
sage: F, dF = prod_with_derivative((f, f.derivative()) for f in fs)
sage: F
24*x^10 + 78*x^9 + 132*x^8 + 90*x^7 + 32*x^4 + 140*x^3 + 293*x^2 + 318*x + 135
sage: dF
240*x^9 + 702*x^8 + 1056*x^7 + 630*x^6 + 128*x^3 + 420*x^2 + 586*x + 318
```

```
>>> from sage.all import *
>>> from sage.rings.generic import prod_with_derivative
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> fs = [x**Integer(2) + Integer(2)*x + Integer(3), Integer(4)*x + Integer(5),␣
↪Integer(6)*x**Integer(7) + Integer(8)*x + Integer(9)]
>>> prod(fs)
24*x^10 + 78*x^9 + 132*x^8 + 90*x^7 + 32*x^4 + 140*x^3 + 293*x^2 + 318*x + 135
>>> prod(fs).derivative()
240*x^9 + 702*x^8 + 1056*x^7 + 630*x^6 + 128*x^3 + 420*x^2 + 586*x + 318
>>> F, dF = prod_with_derivative((f, f.derivative()) for f in fs)
>>> F
24*x^10 + 78*x^9 + 132*x^8 + 90*x^7 + 32*x^4 + 140*x^3 + 293*x^2 + 318*x + 135
>>> dF
240*x^9 + 702*x^8 + 1056*x^7 + 630*x^6 + 128*x^3 + 420*x^2 + 586*x + 318
```

The main reason for this function to exist is that it allows us to *evaluate* the derivative of a product of polynomials at a point α without ever fully expanding the product *as a polynomial*:

```
sage: alpha = 42
sage: F(alpha)
442943981574522759
sage: dF(alpha)
104645261461514994
sage: us = [f(alpha) for f in fs]
sage: vs = [f.derivative()(alpha) for f in fs]
sage: prod_with_derivative(zip(us, vs))
(442943981574522759, 104645261461514994)
```

```
>>> from sage.all import *
>>> alpha = Integer(42)
>>> F(alpha)
442943981574522759
>>> dF(alpha)
104645261461514994
>>> us = [f(alpha) for f in fs]
>>> vs = [f.derivative()(alpha) for f in fs]
>>> prod_with_derivative(zip(us, vs))
(442943981574522759, 104645261461514994)
```

CHAPTER

NINE

UTILITIES

9.1 Big O for various types (power series, p-adics, etc.)

See also

- asymptotic expansions
- p-adic numbers
- power series
- polynomials

sage.rings.big_oh.**O**(*x*, ***kwds*)

Big O constructor for various types.

EXAMPLES:

This is useful for writing power series elements:

```
sage: R.<t> = ZZ[['t']]
sage: (1+t)^10 + O(t^5)
1 + 10*t + 45*t^2 + 120*t^3 + 210*t^4 + O(t^5)
```

```
>>> from sage.all import *
>>> R = ZZ[['t']]; (t,) = R._first_ngens(1)
>>> (Integer(1)+t)**Integer(10) + O(t**Integer(5))
1 + 10*t + 45*t^2 + 120*t^3 + 210*t^4 + O(t^5)
```

A power series ring is created implicitly if a polynomial element is passed:

```
sage: R.<x> = QQ['x']
sage: O(x^100)
O(x^100)
sage: 1/(1+x+O(x^5))
1 - x + x^2 - x^3 + x^4 + O(x^5)
sage: R.<u,v> = QQ[[]]
sage: 1 + u + v^2 + O(u, v)^5
1 + u + v^2 + O(u, v)^5
```

```
>>> from sage.all import *
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> O(x**Integer(100))
O(x^100)
>>> Integer(1)/(Integer(1)+x+O(x**Integer(5)))
1 - x + x^2 - x^3 + x^4 + O(x^5)
>>> R = QQ[['u, v']]; (u, v,) = R._first_ngens(2)
>>> Integer(1) + u + v**Integer(2) + O(u, v)**Integer(5)
1 + u + v^2 + O(u, v)^5
```

This is also useful to create p-adic numbers:

```
sage: O(7^6)                                                                #␣
↪needs sage.rings.padics
O(7^6)
sage: 1/3 + O(7^6)                                                          #␣
↪needs sage.rings.padics
5 + 4*7 + 4*7^2 + 4*7^3 + 4*7^4 + 4*7^5 + O(7^6)
```

```
>>> from sage.all import *
>>> O(Integer(7)**Integer(6))                                                ␣
↪               # needs sage.rings.padics
O(7^6)
>>> Integer(1)/Integer(3) + O(Integer(7)**Integer(6))                        ␣
↪                                  # needs sage.rings.padics
5 + 4*7 + 4*7^2 + 4*7^3 + 4*7^4 + 4*7^5 + O(7^6)
```

It behaves well with respect to adding negative powers of p:

```
sage: a = O(11^-32); a                                                      #␣
↪needs sage.rings.padics
O(11^-32)
sage: a.parent()                                                            #␣
↪needs sage.rings.padics
11-adic Field with capped relative precision 20
```

```
>>> from sage.all import *
>>> a = O(Integer(11)**-Integer(32)); a                                      ␣
↪               # needs sage.rings.padics
O(11^-32)
>>> a.parent()                                                             #␣
↪needs sage.rings.padics
11-adic Field with capped relative precision 20
```

There are problems if you add a rational with very negative valuation to an O-Term:

```
sage: 11^-12 + O(11^15)                                                     #␣
↪needs sage.rings.padics
11^-12 + O(11^8)
```

```
>>> from sage.all import *
>>> Integer(11)**-Integer(12) + O(Integer(11)**Integer(15))                  ␣
```

(continues on next page)

(continued from previous page)

```
↪                                    # needs sage.rings.padics
11^-12 + O(11^8)
```

The reason that this fails is that the constructor doesn't know the right precision cap to use. If you cast explicitly or use other means of element creation, you can get around this issue:

```
sage: K = Qp(11, 30)
sage: K(11^-12) + O(11^15)
11^-12 + O(11^15)
sage: 11^-12 + K(O(11^15))
11^-12 + O(11^15)
sage: K(11^-12, absprec=15)
11^-12 + O(11^15)
sage: K(11^-12, 15)
11^-12 + O(11^15)
```

```
>>> from sage.all import *
>>> K = Qp(Integer(11), Integer(30))
>>> K(Integer(11)**-Integer(12)) + O(Integer(11)**Integer(15))
11^-12 + O(11^15)
>>> Integer(11)**-Integer(12) + K(O(Integer(11)**Integer(15)))
11^-12 + O(11^15)
>>> K(Integer(11)**-Integer(12), absprec=Integer(15))
11^-12 + O(11^15)
>>> K(Integer(11)**-Integer(12), Integer(15))
11^-12 + O(11^15)
```

We can also work with asymptotic expansions:

```
sage: A.<n> = AsymptoticRing(growth_group='QQ^n * n^QQ * log(n)^QQ',            #␣
↪needs sage.symbolic
....:                        coefficient_ring=QQ); A
Asymptotic Ring <QQ^n * n^QQ * log(n)^QQ * Signs^n> over Rational Field
sage: O(n)                                                                     #␣
↪needs sage.symbolic
O(n)
```

```
>>> from sage.all import *
>>> A = AsymptoticRing(growth_group='QQ^n * n^QQ * log(n)^QQ',            # needs␣
↪sage.symbolic
...                         coefficient_ring=QQ, names=('n',)); (n,) = A._first_
↪ngens(1); A
Asymptotic Ring <QQ^n * n^QQ * log(n)^QQ * Signs^n> over Rational Field
>>> O(n)                                                                       #␣
↪needs sage.symbolic
O(n)
```

Application with Puiseux series:

```
sage: P.<y> = PuiseuxSeriesRing(ZZ)
sage: y^(1/5) + O(y^(1/3))
y^(1/5) + O(y^(1/3))
```

(continues on next page)

(continued from previous page)

```
sage: y^(1/3) + O(y^(1/5))
O(y^(1/5))
```

```
>>> from sage.all import *
>>> P = PuiseuxSeriesRing(ZZ, names=('y',)); (y,) = P._first_ngens(1)
>>> y**(Integer(1)/Integer(5)) + O(y**(Integer(1)/Integer(3)))
y^(1/5) + O(y^(1/3))
>>> y**(Integer(1)/Integer(3)) + O(y**(Integer(1)/Integer(5)))
O(y^(1/5))
```

9.2 Signed and Unsigned Infinities

The unsigned infinity "ring" is the set of two elements

1. infinity
2. A number less than infinity

The rules for arithmetic are that the unsigned infinity ring does not canonically coerce to any other ring, and all other rings canonically coerce to the unsigned infinity ring, sending all elements to the single element "a number less than infinity" of the unsigned infinity ring. Arithmetic and comparisons then take place in the unsigned infinity ring, where all arithmetic operations that are well-defined are defined.

The infinity "ring" is the set of five elements

1. plus infinity
2. a positive finite element
3. zero
4. a negative finite element
5. negative infinity

The infinity ring coerces to the unsigned infinity ring, sending the infinite elements to infinity and the non-infinite elements to "a number less than infinity." Any ordered ring coerces to the infinity ring in the obvious way.

Note

The shorthand `oo` is predefined in Sage to be the same as `+Infinity` in the infinity ring. It is considered equal to, but not the same as `Infinity` in the *UnsignedInfinityRing*.

class sage.rings.infinity.**_uniq**(**args*)

sage.rings.infinity.**infinity = +Infinity**

Initialize `self`.

EXAMPLES:

We fetch the unsigned infinity ring and create some elements:

```
sage: P = UnsignedInfinityRing; P
The Unsigned Infinity Ring
sage: P(5)
A number less than infinity
```

(continues on next page)

(continued from previous page)

```
sage: P.ngens()
1
sage: unsigned_oo = P.0; unsigned_oo
Infinity
```

```
>>> from sage.all import *
>>> P = UnsignedInfinityRing; P
The Unsigned Infinity Ring
>>> P(Integer(5))
A number less than infinity
>>> P.ngens()
1
>>> unsigned_oo = P.gen(0); unsigned_oo
Infinity
```

We compare finite numbers with infinity:

```
sage: 5 < unsigned_oo
True
sage: 5 > unsigned_oo
False
sage: unsigned_oo < 5
False
sage: unsigned_oo > 5
True
```

```
>>> from sage.all import *
>>> Integer(5) < unsigned_oo
True
>>> Integer(5) > unsigned_oo
False
>>> unsigned_oo < Integer(5)
False
>>> unsigned_oo > Integer(5)
True
```

Demonstrating the shorthand `oo` versus `Infinity`:

```
sage: oo
+Infinity
sage: oo is InfinityRing.0
True
sage: oo is UnsignedInfinityRing.0
False
sage: oo == UnsignedInfinityRing.0
True
```

```
>>> from sage.all import *
>>> oo
+Infinity
>>> oo is InfinityRing.gen(0)
True
```

(continues on next page)

(continued from previous page)

```
>>> oo is UnsignedInfinityRing.gen(0)
False
>>> oo == UnsignedInfinityRing.gen(0)
True
```

We do arithmetic:

```
sage: unsigned_oo + 5
Infinity
```

```
>>> from sage.all import *
>>> unsigned_oo + Integer(5)
Infinity
```

We make `1 / unsigned_oo` return the integer 0 so that arithmetic of the following type works:

```
sage: (1/unsigned_oo) + 2
2
sage: 32/5 - (2.439/unsigned_oo)
32/5
```

```
>>> from sage.all import *
>>> (Integer(1)/unsigned_oo) + Integer(2)
2
>>> Integer(32)/Integer(5) - (RealNumber('2.439')/unsigned_oo)
32/5
```

Note that many operations are not defined, since the result is not well-defined:

```
sage: unsigned_oo/0
Traceback (most recent call last):
...
ValueError: quotient of number < oo by number < oo not defined
```

```
>>> from sage.all import *
>>> unsigned_oo/Integer(0)
Traceback (most recent call last):
...
ValueError: quotient of number < oo by number < oo not defined
```

What happened above is that 0 is canonically coerced to "A number less than infinity" in the unsigned infinity ring. Next, Sage tries to divide by multiplying with its inverse. Finally, this inverse is not well-defined.

```
sage: 0/unsigned_oo
0
sage: unsigned_oo * 0
Traceback (most recent call last):
...
ValueError: unsigned oo times smaller number not defined
sage: unsigned_oo/unsigned_oo
Traceback (most recent call last):
```

(continues on next page)

(continued from previous page)

```
...
ValueError: unsigned oo times smaller number not defined
```

```
>>> from sage.all import *
>>> Integer(0)/unsigned_oo
0
>>> unsigned_oo * Integer(0)
Traceback (most recent call last):
...
ValueError: unsigned oo times smaller number not defined
>>> unsigned_oo/unsigned_oo
Traceback (most recent call last):
...
ValueError: unsigned oo times smaller number not defined
```

In the infinity ring, we can negate infinity, multiply positive numbers by infinity, etc.

```
sage: P = InfinityRing; P
The Infinity Ring
sage: P(5)
A positive finite number
```

```
>>> from sage.all import *
>>> P = InfinityRing; P
The Infinity Ring
>>> P(Integer(5))
A positive finite number
```

The symbol `oo` is predefined as a shorthand for `+Infinity`:

```
sage: oo
+Infinity
```

```
>>> from sage.all import *
>>> oo
+Infinity
```

We compare finite and infinite elements:

```
sage: 5 < oo
True
sage: P(-5) < P(5)
True
sage: P(2) < P(3)
False
sage: -oo < oo
True
```

```
>>> from sage.all import *
>>> Integer(5) < oo
True
```

(continues on next page)

(continued from previous page)

```
>>> P(-Integer(5)) < P(Integer(5))
True
>>> P(Integer(2)) < P(Integer(3))
False
>>> -oo < oo
True
```

We can do more arithmetic than in the unsigned infinity ring:

```
sage: 2 * oo
+Infinity
sage: -2 * oo
-Infinity
sage: 1 - oo
-Infinity
sage: 1 / oo
0
sage: -1 / oo
0
```

```
>>> from sage.all import *
>>> Integer(2) * oo
+Infinity
>>> -Integer(2) * oo
-Infinity
>>> Integer(1) - oo
-Infinity
>>> Integer(1) / oo
0
>>> -Integer(1) / oo
0
```

We make `1 / oo` and `1 / -oo` return the integer 0 instead of the infinity ring Zero so that arithmetic of the following type works:

```
sage: (1/oo) + 2
2
sage: 32/5 - (2.439/-oo)
32/5
```

```
>>> from sage.all import *
>>> (Integer(1)/oo) + Integer(2)
2
>>> Integer(32)/Integer(5) - (RealNumber('2.439')/-oo)
32/5
```

If we try to subtract infinities or multiply infinity by zero we still get an error:

```
sage: oo - oo
Traceback (most recent call last):
...
SignError: cannot add infinity to minus infinity
```

(continues on next page)

(continued from previous page)

```
sage: 0 * oo
Traceback (most recent call last):
...
SignError: cannot multiply infinity by zero
sage: P(2) + P(-3)
Traceback (most recent call last):
...
SignError: cannot add positive finite value to negative finite value
```

```
>>> from sage.all import *
>>> oo - oo
Traceback (most recent call last):
...
SignError: cannot add infinity to minus infinity
>>> Integer(0) * oo
Traceback (most recent call last):
...
SignError: cannot multiply infinity by zero
>>> P(Integer(2)) + P(-Integer(3))
Traceback (most recent call last):
...
SignError: cannot add positive finite value to negative finite value
```

Signed infinity can also be represented by RR / RDF elements. But unsigned infinity cannot:

```
sage: oo in RR, oo in RDF
(True, True)
sage: unsigned_infinity in RR, unsigned_infinity in RDF
(False, False)
```

```
>>> from sage.all import *
>>> oo in RR, oo in RDF
(True, True)
>>> unsigned_infinity in RR, unsigned_infinity in RDF
(False, False)
```

class sage.rings.infinity.**AnInfinity**

Bases: object

lcm(*x*)

Return the least common multiple of `oo` and `x`, which is by definition oo unless `x` is 0.

EXAMPLES:

```
sage: oo.lcm(0)
0
sage: oo.lcm(oo)
+Infinity
sage: oo.lcm(-oo)
+Infinity
sage: oo.lcm(10)
+Infinity
```

(continues on next page)

(continued from previous page)

```
sage: (-oo).lcm(10)
+Infinity
```

```
>>> from sage.all import *
>>> oo.lcm(Integer(0))
0
>>> oo.lcm(oo)
+Infinity
>>> oo.lcm(-oo)
+Infinity
>>> oo.lcm(Integer(10))
+Infinity
>>> (-oo).lcm(Integer(10))
+Infinity
```

class sage.rings.infinity.**FiniteNumber**(*parent*, *x*)

Bases: RingElement

Initialize `self`.

sign()

Return the sign of `self`.

EXAMPLES:

```
sage: sign(InfinityRing(2))
1
sage: sign(InfinityRing(0))
0
sage: sign(InfinityRing(-2))
-1
```

```
>>> from sage.all import *
>>> sign(InfinityRing(Integer(2)))
1
>>> sign(InfinityRing(Integer(0)))
0
>>> sign(InfinityRing(-Integer(2)))
-1
```

sqrt()

EXAMPLES:

```
sage: InfinityRing(7).sqrt()
A positive finite number
sage: InfinityRing(0).sqrt()
Zero
sage: InfinityRing(-.001).sqrt()
Traceback (most recent call last):
...
SignError: cannot take square root of a negative number
```

```
>>> from sage.all import *
>>> InfinityRing(Integer(7)).sqrt()
A positive finite number
>>> InfinityRing(Integer(0)).sqrt()
Zero
>>> InfinityRing(-RealNumber('.001')).sqrt()
Traceback (most recent call last):
...
SignError: cannot take square root of a negative number
```

class sage.rings.infinity.**InfinityRing_class**

Bases: Singleton, *Ring*

Initialize `self`.

fraction_field()

This isn't really a ring, let alone an integral domain.

gen(*n=0*)

The two generators are plus and minus infinity.

EXAMPLES:

```
sage: InfinityRing.gen(0)
+Infinity
sage: InfinityRing.gen(1)
-Infinity
sage: InfinityRing.gen(2)
Traceback (most recent call last):
...
IndexError: n must be 0 or 1
```

```
>>> from sage.all import *
>>> InfinityRing.gen(Integer(0))
+Infinity
>>> InfinityRing.gen(Integer(1))
-Infinity
>>> InfinityRing.gen(Integer(2))
Traceback (most recent call last):
...
IndexError: n must be 0 or 1
```

gens()

The two generators are plus and minus infinity.

EXAMPLES:

```
sage: InfinityRing.gens()
(+Infinity, -Infinity)
```

```
>>> from sage.all import *
>>> InfinityRing.gens()
(+Infinity, -Infinity)
```

is_commutative()

The Infinity Ring is commutative

EXAMPLES:

```
sage: InfinityRing.is_commutative()
True
```

```
>>> from sage.all import *
>>> InfinityRing.is_commutative()
True
```

is_zero()

The Infinity Ring is not zero

EXAMPLES:

```
sage: InfinityRing.is_zero()
False
```

```
>>> from sage.all import *
>>> InfinityRing.is_zero()
False
```

ngens()

The two generators are plus and minus infinity.

EXAMPLES:

```
sage: InfinityRing.ngens()
2
sage: len(InfinityRing.gens())
2
```

```
>>> from sage.all import *
>>> InfinityRing.ngens()
2
>>> len(InfinityRing.gens())
2
```

class sage.rings.infinity.**LessThanInfinity**(**args*)

Bases: *_uniq*, RingElement

Initialize self.

EXAMPLES:

```
sage: sage.rings.infinity.LessThanInfinity() is UnsignedInfinityRing(5)
True
```

```
>>> from sage.all import *
>>> sage.rings.infinity.LessThanInfinity() is UnsignedInfinityRing(Integer(5))
True
```

sign()

Raise an error because the sign of `self` is not well defined.

EXAMPLES:

```
sage: sign(UnsignedInfinityRing(2))
Traceback (most recent call last):
...
NotImplementedError: sign of number < oo is not well defined
sage: sign(UnsignedInfinityRing(0))
Traceback (most recent call last):
...
NotImplementedError: sign of number < oo is not well defined
sage: sign(UnsignedInfinityRing(-2))
Traceback (most recent call last):
...
NotImplementedError: sign of number < oo is not well defined
```

```
>>> from sage.all import *
>>> sign(UnsignedInfinityRing(Integer(2)))
Traceback (most recent call last):
...
NotImplementedError: sign of number < oo is not well defined
>>> sign(UnsignedInfinityRing(Integer(0)))
Traceback (most recent call last):
...
NotImplementedError: sign of number < oo is not well defined
>>> sign(UnsignedInfinityRing(-Integer(2)))
Traceback (most recent call last):
...
NotImplementedError: sign of number < oo is not well defined
```

class sage.rings.infinity.**MinusInfinity**(**args*)

Bases: *_uniq*, *AnInfinity*, InfinityElement

Initialize `self`.

sqrt()

EXAMPLES:

```
sage: (-oo).sqrt()
Traceback (most recent call last):
...
SignError: cannot take square root of negative infinity
```

```
>>> from sage.all import *
>>> (-oo).sqrt()
Traceback (most recent call last):
...
SignError: cannot take square root of negative infinity
```

class sage.rings.infinity.**PlusInfinity**(**args*)

Bases: *_uniq*, *AnInfinity*, InfinityElement

Initialize `self`.

sqrt()

The square root of `self`.

The square root of infinity is infinity.

EXAMPLES:

```
sage: oo.sqrt()
+Infinity
```

```
>>> from sage.all import *
>>> oo.sqrt()
+Infinity
```

exception sage.rings.infinity.**SignError**

Bases: `ArithmeticError`

Sign error exception.

class sage.rings.infinity.**UnsignedInfinity**(**args*)

Bases: *_uniq*, *AnInfinity*, `InfinityElement`

Initialize `self`.

class sage.rings.infinity.**UnsignedInfinityRing_class**

Bases: `Singleton`, `Parent`

Initialize `self`.

gen(*n=0*)

The "generator" of `self` is the infinity object.

EXAMPLES:

```
sage: UnsignedInfinityRing.gen()
Infinity
sage: UnsignedInfinityRing.gen(1)
Traceback (most recent call last):
...
IndexError: UnsignedInfinityRing only has one generator
```

```
>>> from sage.all import *
>>> UnsignedInfinityRing.gen()
Infinity
>>> UnsignedInfinityRing.gen(Integer(1))
Traceback (most recent call last):
...
IndexError: UnsignedInfinityRing only has one generator
```

gens()

The "generator" of `self` is the infinity object.

EXAMPLES:

```
sage: UnsignedInfinityRing.gens()
(Infinity,)
```

```
>>> from sage.all import *
>>> UnsignedInfinityRing.gens()
(Infinity,)
```

less_than_infinity()

This is the element that represents a finite value.

EXAMPLES:

```
sage: UnsignedInfinityRing.less_than_infinity()
A number less than infinity
sage: UnsignedInfinityRing(5) is UnsignedInfinityRing.less_than_infinity()
True
```

```
>>> from sage.all import *
>>> UnsignedInfinityRing.less_than_infinity()
A number less than infinity
>>> UnsignedInfinityRing(Integer(5)) is UnsignedInfinityRing.less_than_
↪infinity()
True
```

ngens()

The unsigned infinity ring has one "generator."

EXAMPLES:

```
sage: UnsignedInfinityRing.ngens()
1
sage: len(UnsignedInfinityRing.gens())
1
```

```
>>> from sage.all import *
>>> UnsignedInfinityRing.ngens()
1
>>> len(UnsignedInfinityRing.gens())
1
```

sage.rings.infinity.**check_comparison**(*ring*)

Check comparison with infinity.

INPUT:

- `ring` – a sub-ring of the real numbers

OUTPUT:

Various attempts are made to generate elements of `ring`. An assertion is triggered if one of these elements does not compare correctly with plus/minus infinity.

EXAMPLES:

```
sage: from sage.rings.infinity import check_comparison
sage: rings = [ZZ, QQ, RDF]
sage: rings += [RR, RealField(200)]                                          #␣
↪needs sage.rings.real_mpfr
```

(continues on next page)

(continued from previous page)

```
sage: rings += [RLF, RIF]
sage: for R in rings:
....:     print('testing {}'.format(R))
....:     check_comparison(R)
testing Integer Ring
testing Rational Field
testing Real Double Field...
sage: check_comparison(AA)
↪# needs sage.rings.number_field
```

```
>>> from sage.all import *
>>> from sage.rings.infinity import check_comparison
>>> rings = [ZZ, QQ, RDF]
>>> rings += [RR, RealField(Integer(200))]
↪     # needs sage.rings.real_mpfr
>>> rings += [RLF, RIF]
>>> for R in rings:
...     print('testing {}'.format(R))
...     check_comparison(R)
testing Integer Ring
testing Rational Field
testing Real Double Field...
>>> check_comparison(AA)
↪needs sage.rings.number_field
```

Comparison with number fields does not work:

```
sage: x = polygen(ZZ, 'x')
sage: K.<sqrt3> = NumberField(x^2 - 3)
↪needs sage.rings.number_field
sage: (-oo < 1 + sqrt3) and (1 + sqrt3 < oo)     # known bug
↪needs sage.rings.number_field
False
```

```
>>> from sage.all import *
>>> x = polygen(ZZ, 'x')
>>> K = NumberField(x**Integer(2) - Integer(3), names=('sqrt3',)); (sqrt3,) = K._
↪first_ngens(1)# needs sage.rings.number_field
>>> (-oo < Integer(1) + sqrt3) and (Integer(1) + sqrt3 < oo)     # known bug
↪              # needs sage.rings.number_field
False
```

The symbolic ring handles its own infinities, but answers `False` (meaning: cannot decide) already for some very elementary comparisons:

```
sage: check_comparison(SR)                # known bug
↪# needs sage.symbolic
Traceback (most recent call last):
...
AssertionError: testing -1000.0 in Symbolic Ring: id = ...
```

```
>>> from sage.all import *
>>> check_comparison(SR)                # known bug                                #␣
↪needs sage.symbolic
Traceback (most recent call last):
...
AssertionError: testing -1000.0 in Symbolic Ring: id = ...
```

sage.rings.infinity.**check_signed_infinity**(*pos_inf*)

Test consistency of infinity representations.

There are different possible representations of infinity in Sage. These are all consistent with the infinity ring, that is, compare with infinity in the expected way. See also Issue #14045

INPUT:

- `pos_inf` – a representation of positive infinity

OUTPUT:

An assertion error is raised if the representation is not consistent with the infinity ring.

Check that Issue #14045 is fixed:

```
sage: InfinityRing(float('+inf'))
+Infinity
sage: InfinityRing(float('-inf'))
-Infinity
sage: oo > float('+inf')
False
sage: oo == float('+inf')
True
```

```
>>> from sage.all import *
>>> InfinityRing(float('+inf'))
+Infinity
>>> InfinityRing(float('-inf'))
-Infinity
>>> oo > float('+inf')
False
>>> oo == float('+inf')
True
```

EXAMPLES:

```
sage: from sage.rings.infinity import check_signed_infinity
sage: check_signed_infinity(oo)
sage: check_signed_infinity(float('+inf'))
sage: check_signed_infinity(RLF(oo))
sage: check_signed_infinity(RIF(oo))
sage: check_signed_infinity(SR(oo))
↪# needs sage.symbolic
```

```
>>> from sage.all import *
>>> from sage.rings.infinity import check_signed_infinity
>>> check_signed_infinity(oo)
```

(continues on next page)

(continued from previous page)

```
>>> check_signed_infinity(float('+inf'))
>>> check_signed_infinity(RLF(oo))
>>> check_signed_infinity(RIF(oo))
>>> check_signed_infinity(SR(oo))                                    #␣
↪needs sage.symbolic
```

9.3 Support Python's numbers abstract base class

See also

PEP 3141 for more information about `numbers`.

CHAPTER

TEN

DERIVATION

10.1 Derivations

Let A be a ring and B be a bimodule over A. A derivation $d : A \to B$ is an additive map that satisfies the Leibniz rule

$$d(xy) = xd(y) + d(x)y.$$

If B is an algebra over A and if we are given in addition a ring homomorphism $\theta : A \to B$, a twisted derivation with respect to θ (or a θ-derivation) is an additive map $d : A \to B$ such that

$$d(xy) = \theta(x)d(y) + d(x)y.$$

When θ is the morphism defining the structure of A-algebra on B, a θ-derivation is nothing but a derivation. In general, if $\iota : A \to B$ denotes the defining morphism above, one easily checks that $\theta - \iota$ is a θ-derivation.

This file provides support for derivations and twisted derivations over commutative rings with values in algebras (i.e. we require that B is a commutative A-algebra). In this case, the set of derivations (resp. θ-derivations) is a module over B.

Given a ring A, the module of derivations over A can be created as follows:

```
sage: A.<x,y,z> = QQ[]
sage: M = A.derivation_module()
sage: M
Module of derivations over
 Multivariate Polynomial Ring in x, y, z over Rational Field
```

```
>>> from sage.all import *
>>> A = QQ['x, y, z']; (x, y, z,) = A._first_ngens(3)
>>> M = A.derivation_module()
>>> M
Module of derivations over
 Multivariate Polynomial Ring in x, y, z over Rational Field
```

The method *gens()* returns the generators of this module:

```
sage: A.<x,y,z> = QQ[]
sage: M = A.derivation_module()
sage: M.gens()
(d/dx, d/dy, d/dz)
```

```
>>> from sage.all import *
>>> A = QQ['x, y, z']; (x, y, z,) = A._first_ngens(3)
```

(continues on next page)

(continued from previous page)

```
>>> M = A.derivation_module()
>>> M.gens()
(d/dx, d/dy, d/dz)
```

We can combine them in order to create all derivations:

```
sage: d = 2*M.gen(0) + z*M.gen(1) + (x^2 + y^2)*M.gen(2)
sage: d
2*d/dx + z*d/dy + (x^2 + y^2)*d/dz
```

```
>>> from sage.all import *
>>> d = Integer(2)*M.gen(Integer(0)) + z*M.gen(Integer(1)) + (x**Integer(2) +␣
↪y**Integer(2))*M.gen(Integer(2))
>>> d
2*d/dx + z*d/dy + (x^2 + y^2)*d/dz
```

and now play with them:

```
sage: d(x + y + z)
x^2 + y^2 + z + 2
sage: P = A.random_element()
sage: Q = A.random_element()
sage: d(P*Q) == P*d(Q) + d(P)*Q
True
```

```
>>> from sage.all import *
>>> d(x + y + z)
x^2 + y^2 + z + 2
>>> P = A.random_element()
>>> Q = A.random_element()
>>> d(P*Q) == P*d(Q) + d(P)*Q
True
```

Alternatively we can use the method `derivation()` of the ring A to create derivations:

```
sage: Dx = A.derivation(x); Dx
d/dx
sage: Dy = A.derivation(y); Dy
d/dy
sage: Dz = A.derivation(z); Dz
d/dz
sage: A.derivation([2, z, x^2+y^2])
2*d/dx + z*d/dy + (x^2 + y^2)*d/dz
```

```
>>> from sage.all import *
>>> Dx = A.derivation(x); Dx
d/dx
>>> Dy = A.derivation(y); Dy
d/dy
>>> Dz = A.derivation(z); Dz
d/dz
```

(continues on next page)

(continued from previous page)

```
>>> A.derivation([Integer(2), z, x**Integer(2)+y**Integer(2)])
2*d/dx + z*d/dy + (x^2 + y^2)*d/dz
```

Sage knows moreover that M is a Lie algebra:

```
sage: M.category()
Join of
 Category of Lie algebras with basis over Rational Field and
 Category of modules with basis over
  Multivariate Polynomial Ring in x, y, z over Rational Field
```

```
>>> from sage.all import *
>>> M.category()
Join of
 Category of Lie algebras with basis over Rational Field and
 Category of modules with basis over
  Multivariate Polynomial Ring in x, y, z over Rational Field
```

Computations of Lie brackets are implemented as well:

```
sage: Dx.bracket(Dy)
0
sage: d.bracket(Dx)
-2*x*d/dz
```

```
>>> from sage.all import *
>>> Dx.bracket(Dy)
0
>>> d.bracket(Dx)
-2*x*d/dz
```

At the creation of a module of derivations, a codomain can be specified:

```
sage: B = A.fraction_field()
sage: A.derivation_module(B)
Module of derivations from Multivariate Polynomial Ring in x, y, z over Rational Field
 to Fraction Field of Multivariate Polynomial Ring in x, y, z over Rational Field
```

```
>>> from sage.all import *
>>> B = A.fraction_field()
>>> A.derivation_module(B)
Module of derivations from Multivariate Polynomial Ring in x, y, z over Rational Field
 to Fraction Field of Multivariate Polynomial Ring in x, y, z over Rational Field
```

Alternatively, one can specify a morphism f with domain A. In this case, the codomain of the derivations is the codomain of f but the latter is viewed as an algebra over A through the homomorphism f. This construction is useful, for example, if we want to work with derivations on A at a certain point, e.g. $(0, 1, 2)$. Indeed, in order to achieve this, we first define the evaluation map at this point:

```
sage: ev = A.hom([QQ(0), QQ(1), QQ(2)])
sage: ev
Ring morphism:
```

(continues on next page)

(continued from previous page)

```
  From: Multivariate Polynomial Ring in x, y, z over Rational Field
  To:   Rational Field
  Defn: x |--> 0
        y |--> 1
        z |--> 2
```

```
>>> from sage.all import *
>>> ev = A.hom([QQ(Integer(0)), QQ(Integer(1)), QQ(Integer(2))])
>>> ev
Ring morphism:
  From: Multivariate Polynomial Ring in x, y, z over Rational Field
  To:   Rational Field
  Defn: x |--> 0
        y |--> 1
        z |--> 2
```

Now we use this ring homomorphism to define a structure of A-algebra on $\mathbf{Q}$ and then build the following module of derivations:

```
sage: M = A.derivation_module(ev)
sage: M
Module of derivations
 from Multivariate Polynomial Ring in x, y, z over Rational Field
   to Rational Field
sage: M.gens()
(d/dx, d/dy, d/dz)
```

```
>>> from sage.all import *
>>> M = A.derivation_module(ev)
>>> M
Module of derivations
 from Multivariate Polynomial Ring in x, y, z over Rational Field
   to Rational Field
>>> M.gens()
(d/dx, d/dy, d/dz)
```

Elements in M then acts as derivations at $(0, 1, 2)$:

```
sage: Dx = M.gen(0)
sage: Dy = M.gen(1)
sage: Dz = M.gen(2)
sage: f = x^2 + y^2 + z^2
sage: Dx(f)  # = 2*x evaluated at (0,1,2)
0
sage: Dy(f)  # = 2*y evaluated at (0,1,2)
2
sage: Dz(f)  # = 2*z evaluated at (0,1,2)
4
```

```
>>> from sage.all import *
>>> Dx = M.gen(Integer(0))
>>> Dy = M.gen(Integer(1))
```

(continues on next page)

(continued from previous page)

```
>>> Dz = M.gen(Integer(2))
>>> f = x**Integer(2) + y**Integer(2) + z**Integer(2)
>>> Dx(f)  # = 2*x evaluated at (0,1,2)
0
>>> Dy(f)  # = 2*y evaluated at (0,1,2)
2
>>> Dz(f)  # = 2*z evaluated at (0,1,2)
4
```

Twisted derivations are handled similarly:

```
sage: theta = B.hom([B(y),B(z),B(x)])
sage: theta
Ring endomorphism of Fraction Field of
 Multivariate Polynomial Ring in x, y, z over Rational Field
  Defn: x |--> y
        y |--> z
        z |--> x

sage: M = B.derivation_module(twist=theta)
sage: M
Module of twisted derivations over Fraction Field of Multivariate Polynomial Ring
 in x, y, z over Rational Field (twisting morphism: x |--> y, y |--> z, z |--> x)
```

```
>>> from sage.all import *
>>> theta = B.hom([B(y),B(z),B(x)])
>>> theta
Ring endomorphism of Fraction Field of
 Multivariate Polynomial Ring in x, y, z over Rational Field
  Defn: x |--> y
        y |--> z
        z |--> x

>>> M = B.derivation_module(twist=theta)
>>> M
Module of twisted derivations over Fraction Field of Multivariate Polynomial Ring
 in x, y, z over Rational Field (twisting morphism: x |--> y, y |--> z, z |--> x)
```

Over a field, one proves that every θ-derivation is a multiple of $\theta - id$, so that:

```
sage: d = M.gen(); d
[x |--> y, y |--> z, z |--> x] - id
```

```
>>> from sage.all import *
>>> d = M.gen(); d
[x |--> y, y |--> z, z |--> x] - id
```

and then:

```
sage: d(x)
-x + y
sage: d(y)
```

(continues on next page)

(continued from previous page)

```
-y + z
sage: d(z)
x - z
sage: d(x + y + z)
0
```

```
>>> from sage.all import *
>>> d(x)
-x + y
>>> d(y)
-y + z
>>> d(z)
x - z
>>> d(x + y + z)
0
```

AUTHOR:

- Xavier Caruso (2018-09)

class sage.rings.derivation.**RingDerivation**

Bases: ModuleElement

An abstract class for twisted and untwisted derivations over commutative rings.

codomain()

Return the codomain of this derivation.

EXAMPLES:

```
sage: R.<x> = QQ[]
sage: f = R.derivation(); f
d/dx
sage: f.codomain()
Univariate Polynomial Ring in x over Rational Field
sage: f.codomain() is R
True
```

```
>>> from sage.all import *
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> f = R.derivation(); f
d/dx
>>> f.codomain()
Univariate Polynomial Ring in x over Rational Field
>>> f.codomain() is R
True
```

```
sage: S.<y> = R[]
sage: M = R.derivation_module(S)
sage: M.random_element().codomain()
Univariate Polynomial Ring in y over
 Univariate Polynomial Ring in x over Rational Field
sage: M.random_element().codomain() is S
True
```

```
>>> from sage.all import *
>>> S = R['y']; (y,) = S._first_ngens(1)
>>> M = R.derivation_module(S)
>>> M.random_element().codomain()
Univariate Polynomial Ring in y over
 Univariate Polynomial Ring in x over Rational Field
>>> M.random_element().codomain() is S
True
```

domain()

Return the domain of this derivation.

EXAMPLES:

```
sage: R.<x,y> = QQ[]
sage: f = R.derivation(y); f
d/dy
sage: f.domain()
Multivariate Polynomial Ring in x, y over Rational Field
sage: f.domain() is R
True
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> f = R.derivation(y); f
d/dy
>>> f.domain()
Multivariate Polynomial Ring in x, y over Rational Field
>>> f.domain() is R
True
```

class sage.rings.derivation.**RingDerivationModule**(*domain*, *codomain*, *twist=None*)

Bases: Module, UniqueRepresentation

A class for modules of derivations over a commutative ring.

basis()

Return a basis of this module of derivations.

EXAMPLES:

```
sage: R.<x,y> = ZZ[]
sage: M = R.derivation_module()
sage: M.basis()
Family (d/dx, d/dy)
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> M = R.derivation_module()
>>> M.basis()
Family (d/dx, d/dy)
```

codomain()

Return the codomain of the derivations in this module.

EXAMPLES:

```
sage: R.<x,y> = ZZ[]
sage: M = R.derivation_module(); M
Module of derivations over Multivariate Polynomial Ring in x, y over Integer␣
↪Ring
sage: M.codomain()
Multivariate Polynomial Ring in x, y over Integer Ring
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> M = R.derivation_module(); M
Module of derivations over Multivariate Polynomial Ring in x, y over Integer␣
↪Ring
>>> M.codomain()
Multivariate Polynomial Ring in x, y over Integer Ring
```

defining_morphism()

Return the morphism defining the structure of algebra of the codomain over the domain.

EXAMPLES:

```
sage: R.<x> = QQ[]
sage: M = R.derivation_module()
sage: M.defining_morphism()
Identity endomorphism of Univariate Polynomial Ring in x over Rational Field

sage: S.<y> = R[]
sage: M = R.derivation_module(S)
sage: M.defining_morphism()
Polynomial base injection morphism:
  From: Univariate Polynomial Ring in x over Rational Field
  To:   Univariate Polynomial Ring in y over
        Univariate Polynomial Ring in x over Rational Field

sage: ev = R.hom([QQ(0)])
sage: M = R.derivation_module(ev)
sage: M.defining_morphism()
Ring morphism:
  From: Univariate Polynomial Ring in x over Rational Field
  To:   Rational Field
  Defn: x |--> 0
```

```
>>> from sage.all import *
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> M = R.derivation_module()
>>> M.defining_morphism()
Identity endomorphism of Univariate Polynomial Ring in x over Rational Field

>>> S = R['y']; (y,) = S._first_ngens(1)
>>> M = R.derivation_module(S)
>>> M.defining_morphism()
Polynomial base injection morphism:
```

(continues on next page)

(continued from previous page)

```
  From: Univariate Polynomial Ring in x over Rational Field
  To:   Univariate Polynomial Ring in y over
        Univariate Polynomial Ring in x over Rational Field

>>> ev = R.hom([QQ(Integer(0))])
>>> M = R.derivation_module(ev)
>>> M.defining_morphism()
Ring morphism:
  From: Univariate Polynomial Ring in x over Rational Field
  To:   Rational Field
  Defn: x |--> 0
```

domain()

Return the domain of the derivations in this module.

EXAMPLES:

```
sage: R.<x,y> = ZZ[]
sage: M = R.derivation_module(); M
Module of derivations over Multivariate Polynomial Ring in x, y over Integer␣
↪Ring
sage: M.domain()
Multivariate Polynomial Ring in x, y over Integer Ring
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> M = R.derivation_module(); M
Module of derivations over Multivariate Polynomial Ring in x, y over Integer␣
↪Ring
>>> M.domain()
Multivariate Polynomial Ring in x, y over Integer Ring
```

dual_basis()

Return the dual basis of the canonical basis of this module of derivations (which is that returned by the method *basis()*).

Note

The dual basis of $(d_1, \ldots, d_n)$ is a family $(x_1, \ldots, x_n)$ of elements in the domain such that $d_i(x_i) = 1$ and $d_i(x_j) = 0$ if $i \neq j$.

EXAMPLES:

```
sage: R.<x,y> = ZZ[]
sage: M = R.derivation_module()
sage: M.basis()
Family (d/dx, d/dy)
sage: M.dual_basis()
Family (x, y)
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> M = R.derivation_module()
>>> M.basis()
Family (d/dx, d/dy)
>>> M.dual_basis()
Family (x, y)
```

gen(*n=0*)

Return the n-th generator of this module of derivations.

INPUT:

- n – integer (default: 0)

EXAMPLES:

```
sage: R.<x,y> = ZZ[]
sage: M = R.derivation_module(); M
Module of derivations over Multivariate Polynomial Ring in x, y over Integer␣
↪Ring
sage: M.gen()
d/dx
sage: M.gen(1)
d/dy
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> M = R.derivation_module(); M
Module of derivations over Multivariate Polynomial Ring in x, y over Integer␣
↪Ring
>>> M.gen()
d/dx
>>> M.gen(Integer(1))
d/dy
```

gens()

Return the generators of this module of derivations.

EXAMPLES:

```
sage: R.<x,y> = ZZ[]
sage: M = R.derivation_module(); M
Module of derivations over Multivariate Polynomial Ring in x, y over Integer␣
↪Ring
sage: M.gens()
(d/dx, d/dy)
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> M = R.derivation_module(); M
Module of derivations over Multivariate Polynomial Ring in x, y over Integer␣
↪Ring
```

(continues on next page)

(continued from previous page)

```
>>> M.gens()
(d/dx, d/dy)
```

We check that, for a nontrivial twist over a field, the module of twisted derivation is a vector space of dimension 1 generated by `twist - id`:

```
sage: K = R.fraction_field()
sage: theta = K.hom([K(y),K(x)])
sage: M = K.derivation_module(twist=theta); M
Module of twisted derivations over Fraction Field of Multivariate Polynomial
 Ring in x, y over Integer Ring (twisting morphism: x |--> y, y |--> x)
sage: M.gens()
([x |--> y, y |--> x] - id,)
```

```
>>> from sage.all import *
>>> K = R.fraction_field()
>>> theta = K.hom([K(y),K(x)])
>>> M = K.derivation_module(twist=theta); M
Module of twisted derivations over Fraction Field of Multivariate Polynomial
 Ring in x, y over Integer Ring (twisting morphism: x |--> y, y |--> x)
>>> M.gens()
([x |--> y, y |--> x] - id,)
```

ngens()

Return the number of generators of this module of derivations.

EXAMPLES:

```
sage: R.<x,y> = ZZ[]
sage: M = R.derivation_module(); M
Module of derivations over Multivariate Polynomial Ring in x, y over Integer␣
↪Ring
sage: M.ngens()
2
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> M = R.derivation_module(); M
Module of derivations over Multivariate Polynomial Ring in x, y over Integer␣
↪Ring
>>> M.ngens()
2
```

Indeed, generators are:

```
sage: M.gens()
(d/dx, d/dy)
```

```
>>> from sage.all import *
>>> M.gens()
(d/dx, d/dy)
```

We check that, for a nontrivial twist over a field, the module of twisted derivation is a vector space of dimension 1 generated by `twist - id`:

```
sage: K = R.fraction_field()
sage: theta = K.hom([K(y),K(x)])
sage: M = K.derivation_module(twist=theta); M
Module of twisted derivations over Fraction Field of Multivariate Polynomial
 Ring in x, y over Integer Ring (twisting morphism: x |--> y, y |--> x)
sage: M.ngens()
1
sage: M.gen()
[x |--> y, y |--> x] - id
```

```
>>> from sage.all import *
>>> K = R.fraction_field()
>>> theta = K.hom([K(y),K(x)])
>>> M = K.derivation_module(twist=theta); M
Module of twisted derivations over Fraction Field of Multivariate Polynomial
 Ring in x, y over Integer Ring (twisting morphism: x |--> y, y |--> x)
>>> M.ngens()
1
>>> M.gen()
[x |--> y, y |--> x] - id
```

random_element(**args*, ***kwds*)

Return a random derivation in this module.

EXAMPLES:

```
sage: R.<x,y> = ZZ[]
sage: M = R.derivation_module()
sage: M.random_element()  # random
(x^2 + x*y - 3*y^2 + x + 1)*d/dx + (-2*x^2 + 3*x*y + 10*y^2 + 2*x + 8)*d/dy
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> M = R.derivation_module()
>>> M.random_element()  # random
(x^2 + x*y - 3*y^2 + x + 1)*d/dx + (-2*x^2 + 3*x*y + 10*y^2 + 2*x + 8)*d/dy
```

ring_of_constants()

Return the subring of the domain consisting of elements x such that $d(x) = 0$ for all derivation d in this module.

EXAMPLES:

```
sage: R.<x,y> = QQ[]
sage: M = R.derivation_module()
sage: M.basis()
Family (d/dx, d/dy)
sage: M.ring_of_constants()
Rational Field
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> M = R.derivation_module()
>>> M.basis()
Family (d/dx, d/dy)
>>> M.ring_of_constants()
Rational Field
```

some_elements()

Return a list of elements of this module.

EXAMPLES:

```
sage: R.<x,y> = ZZ[]
sage: M = R.derivation_module()
sage: M.some_elements()
[d/dx, d/dy, x*d/dx, x*d/dy, y*d/dx, y*d/dy]
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> M = R.derivation_module()
>>> M.some_elements()
[d/dx, d/dy, x*d/dx, x*d/dy, y*d/dx, y*d/dy]
```

twisting_morphism()

Return the twisting homomorphism of the derivations in this module.

EXAMPLES:

```
sage: R.<x,y> = ZZ[]
sage: theta = R.hom([y,x])
sage: M = R.derivation_module(twist=theta); M
Module of twisted derivations over Multivariate Polynomial Ring in x, y
 over Integer Ring (twisting morphism: x |--> y, y |--> x)
sage: M.twisting_morphism()
Ring endomorphism of Multivariate Polynomial Ring in x, y over Integer Ring
  Defn: x |--> y
        y |--> x
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> theta = R.hom([y,x])
>>> M = R.derivation_module(twist=theta); M
Module of twisted derivations over Multivariate Polynomial Ring in x, y
 over Integer Ring (twisting morphism: x |--> y, y |--> x)
>>> M.twisting_morphism()
Ring endomorphism of Multivariate Polynomial Ring in x, y over Integer Ring
  Defn: x |--> y
        y |--> x
```

When the derivations are untwisted, this method returns nothing:

```
sage: M = R.derivation_module()
sage: M.twisting_morphism()
```

```
>>> from sage.all import *
>>> M = R.derivation_module()
>>> M.twisting_morphism()
```

class sage.rings.derivation.**RingDerivationWithTwist_generic**(*parent*, *scalar=0*)

Bases: *RingDerivation*

The class handles θ-derivations of the form $\lambda(\theta - \iota)$ (where ι is the defining morphism of the codomain over the domain) for a scalar λ varying in the codomain.

extend_to_fraction_field()

Return the extension of this derivation to fraction fields of the domain and the codomain.

EXAMPLES:

```
sage: R.<x,y> = ZZ[]
sage: theta = R.hom([y,x])
sage: d = R.derivation(x, twist=theta)
sage: d
x*([x |--> y, y |--> x] - id)

sage: D = d.extend_to_fraction_field(); D                                   #␣
↪needs sage.libs.singular
x*([x |--> y, y |--> x] - id)
sage: D.domain()                                                            #␣
↪needs sage.libs.singular
Fraction Field of Multivariate Polynomial Ring in x, y over Integer Ring

sage: D(1/x)                                                                #␣
↪needs sage.libs.singular
(x - y)/y
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> theta = R.hom([y,x])
>>> d = R.derivation(x, twist=theta)
>>> d
x*([x |--> y, y |--> x] - id)

>>> D = d.extend_to_fraction_field(); D                                     #␣
↪needs sage.libs.singular
x*([x |--> y, y |--> x] - id)
>>> D.domain()                                                              #␣
↪needs sage.libs.singular
Fraction Field of Multivariate Polynomial Ring in x, y over Integer Ring

>>> D(Integer(1)/x)                                                         ␣
↪     # needs sage.libs.singular
(x - y)/y
```

list()

Return the list of coefficient of this twisted derivation on the canonical basis.

EXAMPLES:

```
sage: R.<x,y> = QQ[]
sage: K = R.fraction_field()
sage: theta = K.hom([y,x])
sage: M = K.derivation_module(twist=theta)
sage: M.basis()
Family (twisting_morphism - id,)
sage: f = (x+y) * M.gen()
sage: f
(x + y)*(twisting_morphism - id)
sage: f.list()
[x + y]
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> K = R.fraction_field()
>>> theta = K.hom([y,x])
>>> M = K.derivation_module(twist=theta)
>>> M.basis()
Family (twisting_morphism - id,)
>>> f = (x+y) * M.gen()
>>> f
(x + y)*(twisting_morphism - id)
>>> f.list()
[x + y]
```

postcompose(*morphism*)

Return the twisted derivation obtained by applying first this twisted derivation and then `morphism`.

INPUT:

- `morphism` – a homomorphism of rings whose domain is the codomain of this derivation or a ring into which the codomain of this derivation

EXAMPLES:

```
sage: R.<x,y> = ZZ[]
sage: theta = R.hom([y,x])
sage: D = R.derivation(x, twist=theta); D
x*([x |--> y, y |--> x] - id)

sage: f = R.hom([x^2, y^3])
sage: g = D.precompose(f); g
x*([x |--> y^2, y |--> x^3] - [x |--> x^2, y |--> y^3])
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> theta = R.hom([y,x])
>>> D = R.derivation(x, twist=theta); D
x*([x |--> y, y |--> x] - id)

>>> f = R.hom([x**Integer(2), y**Integer(3)])
>>> g = D.precompose(f); g
x*([x |--> y^2, y |--> x^3] - [x |--> x^2, y |--> y^3])
```

Observe that the g is no longer a θ-derivation but a $(\theta \circ f)$-derivation:

```
sage: g.parent().twisting_morphism()
Ring endomorphism of Multivariate Polynomial Ring in x, y over Integer Ring
  Defn: x |--> y^2
        y |--> x^3
```

```
>>> from sage.all import *
>>> g.parent().twisting_morphism()
Ring endomorphism of Multivariate Polynomial Ring in x, y over Integer Ring
  Defn: x |--> y^2
        y |--> x^3
```

precompose(*morphism*)

Return the twisted derivation obtained by applying first `morphism` and then this twisted derivation.

INPUT:

- `morphism` – a homomorphism of rings whose codomain is the domain of this derivation or a ring that coerces to the domain of this derivation

EXAMPLES:

```
sage: R.<x,y> = ZZ[]
sage: theta = R.hom([y,x])
sage: D = R.derivation(x, twist=theta); D
x*([x |--> y, y |--> x] - id)

sage: f = R.hom([x^2, y^3])
sage: g = D.postcompose(f); g
x^2*([x |--> y^3, y |--> x^2] - [x |--> x^2, y |--> y^3])
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> theta = R.hom([y,x])
>>> D = R.derivation(x, twist=theta); D
x*([x |--> y, y |--> x] - id)

>>> f = R.hom([x**Integer(2), y**Integer(3)])
>>> g = D.postcompose(f); g
x^2*([x |--> y^3, y |--> x^2] - [x |--> x^2, y |--> y^3])
```

Observe that the g is no longer a θ-derivation but a $(f \circ \theta)$-derivation:

```
sage: g.parent().twisting_morphism()
Ring endomorphism of Multivariate Polynomial Ring in x, y over Integer Ring
  Defn: x |--> y^3
        y |--> x^2
```

```
>>> from sage.all import *
>>> g.parent().twisting_morphism()
Ring endomorphism of Multivariate Polynomial Ring in x, y over Integer Ring
  Defn: x |--> y^3
        y |--> x^2
```

class sage.rings.derivation.**RingDerivationWithoutTwist**

Bases: *RingDerivation*

An abstract class for untwisted derivations.

extend_to_fraction_field()

Return the extension of this derivation to fraction fields of the domain and the codomain.

EXAMPLES:

```
sage: S.<x> = QQ[]
sage: d = S.derivation()
sage: d
d/dx

sage: D = d.extend_to_fraction_field()
sage: D
d/dx
sage: D.domain()
Fraction Field of Univariate Polynomial Ring in x over Rational Field

sage: D(1/x)
-1/x^2
```

```
>>> from sage.all import *
>>> S = QQ['x']; (x,) = S._first_ngens(1)
>>> d = S.derivation()
>>> d
d/dx

>>> D = d.extend_to_fraction_field()
>>> D
d/dx
>>> D.domain()
Fraction Field of Univariate Polynomial Ring in x over Rational Field

>>> D(Integer(1)/x)
-1/x^2
```

is_zero()

Return `True` if this derivation is zero.

EXAMPLES:

```
sage: R.<x,y> = ZZ[]
sage: f = R.derivation(); f
d/dx
sage: f.is_zero()
False

sage: (f-f).is_zero()
True
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> f = R.derivation(); f
d/dx
>>> f.is_zero()
False

>>> (f-f).is_zero()
True
```

list()

Return the list of coefficient of this derivation on the canonical basis.

EXAMPLES:

```
sage: R.<x,y> = QQ[]
sage: M = R.derivation_module()
sage: M.basis()
Family (d/dx, d/dy)

sage: R.derivation(x).list()
[1, 0]
sage: R.derivation(y).list()
[0, 1]

sage: f = x*R.derivation(x) + y*R.derivation(y); f
x*d/dx + y*d/dy
sage: f.list()
[x, y]
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> M = R.derivation_module()
>>> M.basis()
Family (d/dx, d/dy)

>>> R.derivation(x).list()
[1, 0]
>>> R.derivation(y).list()
[0, 1]

>>> f = x*R.derivation(x) + y*R.derivation(y); f
x*d/dx + y*d/dy
>>> f.list()
[x, y]
```

monomial_coefficients(*copy=None*)

Return dictionary of nonzero coordinates (on the canonical basis) of this derivation.

More precisely, this returns a dictionary whose keys are indices of basis elements and whose values are the corresponding coefficients.

EXAMPLES:

```
sage: R.<x,y> = QQ[]
sage: M = R.derivation_module()
sage: M.basis()
Family (d/dx, d/dy)

sage: R.derivation(x).monomial_coefficients()
{0: 1}
sage: R.derivation(y).monomial_coefficients()
{1: 1}

sage: f = x*R.derivation(x) + y*R.derivation(y); f
x*d/dx + y*d/dy
sage: f.monomial_coefficients()
{0: x, 1: y}
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> M = R.derivation_module()
>>> M.basis()
Family (d/dx, d/dy)

>>> R.derivation(x).monomial_coefficients()
{0: 1}
>>> R.derivation(y).monomial_coefficients()
{1: 1}

>>> f = x*R.derivation(x) + y*R.derivation(y); f
x*d/dx + y*d/dy
>>> f.monomial_coefficients()
{0: x, 1: y}
```

postcompose(*morphism*)

Return the derivation obtained by applying first this derivation and then `morphism`.

INPUT:

- `morphism` – a homomorphism of rings whose domain is the codomain of this derivation or a ring into which the codomain of this derivation coerces

EXAMPLES:

```
sage: A.<x,y>= QQ[]
sage: ev = A.hom([QQ(0), QQ(1)])
sage: Dx = A.derivation(x)
sage: Dy = A.derivation(y)
```

```
>>> from sage.all import *
>>> A = QQ['x, y']; (x, y,) = A._first_ngens(2)
>>> ev = A.hom([QQ(Integer(0)), QQ(Integer(1))])
>>> Dx = A.derivation(x)
>>> Dy = A.derivation(y)
```

We can define the derivation at $(0, 1)$ just by postcomposing with `ev`:

```
sage: dx = Dx.postcompose(ev)
sage: dy = Dy.postcompose(ev)
sage: f = x^2 + y^2
sage: dx(f)
0
sage: dy(f)
2
```

```
>>> from sage.all import *
>>> dx = Dx.postcompose(ev)
>>> dy = Dy.postcompose(ev)
>>> f = x**Integer(2) + y**Integer(2)
>>> dx(f)
0
>>> dy(f)
2
```

Note that we cannot avoid the creation of the evaluation morphism: if we pass in `QQ` instead, an error is raised since there is no coercion morphism from `A` to `QQ`:

```
sage: Dx.postcompose(QQ)
Traceback (most recent call last):
...
TypeError: the codomain of the derivation does not coerce to the given ring
```

```
>>> from sage.all import *
>>> Dx.postcompose(QQ)
Traceback (most recent call last):
...
TypeError: the codomain of the derivation does not coerce to the given ring
```

Note that this method cannot be used to compose derivations:

```
sage: Dx.precompose(Dy)
Traceback (most recent call last):
...
TypeError: you must give a homomorphism of rings
```

```
>>> from sage.all import *
>>> Dx.precompose(Dy)
Traceback (most recent call last):
...
TypeError: you must give a homomorphism of rings
```

precompose(*morphism*)

Return the derivation obtained by applying first `morphism` and then this derivation.

INPUT:

- `morphism` – a homomorphism of rings whose codomain is the domain of this derivation or a ring that coerces to the domain of this derivation

EXAMPLES:

```
sage: A.<x> = QQ[]
sage: B.<x,y> = QQ[]
sage: D = B.derivation(x) - 2*x*B.derivation(y); D
d/dx - 2*x*d/dy
```

```
>>> from sage.all import *
>>> A = QQ['x']; (x,) = A._first_ngens(1)
>>> B = QQ['x, y']; (x, y,) = B._first_ngens(2)
>>> D = B.derivation(x) - Integer(2)*x*B.derivation(y); D
d/dx - 2*x*d/dy
```

When restricting to `A`, the term `d/dy` disappears (since it vanishes on `A`):

```
sage: D.precompose(A)
d/dx
```

```
>>> from sage.all import *
>>> D.precompose(A)
d/dx
```

If we restrict to another well chosen subring, the derivation vanishes:

```
sage: C.<t> = QQ[]
sage: f = C.hom([x^2 + y]); f
Ring morphism:
  From: Univariate Polynomial Ring in t over Rational Field
  To:   Multivariate Polynomial Ring in x, y over Rational Field
  Defn: t |--> x^2 + y
sage: D.precompose(f)
0
```

```
>>> from sage.all import *
>>> C = QQ['t']; (t,) = C._first_ngens(1)
>>> f = C.hom([x**Integer(2) + y]); f
Ring morphism:
  From: Univariate Polynomial Ring in t over Rational Field
  To:   Multivariate Polynomial Ring in x, y over Rational Field
  Defn: t |--> x^2 + y
>>> D.precompose(f)
0
```

Note that this method cannot be used to compose derivations:

```
sage: D.precompose(D)
Traceback (most recent call last):
...
TypeError: you must give a homomorphism of rings
```

```
>>> from sage.all import *
>>> D.precompose(D)
Traceback (most recent call last):
...
TypeError: you must give a homomorphism of rings
```

pth_power()

Return the p-th power of this derivation where p is the characteristic of the domain.

> **Note**
>
> Leibniz rule implies that this is again a derivation.

EXAMPLES:

```
sage: R.<x,y> = GF(5)[]
sage: Dx = R.derivation(x)
sage: Dx.pth_power()
0
sage: (x*Dx).pth_power()
x*d/dx
sage: (x^6*Dx).pth_power()
x^26*d/dx

sage: Dy = R.derivation(y)                                                  #␣
↪needs sage.rings.finite_rings
sage: (x*Dx + y*Dy).pth_power()                                             #␣
↪needs sage.rings.finite_rings
x*d/dx + y*d/dy
```

```
>>> from sage.all import *
>>> R = GF(Integer(5))['x, y']; (x, y,) = R._first_ngens(2)
>>> Dx = R.derivation(x)
>>> Dx.pth_power()
0
>>> (x*Dx).pth_power()
x*d/dx
>>> (x**Integer(6)*Dx).pth_power()
x^26*d/dx

>>> Dy = R.derivation(y)                                                    #␣
↪needs sage.rings.finite_rings
>>> (x*Dx + y*Dy).pth_power()                                               #␣
↪needs sage.rings.finite_rings
x*d/dx + y*d/dy
```

An error is raised if the domain has characteristic zero:

```
sage: R.<x,y> = QQ[]
sage: Dx = R.derivation(x)
sage: Dx.pth_power()
Traceback (most recent call last):
...
TypeError: the domain of the derivation must have positive and prime␣
↪characteristic
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
```

(continues on next page)

(continued from previous page)

```
>>> Dx = R.derivation(x)
>>> Dx.pth_power()
Traceback (most recent call last):
...
TypeError: the domain of the derivation must have positive and prime␣
↪characteristic
```

or if the characteristic is not a prime number:

```
sage: R.<x,y> = Integers(10)[]
sage: Dx = R.derivation(x)
sage: Dx.pth_power()
Traceback (most recent call last):
...
TypeError: the domain of the derivation must have positive and prime␣
↪characteristic
```

```
>>> from sage.all import *
>>> R = Integers(Integer(10))['x, y']; (x, y,) = R._first_ngens(2)
>>> Dx = R.derivation(x)
>>> Dx.pth_power()
Traceback (most recent call last):
...
TypeError: the domain of the derivation must have positive and prime␣
↪characteristic
```

class sage.rings.derivation.**RingDerivationWithoutTwist_fraction_field**(*parent*, *arg=None*)

Bases: *RingDerivationWithoutTwist_wrapper*

This class handles derivations over fraction fields.

class sage.rings.derivation.**RingDerivationWithoutTwist_function**(*parent*, *arg=None*)

Bases: *RingDerivationWithoutTwist*

A class for untwisted derivations over rings whose elements are either polynomials, rational fractions, power series or Laurent series.

is_zero()

Return `True` if this derivation is zero.

EXAMPLES:

```
sage: R.<x,y> = ZZ[]
sage: f = R.derivation(); f
d/dx
sage: f.is_zero()
False

sage: (f-f).is_zero()
True
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> f = R.derivation(); f
```

(continues on next page)

(continued from previous page)

```
d/dx
>>> f.is_zero()
False

>>> (f-f).is_zero()
True
```

list()

Return the list of coefficient of this derivation on the canonical basis.

EXAMPLES:

```
sage: R.<x,y> = GF(5)[[]]
sage: M = R.derivation_module()
sage: M.basis()
Family (d/dx, d/dy)

sage: R.derivation(x).list()
[1, 0]
sage: R.derivation(y).list()
[0, 1]

sage: f = x*R.derivation(x) + y*R.derivation(y); f
x*d/dx + y*d/dy
sage: f.list()
[x, y]
```

```
>>> from sage.all import *
>>> R = GF(Integer(5))[['x, y']]; (x, y,) = R._first_ngens(2)
>>> M = R.derivation_module()
>>> M.basis()
Family (d/dx, d/dy)

>>> R.derivation(x).list()
[1, 0]
>>> R.derivation(y).list()
[0, 1]

>>> f = x*R.derivation(x) + y*R.derivation(y); f
x*d/dx + y*d/dy
>>> f.list()
[x, y]
```

class sage.rings.derivation.**RingDerivationWithoutTwist_quotient**(*parent*, *arg=None*)

Bases: *RingDerivationWithoutTwist_wrapper*

This class handles derivations over quotient rings.

class sage.rings.derivation.**RingDerivationWithoutTwist_wrapper**(*parent*, *arg=None*)

Bases: *RingDerivationWithoutTwist*

This class is a wrapper for derivation.

It is useful for changing the parent without changing the computation rules for derivations. It is used for derivations over fraction fields and quotient rings.

list()

Return the list of coefficient of this derivation on the canonical basis.

EXAMPLES:

```
sage: R.<X,Y> = GF(5)[]
sage: S.<x,y> = R.quo([X^5, Y^5])
sage: M = S.derivation_module()
sage: M.basis()
Family (d/dx, d/dy)
sage: S.derivation(x).list()
[1, 0]
sage: S.derivation(y).list()
[0, 1]
sage: f = x*S.derivation(x) + y*S.derivation(y); f
x*d/dx + y*d/dy
sage: f.list()
[x, y]
```

```
>>> from sage.all import *
>>> R = GF(Integer(5))['X, Y']; (X, Y,) = R._first_ngens(2)
>>> S = R.quo([X**Integer(5), Y**Integer(5)], names=('x', 'y',)); (x, y,) = S.
↪_first_ngens(2)
>>> M = S.derivation_module()
>>> M.basis()
Family (d/dx, d/dy)
>>> S.derivation(x).list()
[1, 0]
>>> S.derivation(y).list()
[0, 1]
>>> f = x*S.derivation(x) + y*S.derivation(y); f
x*d/dx + y*d/dy
>>> f.list()
[x, y]
```

class sage.rings.derivation.**RingDerivationWithoutTwist_zero**(*parent*, *arg=None*)

Bases: *RingDerivationWithoutTwist*

This class can only represent the zero derivation.

It is used when the parent is the zero derivation module (e.g., when its domain is ZZ, QQ, a finite field, etc.)

is_zero()

Return True if this derivation vanishes.

EXAMPLES:

```
sage: M = QQ.derivation_module()
sage: M().is_zero()
True
```

```
>>> from sage.all import *
>>> M = QQ.derivation_module()
>>> M().is_zero()
True
```

list()

Return the list of coefficient of this derivation on the canonical basis.

EXAMPLES:

```
sage: M = QQ.derivation_module()
sage: M().list()
[]
```

```
>>> from sage.all import *
>>> M = QQ.derivation_module()
>>> M().list()
[]
```

CHAPTER

ELEVEN

INDICES AND TABLES

- Index
- Module Index
- Search Page

PYTHON MODULE INDEX

r

sage.rings.abc, 5
sage.rings.big_oh, 225
sage.rings.derivation, 243
sage.rings.fraction_field, 119
sage.rings.fraction_field_element, 129
sage.rings.generic, 217
sage.rings.homset, 86
sage.rings.ideal, 19
sage.rings.ideal_monoid, 46
sage.rings.infinity, 228
sage.rings.localization, 139
sage.rings.morphism, 51
sage.rings.noncommutative_ideals, 47
sage.rings.numbers_abc, 242
sage.rings.quotient_ring, 89
sage.rings.quotient_ring_element, 111
sage.rings.ring, 1
sage.rings.ring_extension, 155
sage.rings.ring_extension_element, 190
sage.rings.ring_extension_morphism, 209

INDEX

Non-alphabetical

_uniq (*class in sage.rings.infinity*), 228

A

absolute_base() (*sage.rings.ring_extension.RingExtension_generic method*), 167
absolute_degree() (*sage.rings.ring_extension.RingExtension_generic method*), 168
absolute_norm() (*sage.rings.ideal.Ideal_generic method*), 24
additive_order() (*sage.rings.ring_extension_element.RingExtensionElement method*), 190
Algebra (*class in sage.rings.ring*), 2
AlgebraicField (*class in sage.rings.abc*), 5
AlgebraicField_common (*class in sage.rings.abc*), 6
AlgebraicRealField (*class in sage.rings.abc*), 7
ambient() (*sage.rings.quotient_ring.QuotientRing_nc method*), 98
AnInfinity (*class in sage.rings.infinity*), 233
apply_morphism() (*sage.rings.ideal.Ideal_generic method*), 25
associated_primes() (*sage.rings.ideal.Ideal_generic method*), 25

B

backend() (*sage.rings.ring_extension_element.RingExtensionElement method*), 190
backend() (*sage.rings.ring_extension.RingExtension_generic method*), 168
base() (*sage.rings.ring_extension.RingExtension_generic method*), 169
base_extend() (*sage.rings.ring.Ring method*), 2
base_map() (*sage.rings.morphism.RingHomomorphism_im_gens method*), 83
base_map() (*sage.rings.ring_extension_morphism.RingExtensionHomomorphism method*), 211
base_ring() (*sage.rings.fraction_field.FractionField_generic method*), 124
base_ring() (*sage.rings.ideal.Ideal_generic method*), 26
bases() (*sage.rings.ring_extension.RingExtension_generic method*), 169
basis() (*sage.rings.derivation.RingDerivationModule method*), 249
basis_over() (*sage.rings.ring_extension.RingExtensionWithBasis method*), 159

C

category() (*sage.rings.ideal.Ideal_generic method*), 27
category() (*sage.rings.ring.Ring method*), 2
characteristic() (*sage.rings.fraction_field.FractionField_generic method*), 124
characteristic() (*sage.rings.localization.Localization method*), 147
characteristic() (*sage.rings.quotient_ring.QuotientRing_nc method*), 98
characteristic() (*sage.rings.ring_extension.RingExtension_generic method*), 170
charpoly() (*sage.rings.ring_extension_element.RingExtensionWithBasisElement method*), 197
check_comparison() (*in module sage.rings.infinity*), 239
check_signed_infinity() (*in module sage.rings.infinity*), 241
class_number() (*sage.rings.fraction_field.FractionField_1poly_field method*), 122
codomain() (*sage.rings.derivation.RingDerivation method*), 248
codomain() (*sage.rings.derivation.RingDerivationModule method*), 249
common_base() (*in module sage.rings.ring_extension*), 186
CommutativeAlgebra (*class in sage.rings.ring*), 2
CommutativeRing (*class in sage.rings.ring*), 2
ComplexBallField (*class in sage.rings.abc*), 8
ComplexDoubleField (*class in sage.rings.abc*), 8
ComplexField (*class in sage.rings.abc*), 9
ComplexIntervalField (*class in sage.rings.abc*), 10
construction() (*sage.rings.fraction_field.FractionField_generic method*), 124
construction() (*sage.rings.quotient_ring.QuotientRing_nc method*), 99
construction() (*sage.rings.ring_extension.RingExtension_generic method*), 171

cover() (*sage.rings.quotient_ring.QuotientRing_nc method*), 99
cover_ring() (*sage.rings.quotient_ring.QuotientRing_nc method*), 100
create_key_and_extra_args() (*sage.rings.ring_extension.RingExtensionFactory method*), 158
create_object() (*sage.rings.ring_extension.RingExtensionFactory method*), 158
CRT_bases() (*sage.rings.generic.ProductTree method*), 219
Cyclic() (*in module sage.rings.ideal*), 19

D

DedekindDomain (*class in sage.rings.ring*), 2
defining_ideal() (*sage.rings.quotient_ring.QuotientRing_nc method*), 101
defining_morphism() (*sage.rings.derivation.RingDerivationModule method*), 250
defining_morphism() (*sage.rings.ring_extension.RingExtension_generic method*), 171
degree() (*sage.rings.ring_extension.RingExtension_generic method*), 173
degree_over() (*sage.rings.ring_extension.RingExtension_generic method*), 174
denominator() (*sage.rings.fraction_field_element.FractionFieldElement method*), 129
denominator() (*sage.rings.localization.LocalizationElement method*), 150
denominator() (*sage.rings.ring_extension_element.RingExtensionFractionFieldElement method*), 195
divides() (*sage.rings.ideal.Ideal_principal method*), 43
domain() (*sage.rings.derivation.RingDerivation method*), 249
domain() (*sage.rings.derivation.RingDerivationModule method*), 251
dual_basis() (*sage.rings.derivation.RingDerivationModule method*), 251

E

Element (*sage.rings.homset.RingHomset_generic attribute*), 86
Element (*sage.rings.homset.RingHomset_quo_ring attribute*), 88
Element (*sage.rings.ideal_monoid.IdealMonoid_c attribute*), 47
Element (*sage.rings.localization.Localization attribute*), 147
Element (*sage.rings.quotient_ring.QuotientRing_nc attribute*), 98
Element (*sage.rings.ring_extension.RingExtension_generic attribute*), 167
Element (*sage.rings.ring_extension.RingExtensionFractionField attribute*), 158
Element (*sage.rings.ring_extension.RingExtensionWithBasis attribute*), 159
embedded_primes() (*sage.rings.ideal.Ideal_generic method*), 28
extend_to_fraction_field() (*sage.rings.derivation.RingDerivationWithoutTwist method*), 259
extend_to_fraction_field() (*sage.rings.derivation.RingDerivationWithTwist_generic method*), 256

F

factor() (*sage.rings.localization.LocalizationElement method*), 151
Field (*class in sage.rings.ring*), 2
FieldIdeal() (*in module sage.rings.ideal*), 20
FiniteNumber (*class in sage.rings.infinity*), 234
fraction_field() (*sage.rings.infinity.InfinityRing_class method*), 235
fraction_field() (*sage.rings.localization.Localization method*), 147
fraction_field() (*sage.rings.ring_extension.RingExtension_generic method*), 175
fraction_field() (*sage.rings.ring_extension.RingExtensionWithBasis method*), 160
fraction_field() (*sage.rings.ring_extension.RingExtensionWithGen method*), 164
FractionField() (*in module sage.rings.fraction_field*), 120
FractionField_1poly_field (*class in sage.rings.fraction_field*), 122
FractionField_generic (*class in sage.rings.fraction_field*), 124
FractionFieldElement (*class in sage.rings.fraction_field_element*), 129
FractionFieldElement_1poly_field (*class in sage.rings.fraction_field_element*), 134
FractionFieldEmbedding (*class in sage.rings.fraction_field*), 121
FractionFieldEmbeddingSection (*class in sage.rings.fraction_field*), 122
free_module() (*sage.rings.ring_extension.RingExtensionWithBasis method*), 162
free_resolution() (*sage.rings.ideal.Ideal_generic method*), 28
FrobeniusEndomorphism_generic (*class in sage.rings.morphism*), 63
from_base_ring() (*sage.rings.ring_extension.RingExtension_generic method*), 176
function_field() (*sage.rings.fraction_field.FractionField_1poly_field method*), 123

G

gcd() (*sage.rings.ideal.Ideal_pid method*), 39

gen() (*sage.rings.derivation.RingDerivationModule method*), 252
gen() (*sage.rings.fraction_field.FractionField_generic method*), 125
gen() (*sage.rings.ideal.Ideal_generic method*), 28
gen() (*sage.rings.ideal.Ideal_principal method*), 43
gen() (*sage.rings.infinity.InfinityRing_class method*), 235
gen() (*sage.rings.infinity.UnsignedInfinityRing_class method*), 238
gen() (*sage.rings.localization.Localization method*), 148
gen() (*sage.rings.quotient_ring.QuotientRing_nc method*), 102
gen() (*sage.rings.ring_extension.RingExtension_generic method*), 177
generators() (*in module sage.rings.ring_extension*), 187
gens() (*sage.rings.derivation.RingDerivationModule method*), 252
gens() (*sage.rings.ideal.Ideal_generic method*), 29
gens() (*sage.rings.infinity.InfinityRing_class method*), 235
gens() (*sage.rings.infinity.UnsignedInfinityRing_class method*), 238
gens() (*sage.rings.localization.Localization method*), 148
gens() (*sage.rings.quotient_ring.QuotientRing_nc method*), 103
gens() (*sage.rings.ring_extension.RingExtension_generic method*), 177
gens() (*sage.rings.ring_extension.RingExtensionWithGen method*), 166
gens() (*sage.rings.ring.Ring method*), 3
gens_reduced() (*sage.rings.ideal.Ideal_generic method*), 29
graded_free_resolution() (*sage.rings.ideal.Ideal_generic method*), 30

H

has_coerce_map_from() (*sage.rings.homset.RingHomset_generic method*), 86
hom() (*sage.rings.ring_extension.RingExtension_generic method*), 178

I

Ideal() (*in module sage.rings.ideal*), 21
ideal() (*sage.rings.quotient_ring.QuotientRing_nc method*), 103
Ideal_fractional (*class in sage.rings.ideal*), 24
Ideal_generic (*class in sage.rings.ideal*), 24
Ideal_nc (*class in sage.rings.noncommutative_ideals*), 48
Ideal_pid (*class in sage.rings.ideal*), 38
Ideal_principal (*class in sage.rings.ideal*), 43
IdealMonoid() (*in module sage.rings.ideal_monoid*), 46
IdealMonoid_c (*class in sage.rings.ideal_monoid*), 46
IdealMonoid_nc (*class in sage.rings.noncommutative_ideals*), 48
im_gens() (*sage.rings.morphism.RingHomomorphism_im_gens method*), 84
in_base() (*sage.rings.ring_extension_element.RingExtensionElement method*), 191
infinity (*in module sage.rings.infinity*), 228
InfinityRing_class (*class in sage.rings.infinity*), 235
IntegerModRing (*class in sage.rings.abc*), 10
IntegralDomain (*class in sage.rings.ring*), 2
interpolation() (*sage.rings.generic.ProductTree method*), 220
inverse() (*sage.rings.morphism.RingHomomorphism method*), 63
inverse() (*sage.rings.morphism.RingHomomorphism_from_base method*), 79
inverse() (*sage.rings.morphism.RingHomomorphism_from_fraction_field method*), 80
inverse_image() (*sage.rings.morphism.RingHomomorphism method*), 68
inverse_of_unit() (*sage.rings.localization.LocalizationElement method*), 151
is_commutative() (*sage.rings.infinity.InfinityRing_class method*), 235
is_commutative() (*sage.rings.quotient_ring.QuotientRing_nc method*), 103
is_defined_over() (*sage.rings.ring_extension.RingExtension_generic method*), 180
is_exact() (*sage.rings.fraction_field.FractionField_generic method*), 125
is_field() (*sage.rings.fraction_field.FractionField_generic method*), 126
is_field() (*sage.rings.localization.Localization method*), 149
is_field() (*sage.rings.quotient_ring.QuotientRing_nc method*), 104
is_field() (*sage.rings.ring_extension.RingExtension_generic method*), 181
is_finite() (*sage.rings.fraction_field.FractionField_generic method*), 126
is_finite_over() (*sage.rings.ring_extension.RingExtension_generic method*), 182
is_free_over() (*sage.rings.ring_extension.RingExtension_generic method*), 182
is_identity() (*sage.rings.ring_extension_morphism.RingExtensionHomomorphism method*), 212
is_injective() (*sage.rings.fraction_field.FractionFieldEmbedding method*), 121
is_injective() (*sage.rings.ring_extension_morphism.MapFreeModuleToRelativeRing method*), 209
is_injective() (*sage.rings.ring_extension_morphism.MapRelativeRingToFreeModule method*), 210
is_injective() (*sage.rings.ring_extension_mor-*

phism.RingExtensionHomomorphism method), 213
is_integral() (*sage.rings.fraction_field_element.FractionFieldElement_1poly_field method*), 134
is_integral_domain() (*sage.rings.quotient_ring.QuotientRing_nc method*), 105
is_invertible() (*sage.rings.morphism.RingHomomorphism method*), 71
is_maximal() (*sage.rings.ideal.Ideal_generic method*), 30
is_maximal() (*sage.rings.ideal.Ideal_pid method*), 39
is_nilpotent() (*sage.rings.ring_extension_element.RingExtensionElement method*), 192
is_noetherian() (*sage.rings.quotient_ring.QuotientRing_nc method*), 105
is_one() (*sage.rings.fraction_field_element.FractionFieldElement method*), 130
is_primary() (*sage.rings.ideal.Ideal_generic method*), 31
is_prime() (*sage.rings.ideal.Ideal_generic method*), 32
is_prime() (*sage.rings.ideal.Ideal_pid method*), 40
is_prime() (*sage.rings.ring_extension_element.RingExtensionElement method*), 192
is_principal() (*sage.rings.ideal.Ideal_generic method*), 34
is_principal() (*sage.rings.ideal.Ideal_principal method*), 44
is_square() (*sage.rings.fraction_field_element.FractionFieldElement method*), 130
is_square() (*sage.rings.ring_extension_element.RingExtensionElement method*), 193
is_surjective() (*sage.rings.fraction_field.FractionFieldEmbedding method*), 121
is_surjective() (*sage.rings.morphism.RingHomomorphism method*), 71
is_surjective() (*sage.rings.ring_extension_morphism.MapFreeModuleToRelativeRing method*), 209
is_surjective() (*sage.rings.ring_extension_morphism.MapRelativeRingToFreeModule method*), 210
is_surjective() (*sage.rings.ring_extension_morphism.RingExtensionHomomorphism method*), 214
is_trivial() (*sage.rings.ideal.Ideal_generic method*), 34
is_unit() (*sage.rings.localization.LocalizationElement method*), 152
is_unit() (*sage.rings.quotient_ring_element.QuotientRingElement method*), 113
is_unit() (*sage.rings.ring_extension_element.RingExtensionElement method*), 193
is_zero() (*sage.rings.derivation.RingDerivationWithoutTwist method*), 259
is_zero() (*sage.rings.derivation.RingDerivationWithoutTwist_function method*), 265
is_zero() (*sage.rings.derivation.RingDerivationWithoutTwist_zero method*), 267
is_zero() (*sage.rings.fraction_field_element.FractionFieldElement method*), 131
is_zero() (*sage.rings.infinity.InfinityRing_class method*), 236

K

Katsura() (*in module sage.rings.ideal*), 45
kernel() (*sage.rings.morphism.RingHomomorphism method*), 72
kernel() (*sage.rings.morphism.RingHomomorphism_cover method*), 75
krull_dimension() (*sage.rings.localization.Localization method*), 149

L

lc() (*sage.rings.quotient_ring_element.QuotientRingElement method*), 114
lcm() (*sage.rings.infinity.AnInfinity method*), 233
leaves() (*sage.rings.generic.ProductTree method*), 221
less_than_infinity() (*sage.rings.infinity.UnsignedInfinityRing_class method*), 239
LessThanInfinity (*class in sage.rings.infinity*), 236
lift() (*sage.rings.morphism.RingHomomorphism method*), 73
lift() (*sage.rings.quotient_ring_element.QuotientRingElement method*), 114
lift() (*sage.rings.quotient_ring.QuotientRing_nc method*), 106
lifting_map() (*sage.rings.quotient_ring.QuotientRing_nc method*), 107
list() (*sage.rings.derivation.RingDerivationWithoutTwist method*), 260
list() (*sage.rings.derivation.RingDerivationWithoutTwist_function method*), 266
list() (*sage.rings.derivation.RingDerivationWithoutTwist_wrapper method*), 266
list() (*sage.rings.derivation.RingDerivationWithoutTwist_zero method*), 267
list() (*sage.rings.derivation.RingDerivationWithTwist_generic method*), 256
lm() (*sage.rings.quotient_ring_element.QuotientRingElement method*), 115
Localization (*class in sage.rings.localization*), 144
LocalizationElement (*class in sage.rings.localization*), 150
lt() (*sage.rings.quotient_ring_element.QuotientRingElement method*), 115

M

make_element() (*in module sage.rings.fraction_field_el-*

ement), 136
make_element_old() (*in module sage.rings.fraction_field_element*), 136
MapFreeModuleToRelativeRing (*class in sage.rings.ring_extension_morphism*), 209
MapRelativeRingToFreeModule (*class in sage.rings.ring_extension_morphism*), 210
matrix() (*sage.rings.ring_extension_element.RingExtensionWithBasisElement method*), 198
maximal_order() (*sage.rings.fraction_field.FractionField_1poly_field method*), 123
minimal_associated_primes() (*sage.rings.ideal.Ideal_generic method*), 35
minpoly() (*sage.rings.ring_extension_element.RingExtensionWithBasisElement method*), 200
MinusInfinity (*class in sage.rings.infinity*), 237
module
    sage.rings.abc, 5
    sage.rings.big_oh, 225
    sage.rings.derivation, 243
    sage.rings.fraction_field, 119
    sage.rings.fraction_field_element, 129
    sage.rings.generic, 217
    sage.rings.homset, 86
    sage.rings.ideal, 19
    sage.rings.ideal_monoid, 46
    sage.rings.infinity, 228
    sage.rings.localization, 139
    sage.rings.morphism, 51
    sage.rings.noncommutative_ideals, 47
    sage.rings.numbers_abc, 242
    sage.rings.quotient_ring, 89
    sage.rings.quotient_ring_element, 111
    sage.rings.ring, 1
    sage.rings.ring_extension, 155
    sage.rings.ring_extension_element, 190
    sage.rings.ring_extension_morphism, 209
modulus() (*sage.rings.ring_extension.RingExtensionWithGen method*), 166
monomial_coefficients() (*sage.rings.derivation.RingDerivationWithoutTwist method*), 260
monomials() (*sage.rings.quotient_ring_element.QuotientRingElement method*), 115
morphism_from_cover() (*sage.rings.morphism.RingHomomorphism_from_quotient method*), 82
multiplicative_order() (*sage.rings.ring_extension_element.RingExtensionElement method*), 194

N

natural_map() (*sage.rings.homset.RingHomset_generic method*), 86
ngens() (*sage.rings.derivation.RingDerivationModule method*), 253
ngens() (*sage.rings.fraction_field.FractionField_generic method*), 126
ngens() (*sage.rings.ideal.Ideal_generic method*), 35
ngens() (*sage.rings.infinity.InfinityRing_class method*), 236
ngens() (*sage.rings.infinity.UnsignedInfinityRing_class method*), 239
ngens() (*sage.rings.localization.Localization method*), 149
ngens() (*sage.rings.quotient_ring.QuotientRing_nc method*), 109
ngens() (*sage.rings.ring_extension.RingExtension_generic method*), 183
NoetherianRing (*class in sage.rings.ring*), 2
norm() (*sage.rings.ideal.Ideal_generic method*), 35
norm() (*sage.rings.ring_extension_element.RingExtensionWithBasisElement method*), 202
normalize_extra_units() (*in module sage.rings.localization*), 152
nth_root() (*sage.rings.fraction_field_element.FractionFieldElement method*), 132
NumberField_cyclotomic (*class in sage.rings.abc*), 11
NumberField_quadratic (*class in sage.rings.abc*), 12
numerator() (*sage.rings.fraction_field_element.FractionFieldElement method*), 132
numerator() (*sage.rings.localization.LocalizationElement method*), 152
numerator() (*sage.rings.ring_extension_element.RingExtensionFractionFieldElement method*), 196

O

O() (*in module sage.rings.big_oh*), 225
one() (*sage.rings.ring.Ring method*), 4
Order (*class in sage.rings.abc*), 12
order() (*sage.rings.ring.Ring method*), 4

P

pAdicField (*class in sage.rings.abc*), 17
pAdicRing (*class in sage.rings.abc*), 17
PlusInfinity (*class in sage.rings.infinity*), 237
polynomial() (*sage.rings.ring_extension_element.RingExtensionWithBasisElement method*), 204
postcompose() (*sage.rings.derivation.RingDerivationWithoutTwist method*), 261
postcompose() (*sage.rings.derivation.RingDerivationWithTwist_generic method*), 257
power() (*sage.rings.morphism.FrobeniusEndomorphism_generic method*), 63
precompose() (*sage.rings.derivation.RingDerivationWithoutTwist method*), 262

precompose() (*sage.rings.derivation.RingDerivationWithTwist_generic method*), 258
primary_decomposition() (*sage.rings.ideal.Ideal_generic method*), 36
PrincipalIdealDomain (*class in sage.rings.ring*), 2
print_options() (*sage.rings.ring_extension.RingExtension_generic method*), 184
prod_with_derivative() (*in module sage.rings.generic*), 223
ProductTree (*class in sage.rings.generic*), 217
pth_power() (*sage.rings.derivation.RingDerivationWithoutTwist method*), 263
pushforward() (*sage.rings.morphism.RingHomomorphism method*), 74
Python Enhancement Proposals
PEP 3141, 242

Q

QuotientRing() (*in module sage.rings.quotient_ring*), 92
QuotientRing_generic (*class in sage.rings.quotient_ring*), 95
QuotientRing_nc (*class in sage.rings.quotient_ring*), 96
QuotientRingElement (*class in sage.rings.quotient_ring_element*), 111
QuotientRingIdeal_generic (*class in sage.rings.quotient_ring*), 95
QuotientRingIdeal_principal (*class in sage.rings.quotient_ring*), 95

R

radical() (*sage.rings.ideal.Ideal_pid method*), 41
radical() (*sage.rings.quotient_ring.QuotientRingIdeal_generic method*), 95
random_element() (*sage.rings.derivation.RingDerivationModule method*), 254
random_element() (*sage.rings.fraction_field.FractionField_generic method*), 127
random_element() (*sage.rings.ideal.Ideal_generic method*), 36
random_element() (*sage.rings.quotient_ring.QuotientRing_nc method*), 110
random_element() (*sage.rings.ring_extension.RingExtension_generic method*), 186
RealBallField (*class in sage.rings.abc*), 13
RealDoubleField (*class in sage.rings.abc*), 14
RealField (*class in sage.rings.abc*), 15
RealIntervalField (*class in sage.rings.abc*), 15
reduce() (*sage.rings.fraction_field_element.FractionFieldElement method*), 133
reduce() (*sage.rings.fraction_field_element.FractionFieldElement_1poly_field method*), 135
reduce() (*sage.rings.ideal.Ideal_generic method*), 37
reduce() (*sage.rings.ideal.Ideal_pid method*), 41
reduce() (*sage.rings.quotient_ring_element.QuotientRingElement method*), 116
relative_degree() (*sage.rings.ring_extension.RingExtension_generic method*), 186
remainders() (*sage.rings.generic.ProductTree method*), 222
residue_field() (*sage.rings.ideal.Ideal_pid method*), 42
retract() (*sage.rings.quotient_ring.QuotientRing_nc method*), 110
Ring (*class in sage.rings.ring*), 2
ring() (*sage.rings.fraction_field.FractionField_generic method*), 127
ring() (*sage.rings.ideal_monoid.IdealMonoid_c method*), 47
ring() (*sage.rings.ideal.Ideal_generic method*), 37
ring() (*sage.rings.ring_extension.RingExtensionFractionField method*), 158
ring_of_constants() (*sage.rings.derivation.RingDerivationModule method*), 254
ring_of_integers() (*sage.rings.fraction_field.FractionField_1poly_field method*), 123
RingDerivation (*class in sage.rings.derivation*), 248
RingDerivationModule (*class in sage.rings.derivation*), 249
RingDerivationWithoutTwist (*class in sage.rings.derivation*), 258
RingDerivationWithoutTwist_fraction_field (*class in sage.rings.derivation*), 265
RingDerivationWithoutTwist_function (*class in sage.rings.derivation*), 265
RingDerivationWithoutTwist_quotient (*class in sage.rings.derivation*), 266
RingDerivationWithoutTwist_wrapper (*class in sage.rings.derivation*), 266
RingDerivationWithoutTwist_zero (*class in sage.rings.derivation*), 267
RingDerivationWithTwist_generic (*class in sage.rings.derivation*), 256
RingExtension_generic (*class in sage.rings.ring_extension*), 167
RingExtensionBackendIsomorphism (*class in sage.rings.ring_extension_morphism*), 211
RingExtensionBackendReverseIsomorphism (*class in sage.rings.ring_extension_morphism*), 211
RingExtensionElement (*class in sage.rings.ring_extension_element*), 190
RingExtensionFactory (*class in sage.rings.ring_extension*), 158
RingExtensionFractionField (*class in sage.rings.ring_extension*), 158
RingExtensionFractionFieldElement (*class in sage.rings.ring_extension_element*), 195
RingExtensionHomomorphism (*class in*

sage.rings.ring_extension_morphism), 211
RingExtensionWithBasis (*class in sage.rings.ring_extension*), 159
RingExtensionWithBasisElement (*class in sage.rings.ring_extension_element*), 197
RingExtensionWithGen (*class in sage.rings.ring_extension*), 164
RingHomomorphism (*class in sage.rings.morphism*), 63
RingHomomorphism_cover (*class in sage.rings.morphism*), 74
RingHomomorphism_from_base (*class in sage.rings.morphism*), 75
RingHomomorphism_from_fraction_field (*class in sage.rings.morphism*), 80
RingHomomorphism_from_quotient (*class in sage.rings.morphism*), 81
RingHomomorphism_im_gens (*class in sage.rings.morphism*), 82
RingHomset() (*in module sage.rings.homset*), 86
RingHomset_generic (*class in sage.rings.homset*), 86
RingHomset_quo_ring (*class in sage.rings.homset*), 87
RingMap (*class in sage.rings.morphism*), 84
RingMap_lift (*class in sage.rings.morphism*), 84
root() (*sage.rings.generic.ProductTree method*), 222

S

sage.rings.abc
    module, 5
sage.rings.big_oh
    module, 225
sage.rings.derivation
    module, 243
sage.rings.fraction_field
    module, 119
sage.rings.fraction_field_element
    module, 129
sage.rings.generic
    module, 217
sage.rings.homset
    module, 86
sage.rings.ideal
    module, 19
sage.rings.ideal_monoid
    module, 46
sage.rings.infinity
    module, 228
sage.rings.localization
    module, 139
sage.rings.morphism
    module, 51
sage.rings.noncommutative_ideals
    module, 47
sage.rings.numbers_abc
    module, 242
sage.rings.quotient_ring
    module, 89
sage.rings.quotient_ring_element
    module, 111
sage.rings.ring
    module, 1
sage.rings.ring_extension
    module, 155
sage.rings.ring_extension_element
    module, 190
sage.rings.ring_extension_morphism
    module, 209
section() (*sage.rings.fraction_field.FractionFieldEmbedding method*), 122
side() (*sage.rings.noncommutative_ideals.Ideal_nc method*), 50
sign() (*sage.rings.infinity.FiniteNumber method*), 234
sign() (*sage.rings.infinity.LessThanInfinity method*), 236
SignError, 238
some_elements() (*sage.rings.derivation.RingDerivationModule method*), 255
some_elements() (*sage.rings.fraction_field.FractionField_generic method*), 128
specialization() (*sage.rings.fraction_field_element.FractionFieldElement method*), 133
sqrt() (*sage.rings.infinity.FiniteNumber method*), 234
sqrt() (*sage.rings.infinity.MinusInfinity method*), 237
sqrt() (*sage.rings.infinity.PlusInfinity method*), 237
sqrt() (*sage.rings.ring_extension_element.RingExtensionElement method*), 194
subs() (*sage.rings.fraction_field_element.FractionFieldElement method*), 133
support() (*sage.rings.fraction_field_element.FractionFieldElement_1poly_field method*), 135

T

term_order() (*sage.rings.quotient_ring.QuotientRing_nc method*), 111
tower_bases() (*in module sage.rings.ring_extension*), 188
trace() (*sage.rings.ring_extension_element.RingExtensionWithBasisElement method*), 206
twisting_morphism() (*sage.rings.derivation.RingDerivationModule method*), 255

U

underlying_map() (*sage.rings.morphism.RingHomomorphism_from_base method*), 80
UniversalCyclotomicField (*class in sage.rings.abc*), 16
UnsignedInfinity (*class in sage.rings.infinity*), 238
UnsignedInfinityRing_class (*class in sage.rings.infinity*), 238

V

valuation() (*sage.rings.fraction_field_element.FractionFieldElement method*), 134
variable_names() (*in module sage.rings.ring_extension*), 189
variable_names_recursive() (*sage.rings.fraction_field.FractionField_generic method*), 128
variables() (*sage.rings.quotient_ring_element.QuotientRingElement method*), 117
vector() (*sage.rings.ring_extension_element.RingExtensionWithBasisElement method*), 208

Z

zero() (*sage.rings.homset.RingHomset_generic method*), 87
zero() (*sage.rings.ring.Ring method*), 5